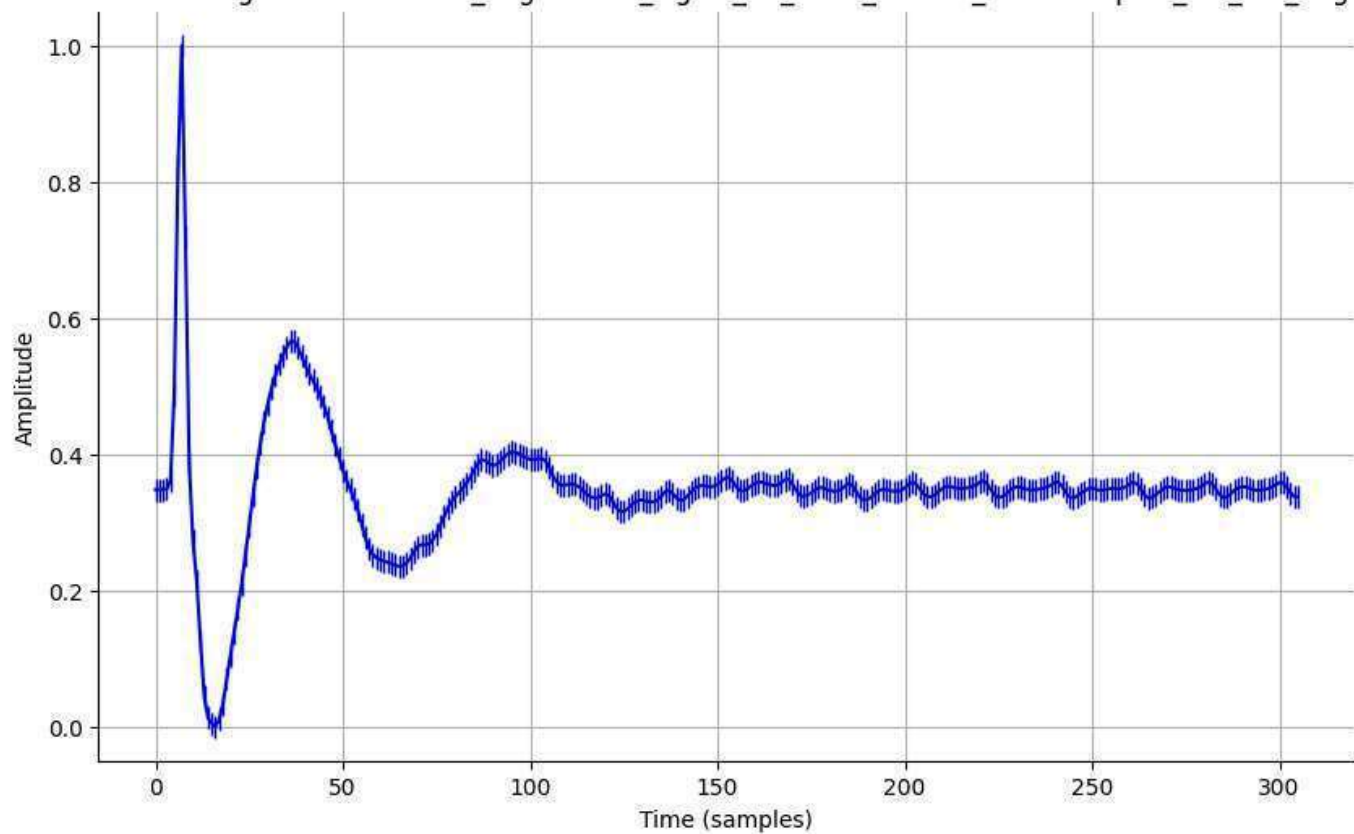
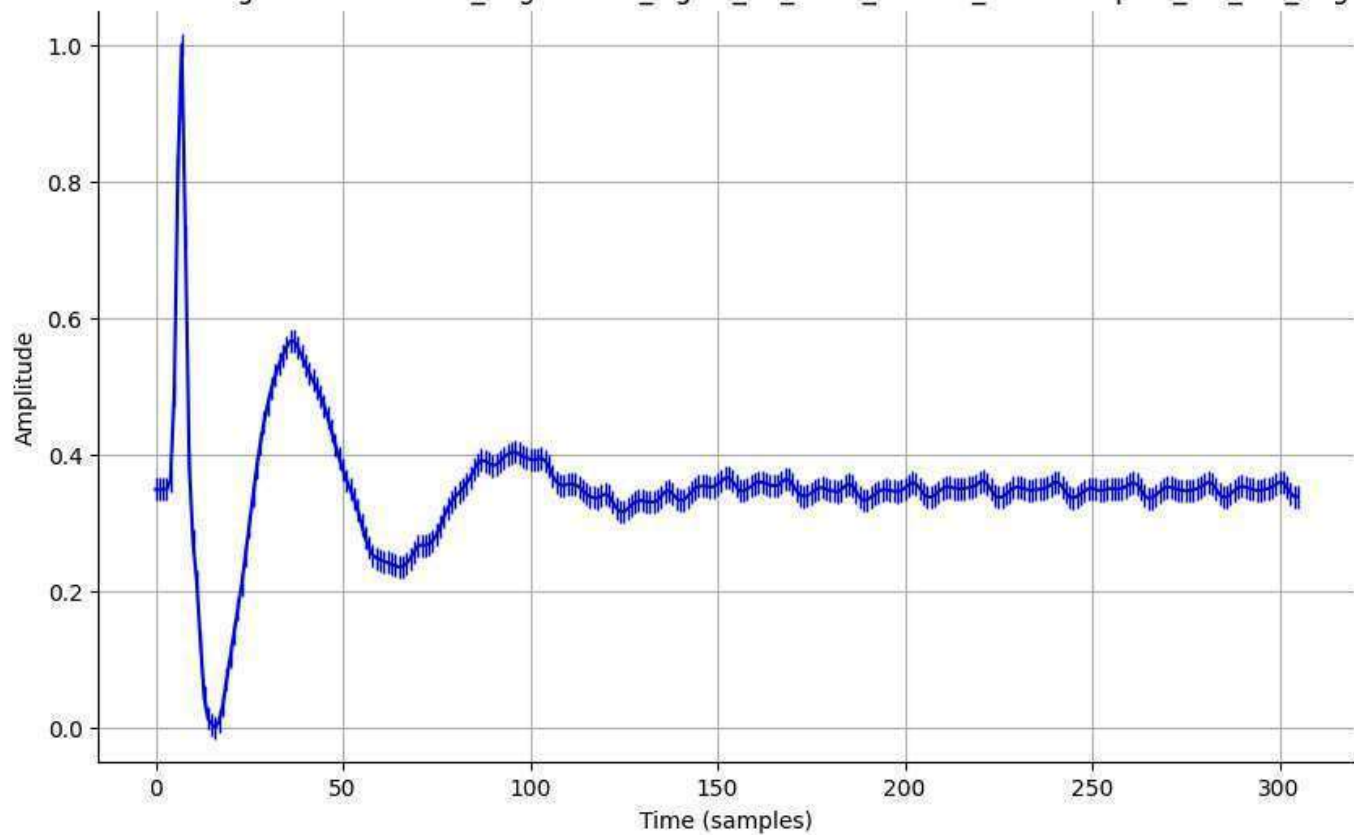


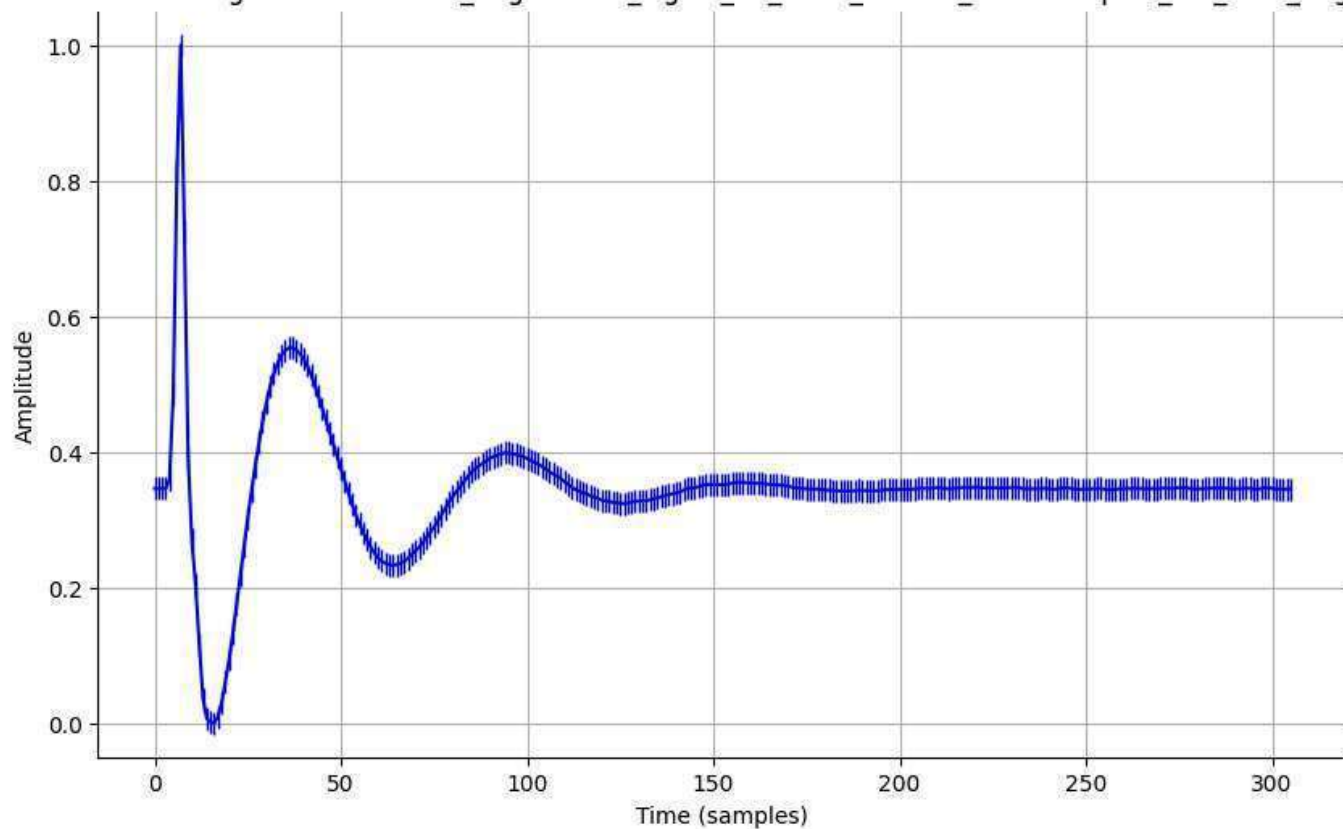
Normalized Signal - normalized_augmented_signal_16_0002_filtered_downsampled_std_0.1_aug.csv



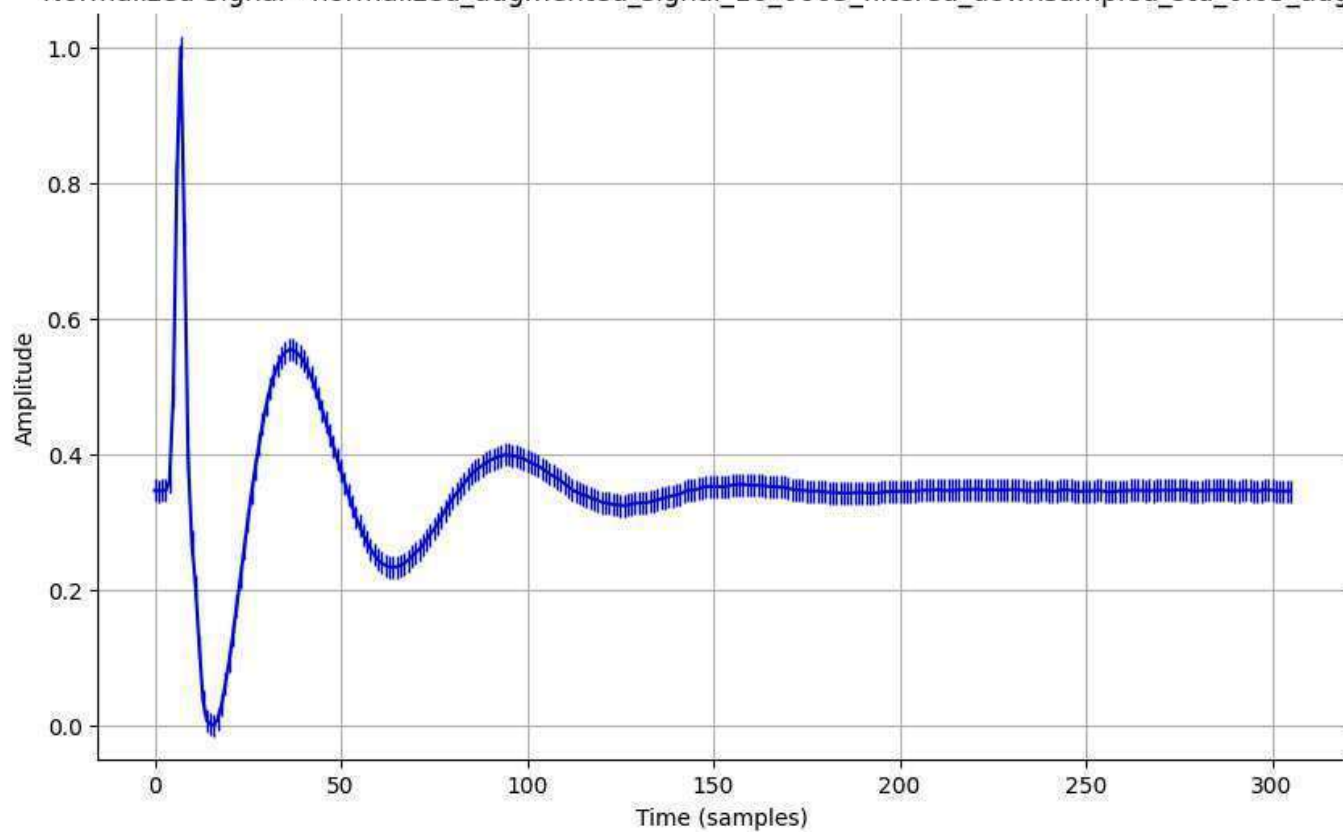
Normalized Signal - normalized_augmented_signal_16_0002_filtered_downsampled_std_0.2_aug.csv



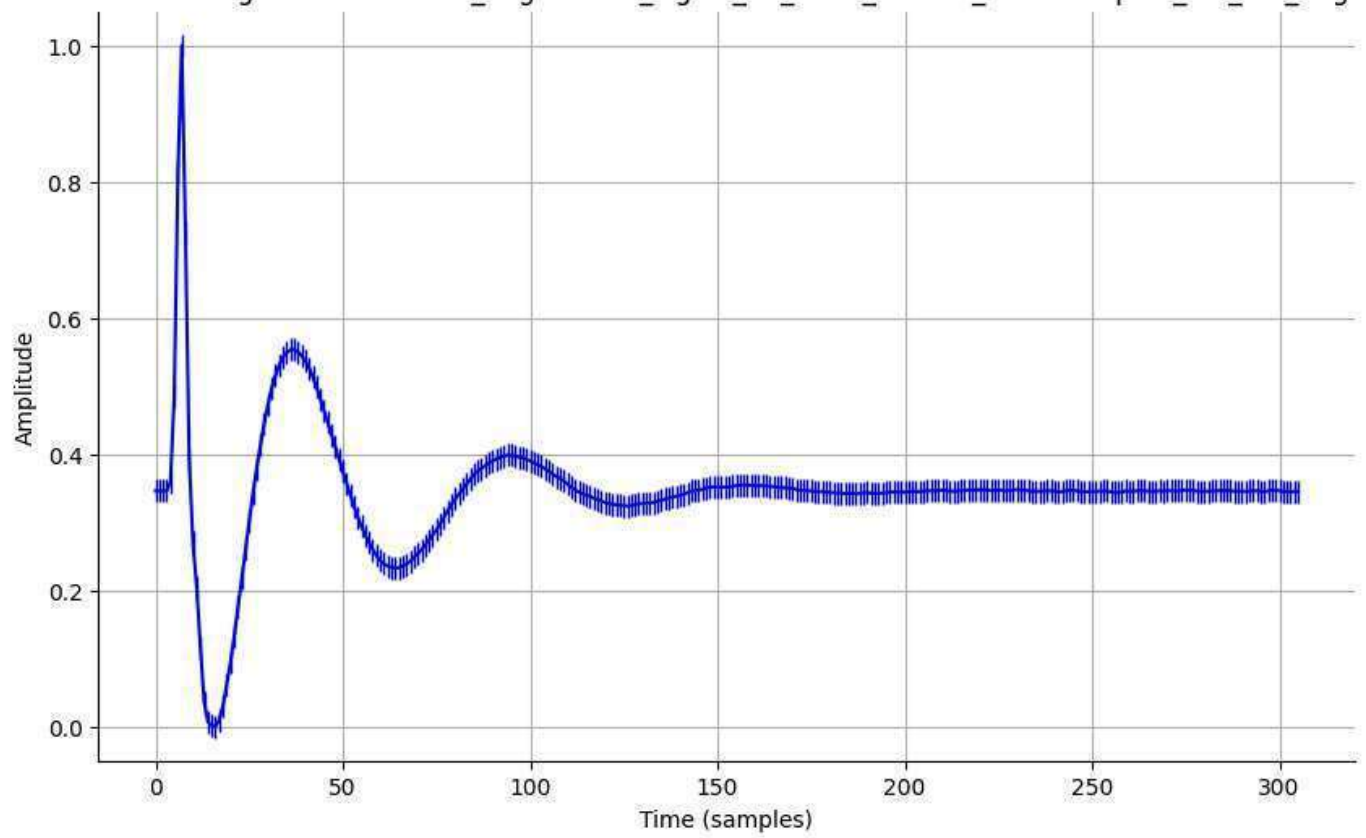
Normalized Signal - normalized_augmented_signal_16_0003_filtered_downsampled_std_0.01_aug.csv



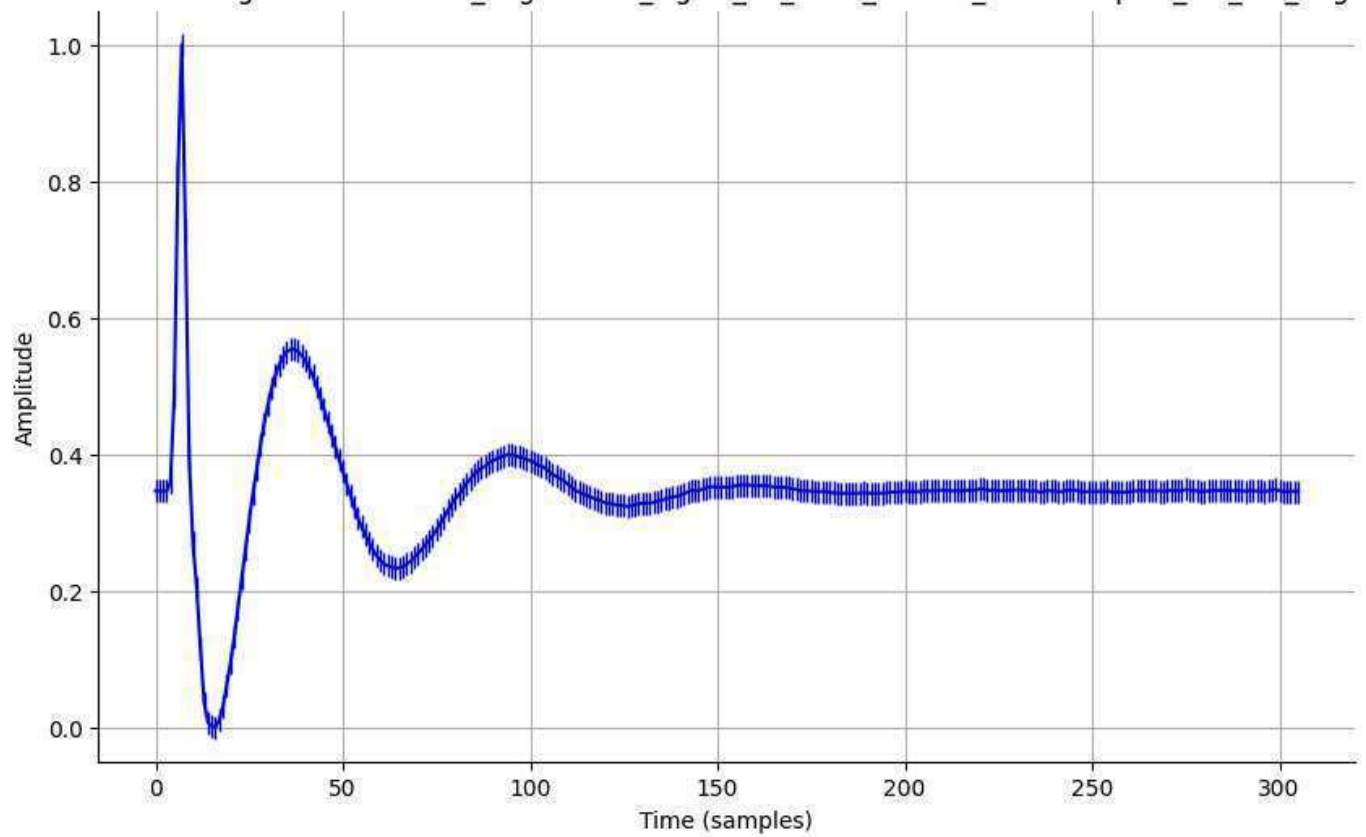
Normalized Signal - normalized_augmented_signal_16_0003_filtered_downsampled_std_0.05_aug.csv



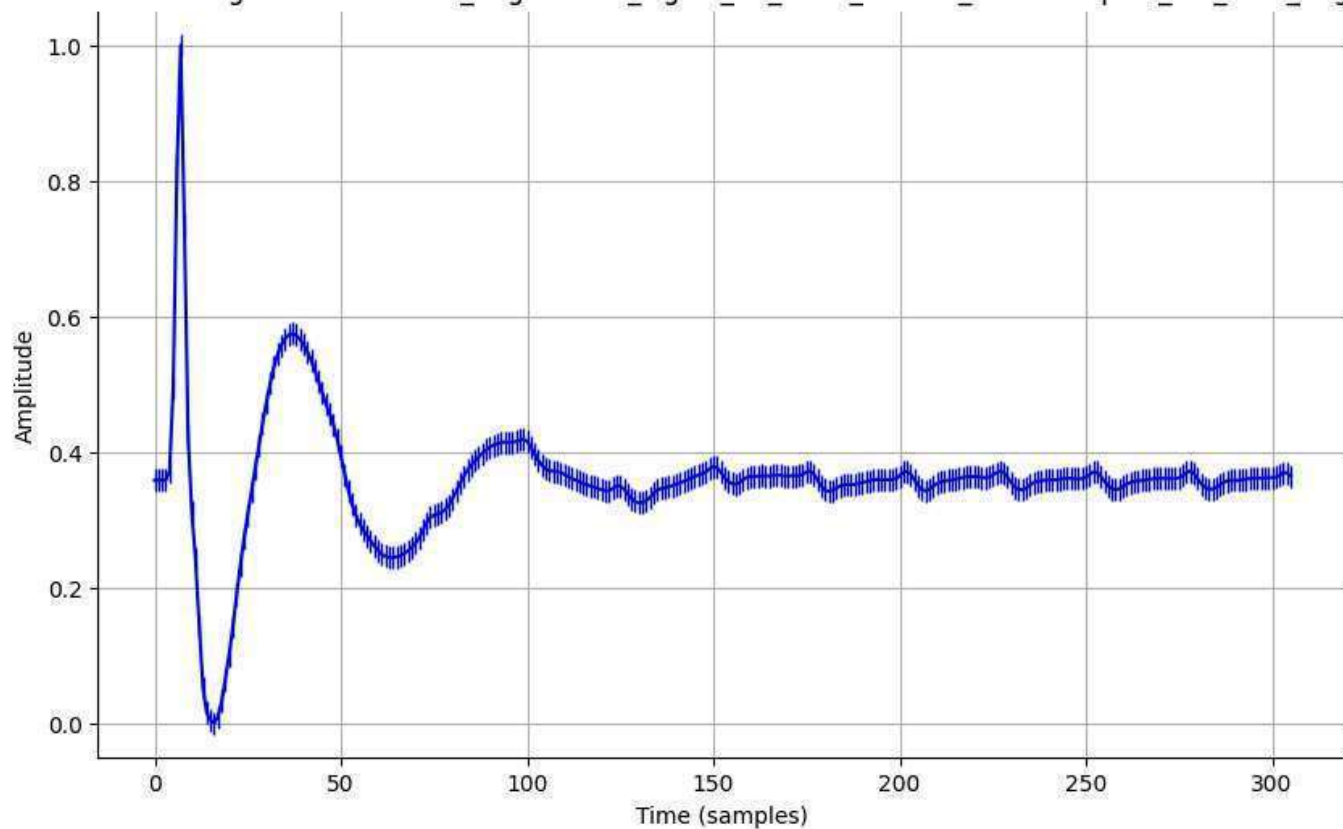
Normalized Signal - normalized_augmented_signal_16_0003_filtered_downsampled_std_0.1_aug.csv



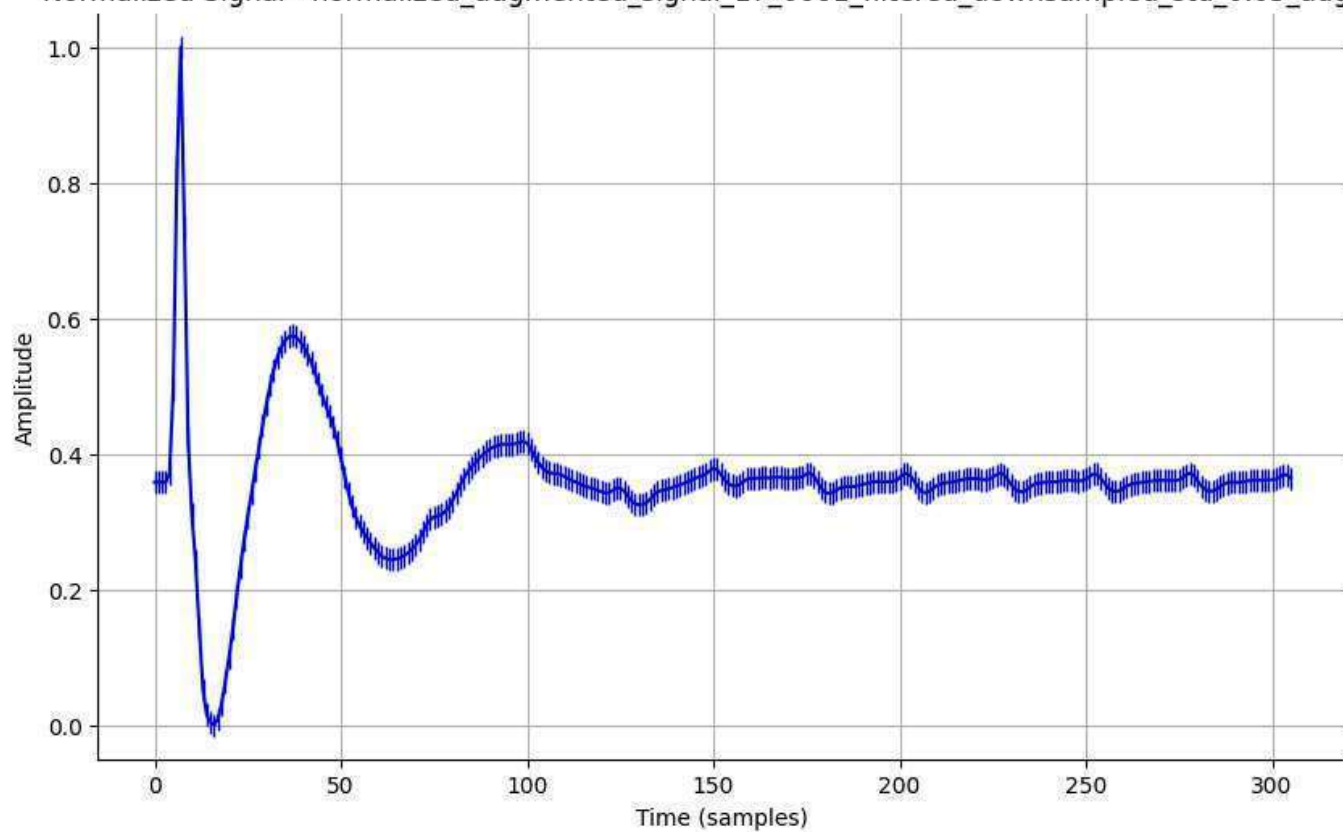
Normalized Signal - normalized_augmented_signal_16_0003_filtered_downsampled_std_0.2_aug.csv



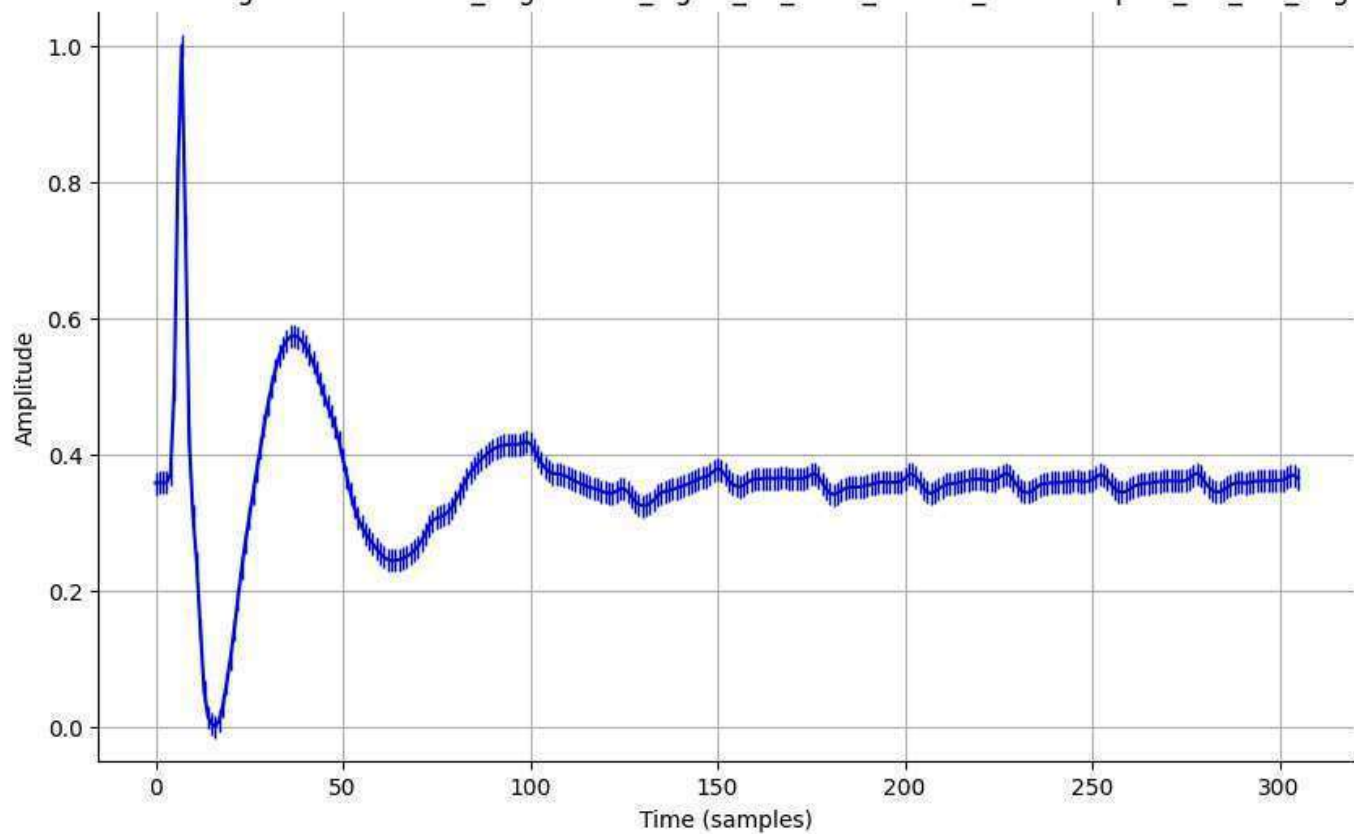
Normalized Signal - normalized_augmented_signal_17_0001_filtered_downsampled_std_0.01_aug.csv



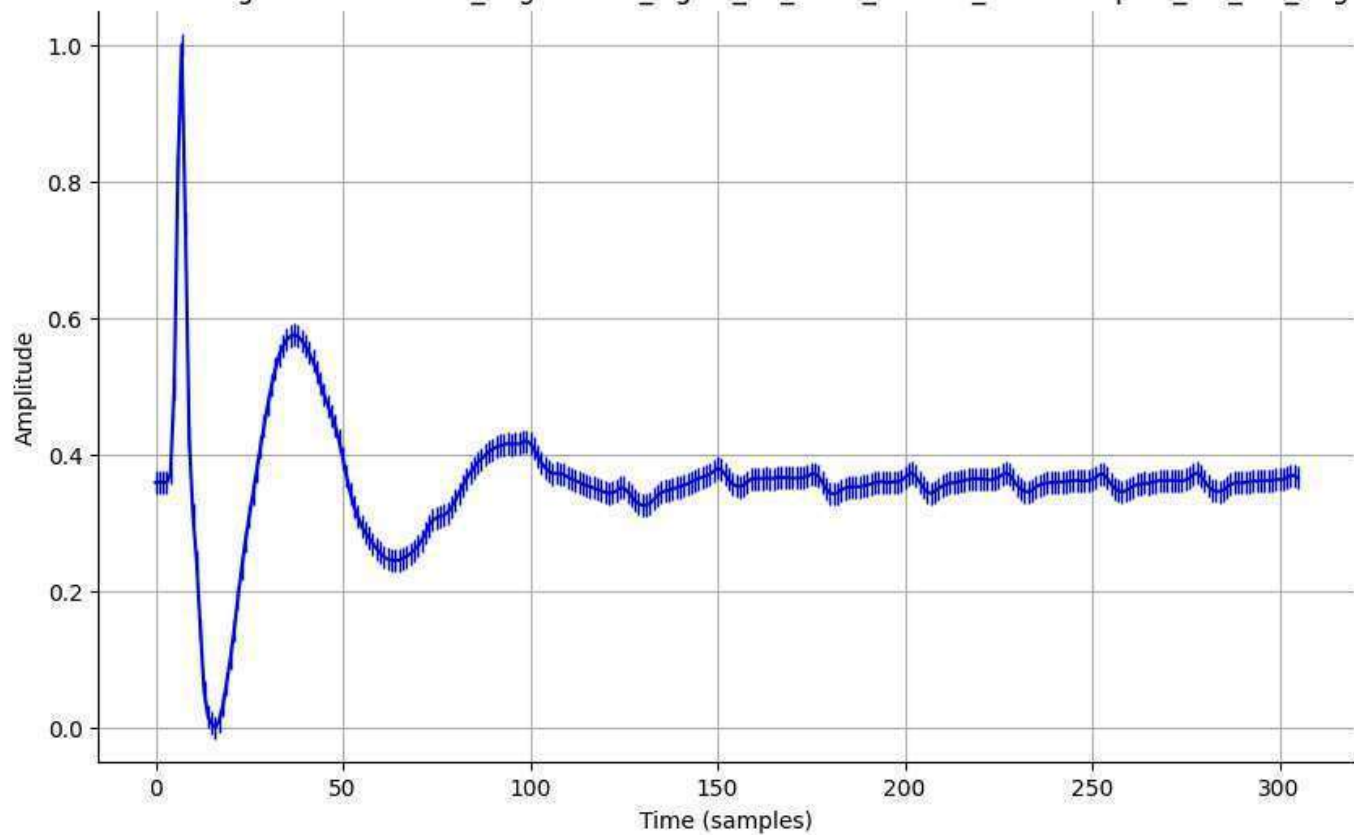
Normalized Signal - normalized_augmented_signal_17_0001_filtered_downsampled_std_0.05_aug.csv



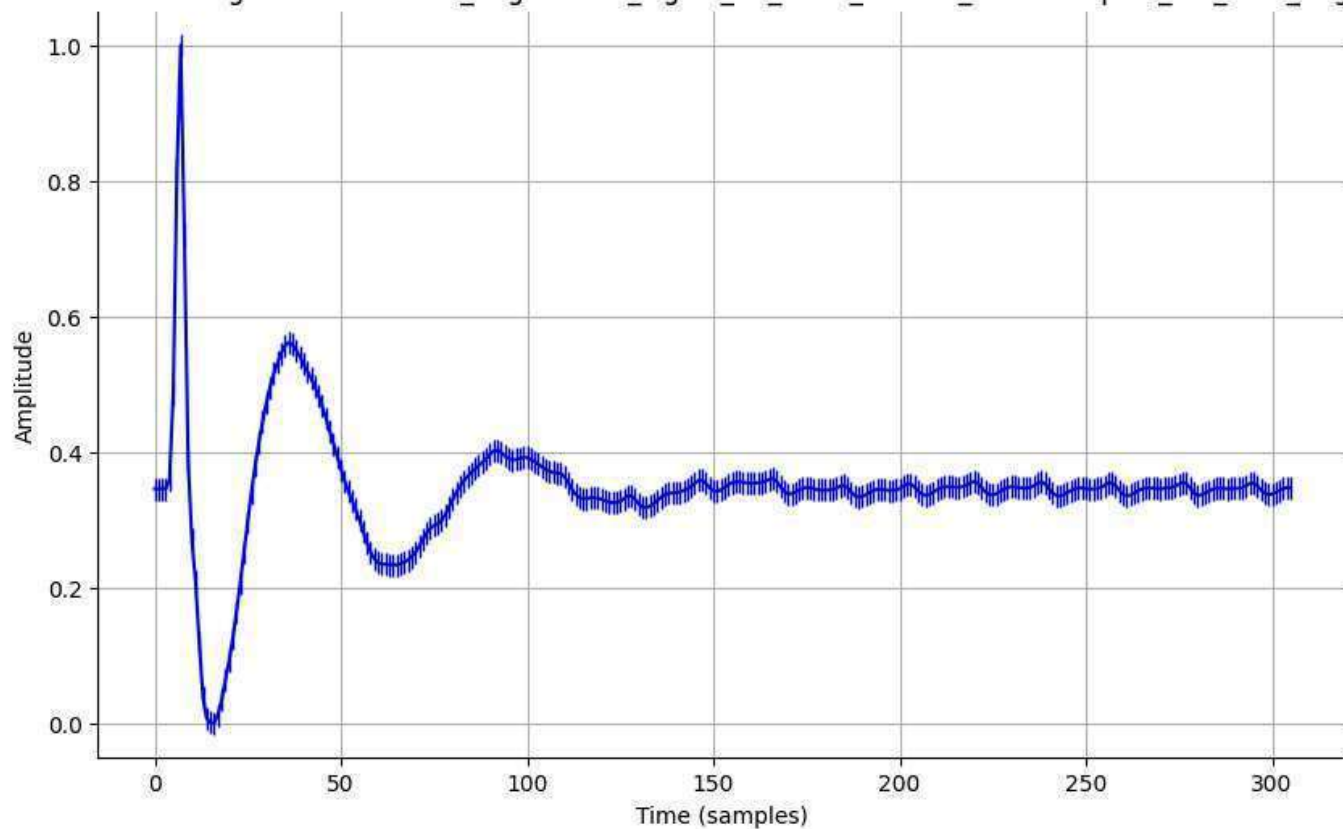
Normalized Signal - normalized_augmented_signal_17_0001_filtered_downsampled_std_0.1_aug.csv



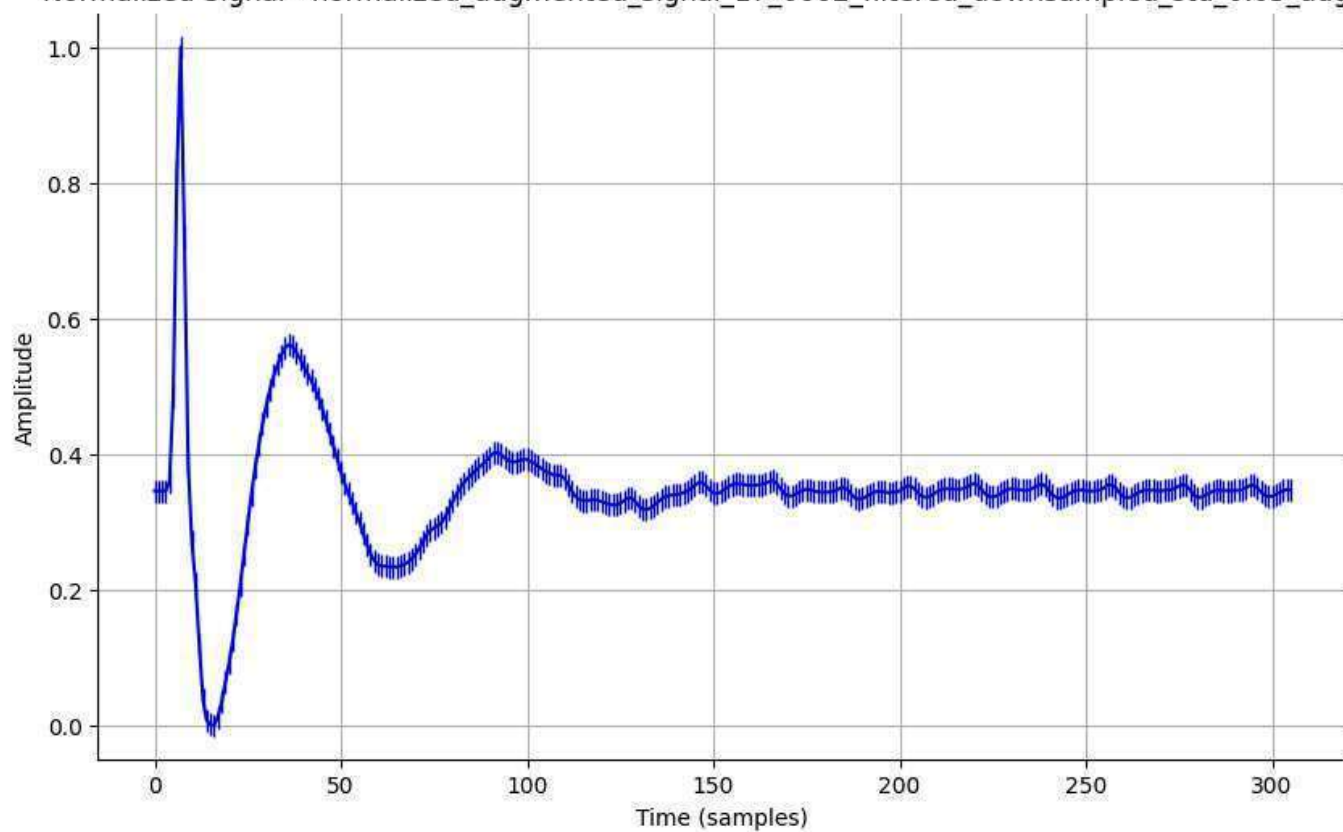
Normalized Signal - normalized_augmented_signal_17_0001_filtered_downsampled_std_0.2_aug.csv



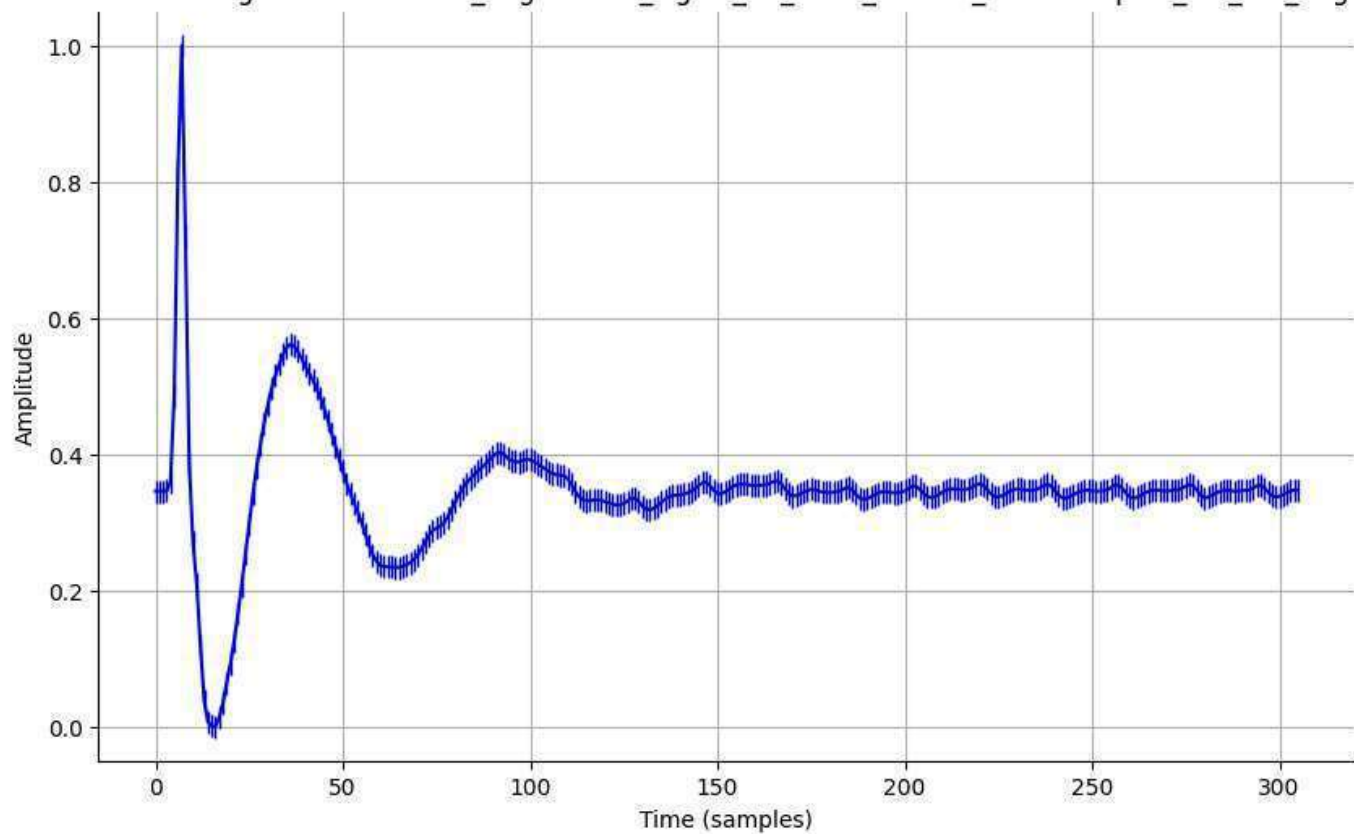
Normalized Signal - normalized_augmented_signal_17_0002_filtered_downsampled_std_0.01_aug.csv



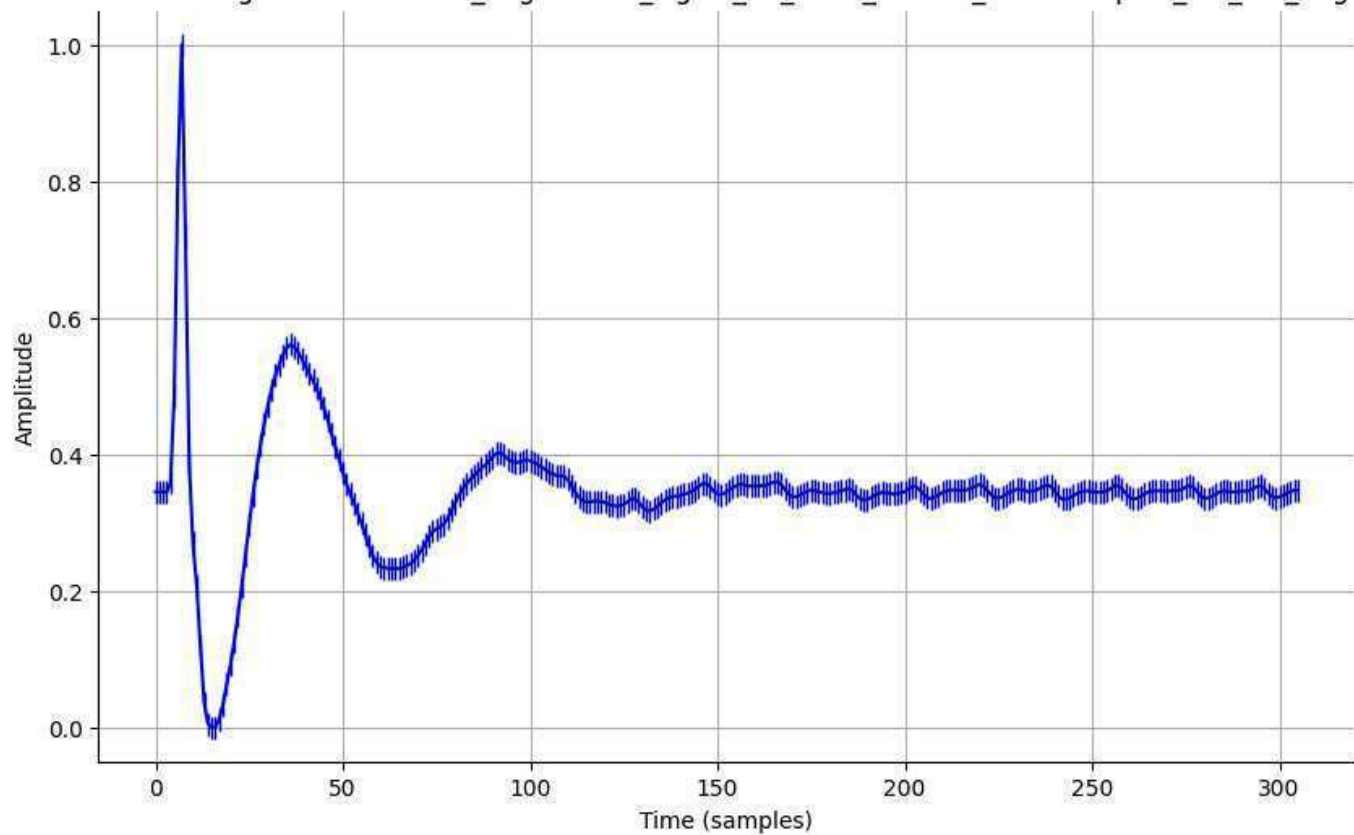
Normalized Signal - normalized_augmented_signal_17_0002_filtered_downsampled_std_0.05_aug.csv



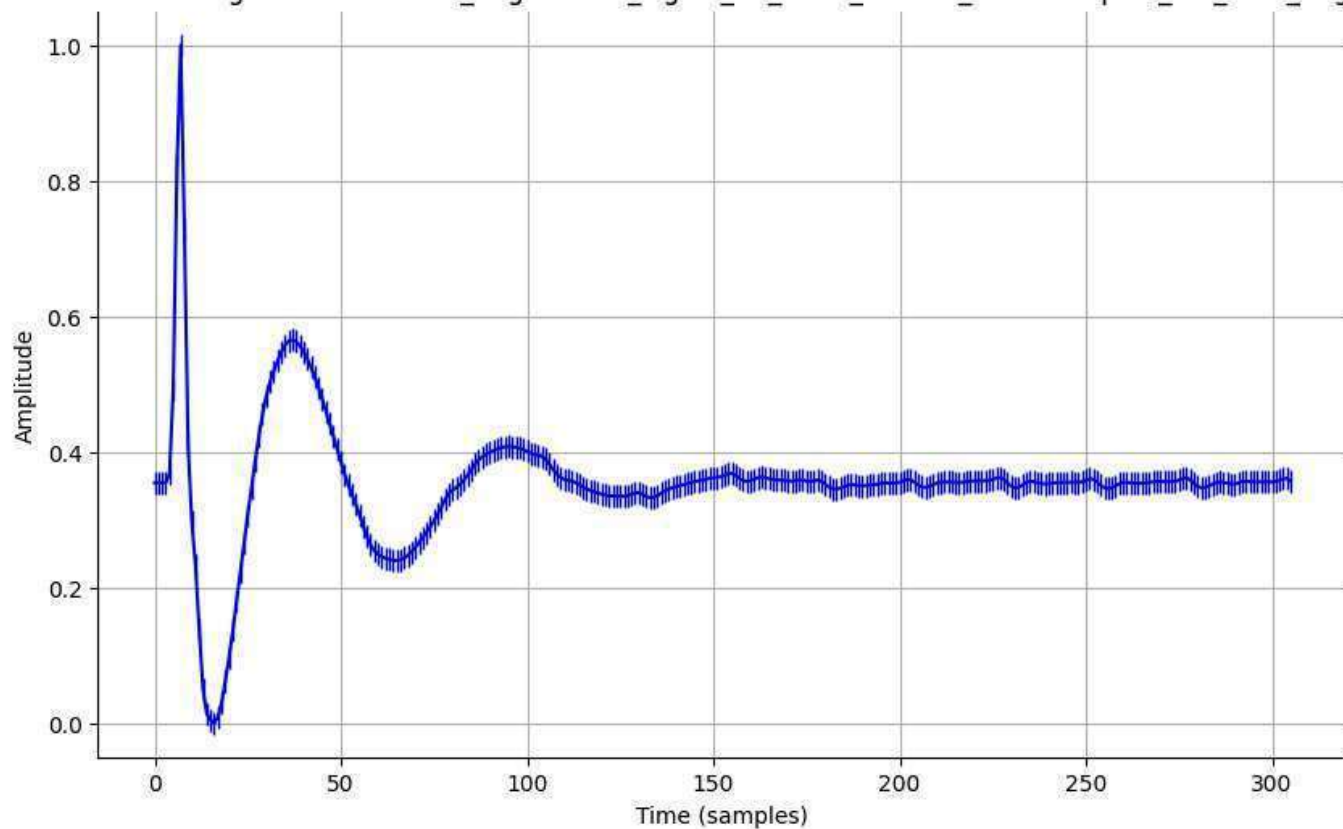
Normalized Signal - normalized_augmented_signal_17_0002_filtered_downsampled_std_0.1_aug.csv



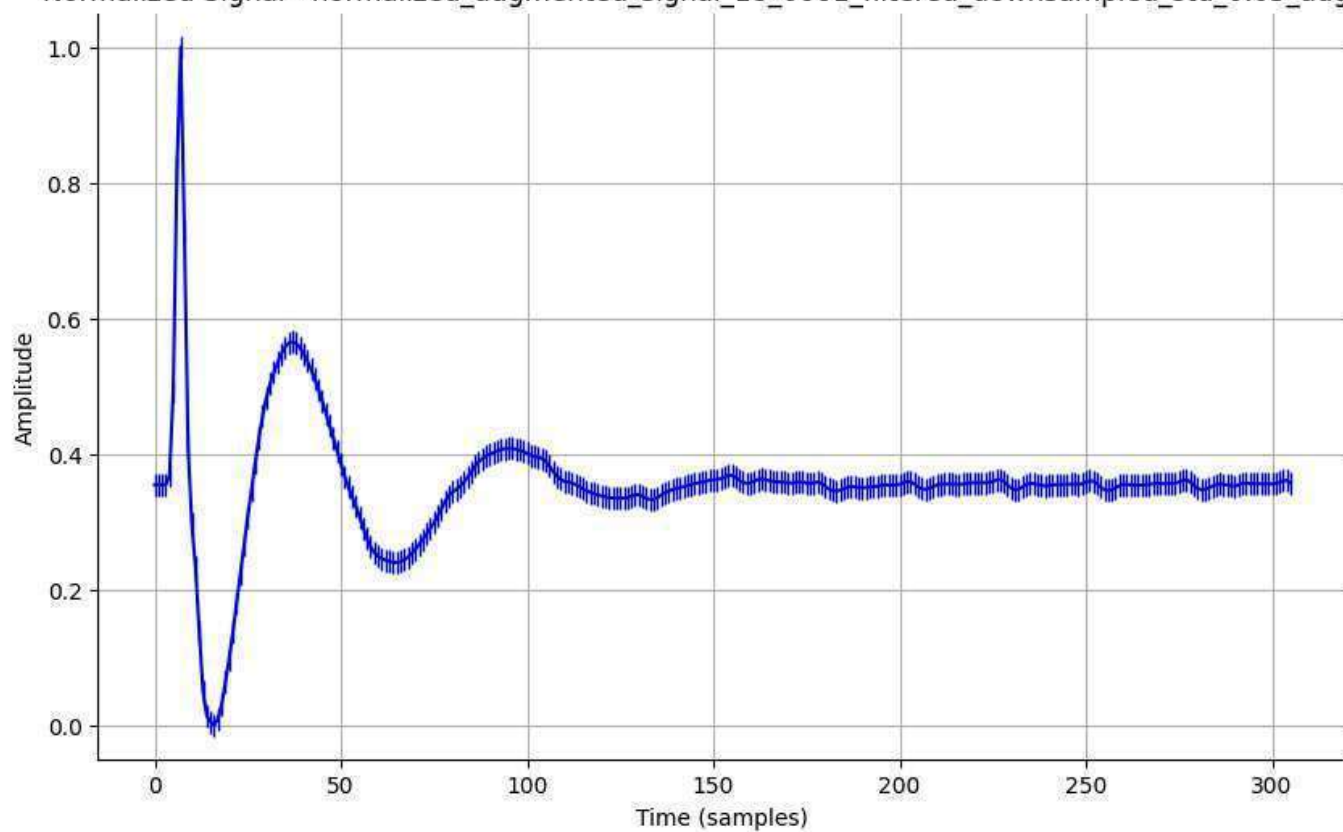
Normalized Signal - normalized_augmented_signal_17_0002_filtered_downsampled_std_0.2_aug.csv



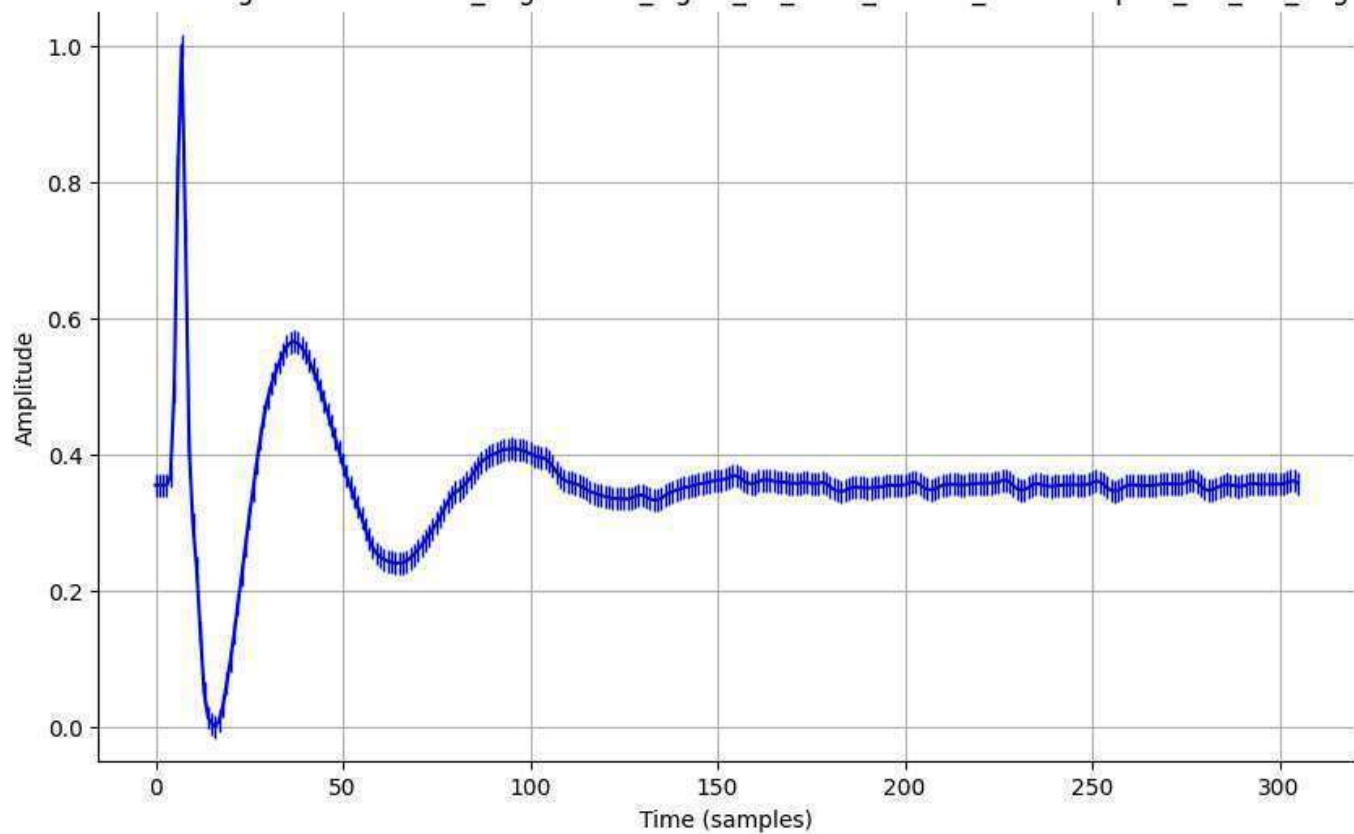
Normalized Signal - normalized_augmented_signal_18_0001_filtered_downsampled_std_0.01_aug.csv



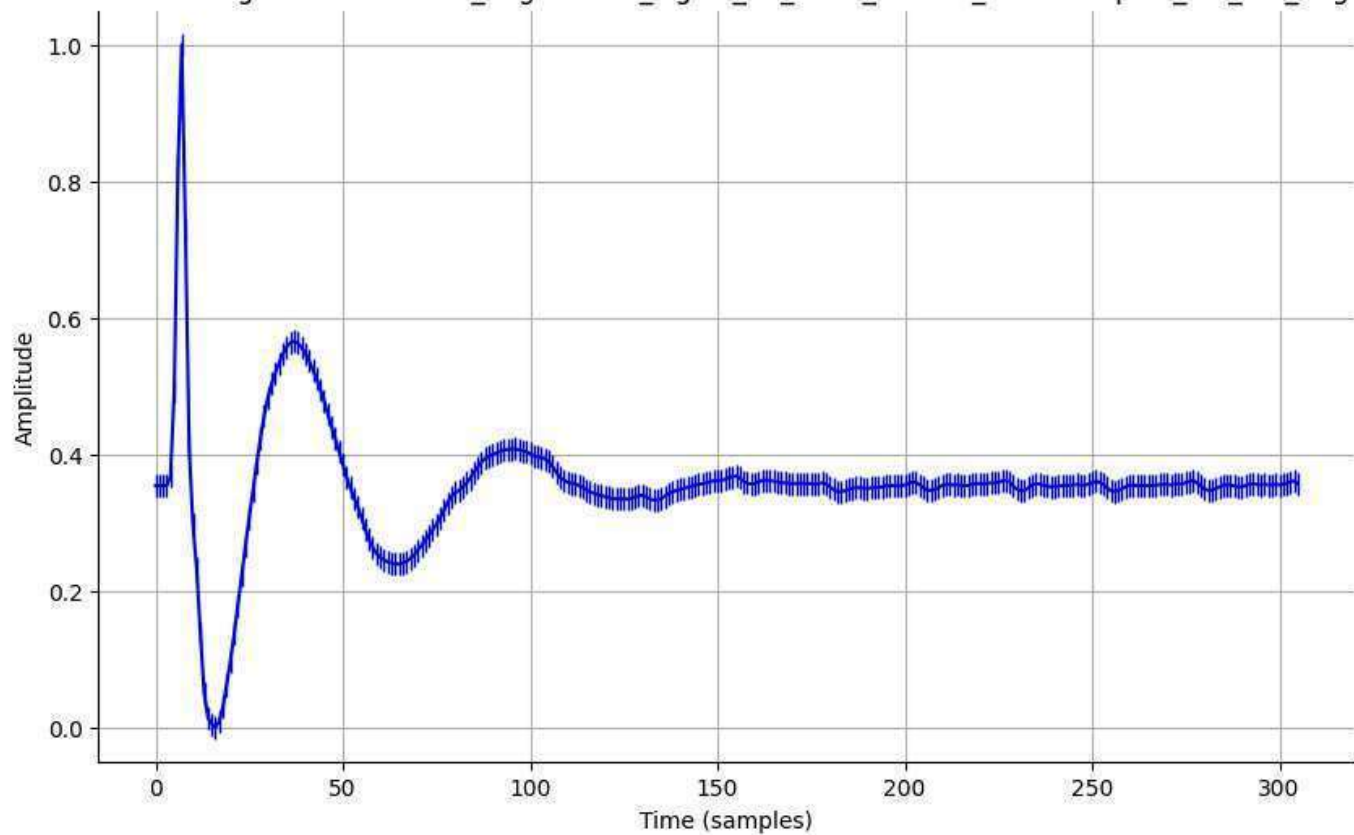
Normalized Signal - normalized_augmented_signal_18_0001_filtered_downsampled_std_0.05_aug.csv



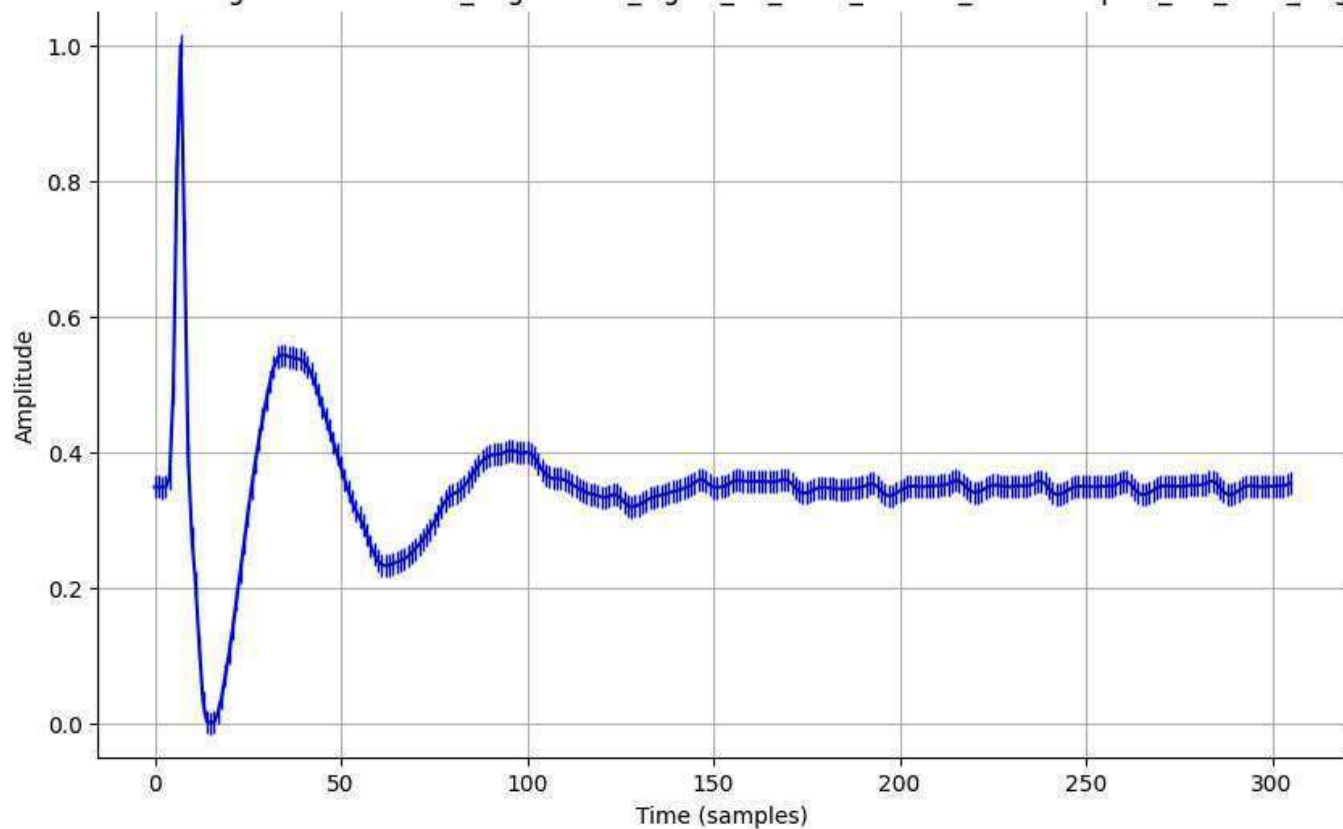
Normalized Signal - normalized_augmented_signal_18_0001_filtered_downsampled_std_0.1_aug.csv



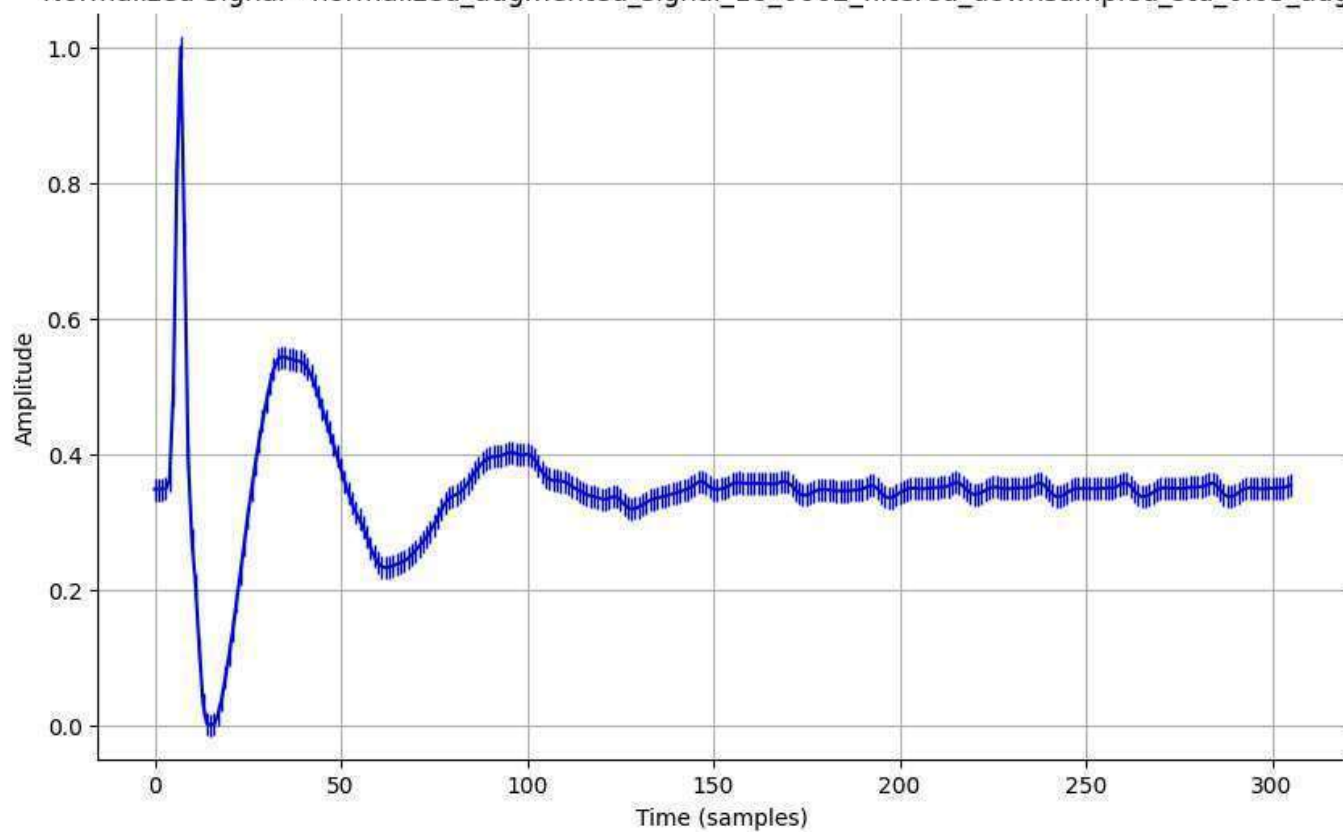
Normalized Signal - normalized_augmented_signal_18_0001_filtered_downsampled_std_0.2_aug.csv



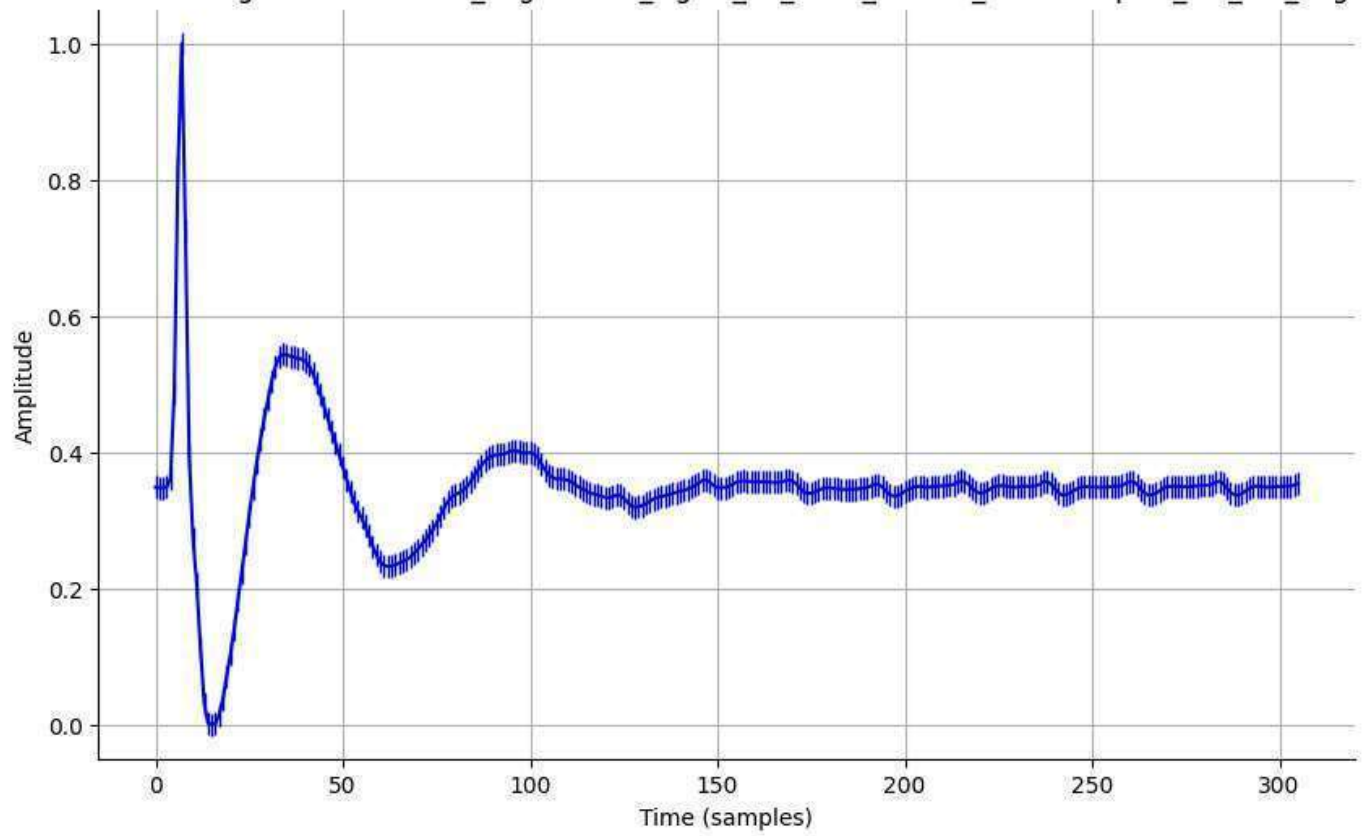
Normalized Signal - normalized_augmented_signal_18_0002_filtered_downsampled_std_0.01_aug.csv



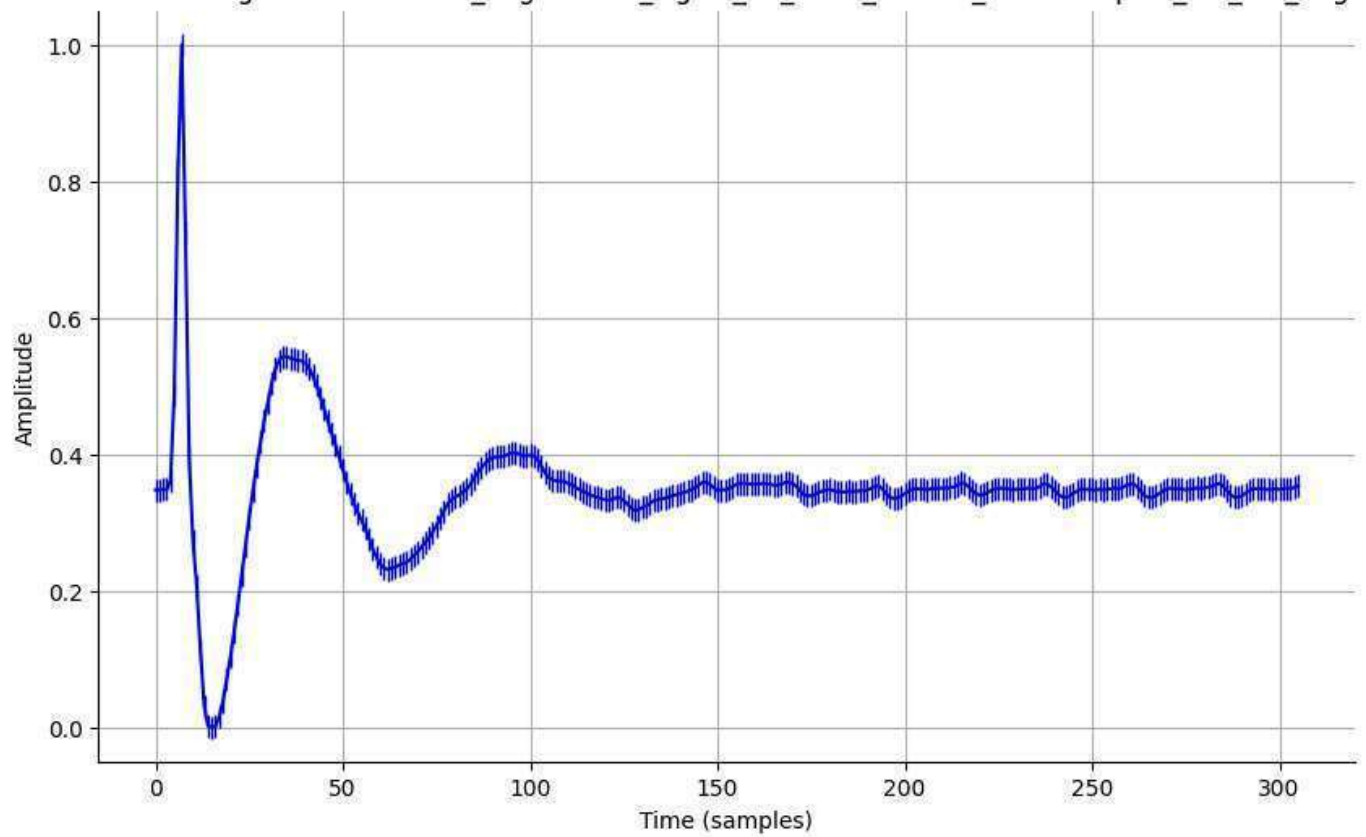
Normalized Signal - normalized_augmented_signal_18_0002_filtered_downsampled_std_0.05_aug.csv



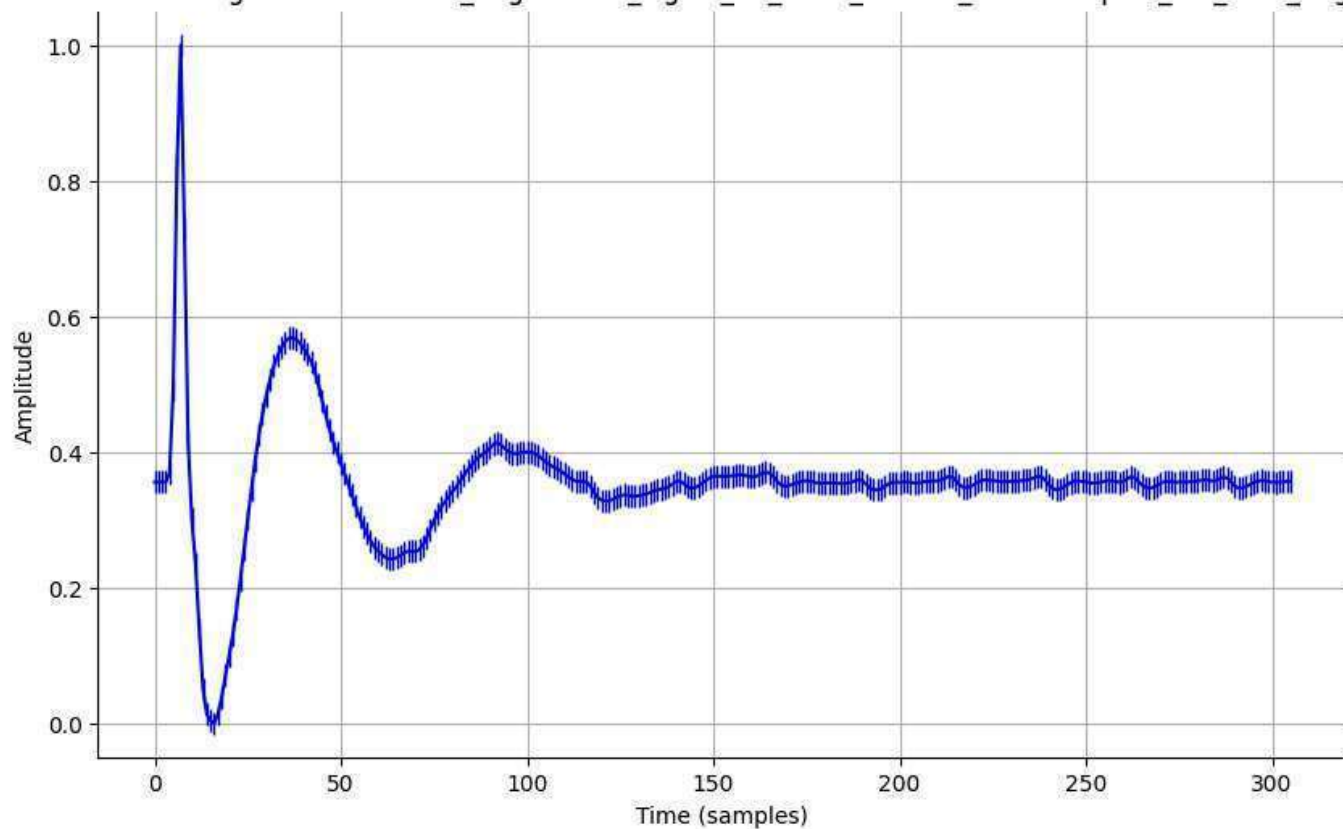
Normalized Signal - normalized_augmented_signal_18_0002_filtered_downsampled_std_0.1_aug.csv



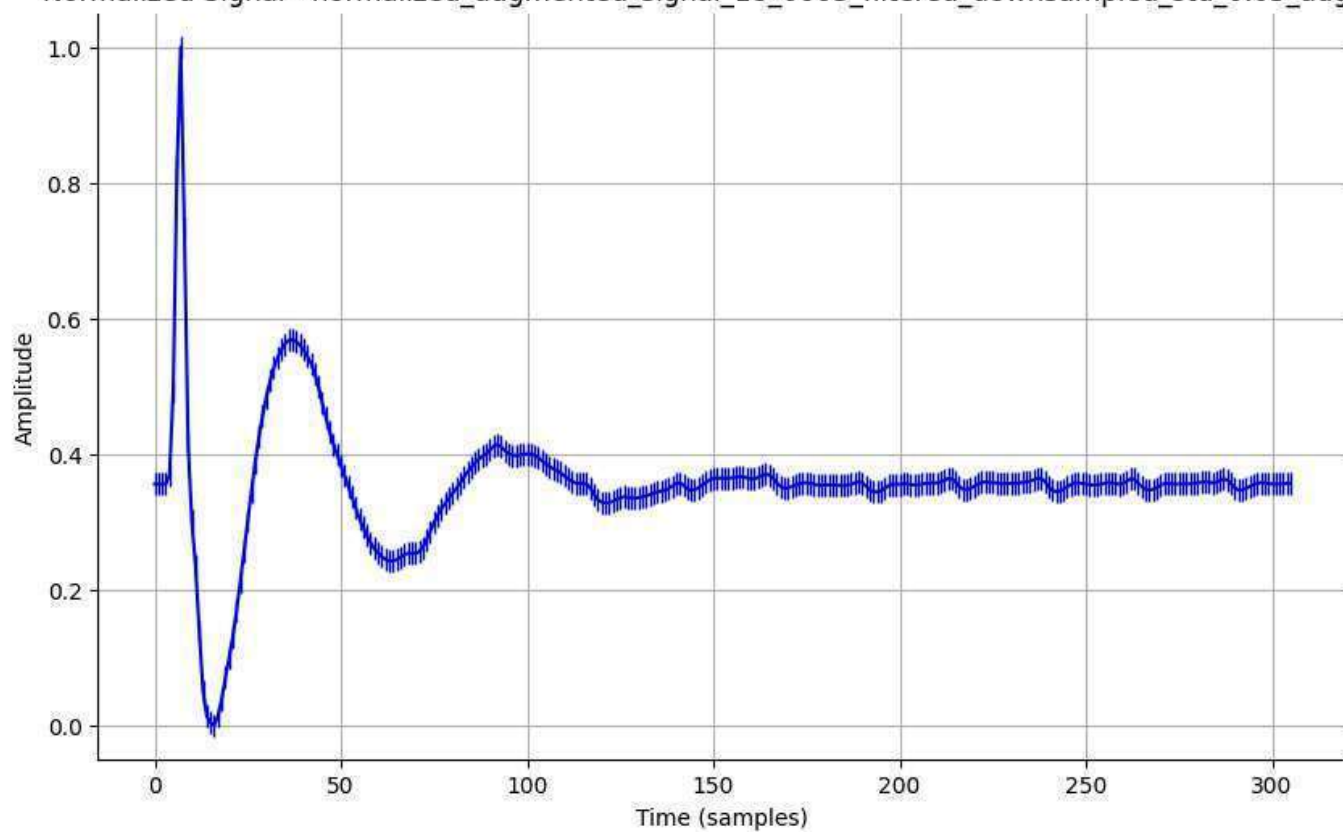
Normalized Signal - normalized_augmented_signal_18_0002_filtered_downsampled_std_0.2_aug.csv



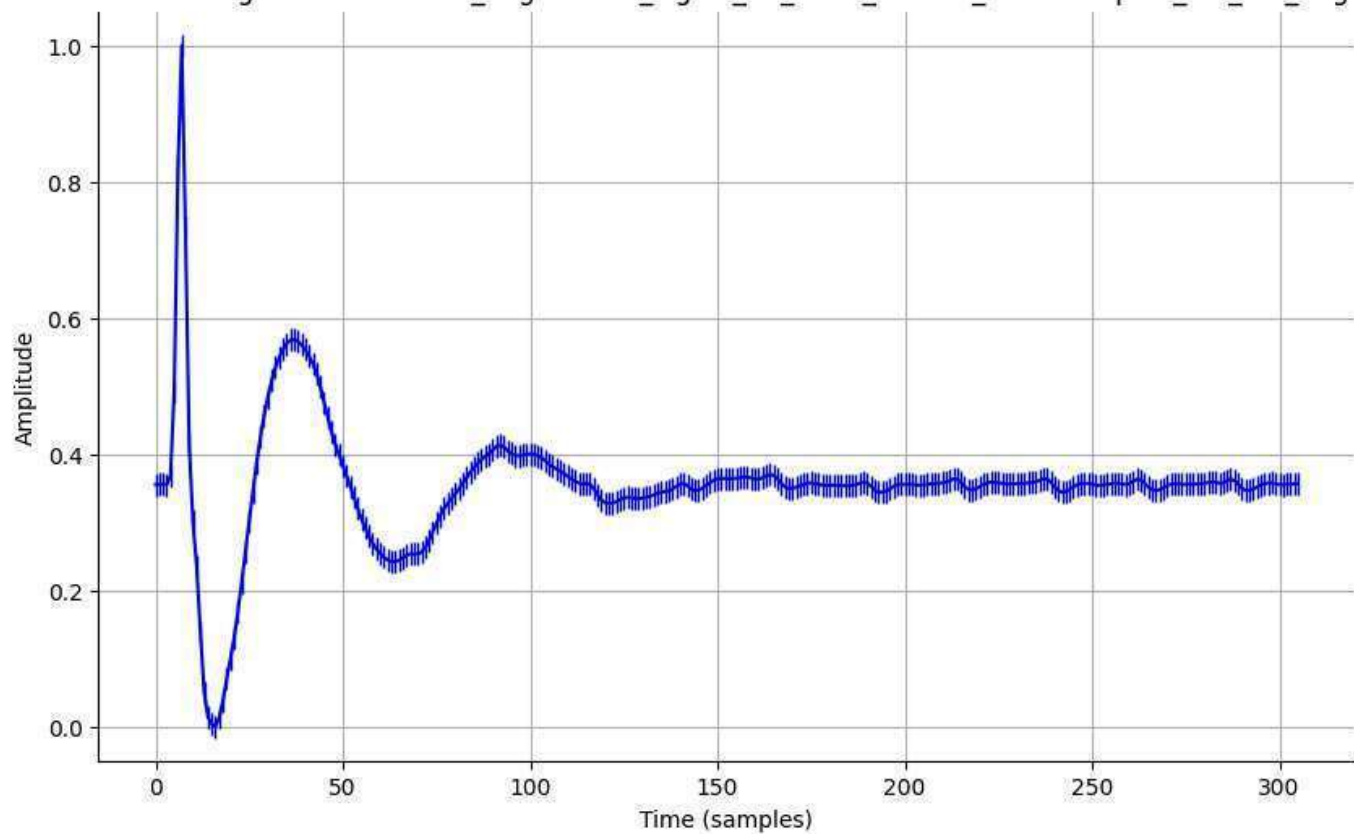
Normalized Signal - normalized_augmented_signal_18_0003_filtered_downsampled_std_0.01_aug.csv



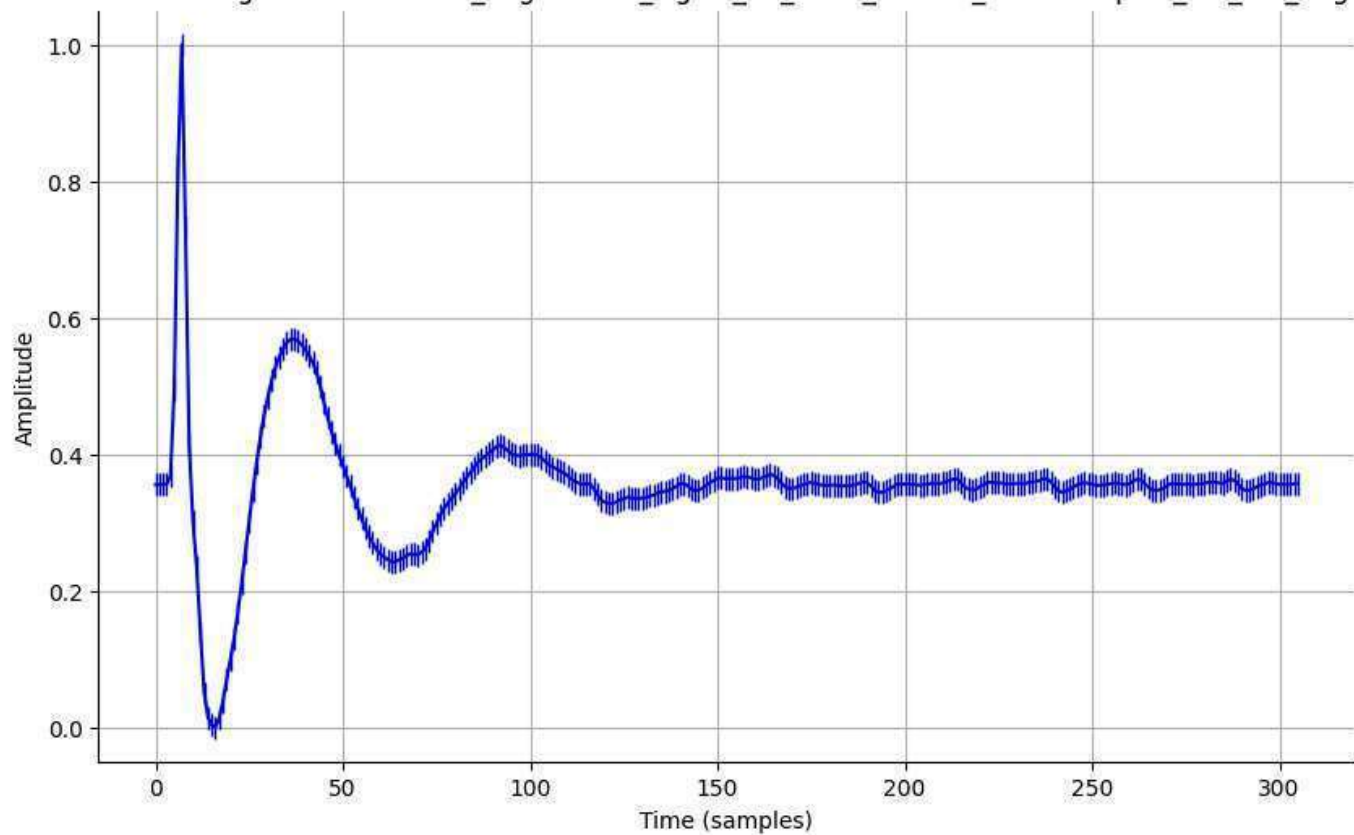
Normalized Signal - normalized_augmented_signal_18_0003_filtered_downsampled_std_0.05_aug.csv



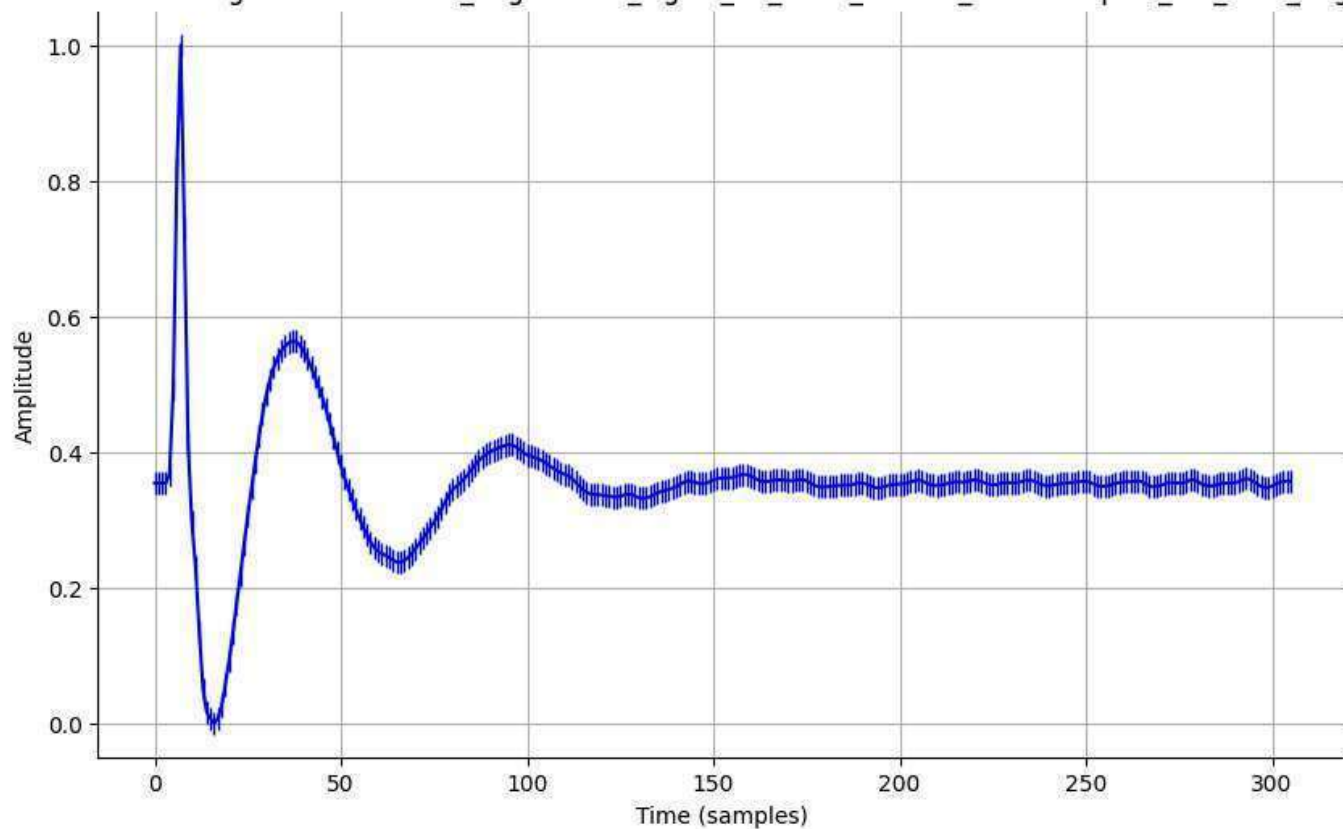
Normalized Signal - normalized_augmented_signal_18_0003_filtered_downsampled_std_0.1_aug.csv



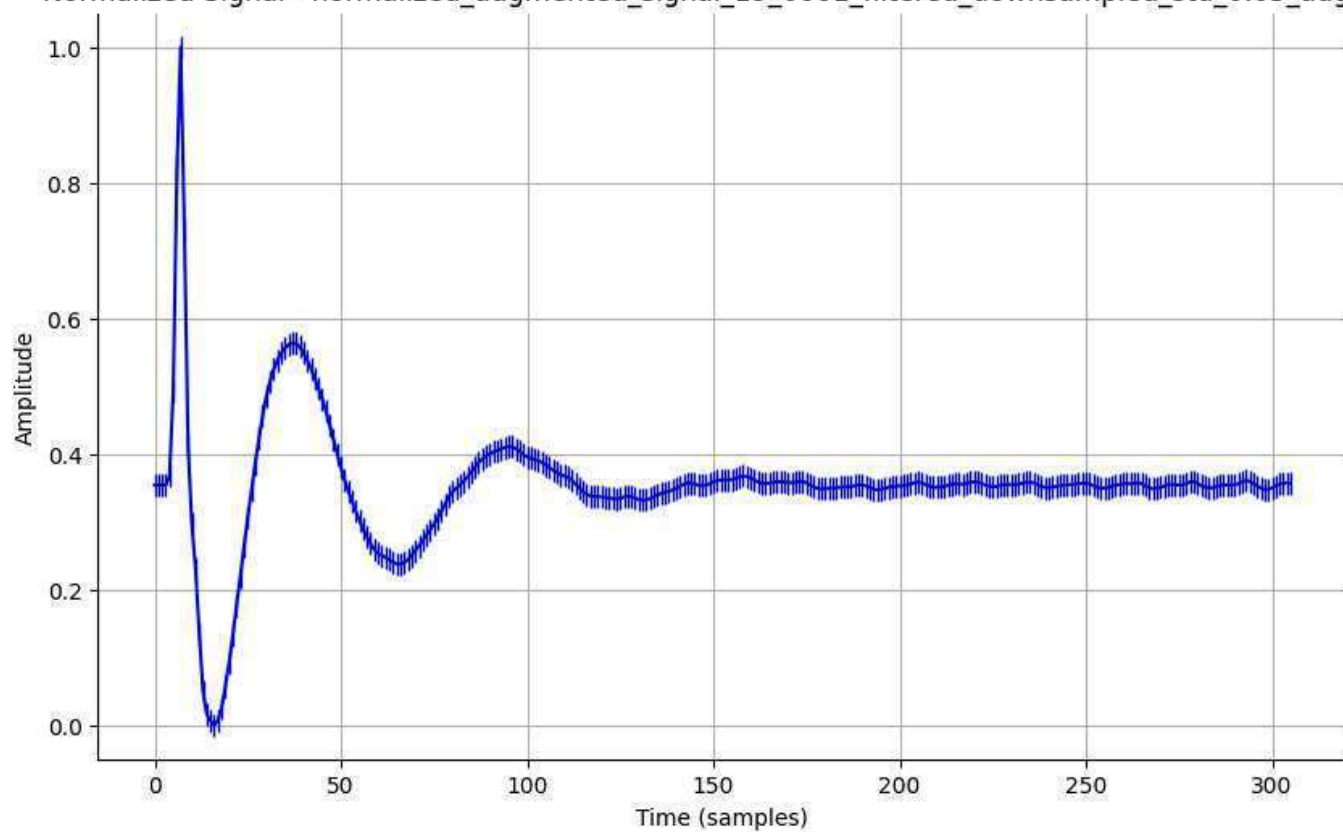
Normalized Signal - normalized_augmented_signal_18_0003_filtered_downsampled_std_0.2_aug.csv



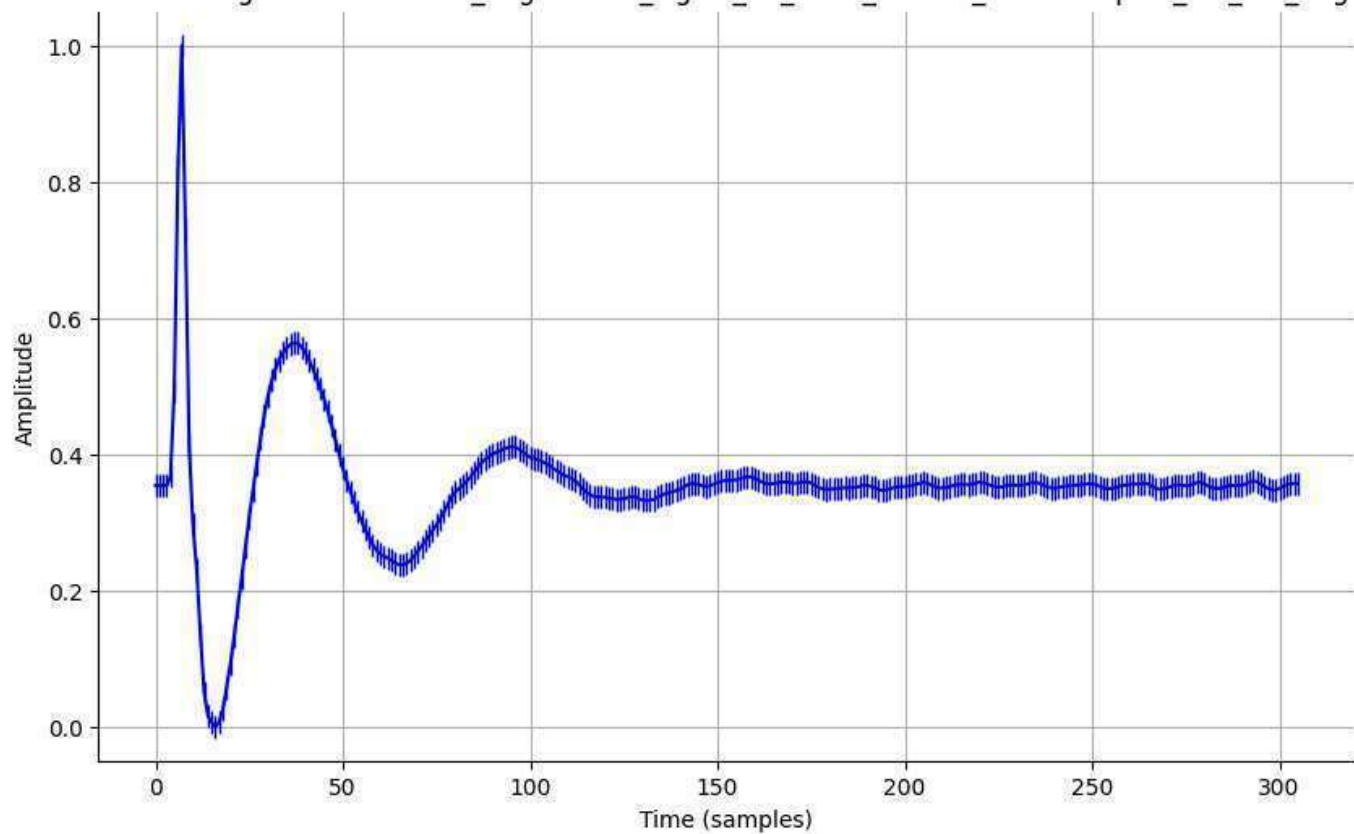
Normalized Signal - normalized_augmented_signal_19_0001_filtered_downsampled_std_0.01_aug.csv



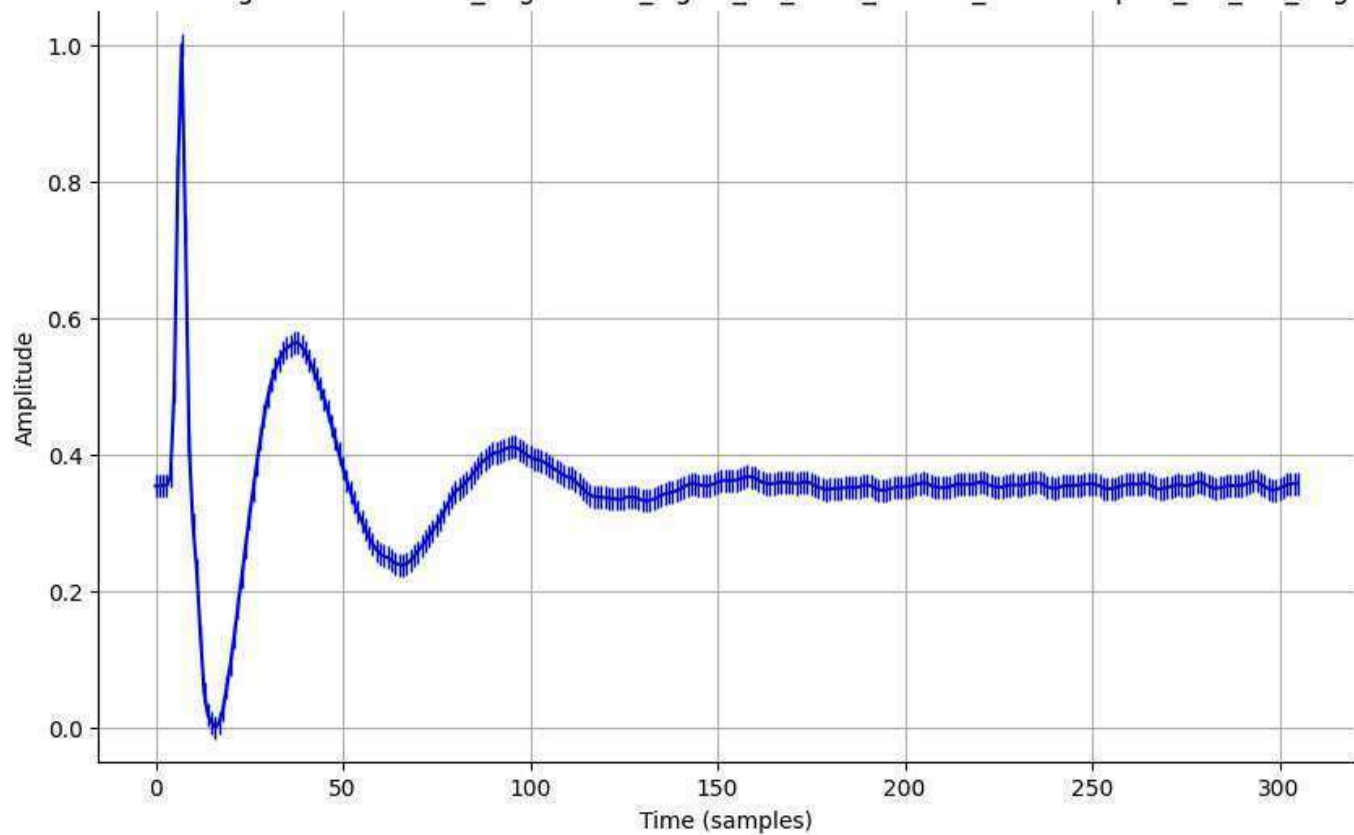
Normalized Signal - normalized_augmented_signal_19_0001_filtered_downsampled_std_0.05_aug.csv



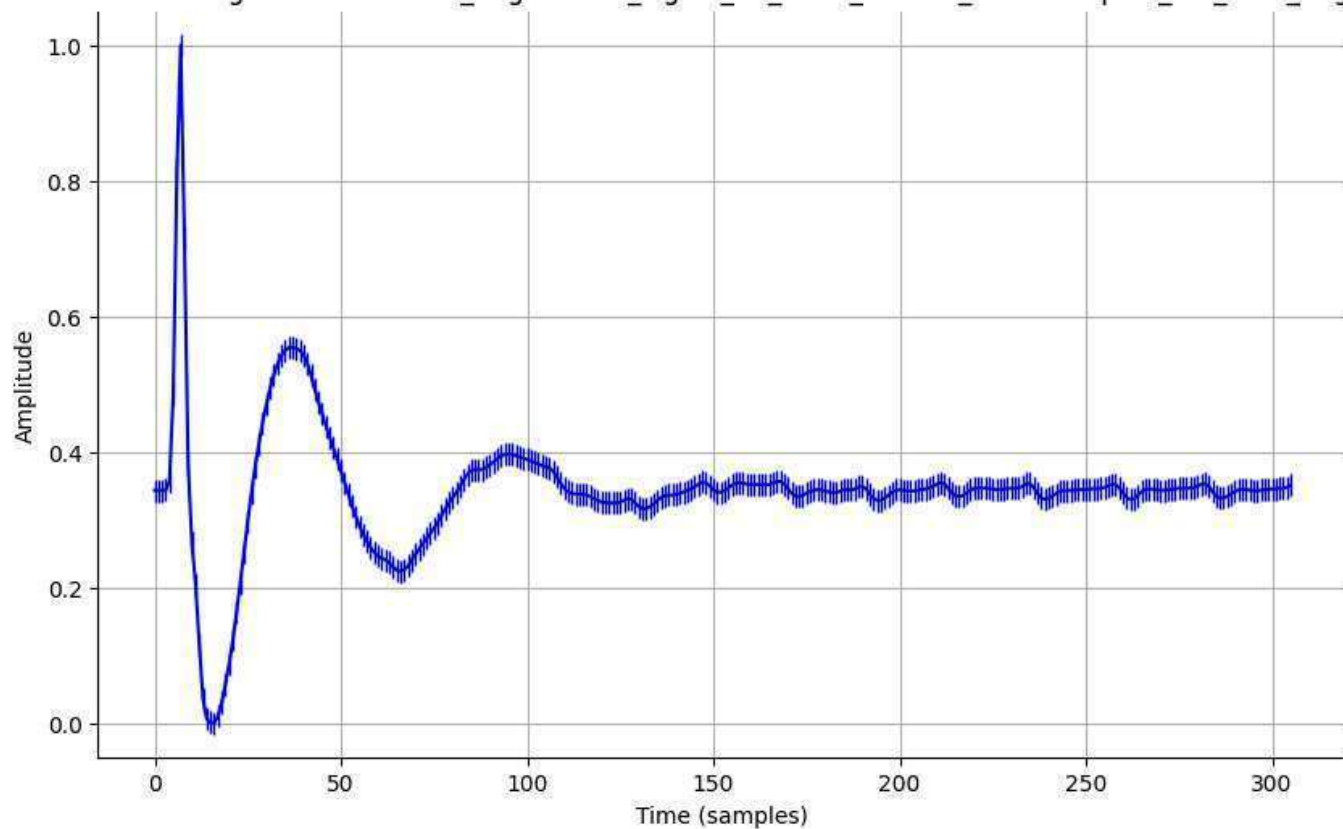
Normalized Signal - normalized_augmented_signal_19_0001_filtered_downsampled_std_0.1_aug.csv



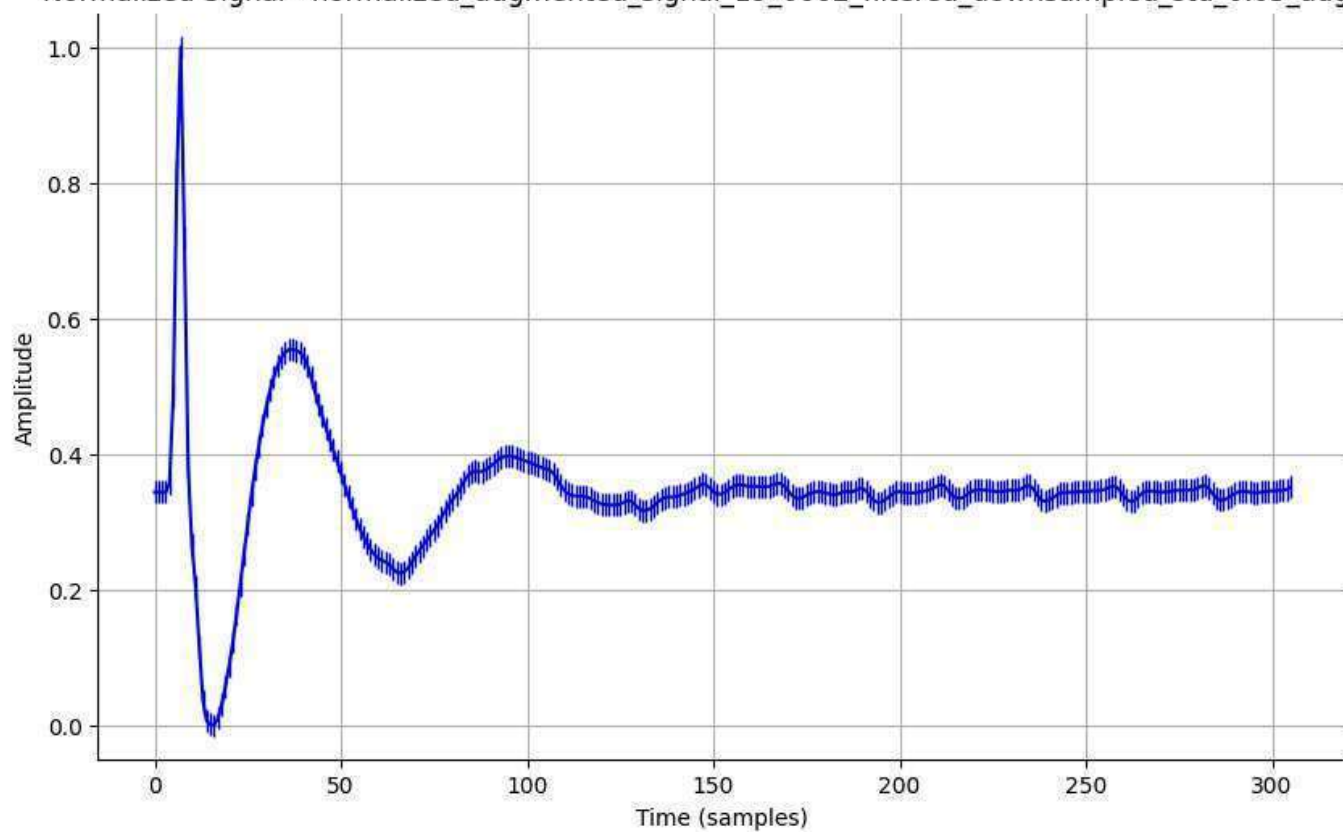
Normalized Signal - normalized_augmented_signal_19_0001_filtered_downsampled_std_0.2_aug.csv



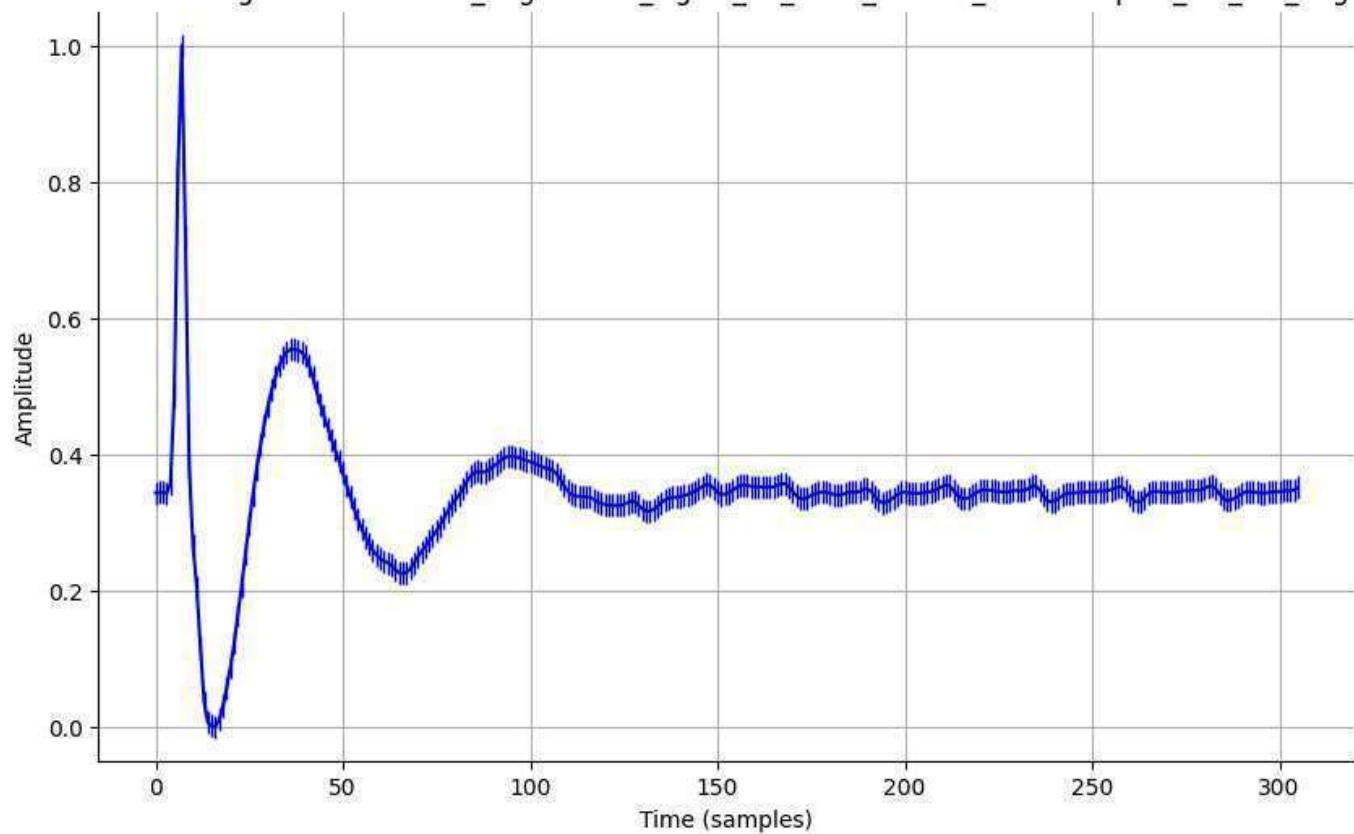
Normalized Signal - normalized_augmented_signal_19_0002_filtered_downsampled_std_0.01_aug.csv



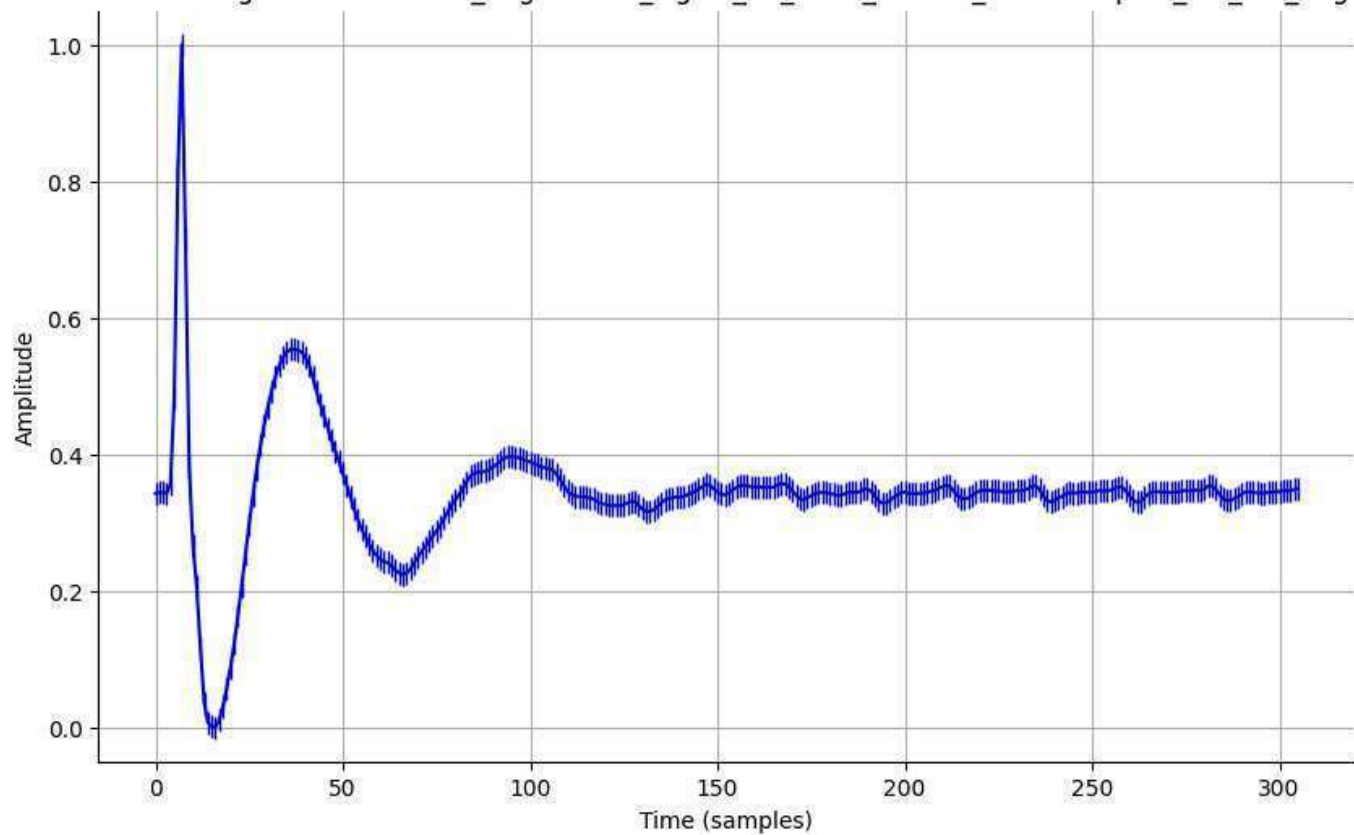
Normalized Signal - normalized_augmented_signal_19_0002_filtered_downsampled_std_0.05_aug.csv



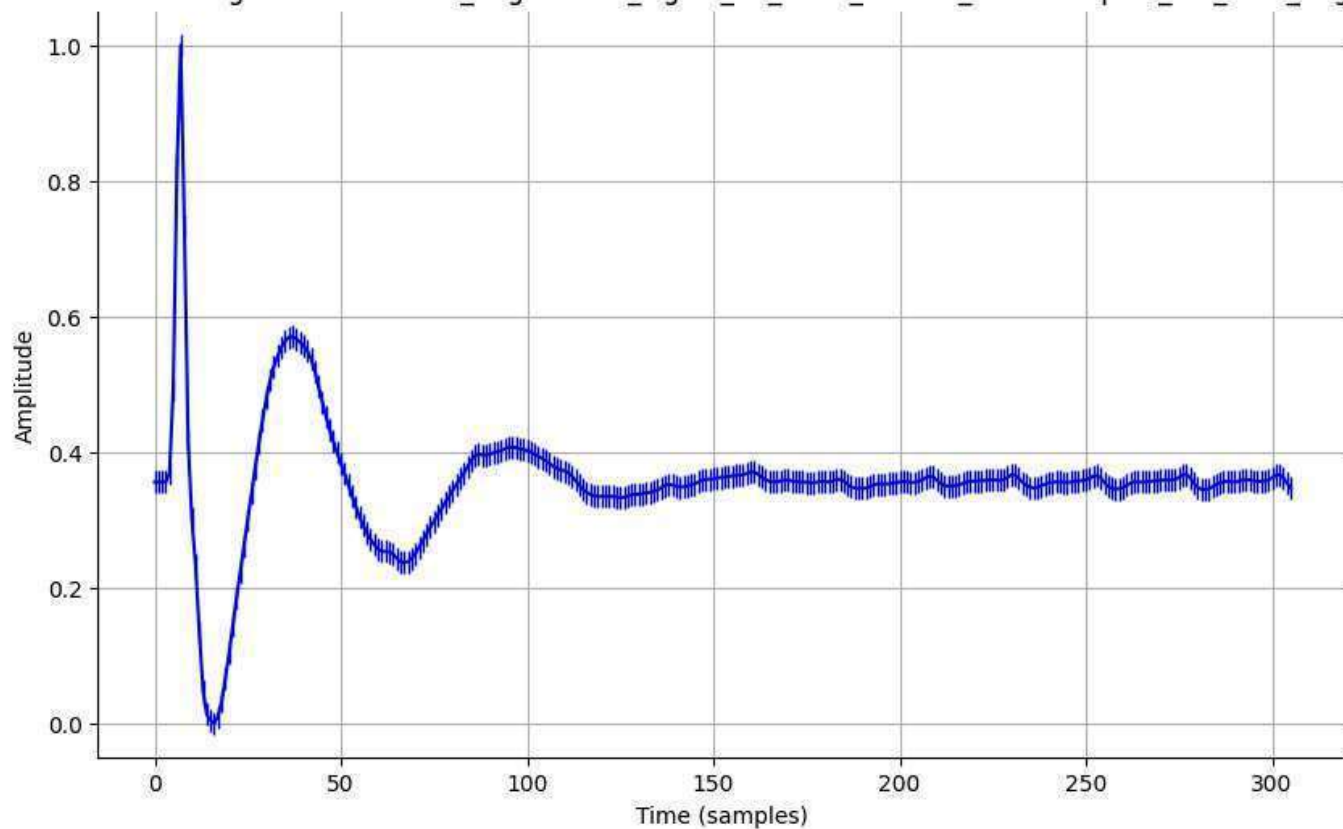
Normalized Signal - normalized_augmented_signal_19_0002_filtered_downsampled_std_0.1_aug.csv



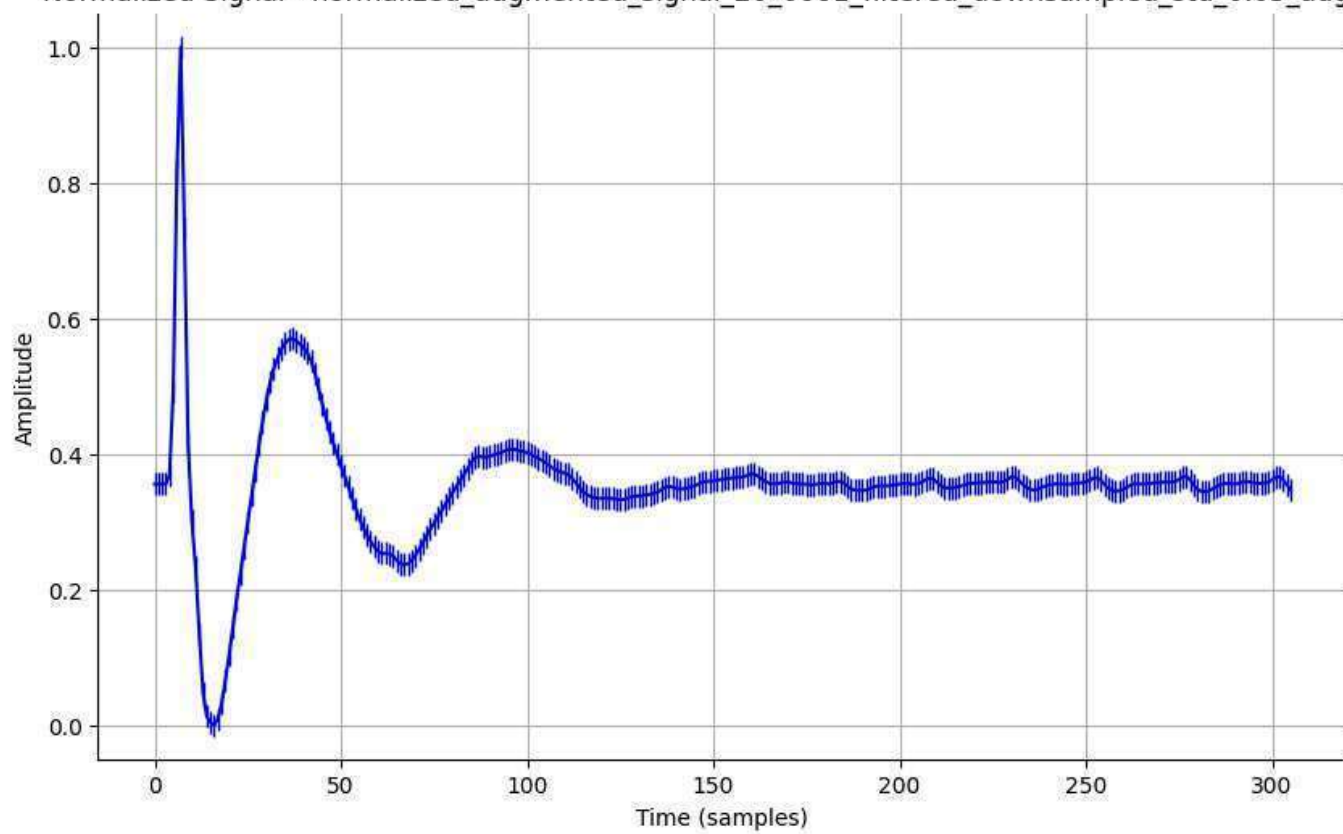
Normalized Signal - normalized_augmented_signal_19_0002_filtered_downsampled_std_0.2_aug.csv



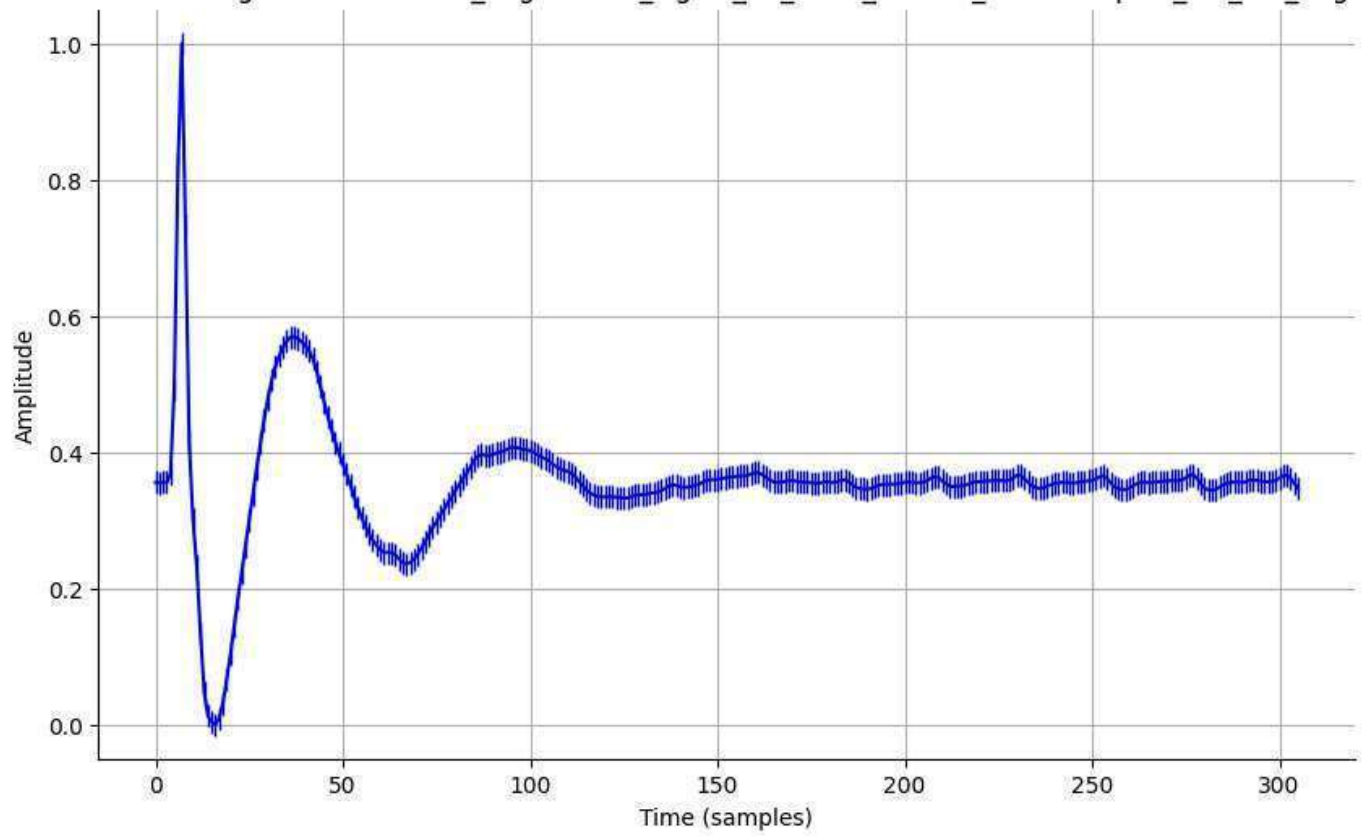
Normalized Signal - normalized_augmented_signal_20_0001_filtered_downsampled_std_0.01_aug.csv



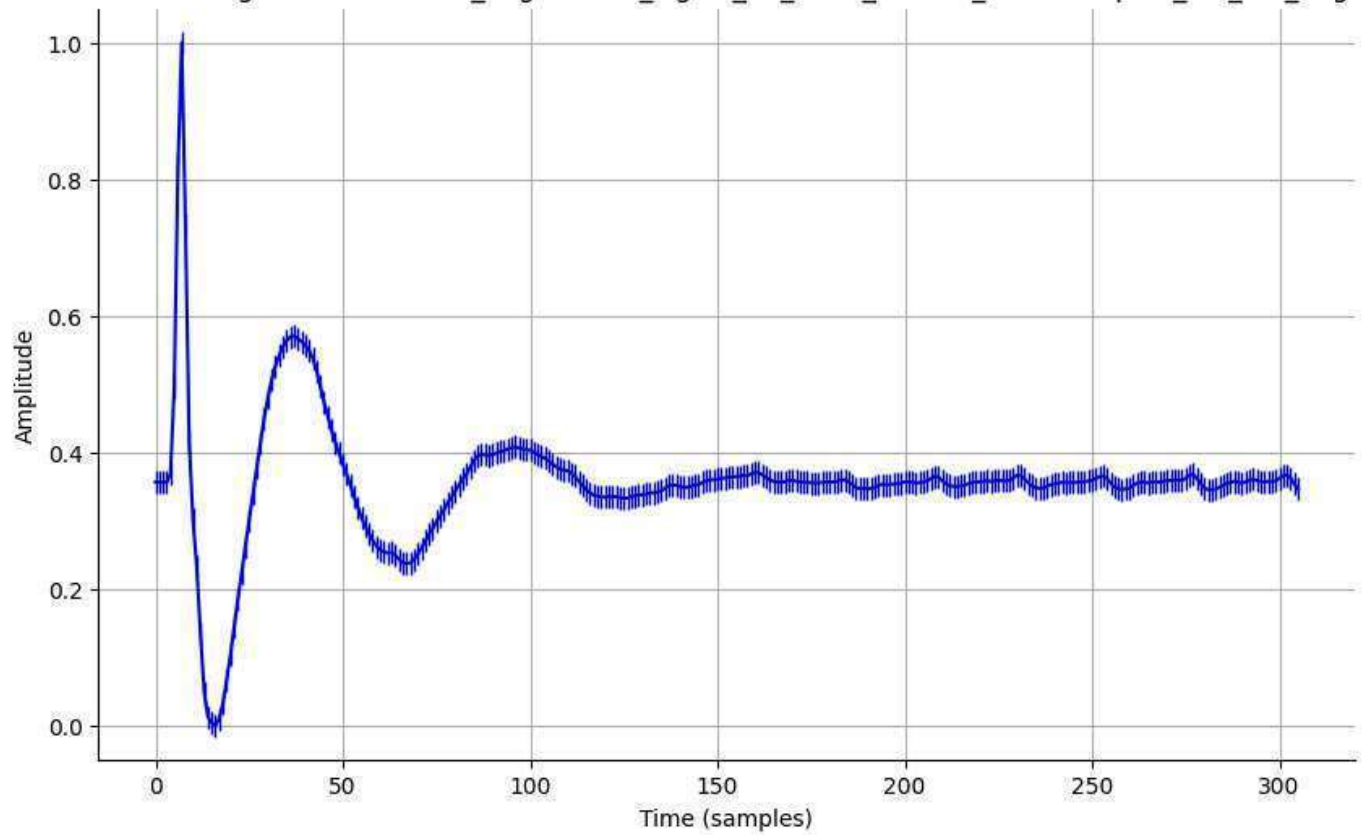
Normalized Signal - normalized_augmented_signal_20_0001_filtered_downsampled_std_0.05_aug.csv



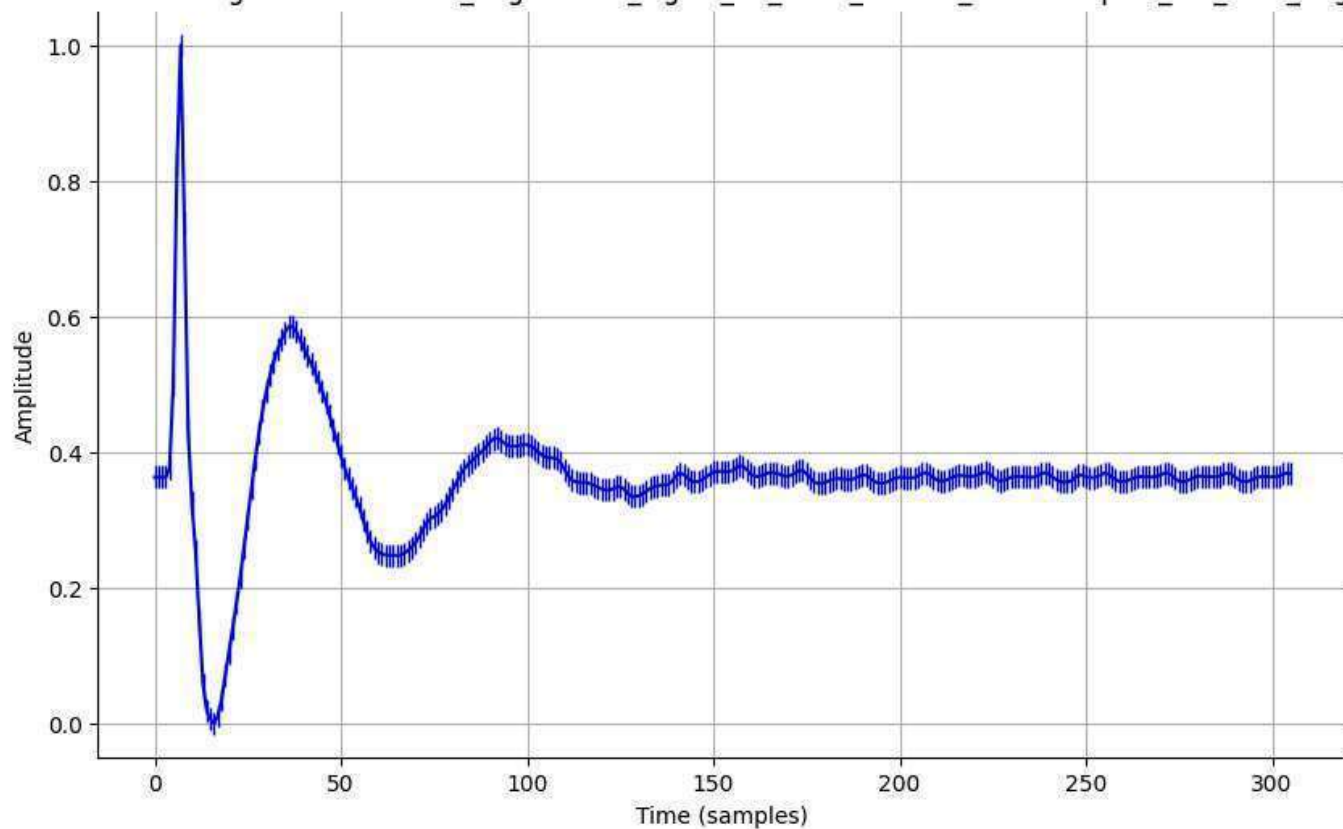
Normalized Signal - normalized_augmented_signal_20_0001_filtered_downsampled_std_0.1_aug.csv



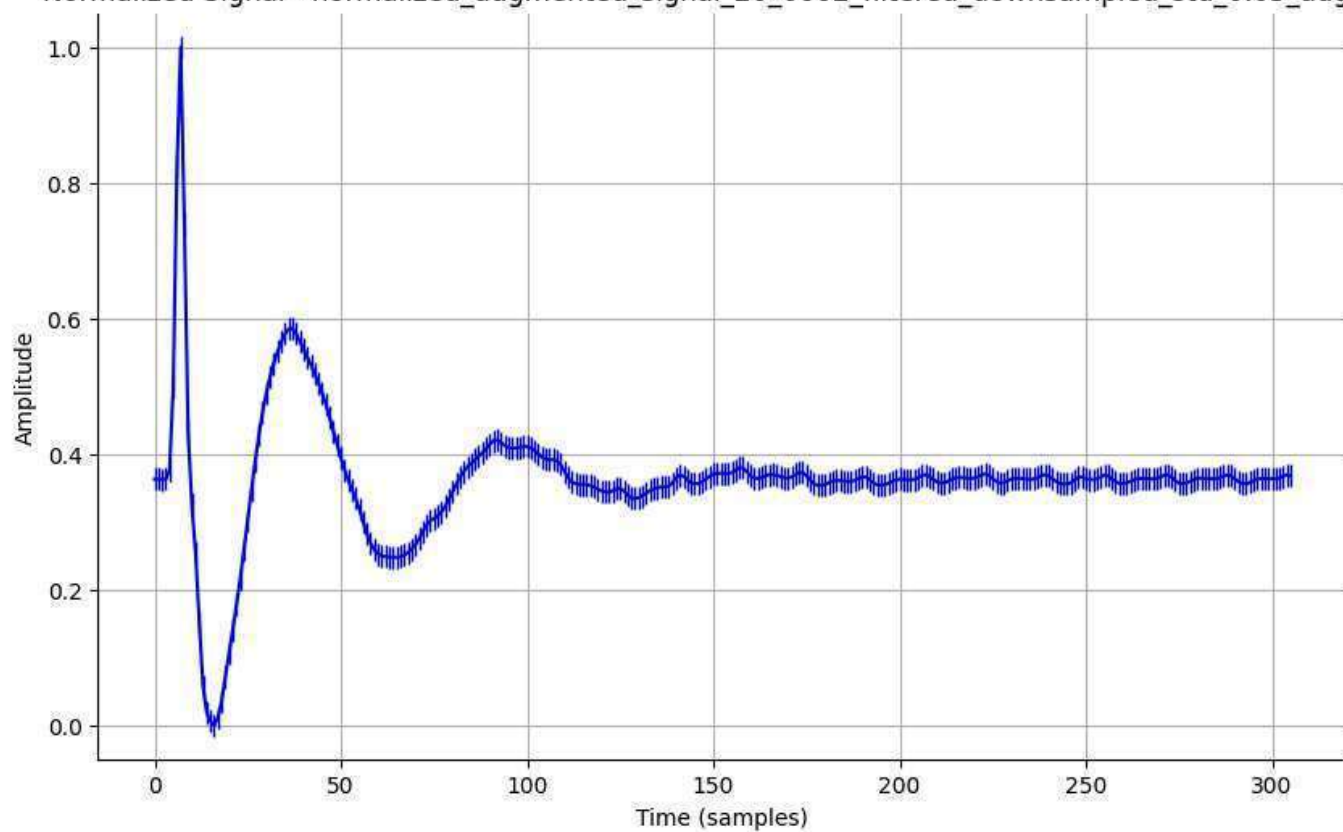
Normalized Signal - normalized_augmented_signal_20_0001_filtered_downsampled_std_0.2_aug.csv



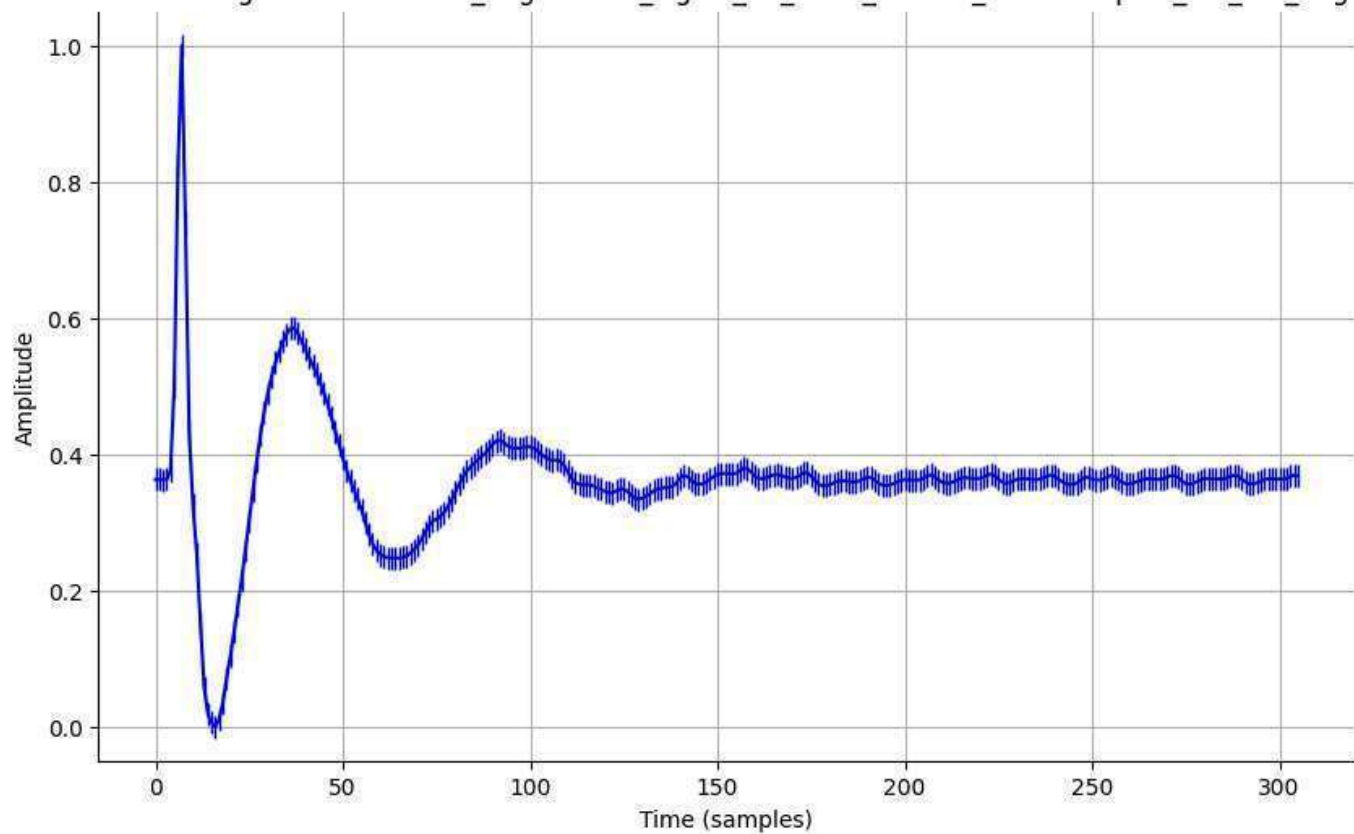
Normalized Signal - normalized_augmented_signal_20_0002_filtered_downsampled_std_0.01_aug.csv



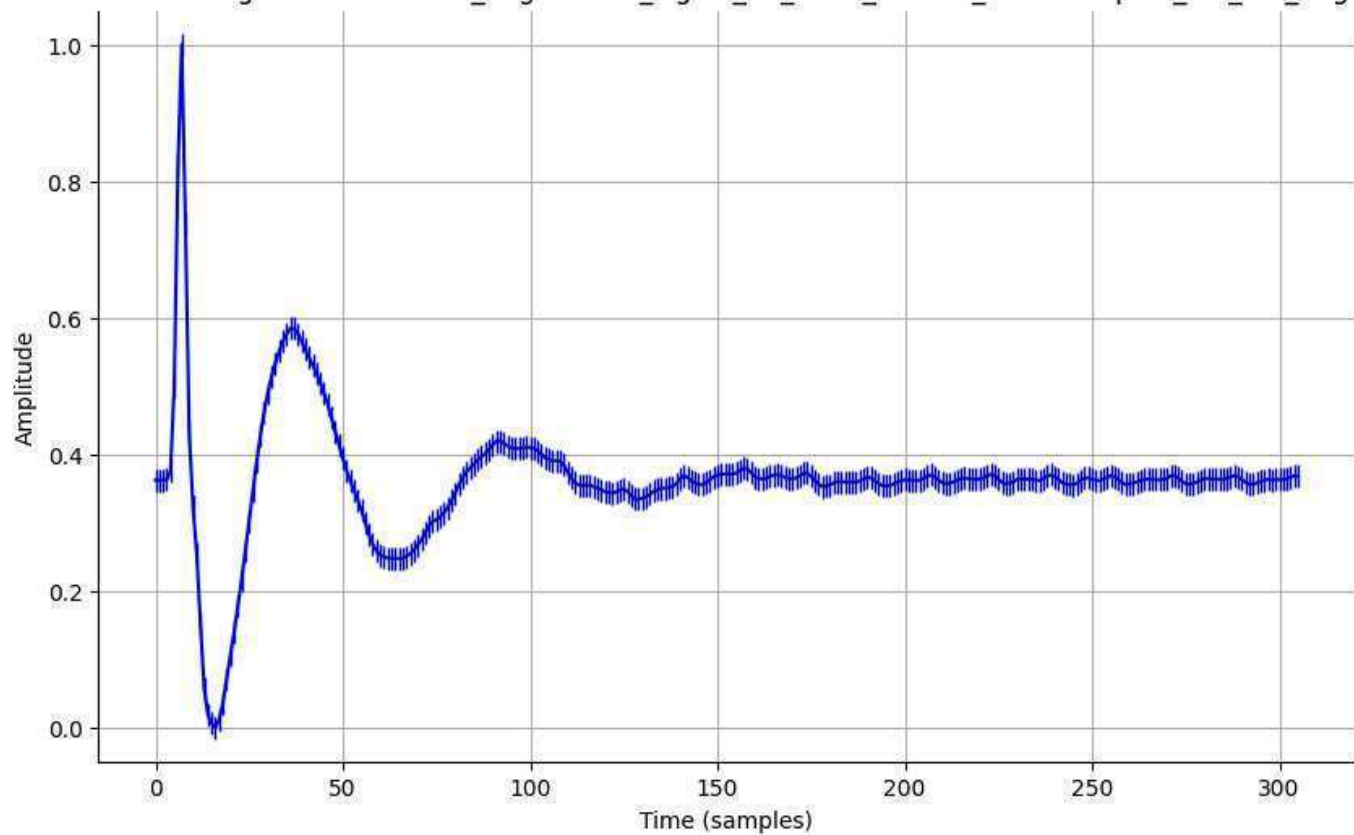
Normalized Signal - normalized_augmented_signal_20_0002_filtered_downsampled_std_0.05_aug.csv



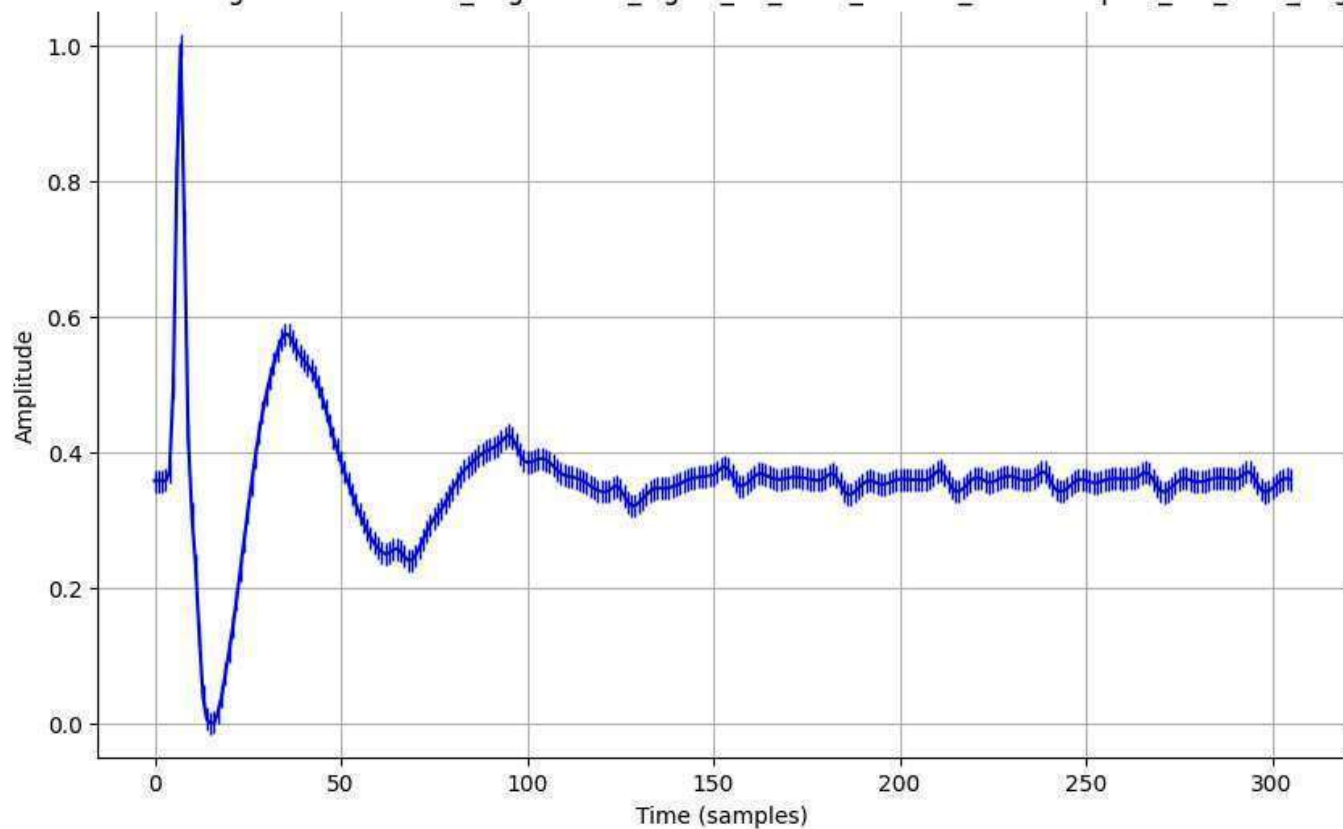
Normalized Signal - normalized_augmented_signal_20_0002_filtered_downsampled_std_0.1_aug.csv



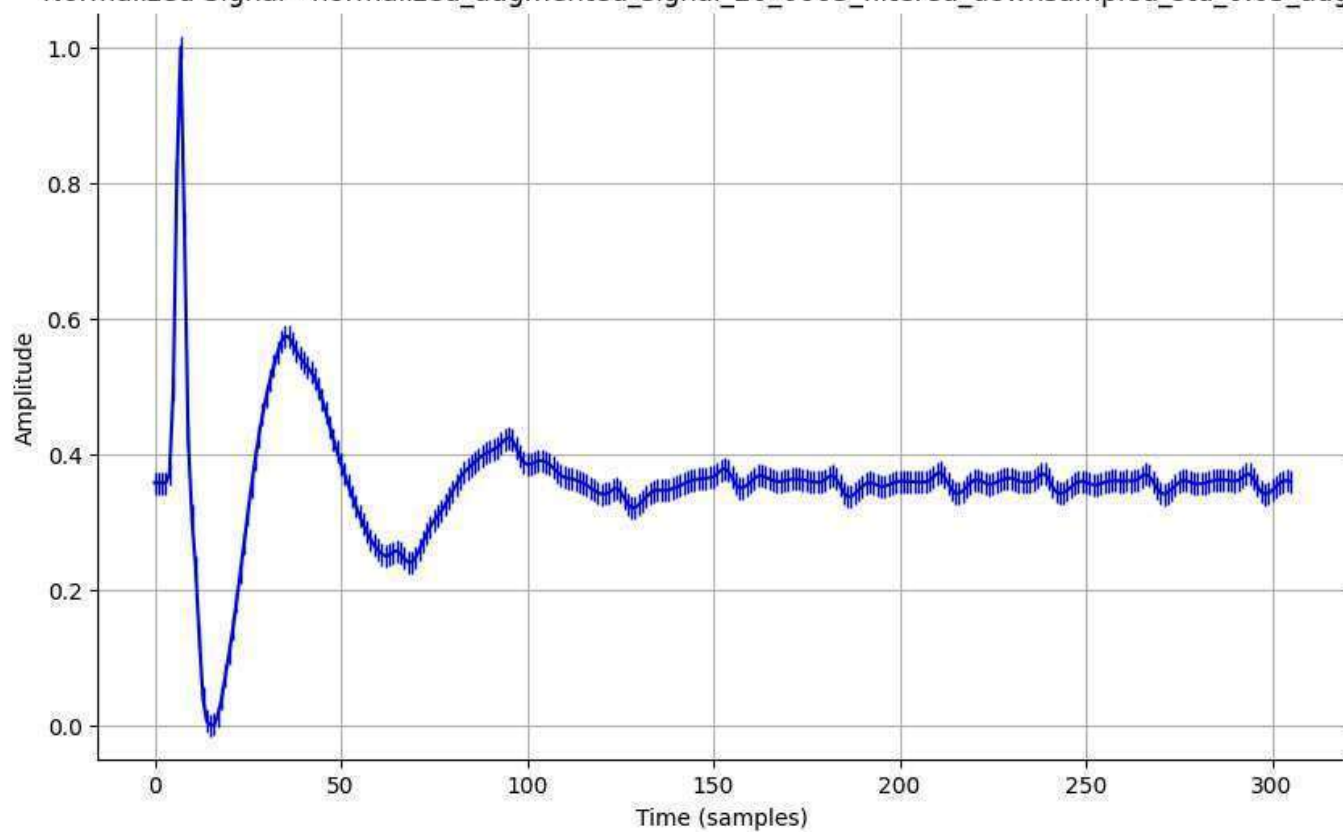
Normalized Signal - normalized_augmented_signal_20_0002_filtered_downsampled_std_0.2_aug.csv



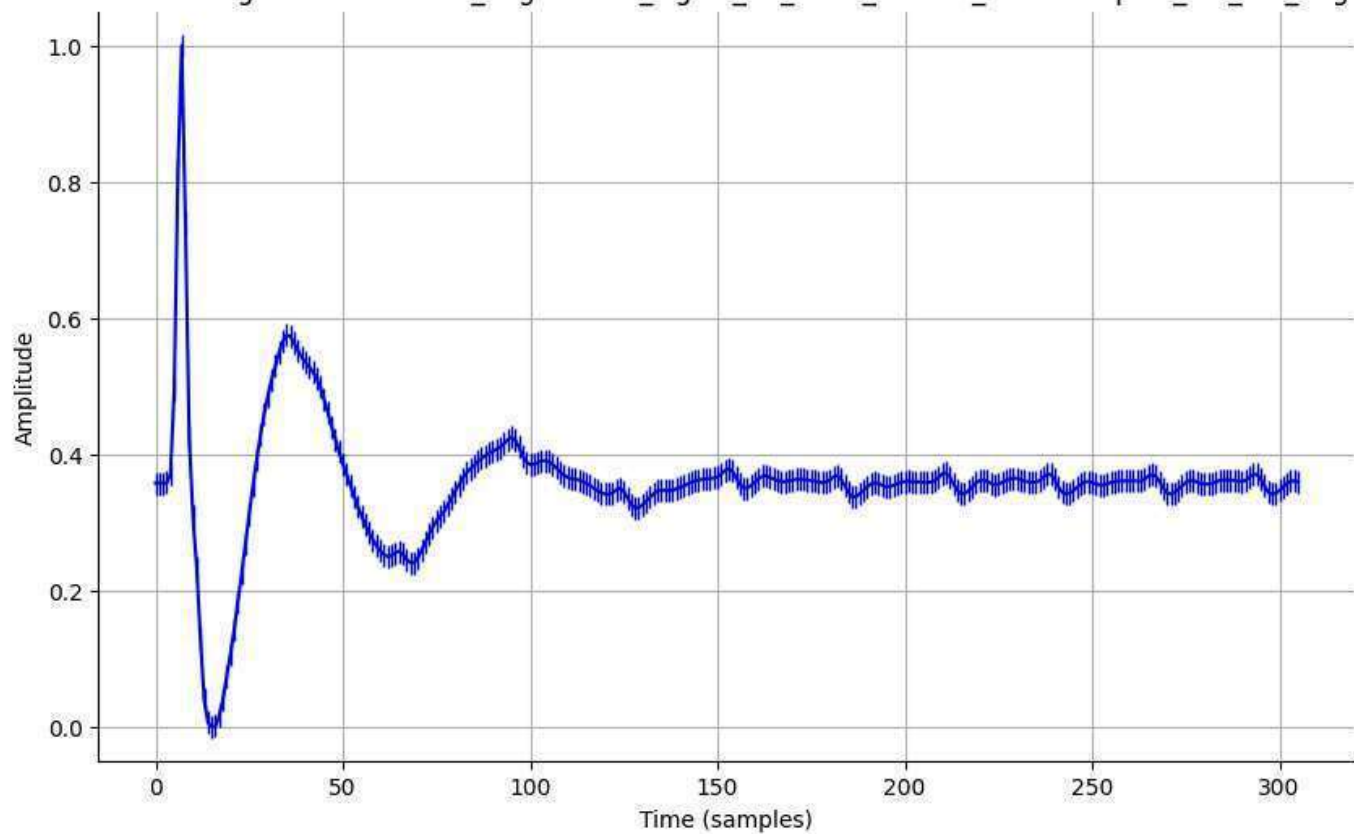
Normalized Signal - normalized_augmented_signal_20_0003_filtered_downsampled_std_0.01_aug.csv



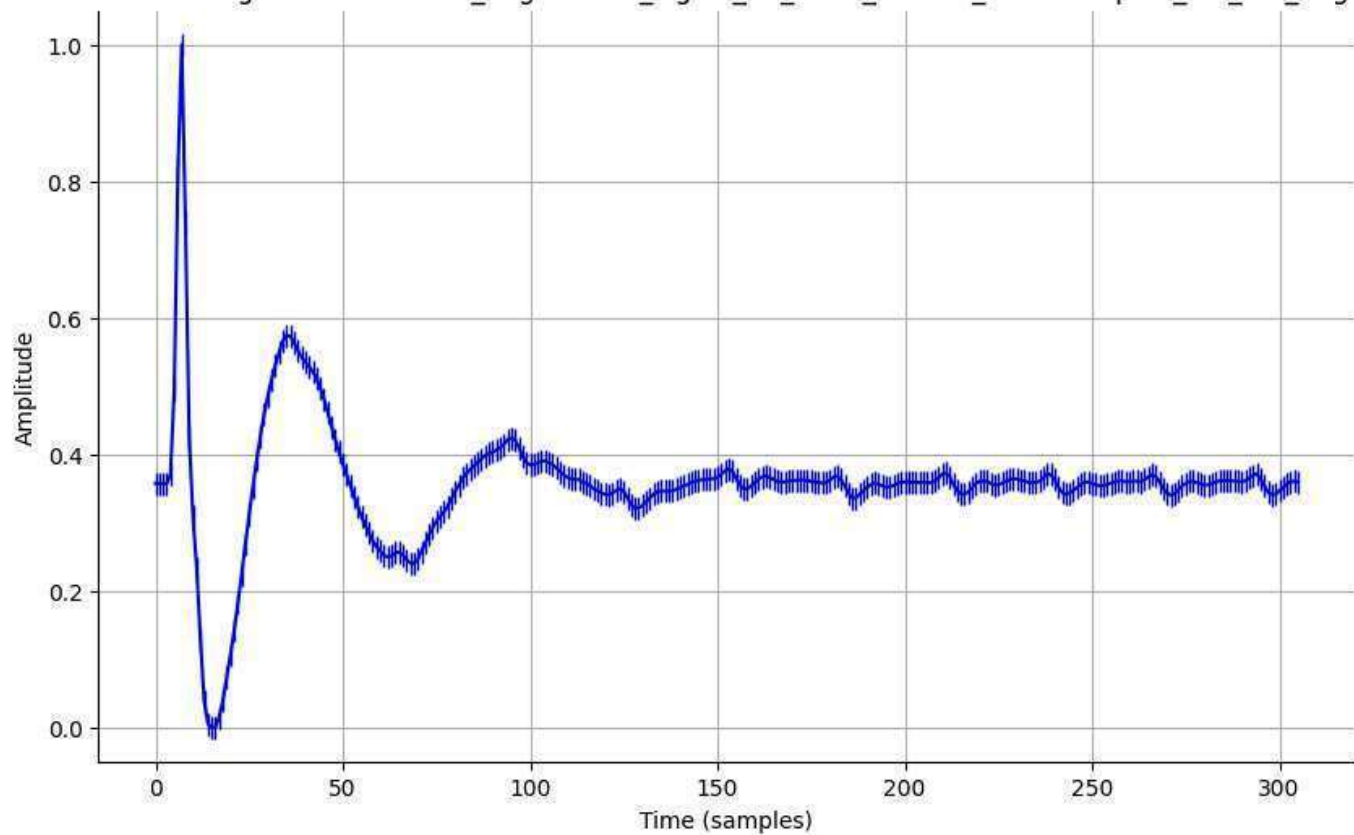
Normalized Signal - normalized_augmented_signal_20_0003_filtered_downsampled_std_0.05_aug.csv



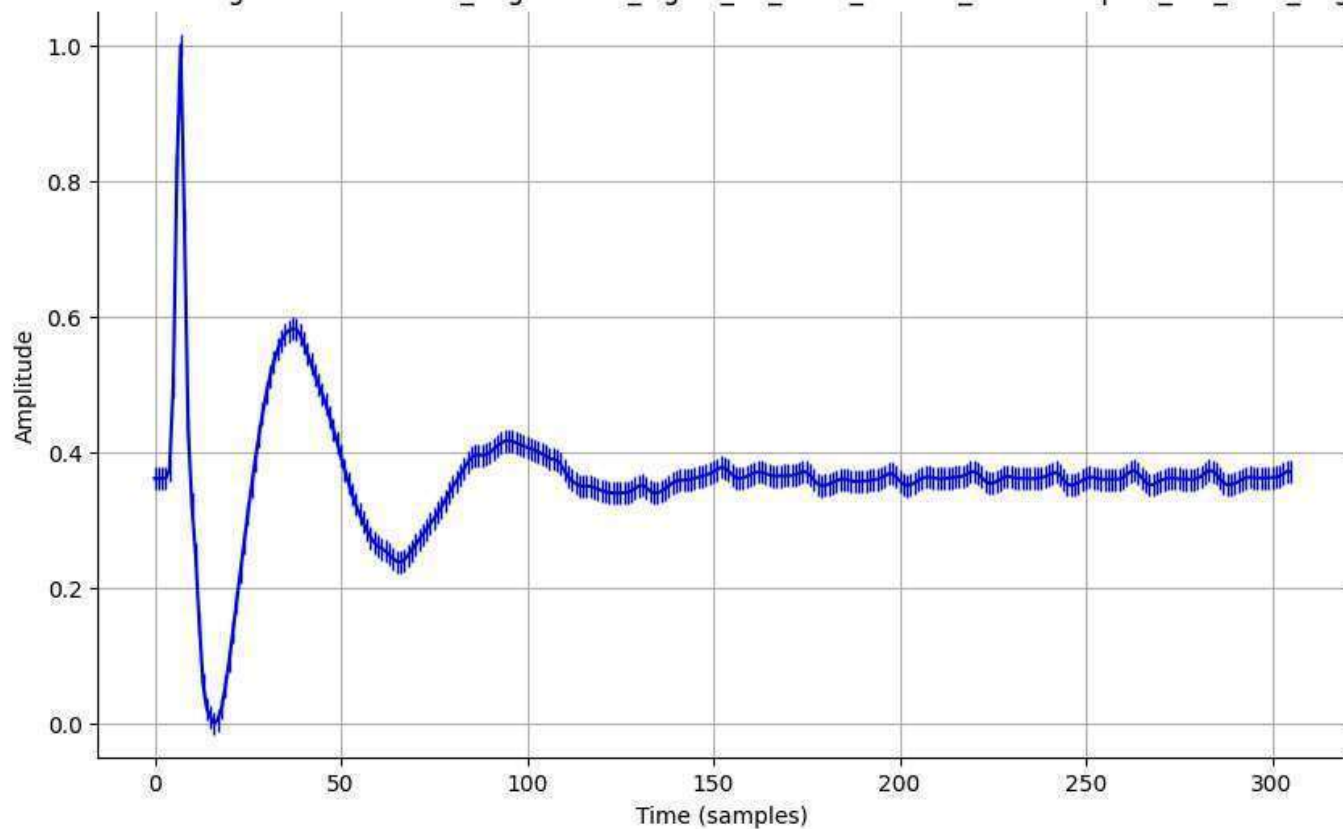
Normalized Signal - normalized_augmented_signal_20_0003_filtered_downsampled_std_0.1_aug.csv



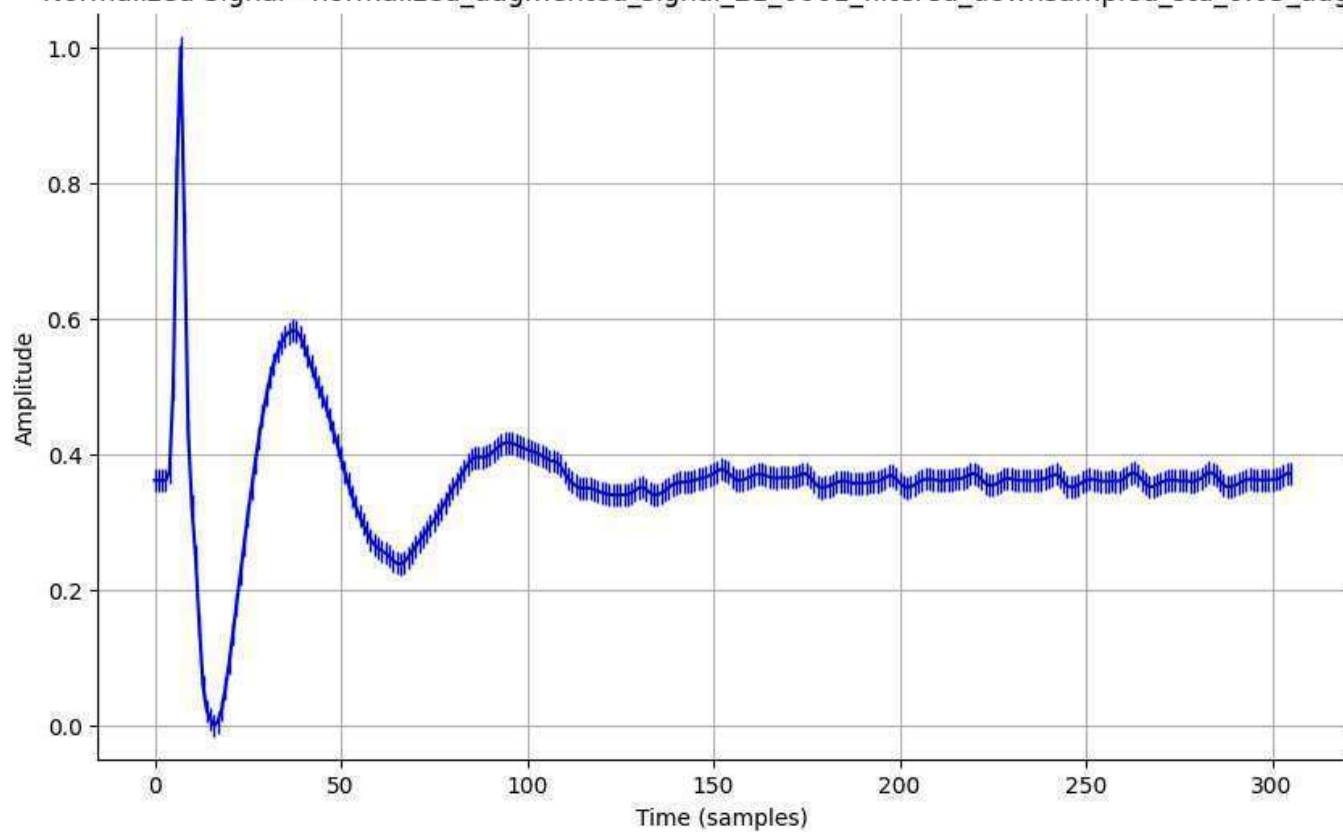
Normalized Signal - normalized_augmented_signal_20_0003_filtered_downsampled_std_0.2_aug.csv



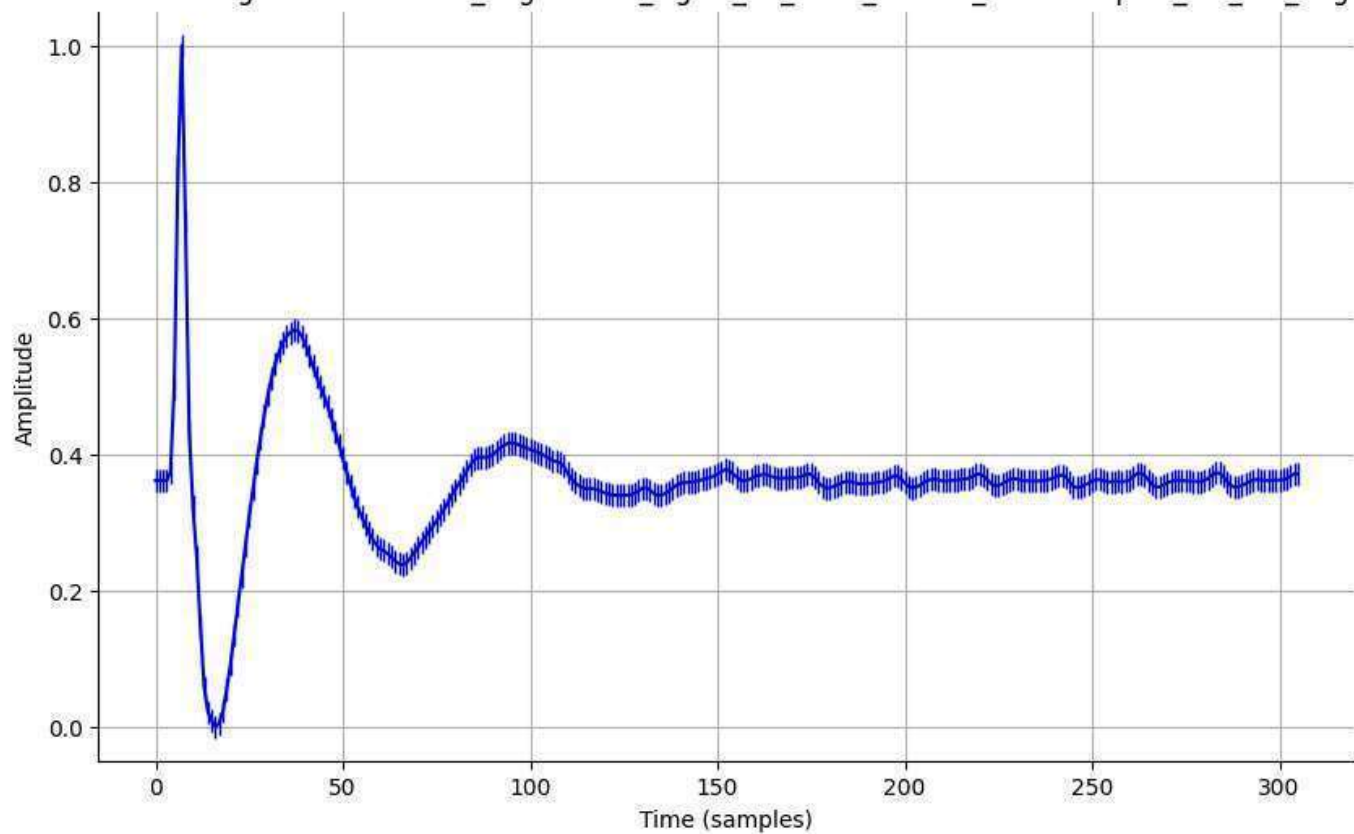
Normalized Signal - normalized_augmented_signal_21_0001_filtered_downsampled_std_0.01_aug.csv



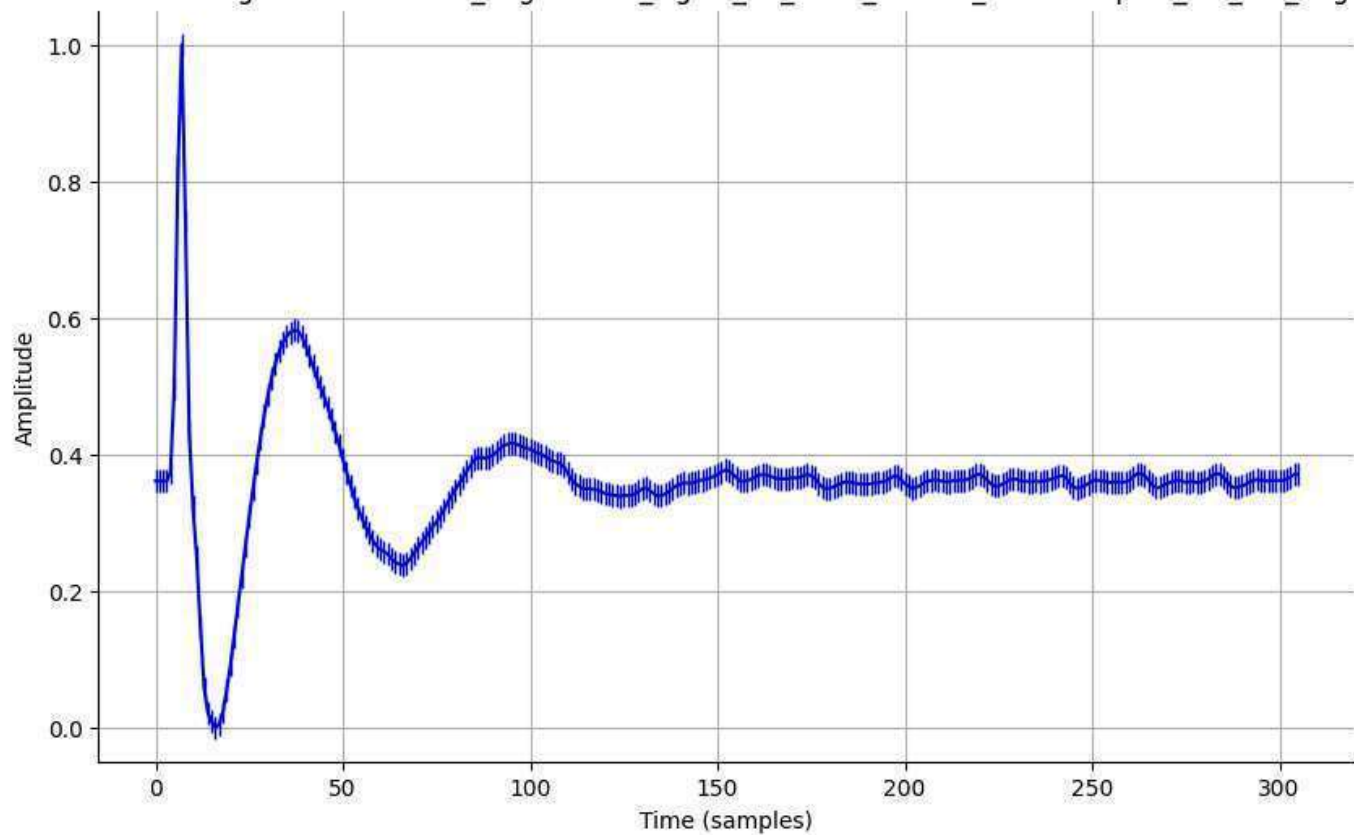
Normalized Signal - normalized_augmented_signal_21_0001_filtered_downsampled_std_0.05_aug.csv



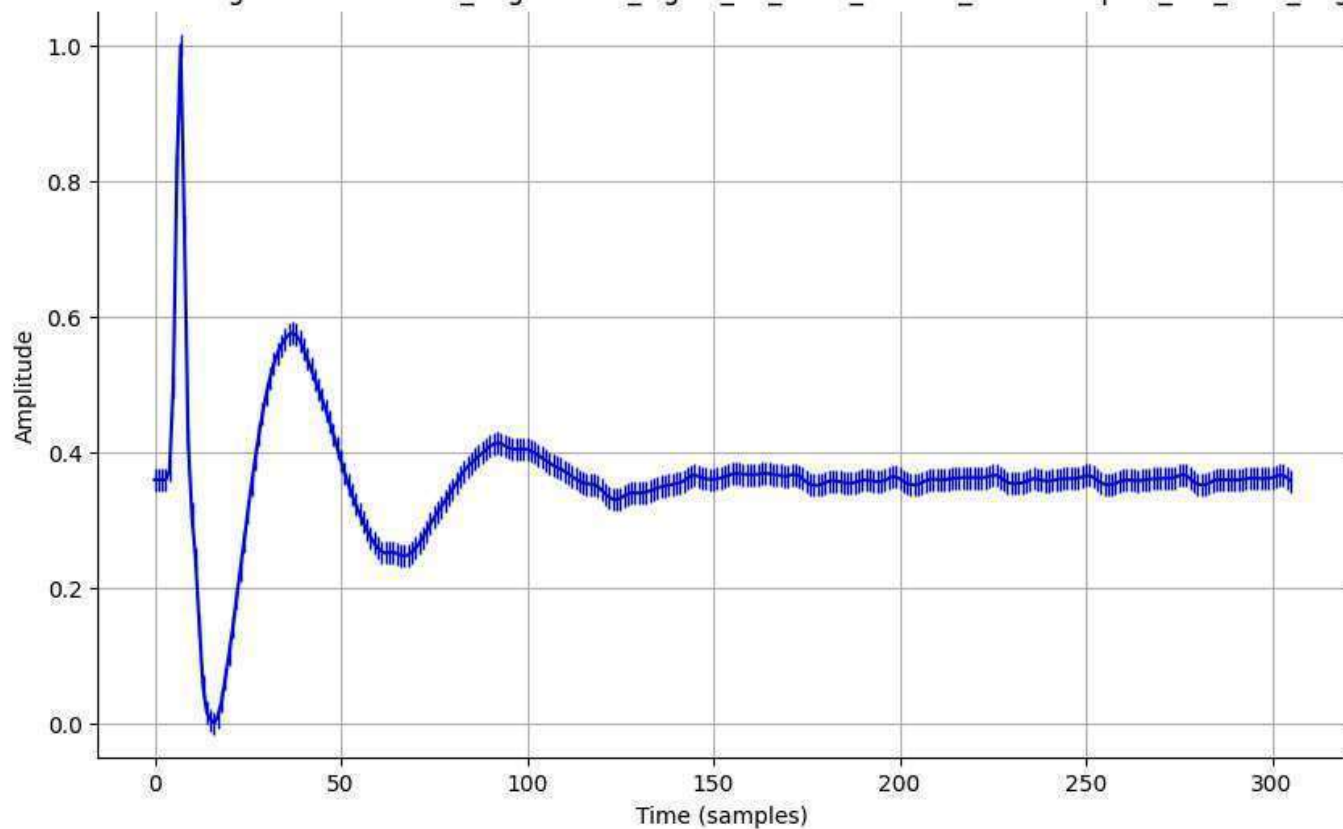
Normalized Signal - normalized_augmented_signal_21_0001_filtered_downsampled_std_0.1_aug.csv



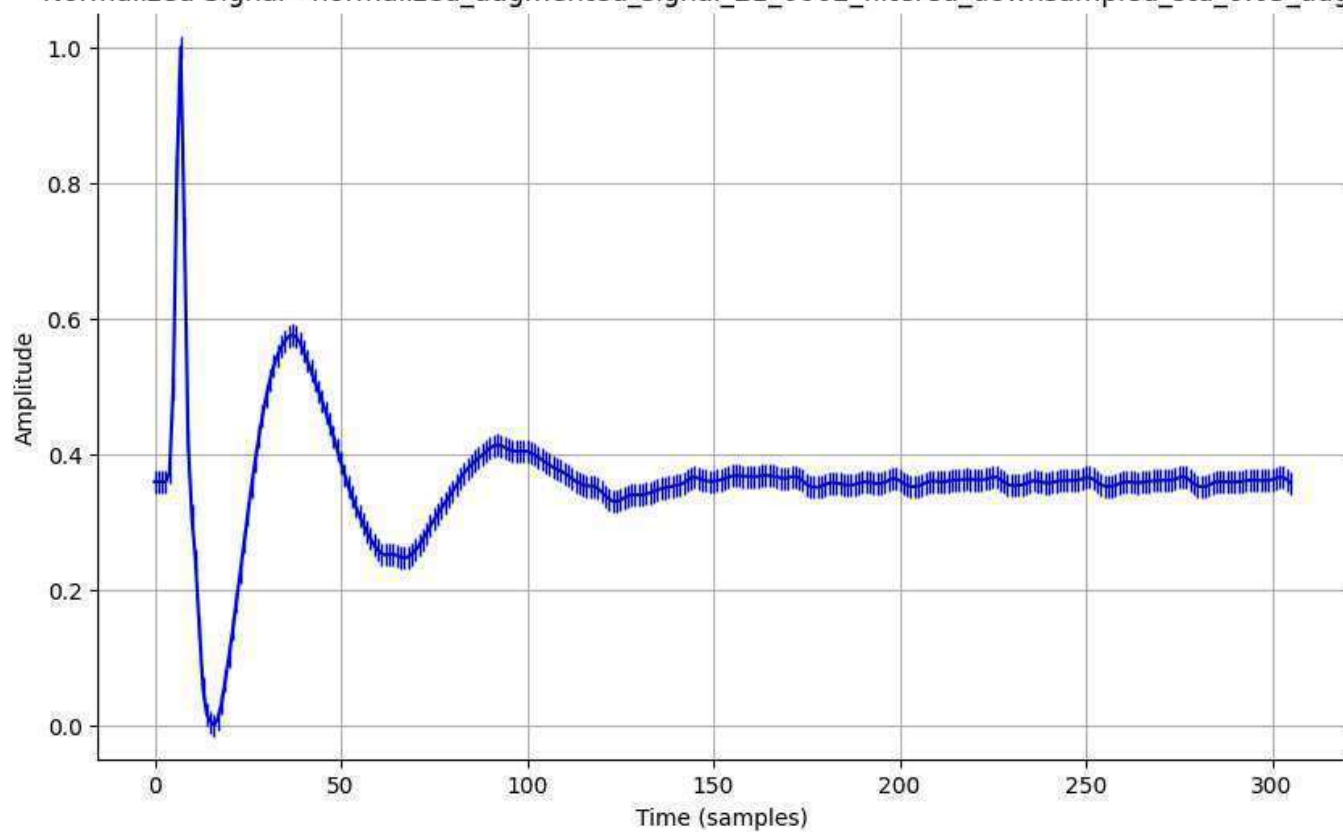
Normalized Signal - normalized_augmented_signal_21_0001_filtered_downsampled_std_0.2_aug.csv



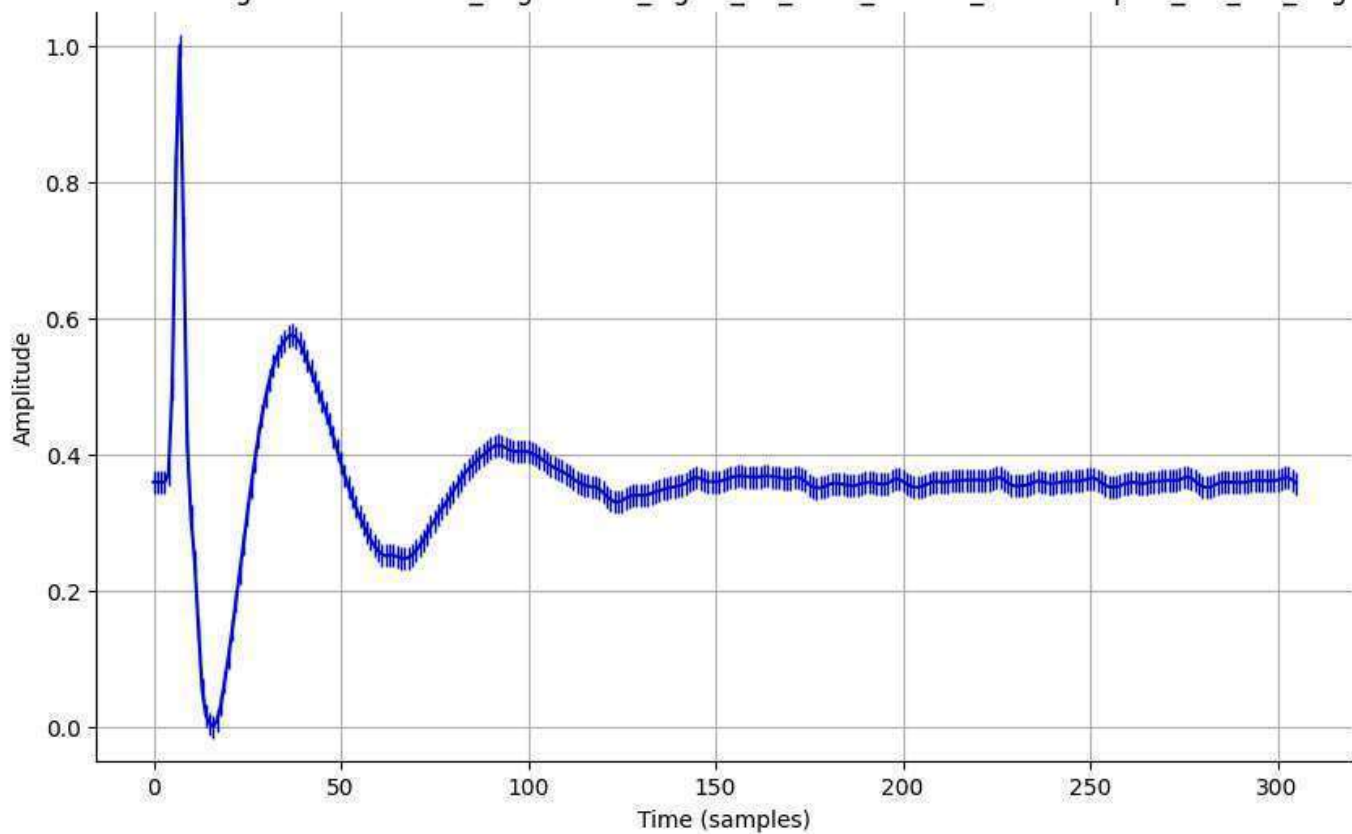
Normalized Signal - normalized_augmented_signal_21_0002_filtered_downsampled_std_0.01_aug.csv



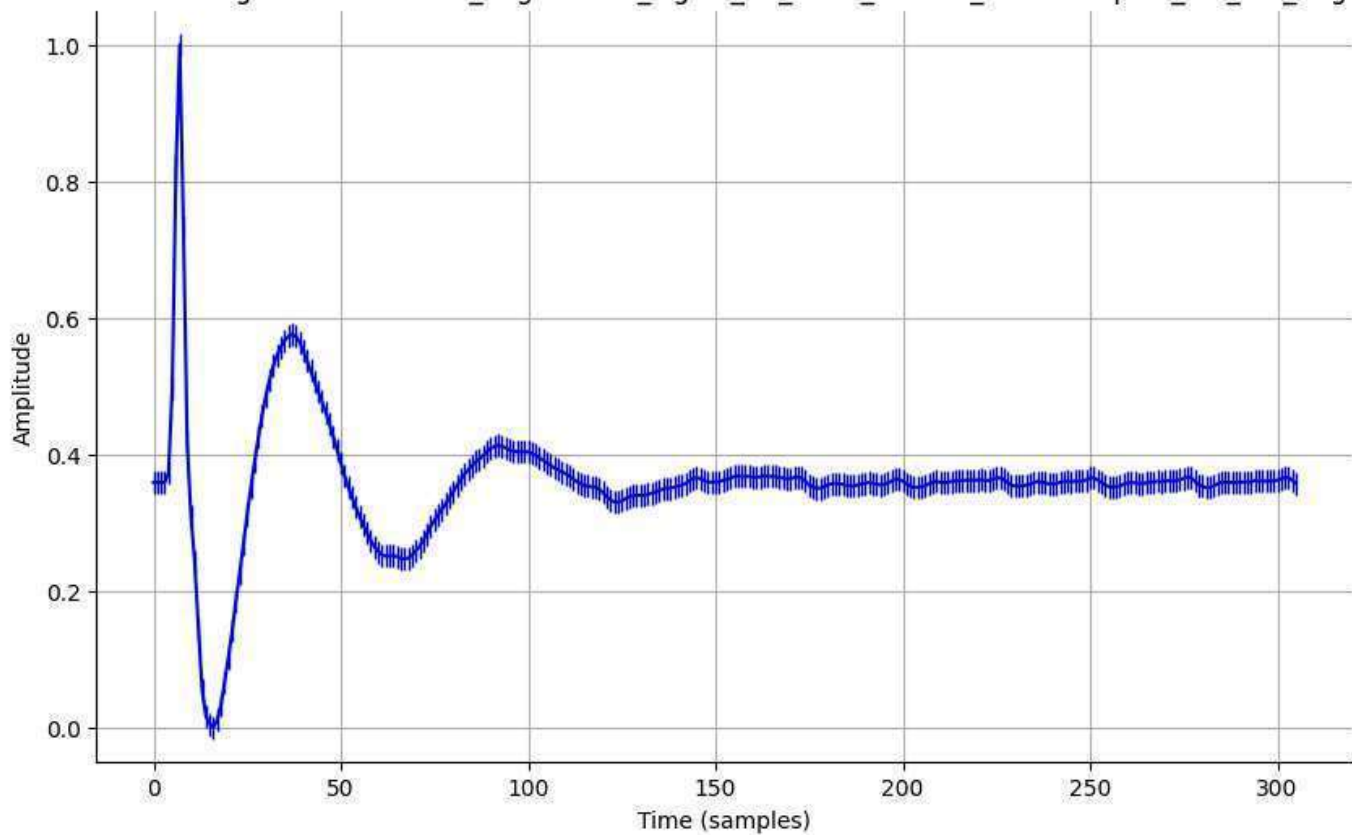
Normalized Signal - normalized_augmented_signal_21_0002_filtered_downsampled_std_0.05_aug.csv



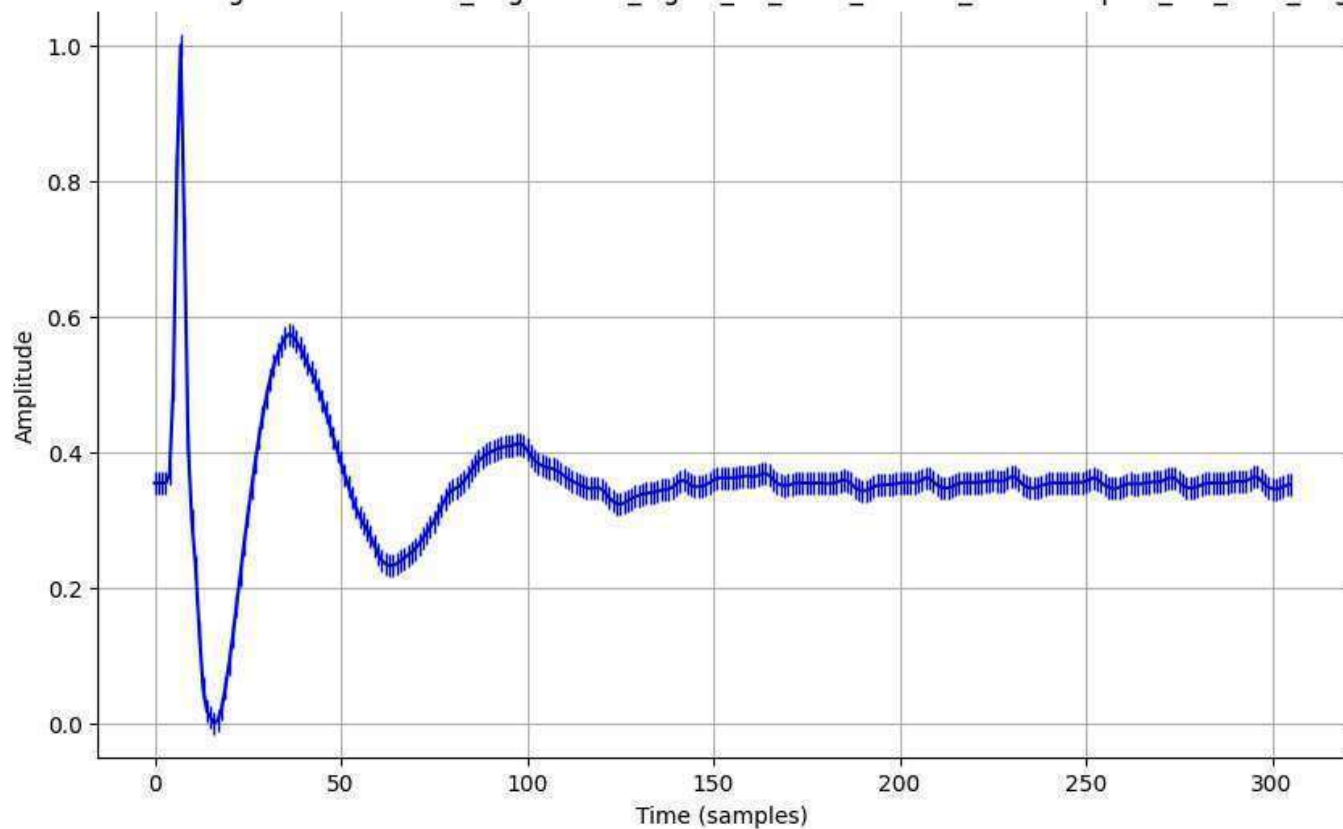
Normalized Signal - normalized_augmented_signal_21_0002_filtered_downsampled_std_0.1_aug.csv



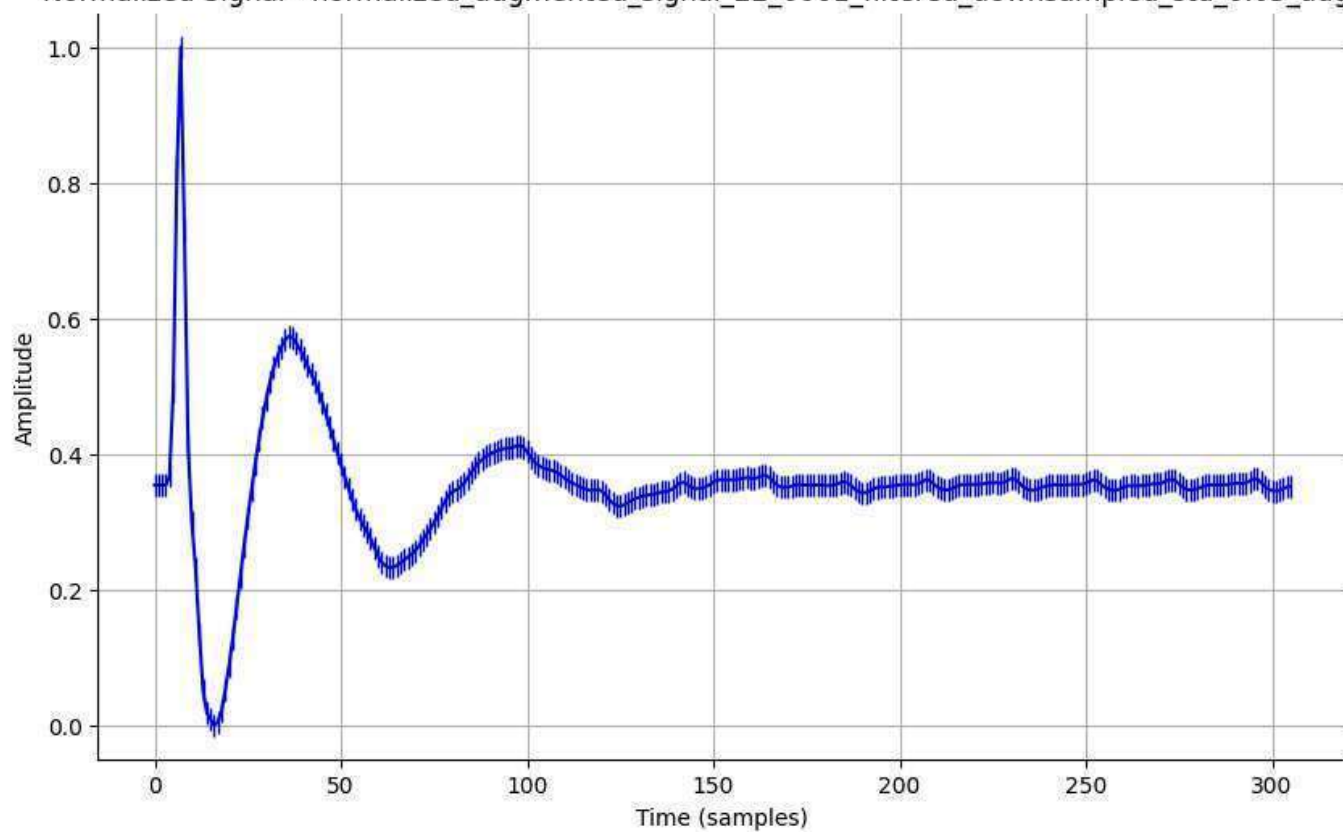
Normalized Signal - normalized_augmented_signal_21_0002_filtered_downsampled_std_0.2_aug.csv



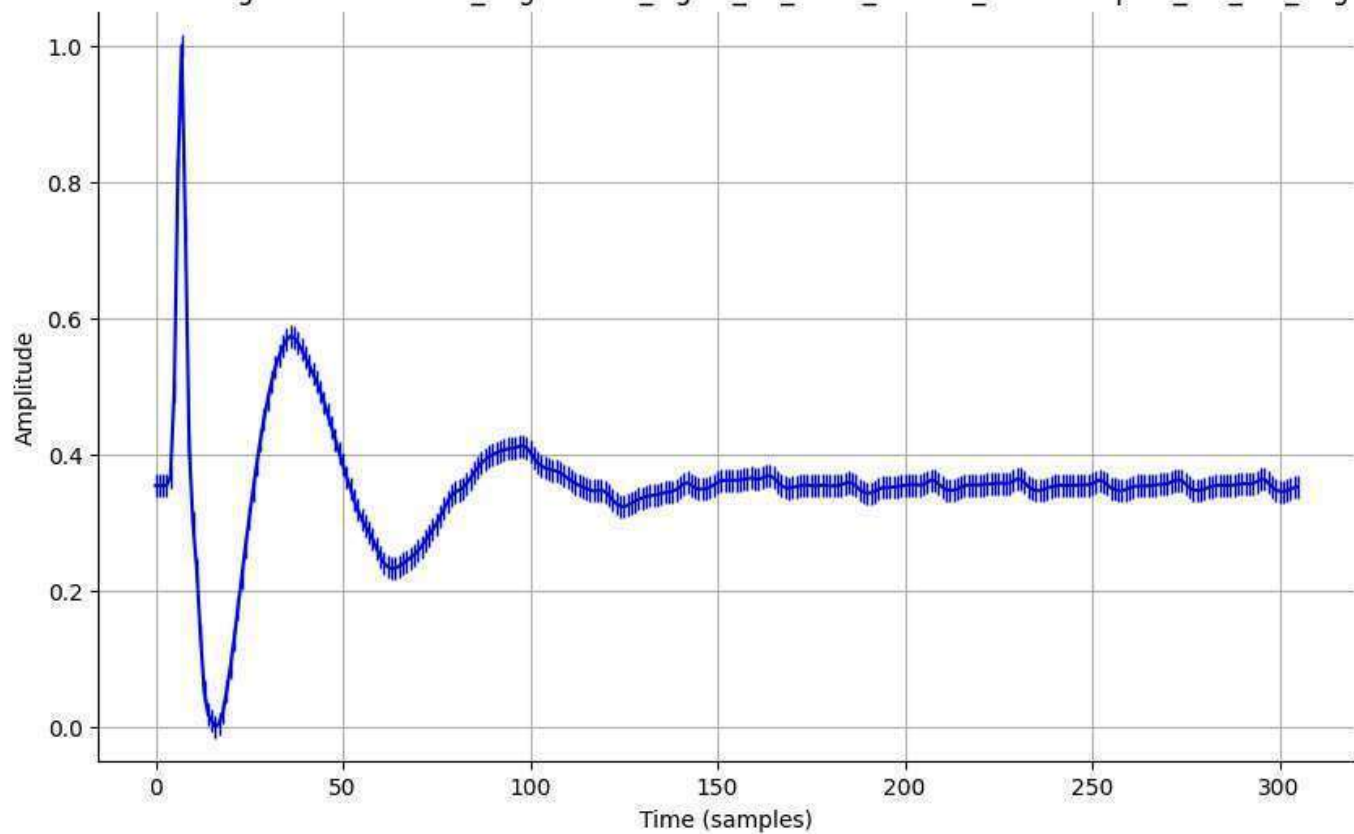
Normalized Signal - normalized_augmented_signal_22_0001_filtered_downsampled_std_0.01_aug.csv



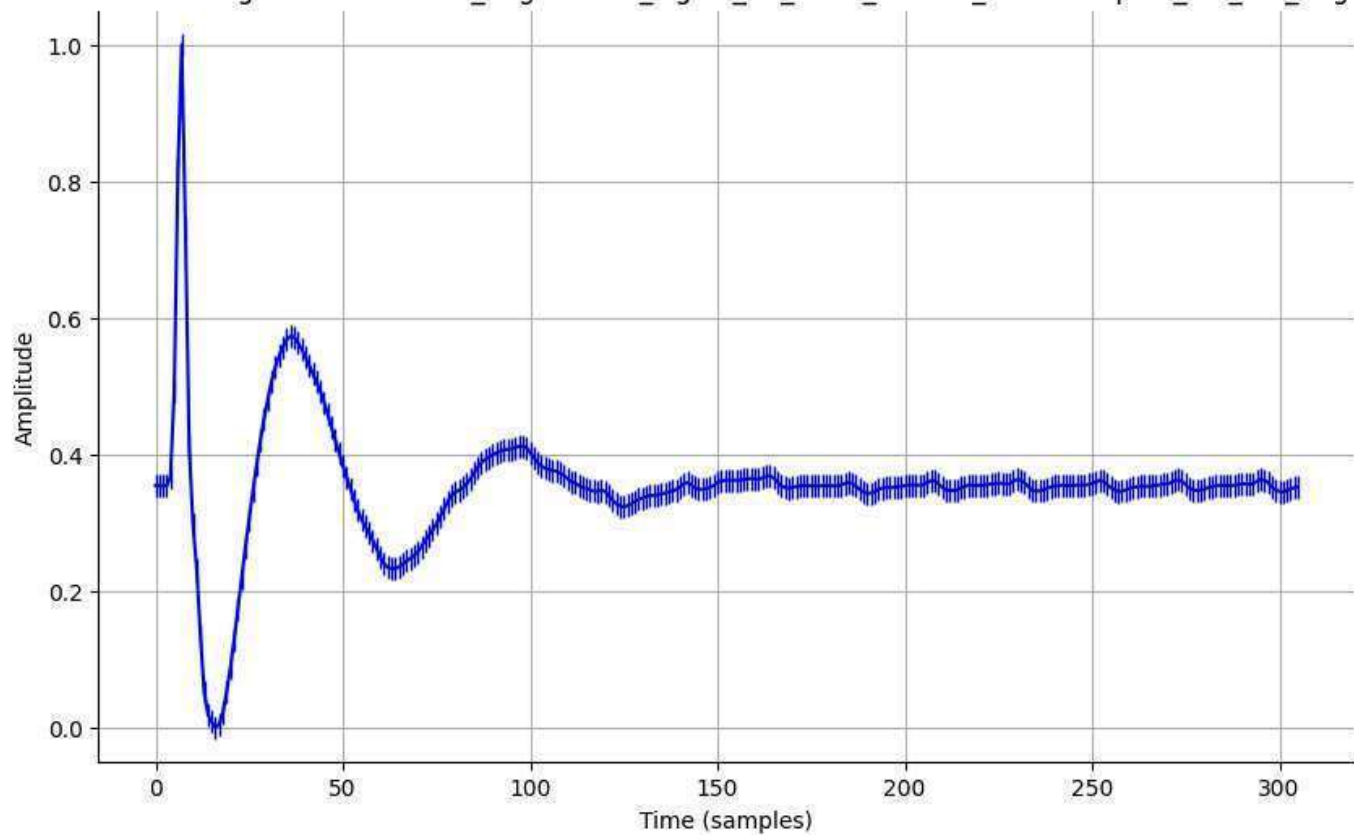
Normalized Signal - normalized_augmented_signal_22_0001_filtered_downsampled_std_0.05_aug.csv



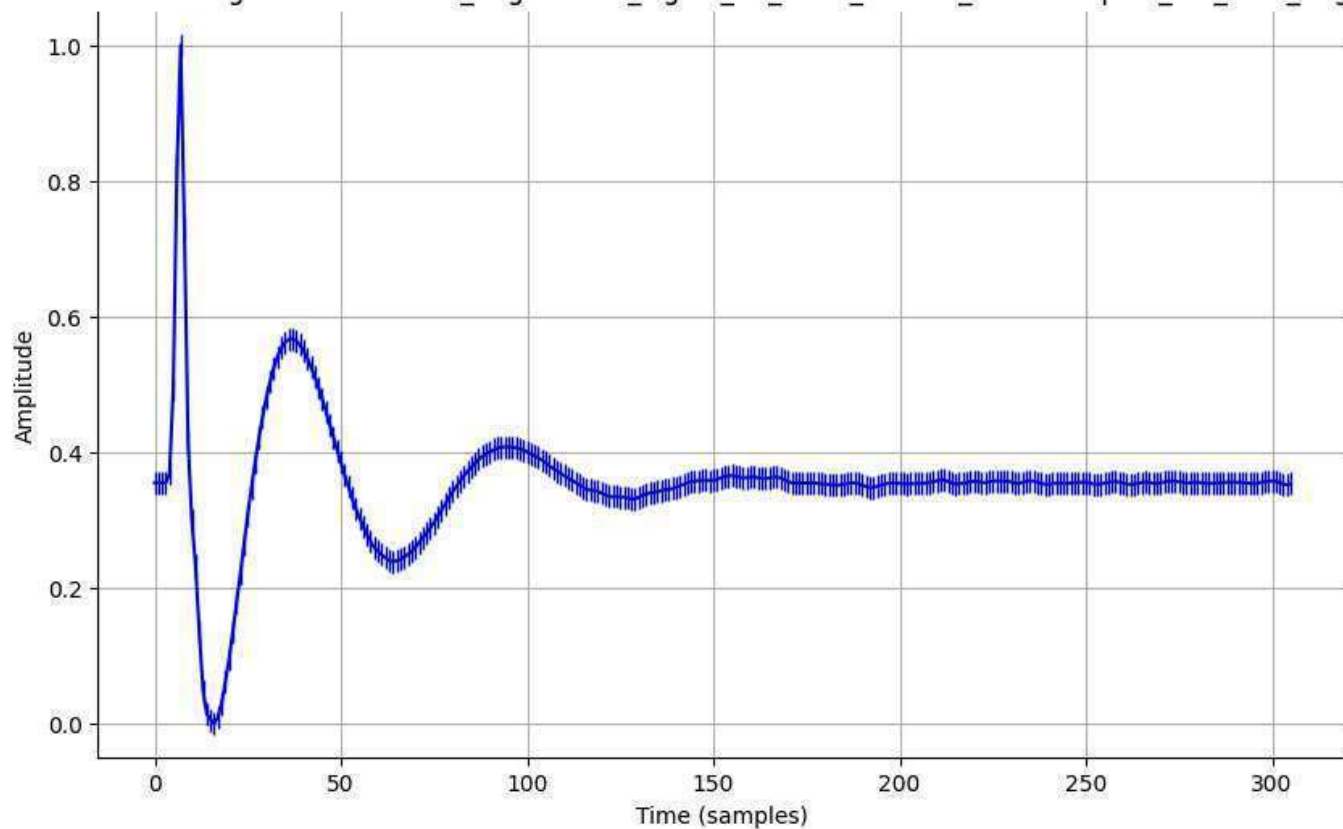
Normalized Signal - normalized_augmented_signal_22_0001_filtered_downsampled_std_0.1_aug.csv



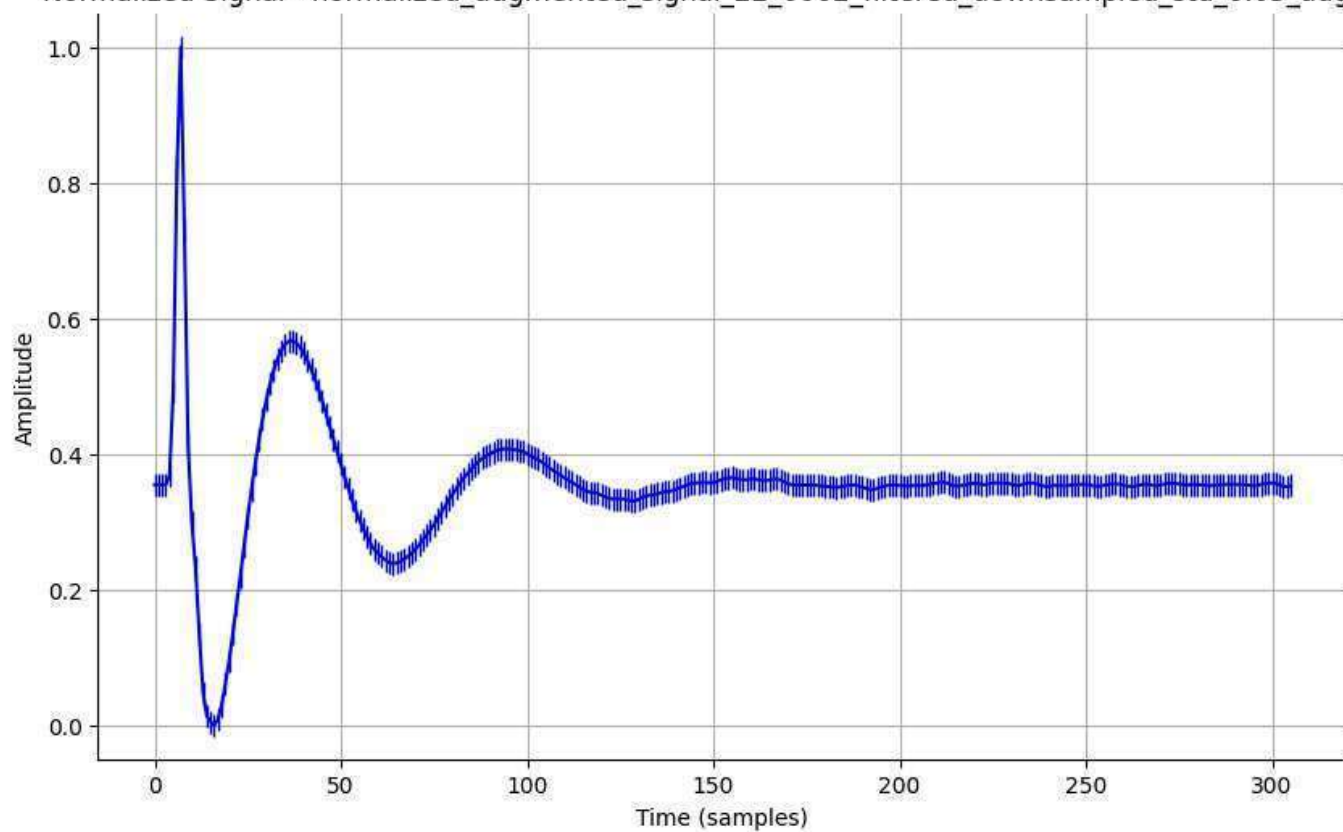
Normalized Signal - normalized_augmented_signal_22_0001_filtered_downsampled_std_0.2_aug.csv



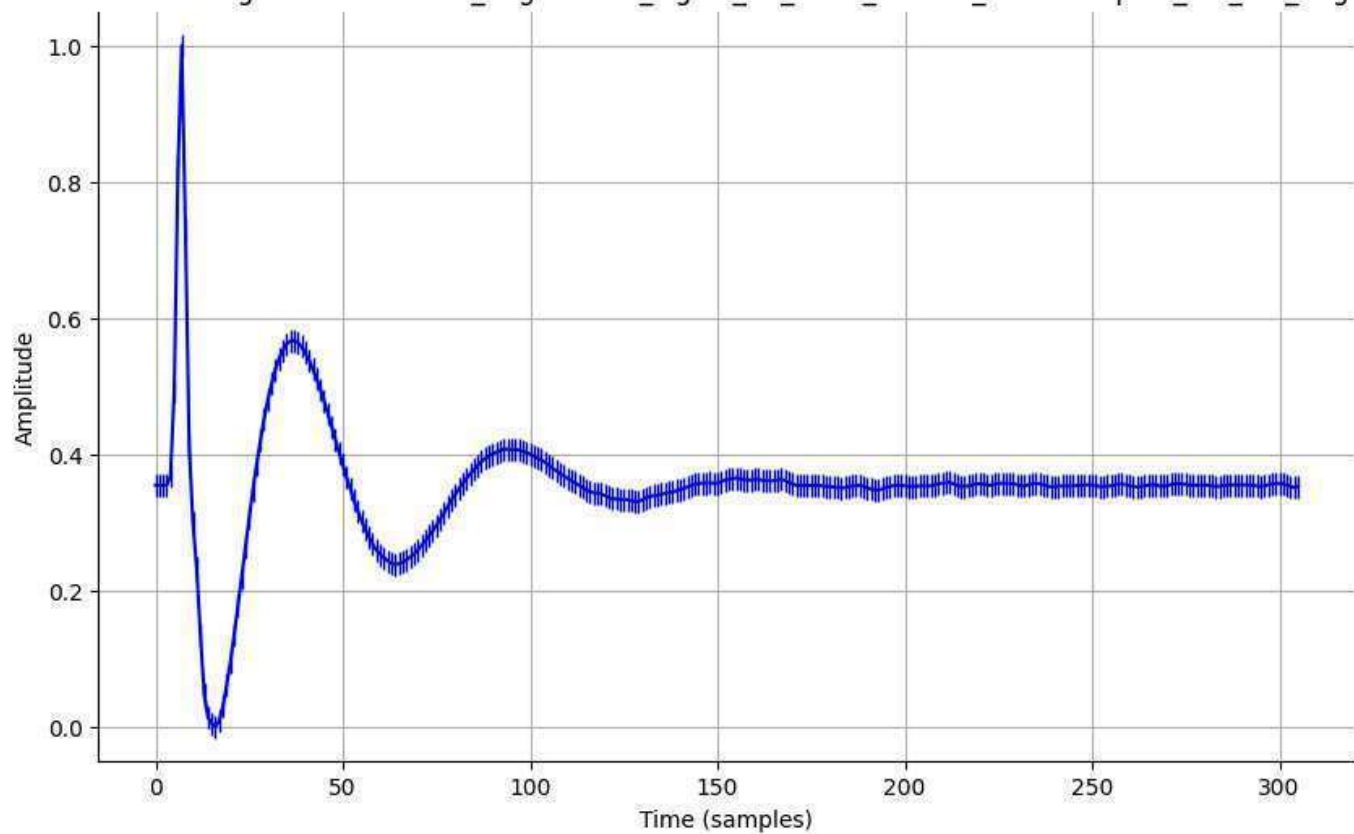
Normalized Signal - normalized_augmented_signal_22_0002_filtered_downsampled_std_0.01_aug.csv



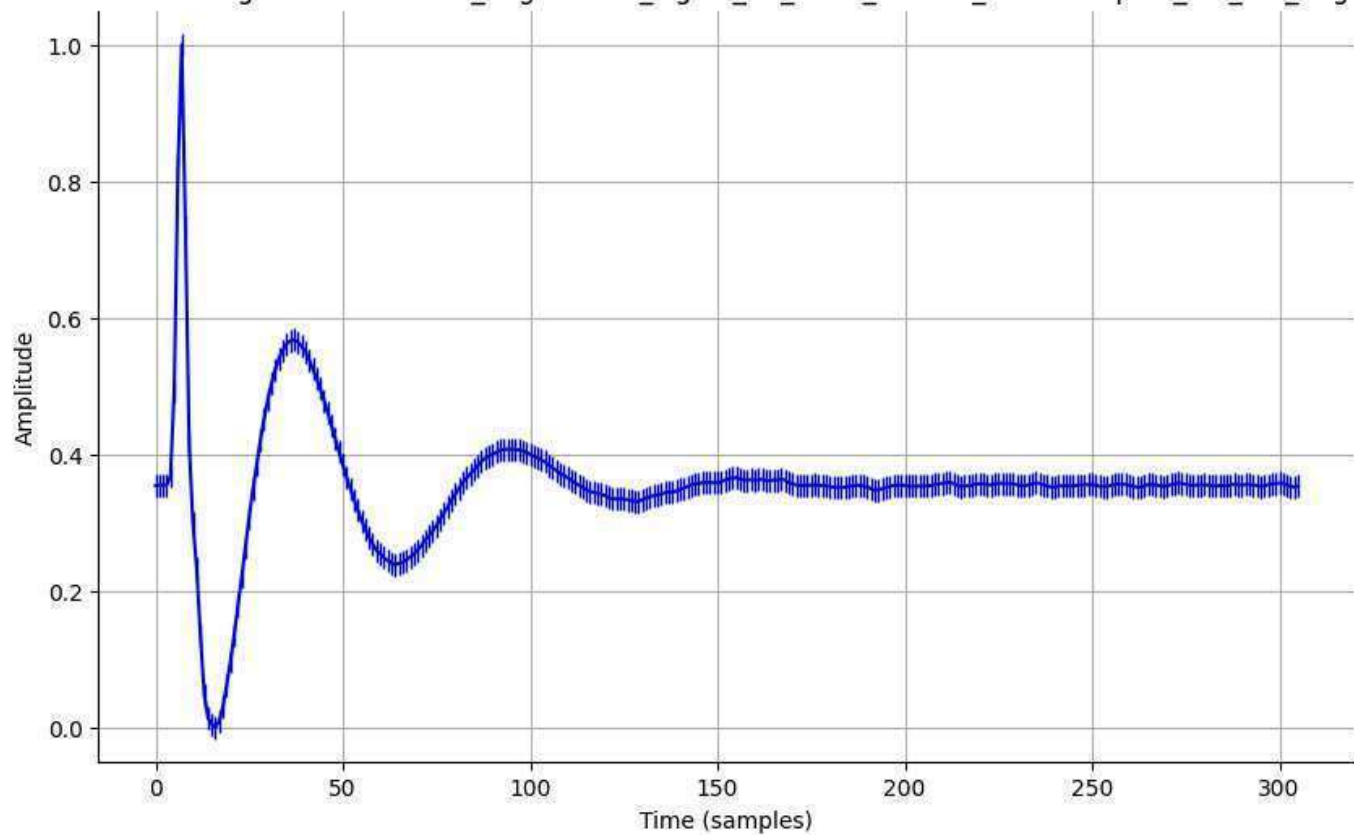
Normalized Signal - normalized_augmented_signal_22_0002_filtered_downsampled_std_0.05_aug.csv



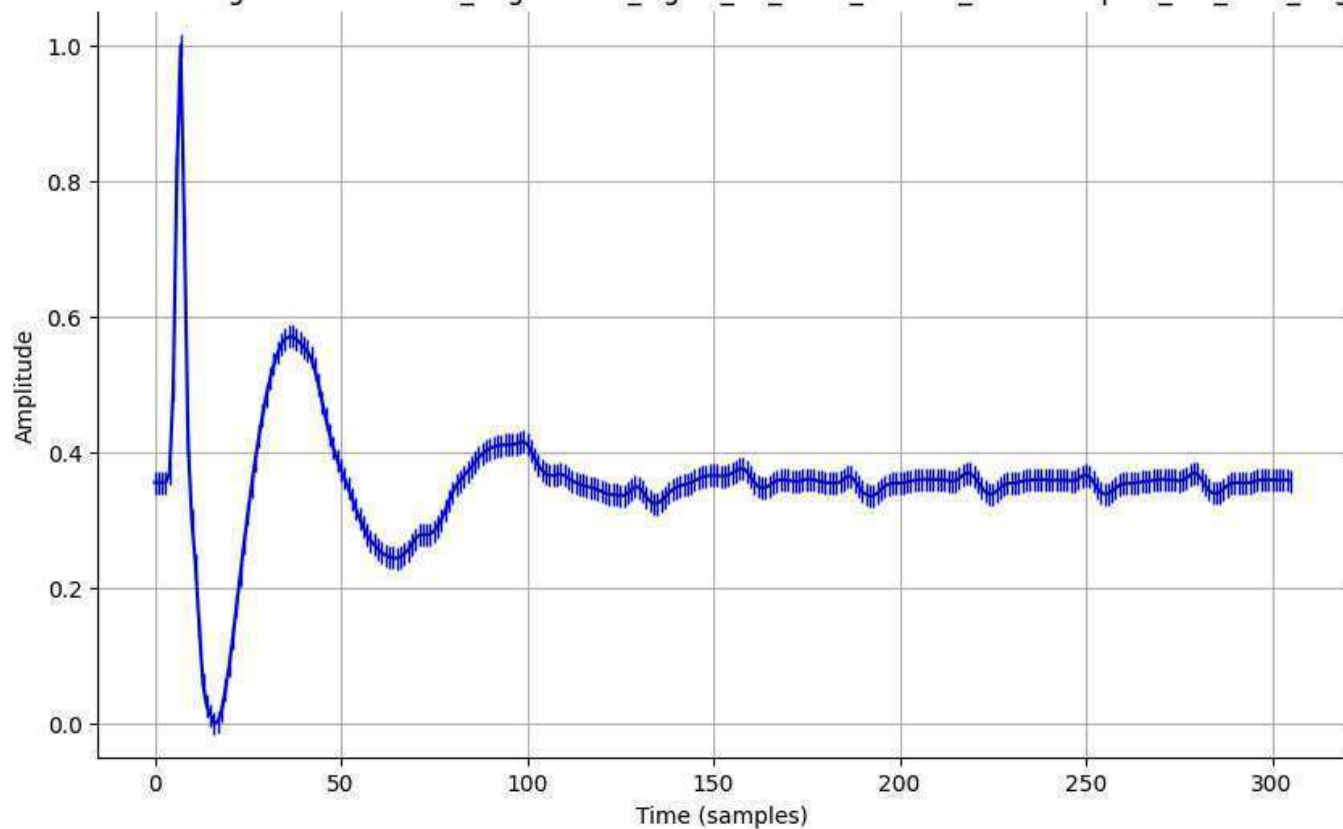
Normalized Signal - normalized_augmented_signal_22_0002_filtered_downsampled_std_0.1_aug.csv



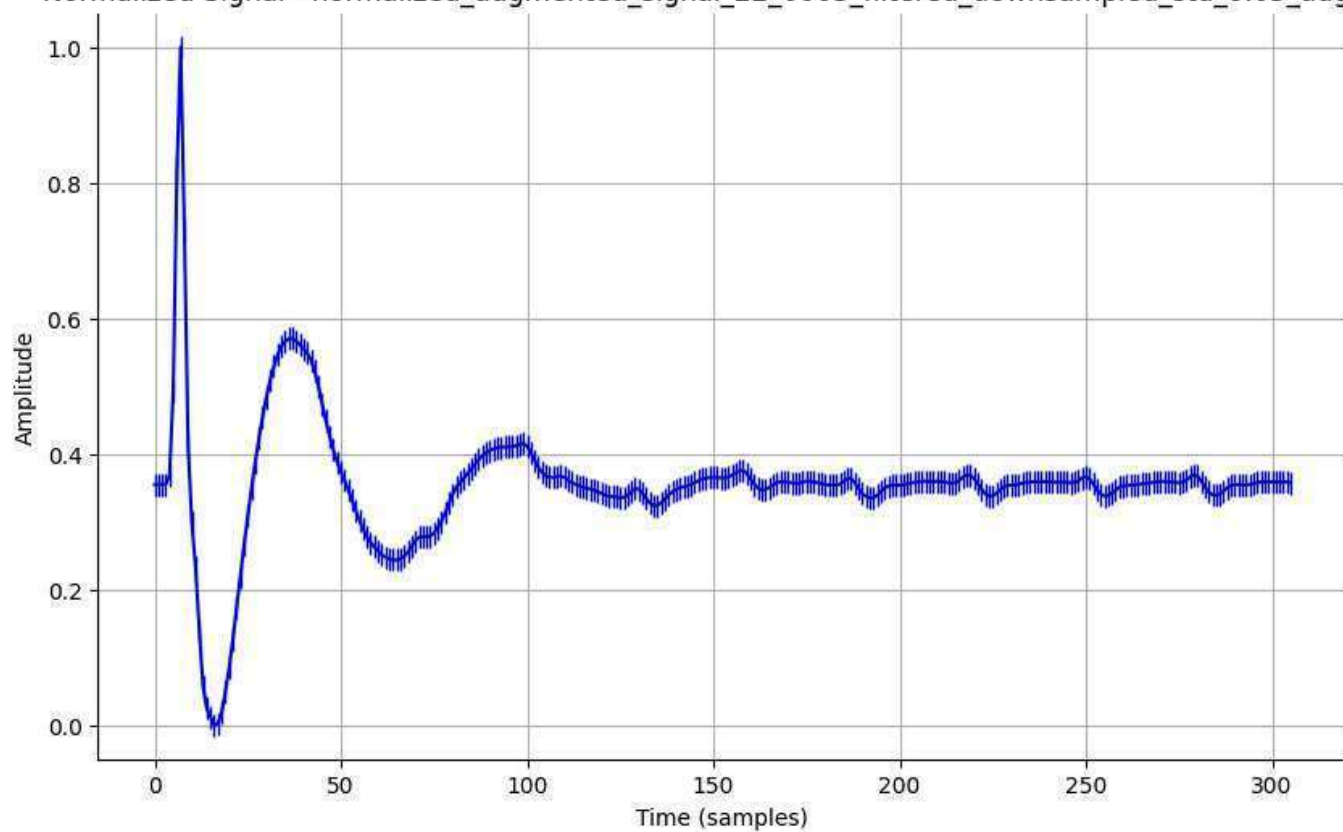
Normalized Signal - normalized_augmented_signal_22_0002_filtered_downsampled_std_0.2_aug.csv



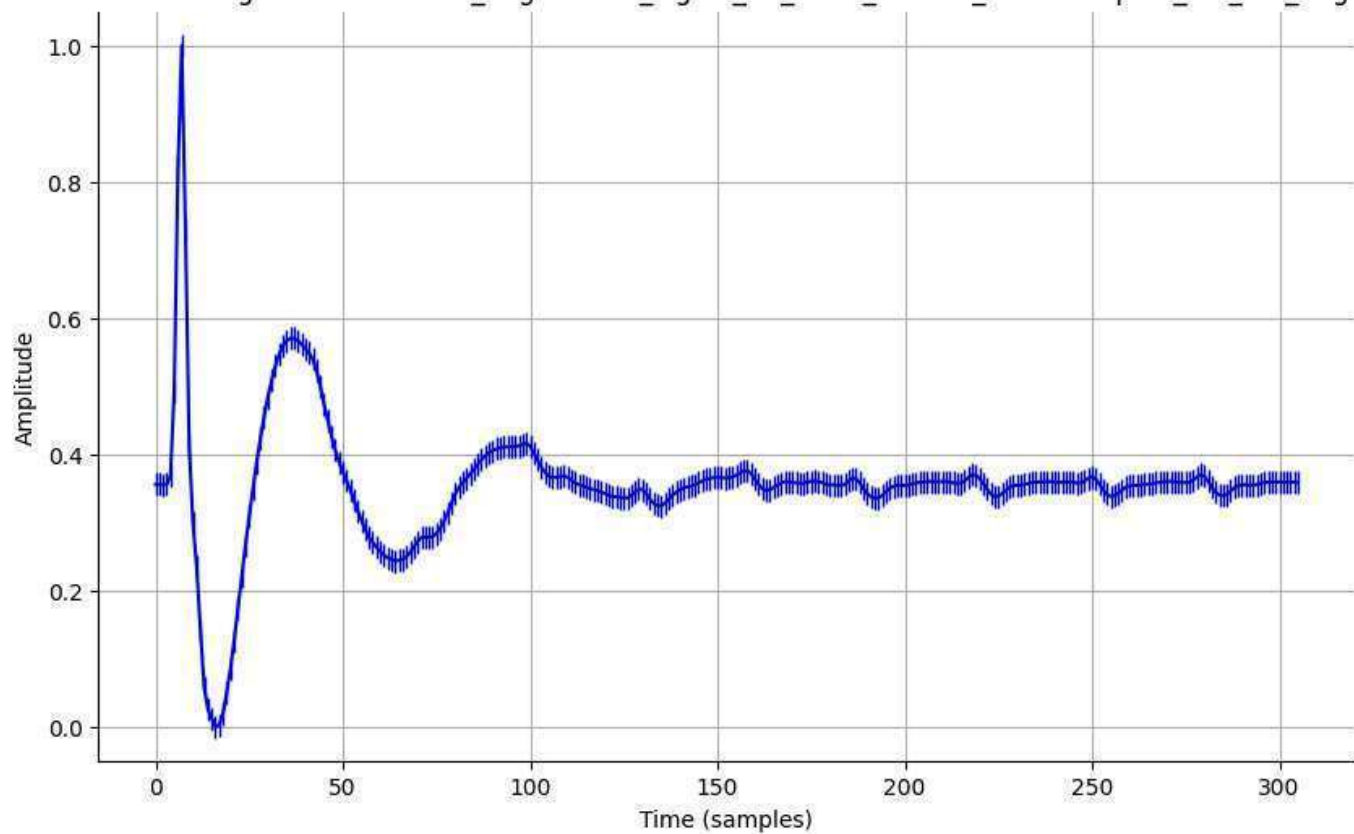
Normalized Signal - normalized_augmented_signal_22_0003_filtered_downsampled_std_0.01_aug.csv



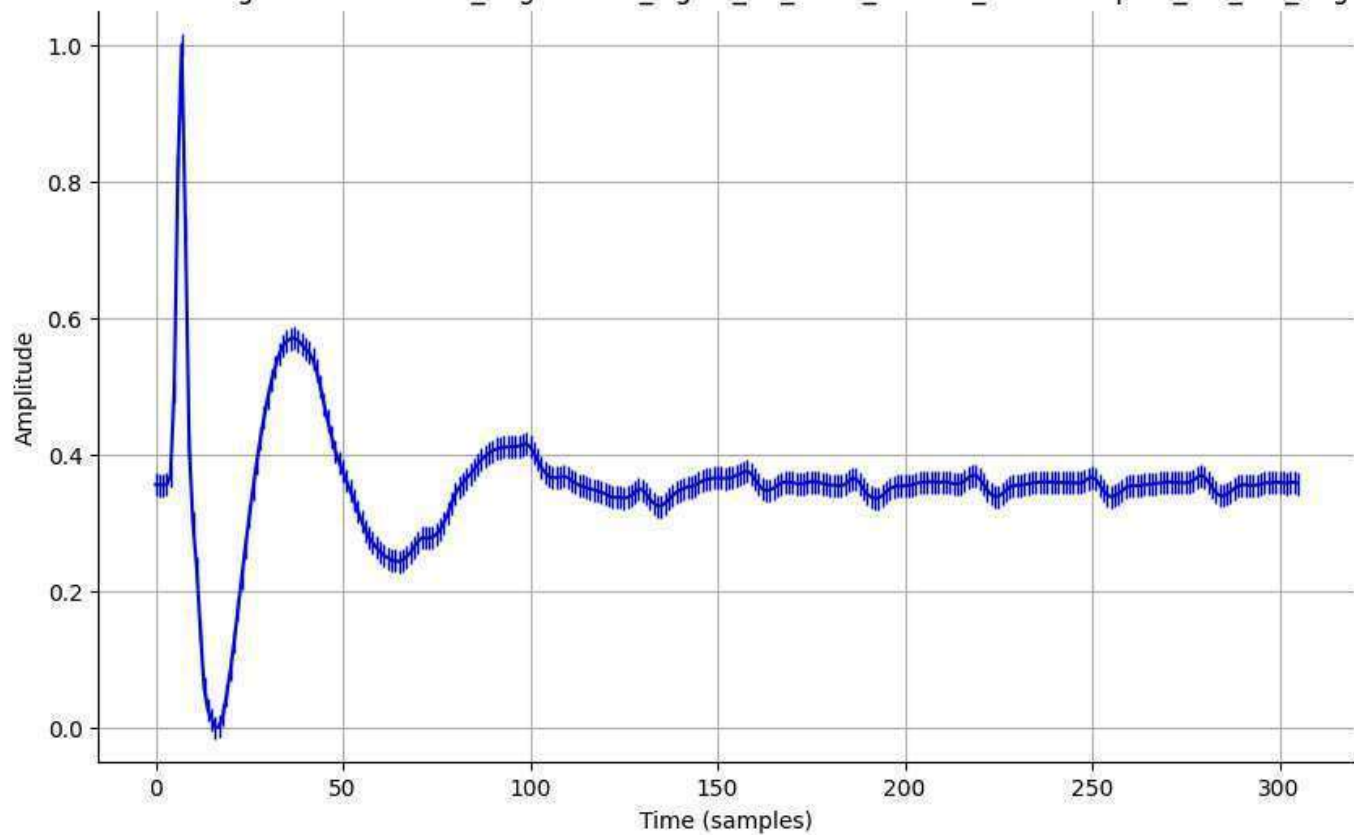
Normalized Signal - normalized_augmented_signal_22_0003_filtered_downsampled_std_0.05_aug.csv



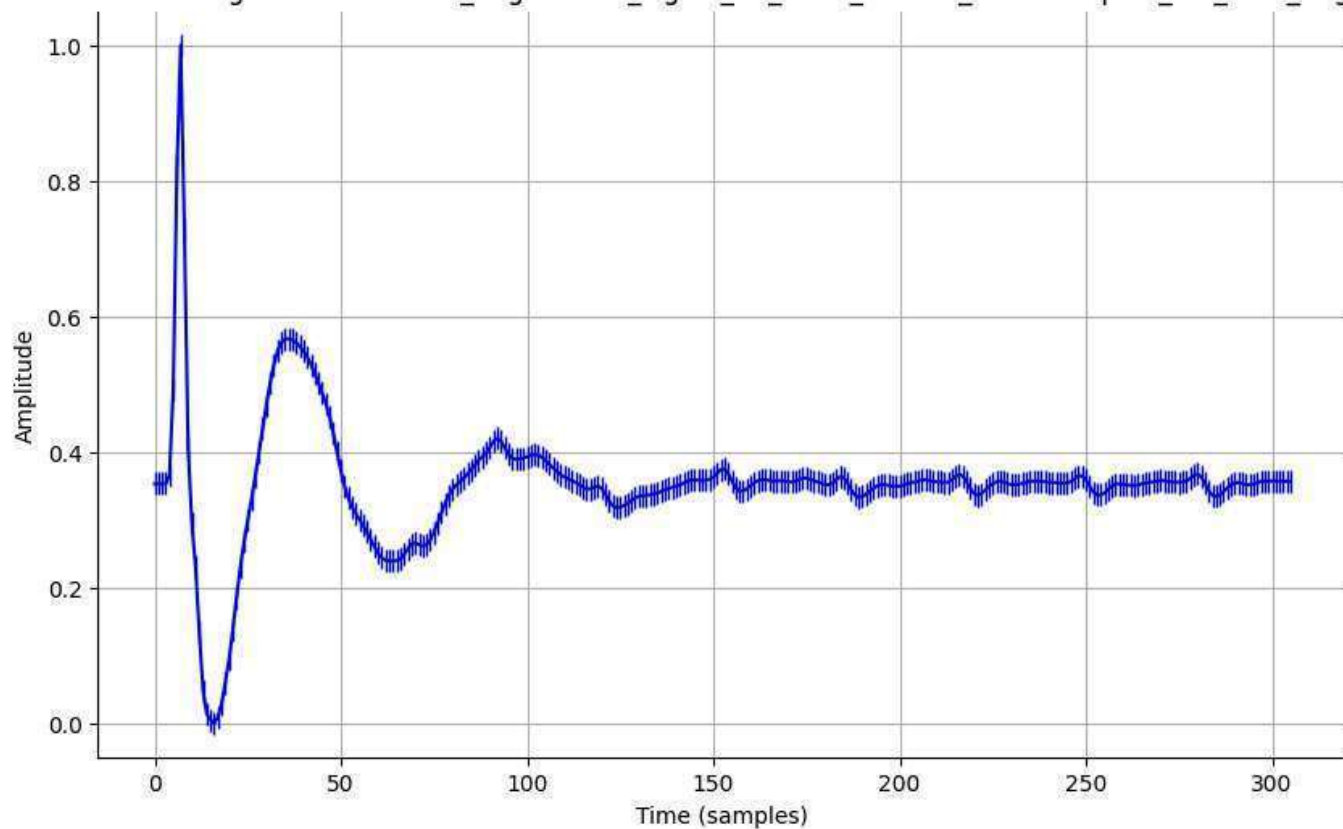
Normalized Signal - normalized_augmented_signal_22_0003_filtered_downsampled_std_0.1_aug.csv



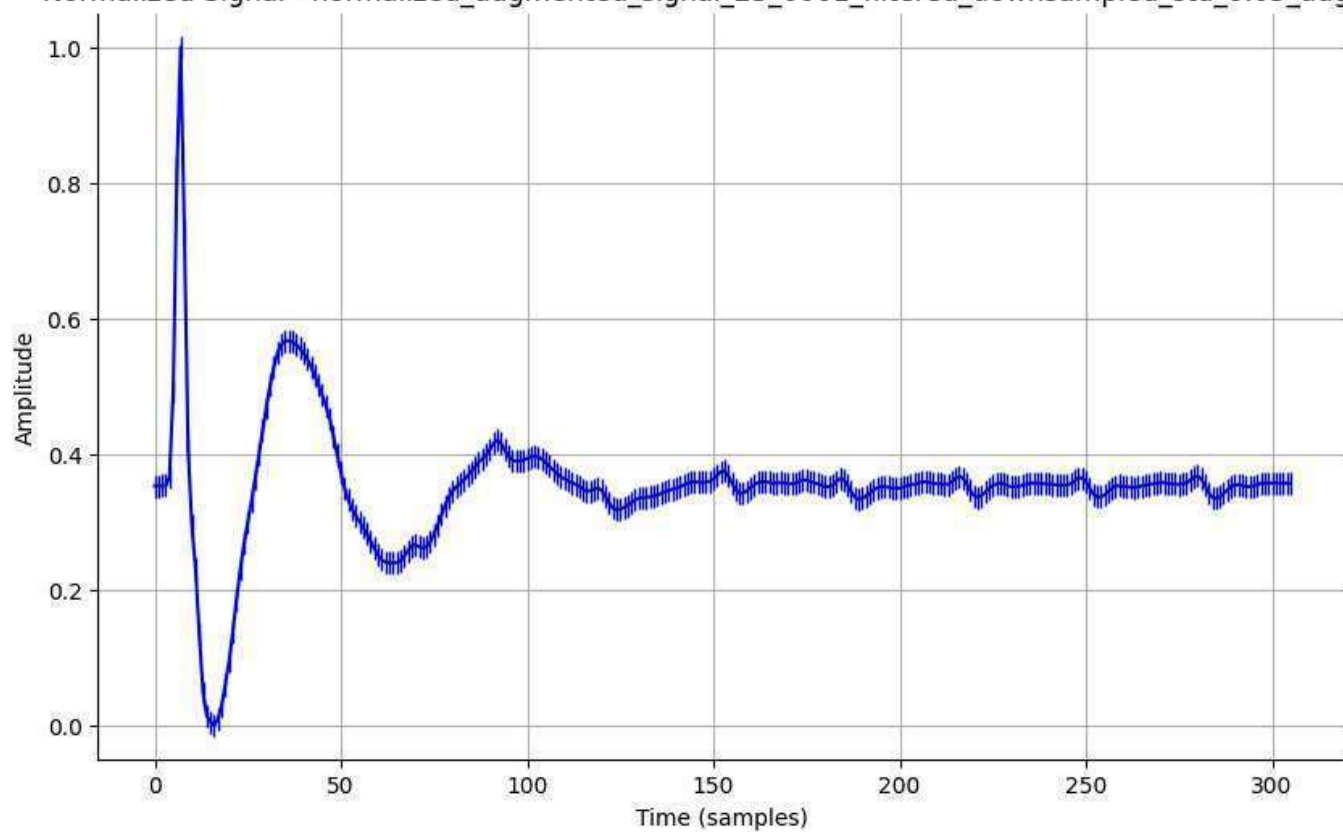
Normalized Signal - normalized_augmented_signal_22_0003_filtered_downsampled_std_0.2_aug.csv



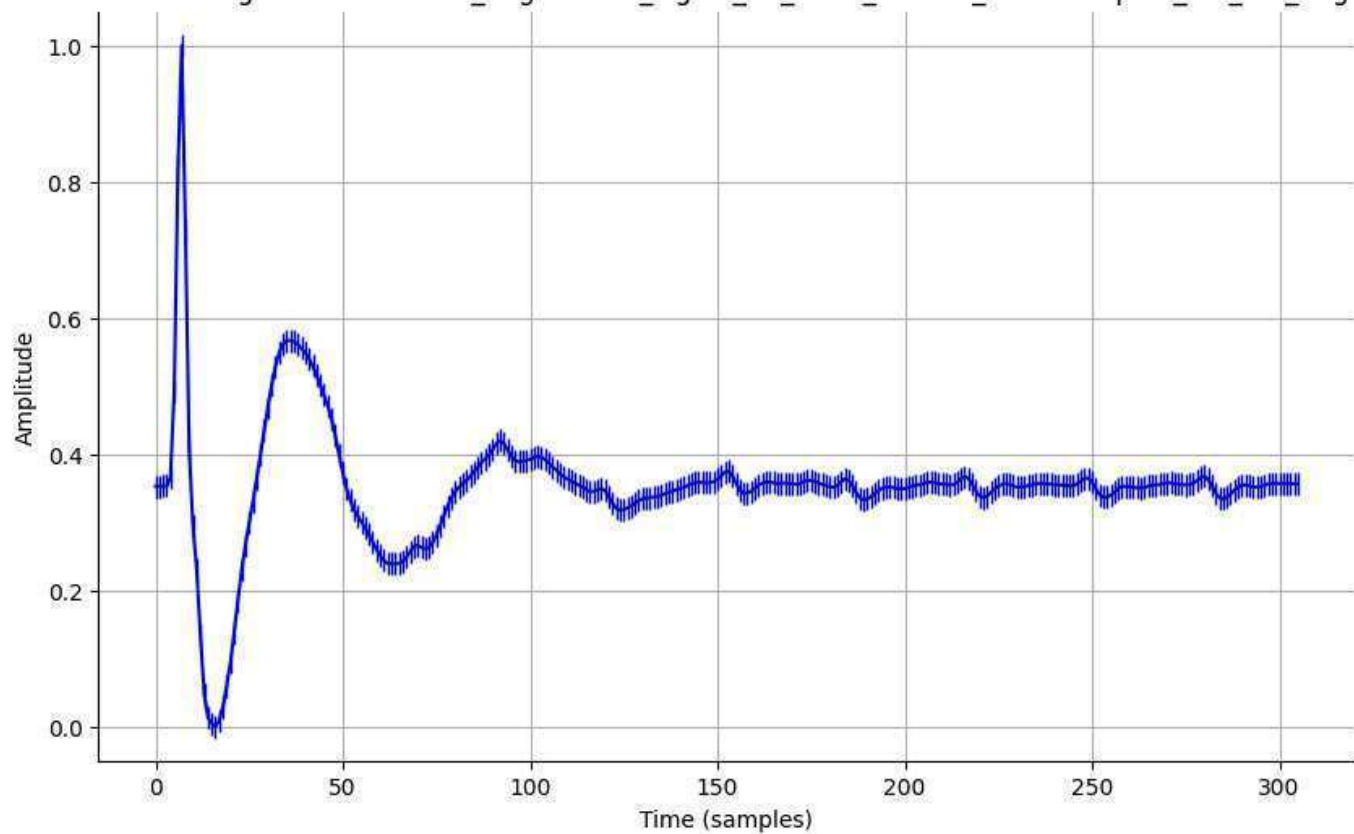
Normalized Signal - normalized_augmented_signal_23_0001_filtered_downsampled_std_0.01_aug.csv



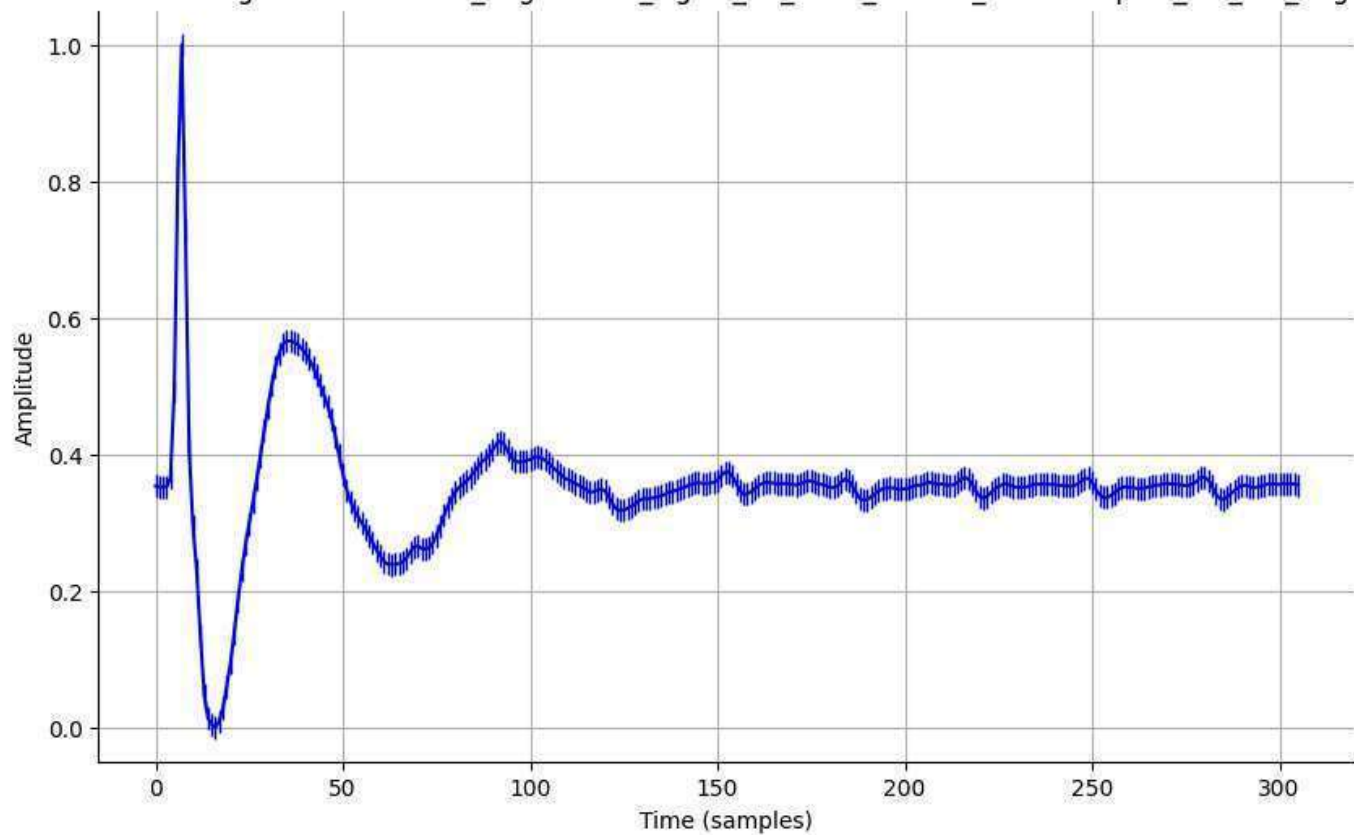
Normalized Signal - normalized_augmented_signal_23_0001_filtered_downsampled_std_0.05_aug.csv



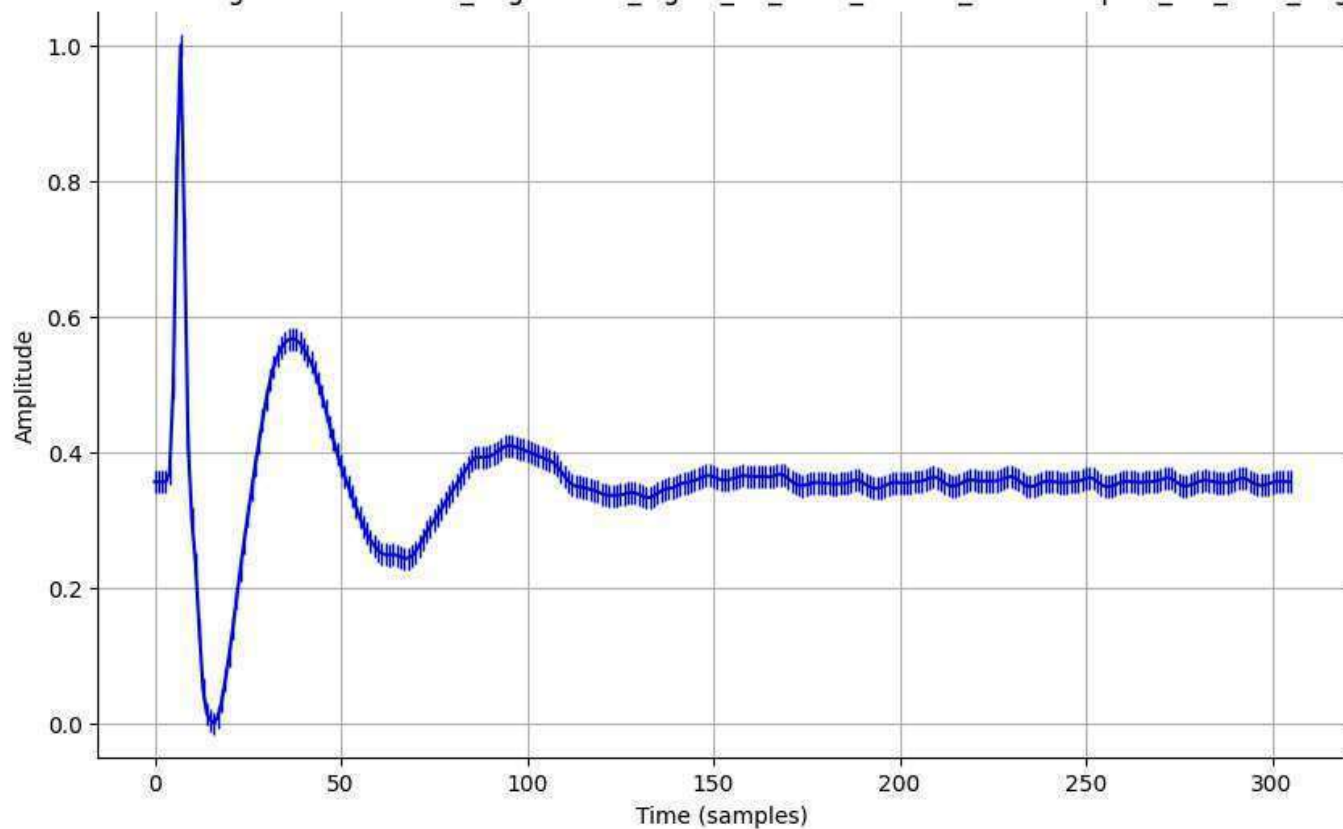
Normalized Signal - normalized_augmented_signal_23_0001_filtered_downsampled_std_0.1_aug.csv



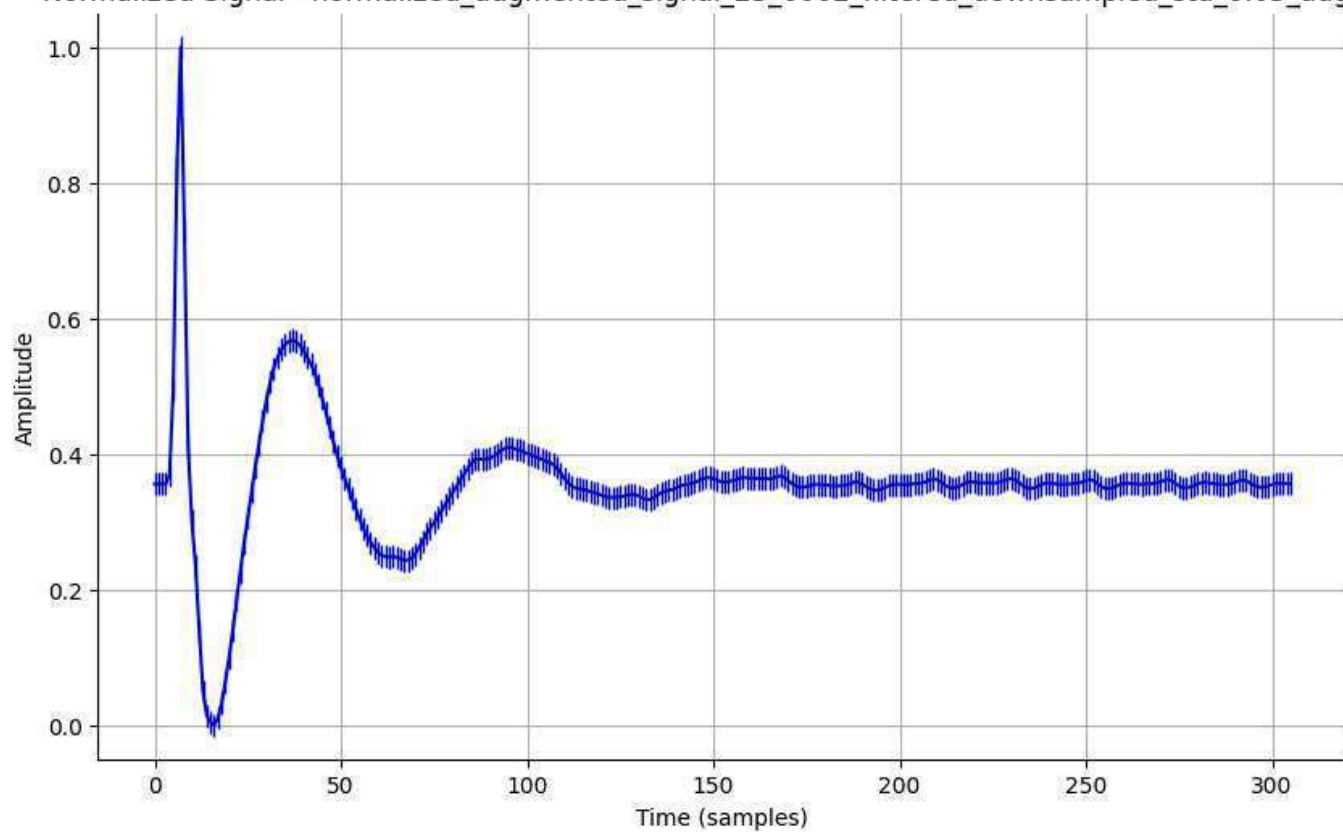
Normalized Signal - normalized_augmented_signal_23_0001_filtered_downsampled_std_0.2_aug.csv

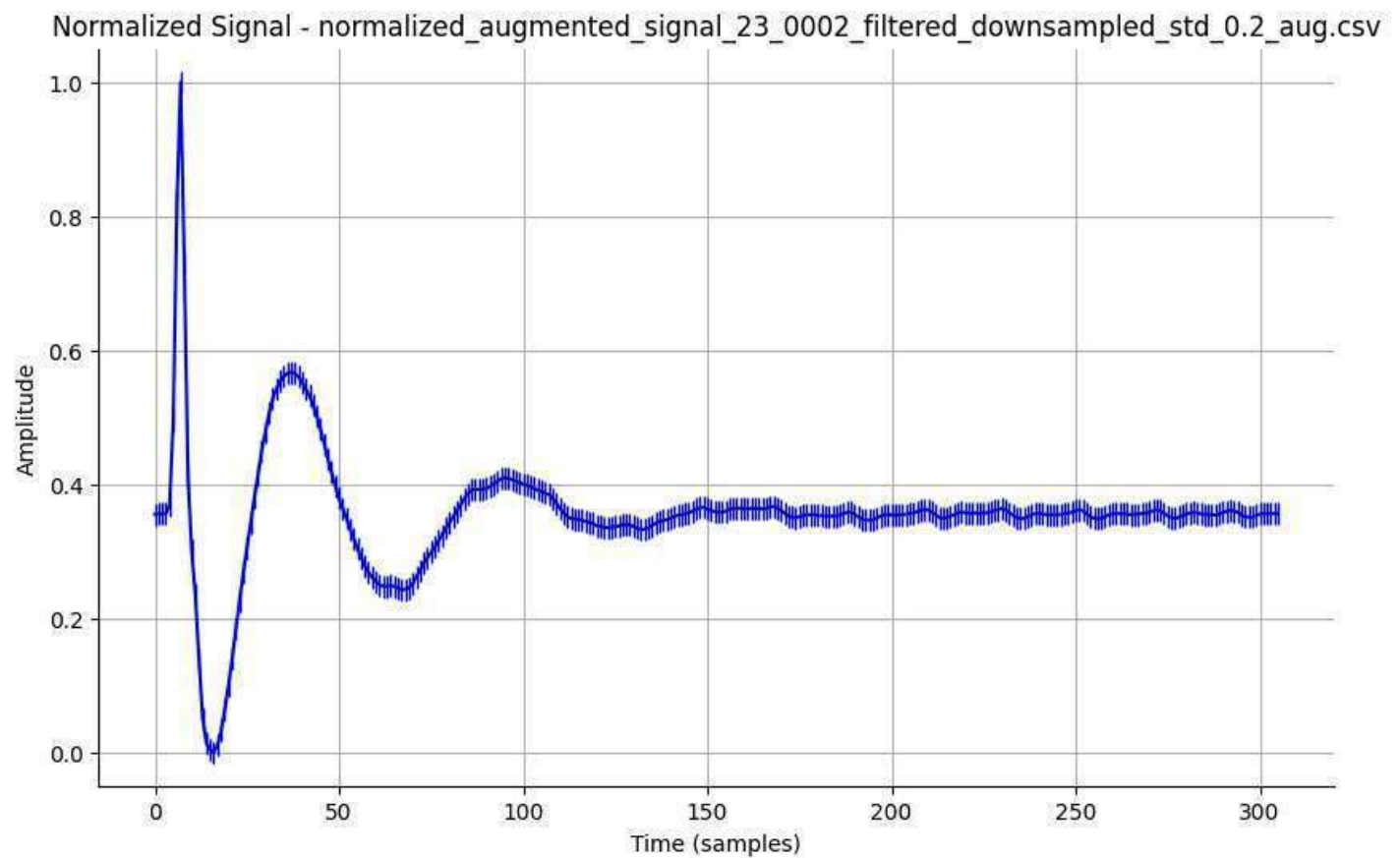
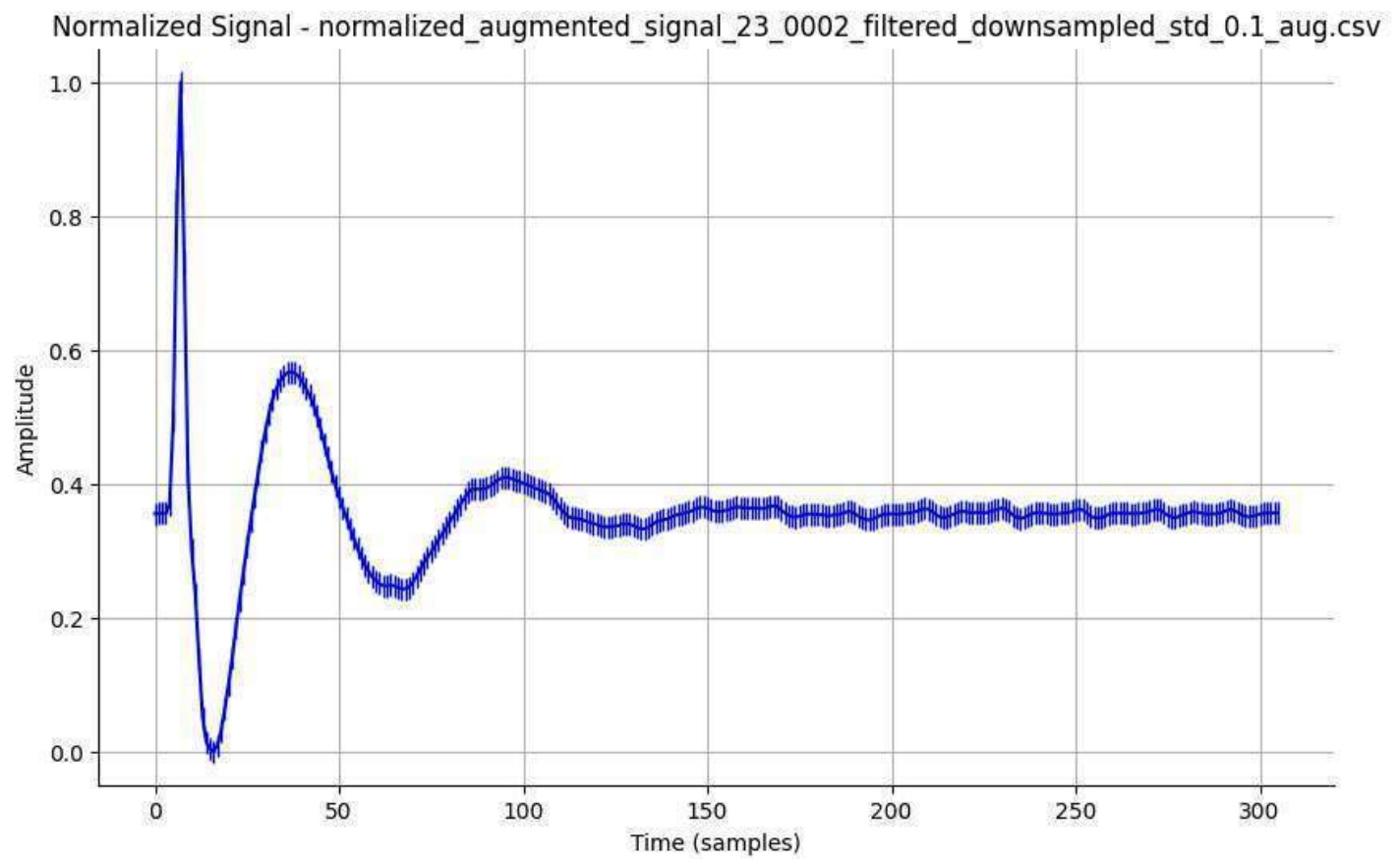


Normalized Signal - normalized_augmented_signal_23_0002_filtered_downsampled_std_0.01_aug.csv



Normalized Signal - normalized_augmented_signal_23_0002_filtered_downsampled_std_0.05_aug.csv





```
In [ ]: #####check it#####
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
import os
```

```

# Folder containing the augmented data with labels
augmented_folder = '/content/PPG_Data/AugmentedDatawithlabels' # Change this to your path

# List all CSV files containing the augmented data with labels
augmented_files = [f for f in os.listdir(augmented_folder) if f.endswith('.csv')]
augmented_files.sort()

# Function to Load augmented data and Labels
def load_data_and_labels(file_list, folder):
    data = []
    labels = []

    for file in file_list:
        # Read the augmented data
        file_path = os.path.join(folder, file)
        df = pd.read_csv(file_path)

        # Extract signal column
        signals = df['signal'].values

        # Extract multiple Labels (ID, Gender, Glucose, Age, etc.)
        # Assume all columns except 'signal' are the Label columns
        label_data = df.drop(columns=['signal'])

        # Append signal and label(s) to respective lists
        data.append(signals)
        labels.append(label_data.iloc[0].to_dict()) # Get the first row as a dictionary of Labels

    # Convert Lists to numpy arrays for further processing
    return np.array(data), labels

# Load augmented data and Labels
data, labels = load_data_and_labels(augmented_files, augmented_folder)

# Split the data into training (60%), validation (20%), and test (20%) sets
X_train, X_temp, y_train, y_temp = train_test_split(data, labels, test_size=0.4, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42)

# Verify the split
print(f"Training data shape: {X_train.shape}, Training labels shape: {len(y_train)}")
print(f"Validation data shape: {X_val.shape}, Validation labels shape: {len(y_val)}")
print(f"Test data shape: {X_test.shape}, Test labels shape: {len(y_test)}")

# Optionally save the data splits to CSV (if needed for later use)
def save_split_data(X_train, X_val, X_test, y_train, y_val, y_test):
    # Save the split data to CSV files
    pd.DataFrame(X_train).to_csv('/content/PPG_Data/split_X_train.csv', index=False)
    pd.DataFrame(X_val).to_csv('/content/PPG_Data/split_X_val.csv', index=False)
    pd.DataFrame(X_test).to_csv('/content/PPG_Data/split_X_test.csv', index=False)

    # Save the Labels to CSV
    pd.DataFrame(y_train).to_csv('/content/PPG_Data/split_y_train.csv', index=False)
    pd.DataFrame(y_val).to_csv('/content/PPG_Data/split_y_val.csv', index=False)
    pd.DataFrame(y_test).to_csv('/content/PPG_Data/split_y_test.csv', index=False)

# Uncomment to save the splits to CSV files
save_split_data(X_train, X_val, X_test, y_train, y_val, y_test)

```

Training data shape: (160, 306), Training labels shape: 160
Validation data shape: (54, 306), Validation labels shape: 54
Test data shape: (54, 306), Test labels shape: 54

```
In [ ]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
import os

# Folder containing the augmented data with labels
augmented_folder = '/content/PPG_Data/AugmentedDatawithlabels' # Change this to your path

# List all CSV files containing the augmented data with labels
augmented_files = [f for f in os.listdir(augmented_folder) if f.endswith('.csv')]
augmented_files.sort()

# Function to Load augmented data and Labels
def load_data_and_labels(file_list, folder):
    data = []
    labels = []

    for file in file_list:
        # Read the augmented data
        file_path = os.path.join(folder, file)
        df = pd.read_csv(file_path)

        # Extract signal column (the signal is assumed to be in the 'signal' column)
        signals = df['signal'].values

        # Extract multiple Labels (Gender, Age, Glucose, etc.)
        # Gender, Age, and other features should be considered as the features,
        # and Glucose should be the target Label.
        features = df.drop(columns=['Glucose']) # Exclude 'signal' and 'Glucose'

        # Extract Glucose as the target
        glucose = df['Glucose'].values[0] # Assuming the Glucose Label is the same for all rows

        # Append signal and Label(s) to respective lists
        data.append(np.hstack([signals, features.values.flatten()])) # Combine signal and features
        labels.append(glucose) # Glucose as the Label

    # Convert Lists to numpy arrays for further processing
    return np.array(data), np.array(labels)

# Load augmented data and Labels
data, labels = load_data_and_labels(augmented_files, augmented_folder)

# Split the data into training (60%), validation (20%), and test (20%) sets
X_train, X_temp, y_train, y_temp = train_test_split(data, labels, test_size=0.4, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42)

# Verify the split
print(f"Training data shape: {X_train.shape}, Training labels shape: {y_train.shape}")
print(f"Validation data shape: {X_val.shape}, Validation labels shape: {y_val.shape}")
print(f"Test data shape: {X_test.shape}, Test labels shape: {y_test.shape}")

# Optionally save the data splits to CSV (if needed for later use)
def save_split_data(X_train, X_val, X_test, y_train, y_val, y_test):
    # Save the split data to CSV files
    pd.DataFrame(X_train).to_csv('/content/PPG_Data/split_X_train.csv', index=False)
```

```

pd.DataFrame(X_val).to_csv('/content/PPG_Data/split_X_val.csv', index=False)
pd.DataFrame(X_test).to_csv('/content/PPG_Data/split_X_test.csv', index=False)

# Save the Labels to CSV
pd.DataFrame(y_train).to_csv('/content/PPG_Data/split_y_train.csv', index=False)
pd.DataFrame(y_val).to_csv('/content/PPG_Data/split_y_val.csv', index=False)
pd.DataFrame(y_test).to_csv('/content/PPG_Data/split_y_test.csv', index=False)

# Uncomment to save the splits to CSV files
save_split_data(X_train, X_val, X_test, y_train, y_val, y_test)

```

Training data shape: (160, 2142), Training labels shape: (160,)
 Validation data shape: (54, 2142), Validation labels shape: (54,)
 Test data shape: (54, 2142), Test labels shape: (54,)

```

In [ ]: df=pd.read_csv('/content/PPG_Data/AugmentedDatawithlabels/augmented_signal_01_0001_filtered_downs
df

```

```

Out[ ]:

```

	signal	ID	Gender	Age	Height	Weight	Glucose
0	-0.001689	1	0	38	180	53	99
1	0.006433	1	0	38	180	53	99
2	0.005465	1	0	38	180	53	99
3	0.113350	1	0	38	180	53	99
4	7.175986	1	0	38	180	53	99
...	...	...	...	...	...	...	...
301	4.445192	1	0	38	180	53	99
302	6.387539	1	0	38	180	53	99
303	5.492667	1	0	38	180	53	99
304	1.655626	1	0	38	180	53	99
305	-2.909335	1	0	38	180	53	99

306 rows × 7 columns

```

In [ ]: df=pd.read_csv('/content/PPG_Data/split_y_test.csv')
df

```


Out[]:

0	
0	103
1	102
2	110
3	99
4	130
5	112
6	127
7	96
8	94
9	110
10	103
11	124
12	110
13	106
14	109
15	130
16	88
17	88
18	96
19	183
20	106
21	110
22	110
23	183
24	96
25	100
26	128
27	99
28	94
29	124
30	112
31	106

	0
32	109
33	103
34	183
35	146
36	100
37	130
38	95
39	110
40	110
41	102
42	129
43	112
44	118
45	113
46	103
47	146
48	136
49	129
50	96
51	127
52	110
53	100

```
In [ ]: import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.models import Model

# Model Architecture

def build_model(input_shape):
    # Input Layer
    input_signal = layers.Input(shape=input_shape, name='input_signal')

    # CNN Block 1 (for signal feature extraction)
    cnn_block_1 = layers.Conv1D(32, kernel_size=3, activation='relu')(input_signal)
    cnn_block_1 = layers.BatchNormalization()(cnn_block_1)
    cnn_block_1 = layers.MaxPooling1D(pool_size=2)(cnn_block_1)

    # CNN Block 2 (for additional feature extraction)
    cnn_block_2 = layers.Conv1D(64, kernel_size=5, activation='relu')(cnn_block_1)
```

```

cnn_block_2 = layers.BatchNormalization()(cnn_block_2)
cnn_block_2 = layers.MaxPooling1D(pool_size=2)(cnn_block_2)

# GRU Block (for sequential pattern recognition)
gru_block = layers.GRU(64, return_sequences=False)(cnn_block_2)

# Flatten the outputs from CNN and GRU blocks
flatten_cnn = layers.Flatten()(cnn_block_2)
flatten_gru = layers.Flatten()(gru_block)

# Fully connected layers
fc1 = layers.Dense(128, activation='relu')(flatten_cnn)
fc2 = layers.Dense(128, activation='relu')(flatten_gru)

# Concatenate the outputs from CNN and GRU
concatenated = layers.concatenate([fc1, fc2], axis=-1)

# Final regression output
output = layers.Dense(1, activation='linear')(concatenated)

# Create the model
model = Model(inputs=input_signal, outputs=output)

# Compile the model
model.compile(optimizer='adam', loss='mean_squared_error', metrics=['mae'])

return model

# Define input shape based on the number of features and signal length
input_shape = (X_train.shape[1], 1) # Adjust based on your signal length

# Build the model
model = build_model(input_shape)

# Summary of the model
model.summary()

# Train the model
model.fit(X_train, y_train, epochs=5, batch_size=32, validation_data=(X_val, y_val))

# Evaluate the model on the test set
#test_loss, test_mae = model.evaluate(X_test, y_test)
print(f"Test Loss: {test_loss}, Test MAE: {test_mae}")

```

Model: "functional_23"

Layer (type)	Output Shape	Param #	Connected to
input_signal (InputLayer)	(None, 2142, 1)	0	-
conv1d_14 (Conv1D)	(None, 2140, 32)	128	input_signal[0][0]
batch_normalization_14 (BatchNormalization)	(None, 2140, 32)	128	conv1d_14[0][0]
max_pooling1d_14 (MaxPooling1D)	(None, 1070, 32)	0	batch_normalizati
conv1d_15 (Conv1D)	(None, 1066, 64)	10,304	max_pooling1d_14[
batch_normalization_15 (BatchNormalization)	(None, 1066, 64)	256	conv1d_15[0][0]
max_pooling1d_15 (MaxPooling1D)	(None, 533, 64)	0	batch_normalizati
gru_13 (GRU)	(None, 64)	24,960	max_pooling1d_15[
flatten_14 (Flatten)	(None, 34112)	0	max_pooling1d_15[
flatten_15 (Flatten)	(None, 64)	0	gru_13[0][0]
dense_29 (Dense)	(None, 128)	4,366,464	flatten_14[0][0]
dense_30 (Dense)	(None, 128)	8,320	flatten_15[0][0]
concatenate_7 (Concatenate)	(None, 256)	0	dense_29[0][0], dense_30[0][0]
dense_31 (Dense)	(None, 1)	257	concatenate_7[0][

Total params: 4,410,817 (16.83 MB)

Trainable params: 4,410,625 (16.83 MB)

Non-trainable params: 192 (768.00 B)

Epoch 1/5

4/4 ————— 5s 161ms/step - loss: 8221.5732 - mae: 77.8714 - val_loss: 135517.6250 - val_mae: 366.6783

Epoch 2/5

4/4 ————— 0s 62ms/step - loss: 946.9780 - mae: 25.8566 - val_loss: 30773.9277 - val_mae: 173.7663

Epoch 3/5

4/4 ————— 0s 59ms/step - loss: 540.4016 - mae: 17.2423 - val_loss: 41828.5742 - val_mae: 203.0228

Epoch 4/5

4/4 ————— 0s 55ms/step - loss: 572.8195 - mae: 20.4578 - val_loss: 9268.7783 - val_mae: 93.9294

Epoch 5/5

4/4 ————— 0s 42ms/step - loss: 489.2506 - mae: 16.0450 - val_loss: 11560.1133 - val_mae: 105.3837

Test Loss: 386.7946472167969, Test MAE: 14.682648658752441

```
In [ ]: # Calculate RMSE
rmse = np.sqrt(test_loss)

# Print the RMSE
print(f"Test RMSE: {rmse}")
```

Test RMSE: 19.66709554603315

```
In [ ]: #My model
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.models import Model
from tensorflow.keras.callbacks import LearningRateScheduler, EarlyStopping

# Model Architecture with Improvements
def build_model(input_shape):
    # Input Layer
    input_signal = layers.Input(shape=input_shape, name='input_signal')

    # CNN Block 1 (for signal feature extraction)
    cnn_block_1 = layers.Conv1D(32, kernel_size=3, activation='relu', kernel_initializer='he_normal')(input_signal)
    cnn_block_1 = layers.BatchNormalization()(cnn_block_1)
    cnn_block_1 = layers.MaxPooling1D(pool_size=2)(cnn_block_1)
    cnn_block_1 = layers.Dropout(0.3)(cnn_block_1) # Adding dropout

    # CNN Block 2 (for additional feature extraction)
    cnn_block_2 = layers.Conv1D(64, kernel_size=5, activation='relu', kernel_initializer='he_normal')(cnn_block_1)
    cnn_block_2 = layers.BatchNormalization()(cnn_block_2)
    cnn_block_2 = layers.MaxPooling1D(pool_size=2)(cnn_block_2)
    cnn_block_2 = layers.Dropout(0.3)(cnn_block_2) # Adding dropout

    # GRU Block (for sequential pattern recognition)
    gru_block = layers.GRU(64, return_sequences=True)(cnn_block_2) # Allowing the GRU to return sequences
    gru_block = layers.GRU(64, return_sequences=False)(gru_block) # Adding another GRU layer

    # Flatten the outputs from CNN and GRU blocks
    flatten_cnn = layers.Flatten()(cnn_block_2)
    flatten_gru = layers.Flatten()(gru_block)

    # Fully connected layers
    fc1 = layers.Dense(128, activation='relu', kernel_initializer='he_normal')(flatten_cnn)
    fc1 = layers.Dropout(0.4)(fc1) # Adding dropout to fully connected layer
    fc2 = layers.Dense(128, activation='relu', kernel_initializer='he_normal')(flatten_gru)
    fc2 = layers.Dropout(0.4)(fc2) # Adding dropout to fully connected layer

    # Concatenate the outputs from CNN and GRU
    concatenated = layers.concatenate([fc1, fc2], axis=-1)

    # Final regression output
    output = layers.Dense(1, activation='linear')(concatenated)

    # Create the model
    model = Model(inputs=input_signal, outputs=output)

    # Compile the model
    model.compile(optimizer='adam', loss='mean_squared_error', metrics=['mae'])

    return model
```

```

# Learning Rate Scheduler Function (if needed)
def scheduler(epoch, lr):
    if epoch < 10:
        return lr
    else:
        return lr * 0.9 # Reduce LR by 10% every 10 epochs

# Define input shape based on the number of features and signal length
input_shape = (X_train.shape[1], 1) # Adjust based on your signal length

# Build the model
model = build_model(input_shape)

# Summary of the model
model.summary()

# Callbacks
lr_scheduler = LearningRateScheduler(scheduler)
early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

# Train the model with callbacks
model.fit(X_train, y_train, epochs=50, batch_size=32, validation_data=(X_val, y_val),
        callbacks=[lr_scheduler, early_stopping])

# Evaluate the model on the test set
test_loss, test_mae = model.evaluate(X_test, y_test)
print(f"Test Loss: {test_loss}, Test MAE: {test_mae}")

```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_signal (InputLayer)	(None, 2142, 1)	0	-
conv1d (Conv1D)	(None, 2140, 32)	128	input_signal[0][0]
batch_normalization (BatchNormalization)	(None, 2140, 32)	128	conv1d[0][0]
max_pooling1d (MaxPooling1D)	(None, 1070, 32)	0	batch_normalizati
dropout (Dropout)	(None, 1070, 32)	0	max_pooling1d[0][
conv1d_1 (Conv1D)	(None, 1066, 64)	10,304	dropout[0][0]
batch_normalization_1 (BatchNormalization)	(None, 1066, 64)	256	conv1d_1[0][0]
max_pooling1d_1 (MaxPooling1D)	(None, 533, 64)	0	batch_normalizati
dropout_1 (Dropout)	(None, 533, 64)	0	max_pooling1d_1[0
gru (GRU)	(None, 533, 64)	24,960	dropout_1[0][0]
gru_1 (GRU)	(None, 64)	24,960	gru[0][0]
flatten (Flatten)	(None, 34112)	0	dropout_1[0][0]
flatten_1 (Flatten)	(None, 64)	0	gru_1[0][0]
dense (Dense)	(None, 128)	4,366,464	flatten[0][0]
dense_1 (Dense)	(None, 128)	8,320	flatten_1[0][0]
dropout_2 (Dropout)	(None, 128)	0	dense[0][0]
dropout_3 (Dropout)	(None, 128)	0	dense_1[0][0]
concatenate (Concatenate)	(None, 256)	0	dropout_2[0][0], dropout_3[0][0]
dense_2 (Dense)	(None, 1)	257	concatenate[0][0]



Total params: 4,435,777 (16.92 MB)

Trainable params: 4,435,585 (16.92 MB)

Non-trainable params: 192 (768.00 B)

Epoch 1/50
5/5  10s 381ms/step - loss: 6795.5625 - mae: 72.0402 - val_loss: 500917.6562 - val_mae: 706.6909 - learning_rate: 0.0010

Epoch 2/50
5/5  0s 84ms/step - loss: 1210.9675 - mae: 28.6120 - val_loss: 110592.8672 - val_mae: 331.7668 - learning_rate: 0.0010

Epoch 3/50
5/5  0s 57ms/step - loss: 797.0405 - mae: 21.6893 - val_loss: 175273.0938 - val_mae: 417.9227 - learning_rate: 0.0010

Epoch 4/50
5/5  0s 68ms/step - loss: 1094.3379 - mae: 28.0080 - val_loss: 23429.2324 - val_mae: 152.0151 - learning_rate: 0.0010

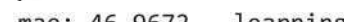
Epoch 5/50
5/5  0s 62ms/step - loss: 1130.4368 - mae: 26.9098 - val_loss: 29387.4492 - val_mae: 170.5054 - learning_rate: 0.0010

Epoch 6/50
5/5  1s 62ms/step - loss: 592.0296 - mae: 19.2395 - val_loss: 28857.5039 - val_mae: 168.9699 - learning_rate: 0.0010

Epoch 7/50
5/5  0s 72ms/step - loss: 655.3947 - mae: 20.0847 - val_loss: 7808.9199 - val_mae: 86.8109 - learning_rate: 0.0010

Epoch 8/50
5/5  0s 68ms/step - loss: 556.7805 - mae: 18.0451 - val_loss: 7446.5000 - val_mae: 84.7133 - learning_rate: 0.0010

Epoch 9/50
5/5  0s 68ms/step - loss: 656.9007 - mae: 21.1067 - val_loss: 6338.3037 - val_mae: 77.9061 - learning_rate: 0.0010

Epoch 10/50
5/5  0s 71ms/step - loss: 548.9042 - mae: 18.6389 - val_loss: 2388.4124 - val_mae: 46.9672 - learning_rate: 0.0010

Epoch 11/50
5/5  1s 58ms/step - loss: 496.6623 - mae: 17.2182 - val_loss: 2861.0786 - val_mae: 51.6792 - learning_rate: 9.0000e-04

Epoch 12/50
5/5  0s 67ms/step - loss: 711.3015 - mae: 21.1748 - val_loss: 1259.4554 - val_mae: 33.0561 - learning_rate: 8.1000e-04

Epoch 13/50
5/5  0s 69ms/step - loss: 466.5118 - mae: 16.9522 - val_loss: 934.3157 - val_mae: 27.7025 - learning_rate: 7.2900e-04

Epoch 14/50
5/5  1s 71ms/step - loss: 431.3033 - mae: 16.6462 - val_loss: 878.0089 - val_mae: 26.6772 - learning_rate: 6.5610e-04

Epoch 15/50
5/5  0s 69ms/step - loss: 556.0900 - mae: 17.6286 - val_loss: 383.4779 - val_mae: 17.0197 - learning_rate: 5.9049e-04

Epoch 16/50
5/5  0s 69ms/step - loss: 543.2729 - mae: 17.9084 - val_loss: 368.2104 - val_mae: 16.6681 - learning_rate: 5.3144e-04

Epoch 17/50
5/5  1s 64ms/step - loss: 639.7583 - mae: 19.6370 - val_loss: 578.8574 - val_mae: 21.0389 - learning_rate: 4.7830e-04

Epoch 18/50
5/5  1s 64ms/step - loss: 596.1323 - mae: 19.7965 - val_loss: 269.6535 - val_mae: 13.5063 - learning_rate: 4.3047e-04

Epoch 19/50
5/5  0s 70ms/step - loss: 462.8098 - mae: 16.6174 - val_loss: 251.4798 - val_mae: 11.9862 - learning_rate: 3.8742e-04

Epoch 20/50
5/5  0s 60ms/step - loss: 521.2361 - mae: 17.6520 - val_loss: 255.3566 - val_mae: 11.9862 - learning_rate: 3.8742e-04

```

mae: 12.5758 - learning_rate: 3.4868e-04
Epoch 21/50
5/5 ————— 0s 62ms/step - loss: 556.8442 - mae: 18.4646 - val_loss: 253.3802 - val_
mae: 11.4599 - learning_rate: 3.1381e-04
Epoch 22/50
5/5 ————— 0s 66ms/step - loss: 578.6909 - mae: 19.0837 - val_loss: 285.6198 - val_
mae: 11.0177 - learning_rate: 2.8243e-04
Epoch 23/50
5/5 ————— 1s 58ms/step - loss: 516.5750 - mae: 17.6800 - val_loss: 292.1223 - val_
mae: 11.0584 - learning_rate: 2.5419e-04
Epoch 24/50
5/5 ————— 0s 57ms/step - loss: 563.7471 - mae: 19.6699 - val_loss: 339.5040 - val_
mae: 12.0885 - learning_rate: 2.2877e-04
2/2 ————— 0s 32ms/step - loss: 386.8087 - mae: 14.3898
Test Loss: 386.7946472167969, Test MAE: 14.682648658752441

```

```

In [ ]: # Calculate RMSE
rmse = np.sqrt(test_loss)

# Print the RMSE
print(f"Test RMSE: {rmse}")

```

Test RMSE: 19.66709554603315

```

In [ ]: #My model 1
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.models import Sequential
from tensorflow.keras.callbacks import LearningRateScheduler, EarlyStopping
from sklearn.model_selection import KFold
import numpy as np
import xgboost as xgb
from sklearn.metrics import mean_squared_error
from tensorflow.keras.optimizers import Adam

# LSTM Model for feature extraction
def build_lstm_model(input_shape):
    lstm_model = Sequential()
    lstm_model.add(layers.LSTM(64, input_shape=(input_shape[1], input_shape[2]), return_sequences=False))
    lstm_model.add(layers.Dense(1)) # Output layer for regression task

    lstm_model.compile(optimizer=Adam(), loss='mean_squared_error')
    return lstm_model

# Learning Rate Scheduler Function
def scheduler(epoch, lr):
    if epoch < 10:
        return lr
    else:
        return lr * 0.9 # Reduce LR by 10% after 10 epochs

# RMSE Calculation function
def rmse(y_true, y_pred):
    return np.sqrt(np.mean(np.square(y_pred - y_true)))

# Define input shape based on the number of features and signal length
input_shape = (X_train.shape[1], 1) # Adjust based on your signal length

# K-Fold Cross-Validation
kf = KFold(n_splits=5, shuffle=True, random_state=42) # Define number of splits (K=5)

```

```

fold = 1
results = []

# Callbacks
lr_scheduler = LearningRateScheduler(scheduler)
early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

# Perform K-Fold Cross-Validation
for train_index, val_index in kf.split(X_train):
    print(f"\nTraining on fold {fold}...")

    # Split data into training and validation based on the fold indices
    X_train_fold, X_val_fold = X_train[train_index], X_train[val_index]
    y_train_fold, y_val_fold = y_train[train_index], y_train[val_index]

    # Reshape data for LSTM (samples, timesteps, features)
    X_train_fold_resaped = X_train_fold.reshape((X_train_fold.shape[0], 1, X_train_fold.shape[1]))
    X_val_fold_resaped = X_val_fold.reshape((X_val_fold.shape[0], 1, X_val_fold.shape[1]))

    # Build the LSTM model for the current fold
    lstm_model = build_lstm_model(X_train_fold_resaped.shape)

    # Train the LSTM model for feature extraction
    history = lstm_model.fit(X_train_fold_resaped, y_train_fold, epochs=50, batch_size=32,
                             validation_data=(X_val_fold_resaped, y_val_fold),
                             callbacks=[lr_scheduler, early_stopping])

    # Extract LSTM features for XGBoost
    lstm_features_train = lstm_model.predict(X_train_fold_resaped)
    lstm_features_val = lstm_model.predict(X_val_fold_resaped)

    # Combine LSTM features with original PPG data for XGBoost
    X_train_combined = np.hstack((X_train_fold, lstm_features_train))
    X_val_combined = np.hstack((X_val_fold, lstm_features_val))

    # Train XGBoost on the combined features
    dtrain = xgb.DMatrix(X_train_combined, label=y_train_fold)
    dval = xgb.DMatrix(X_val_combined, label=y_val_fold)

    params = {
        'objective': 'reg:squarederror',
        'max_depth': 6,
        'learning_rate': 0.1,
        'eval_metric': 'rmse'
    }

    xgb_model = xgb.train(params, dtrain, num_boost_round=500, early_stopping_rounds=50, evals=[

    # Predict using XGBoost model
    y_val_pred = xgb_model.predict(dval)
    fold_rmse = rmse(y_val_fold, y_val_pred)
    print(f"Fold {fold} - RMSE: {fold_rmse}")

    # Save the result for this fold (Loss, RMSE)
    results.append(fold_rmse)

    fold += 1

# After K-Fold, calculate the average, min, and max results

```

```

rmse_values = results

print("\nK-Fold Cross-Validation Results:")
print(f"Average RMSE: {np.mean(rmse_values)}")
print(f"Min RMSE: {np.min(rmse_values)}")
print(f"Max RMSE: {np.max(rmse_values)}")

# Optionally, retrain the model on the entire dataset after K-Fold (not necessary for validation,
# Reshape for LSTM
X_train_resaped = X_train.reshape((X_train.shape[0], 1, X_train.shape[1]))
X_test_resaped = X_test.reshape((X_test.shape[0], 1, X_test.shape[1]))

# Build and train the final LSTM model
final_lstm_model = build_lstm_model(X_train_resaped.shape)
final_lstm_model.fit(X_train_resaped, y_train, epochs=50, batch_size=32, validation_data=(X_test, y_test),
                    callbacks=[lr_scheduler, early_stopping])

# Extract features from the final LSTM model for XGBoost
lstm_features_train_final = final_lstm_model.predict(X_train_resaped)
lstm_features_test_final = final_lstm_model.predict(X_test_resaped)

# Combine LSTM features with original PPG data for XGBoost
X_train_combined_final = np.hstack((X_train, lstm_features_train_final))
X_test_combined_final = np.hstack((X_test, lstm_features_test_final))

# Train XGBoost on the final combined features
dtrain_final = xgb.DMatrix(X_train_combined_final, label=y_train)
dtest_final = xgb.DMatrix(X_test_combined_final, label=y_test)

xgb_model_final = xgb.train(params, dtrain_final, num_boost_round=500, early_stopping_rounds=50,
                             callbacks=[early_stopping])

# Predict and calculate RMSE for the final model
y_test_pred = xgb_model_final.predict(dtest_final)
test_rmse = rmse(y_test, y_test_pred)

print(f"\nFinal Test RMSE: {test_rmse}")

```

Training on fold 1...

Epoch 1/50

```

/usr/local/lib/python3.11/dist-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass
an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `
Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)

```


4/4  1s 80ms/step - loss: 13339.0996 - val_loss: 14321.1738 - learning_rate: 0.0010
Epoch 2/50

4/4  0s 22ms/step - loss: 13309.7012 - val_loss: 14308.8223 - learning_rate: 0.0010
Epoch 3/50

4/4  0s 21ms/step - loss: 13347.5986 - val_loss: 14296.3730 - learning_rate: 0.0010
Epoch 4/50

4/4  0s 20ms/step - loss: 12934.8594 - val_loss: 14283.9102 - learning_rate: 0.0010
Epoch 5/50

4/4  0s 19ms/step - loss: 13239.5166 - val_loss: 14271.4199 - learning_rate: 0.0010
Epoch 6/50

4/4  0s 22ms/step - loss: 12426.1709 - val_loss: 14258.9658 - learning_rate: 0.0010
Epoch 7/50

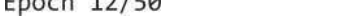
4/4  0s 21ms/step - loss: 13218.6865 - val_loss: 14246.4668 - learning_rate: 0.0010
Epoch 8/50

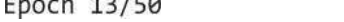
4/4  0s 21ms/step - loss: 12734.2637 - val_loss: 14234.0078 - learning_rate: 0.0010
Epoch 9/50

4/4  0s 21ms/step - loss: 13059.3154 - val_loss: 14221.5332 - learning_rate: 0.0010
Epoch 10/50

4/4  0s 21ms/step - loss: 12961.4180 - val_loss: 14209.0781 - learning_rate: 0.0010
Epoch 11/50

4/4  0s 21ms/step - loss: 12980.7041 - val_loss: 14197.8760 - learning_rate: 9.0000e-04
Epoch 12/50

4/4  0s 22ms/step - loss: 12785.9912 - val_loss: 14187.8125 - learning_rate: 8.1000e-04
Epoch 13/50

4/4  0s 25ms/step - loss: 13417.9980 - val_loss: 14178.7344 - learning_rate: 7.2900e-04
Epoch 14/50

4/4  0s 22ms/step - loss: 12941.9023 - val_loss: 14170.5928 - learning_rate: 6.5610e-04
Epoch 15/50

4/4  0s 20ms/step - loss: 13151.3799 - val_loss: 14163.2617 - learning_rate: 5.9049e-04
Epoch 16/50

4/4  0s 20ms/step - loss: 13018.6250 - val_loss: 14156.6719 - learning_rate: 5.3144e-04
Epoch 17/50

4/4  0s 19ms/step - loss: 12746.0615 - val_loss: 14150.7539 - learning_rate: 4.7830e-04
Epoch 18/50

4/4  0s 21ms/step - loss: 13367.3721 - val_loss: 14145.4102 - learning_rate: 4.3047e-04
Epoch 19/50

4/4  0s 21ms/step - loss: 12923.2012 - val_loss: 14140.6182 - learning_rate: 3.8742e-04
Epoch 20/50

4/4  0s 19ms/step - loss: 13274.5137 - val_loss: 14136.2969 - learning_rate: 3.4868e-04

Epoch 21/50
4/4  0s 22ms/step - loss: 13046.2461 - val_loss: 14132.4141 - learning_rate: 3.1381e-04

Epoch 22/50
4/4  0s 24ms/step - loss: 13034.9678 - val_loss: 14128.9229 - learning_rate: 2.8243e-04

Epoch 23/50
4/4  0s 22ms/step - loss: 12926.9834 - val_loss: 14125.7803 - learning_rate: 2.5419e-04

Epoch 24/50
4/4  0s 23ms/step - loss: 12907.1846 - val_loss: 14122.9512 - learning_rate: 2.2877e-04

Epoch 25/50
4/4  0s 22ms/step - loss: 13462.4473 - val_loss: 14120.4004 - learning_rate: 2.0589e-04

Epoch 26/50
4/4  0s 20ms/step - loss: 12786.4141 - val_loss: 14118.1152 - learning_rate: 1.8530e-04

Epoch 27/50
4/4  0s 19ms/step - loss: 12634.8457 - val_loss: 14116.0576 - learning_rate: 1.6677e-04

Epoch 28/50
4/4  0s 24ms/step - loss: 13092.5127 - val_loss: 14114.2012 - learning_rate: 1.5009e-04

Epoch 29/50
4/4  0s 20ms/step - loss: 12876.1172 - val_loss: 14112.5322 - learning_rate: 1.3509e-04

Epoch 30/50
4/4  0s 23ms/step - loss: 13054.5918 - val_loss: 14111.0283 - learning_rate: 1.2158e-04

Epoch 31/50
4/4  0s 23ms/step - loss: 13088.8623 - val_loss: 14109.6777 - learning_rate: 1.0942e-04

Epoch 32/50
4/4  0s 22ms/step - loss: 13154.6895 - val_loss: 14108.4590 - learning_rate: 9.8477e-05

Epoch 33/50
4/4  0s 35ms/step - loss: 13072.0996 - val_loss: 14107.3643 - learning_rate: 8.8629e-05

Epoch 34/50
4/4  0s 22ms/step - loss: 13104.2920 - val_loss: 14106.3770 - learning_rate: 7.9766e-05

Epoch 35/50
4/4  0s 19ms/step - loss: 12922.0742 - val_loss: 14105.4922 - learning_rate: 7.1790e-05

Epoch 36/50
4/4  0s 21ms/step - loss: 12766.4668 - val_loss: 14104.6953 - learning_rate: 6.4611e-05

Epoch 37/50
4/4  0s 20ms/step - loss: 13378.4551 - val_loss: 14103.9746 - learning_rate: 5.8150e-05

Epoch 38/50
4/4  0s 22ms/step - loss: 13165.0166 - val_loss: 14103.3271 - learning_rate: 5.2335e-05

Epoch 39/50
4/4  0s 22ms/step - loss: 12905.7363 - val_loss: 14102.7480 - learning_rate: 4.7101e-05

Epoch 40/50
4/4  0s 19ms/step - loss: 12982.2744 - val_loss: 14102.2227 - learning_rate:

```
4.2391e-05
Epoch 41/50
4/4  0s 19ms/step - loss: 13343.3135 - val_loss: 14101.7520 - learning_rate:
3.8152e-05
Epoch 42/50
4/4  0s 21ms/step - loss: 13098.3740 - val_loss: 14101.3281 - learning_rate:
3.4337e-05
Epoch 43/50
4/4  0s 23ms/step - loss: 13332.2129 - val_loss: 14100.9453 - learning_rate:
3.0903e-05
Epoch 44/50
4/4  0s 19ms/step - loss: 12993.0059 - val_loss: 14100.6016 - learning_rate:
2.7813e-05
Epoch 45/50
4/4  0s 22ms/step - loss: 13141.6689 - val_loss: 14100.2920 - learning_rate:
2.5032e-05
Epoch 46/50
4/4  0s 19ms/step - loss: 12875.1787 - val_loss: 14100.0146 - learning_rate:
2.2528e-05
Epoch 47/50
4/4  0s 22ms/step - loss: 12624.4229 - val_loss: 14099.7656 - learning_rate:
2.0276e-05
Epoch 48/50
4/4  0s 21ms/step - loss: 12707.4502 - val_loss: 14099.5410 - learning_rate:
1.8248e-05
Epoch 49/50
4/4  0s 20ms/step - loss: 12881.6270 - val_loss: 14099.3359 - learning_rate:
1.6423e-05
Epoch 50/50
4/4  0s 22ms/step - loss: 13164.4551 - val_loss: 14099.1543 - learning_rate:
1.4781e-05
4/4  0s 9ms/step
1/1  0s 47ms/step
[0] eval-rmse:18.44808
[1] eval-rmse:17.91235
[2] eval-rmse:17.07084
[3] eval-rmse:16.39592
[4] eval-rmse:15.84243
[5] eval-rmse:15.29586
[6] eval-rmse:14.79814
[7] eval-rmse:14.37955
[8] eval-rmse:14.01766
[9] eval-rmse:13.47722
[10] eval-rmse:13.27352
[11] eval-rmse:13.01853
[12] eval-rmse:12.80995
[13] eval-rmse:12.60174
[14] eval-rmse:12.44518
[15] eval-rmse:12.32378
[16] eval-rmse:12.23417
[17] eval-rmse:12.16060
[18] eval-rmse:12.11531
[19] eval-rmse:12.06693
[20] eval-rmse:12.01014
[21] eval-rmse:11.96000
[22] eval-rmse:11.93099
[23] eval-rmse:11.89244
[24] eval-rmse:11.85259
[25] eval-rmse:11.83520
```

[26] eval-rmse:11.82069
[27] eval-rmse:11.78432
[28] eval-rmse:11.73367
[29] eval-rmse:11.68813
[30] eval-rmse:11.64868
[31] eval-rmse:11.62919
[32] eval-rmse:11.56647
[33] eval-rmse:11.53489
[34] eval-rmse:11.51443
[35] eval-rmse:11.50031
[36] eval-rmse:11.45949
[37] eval-rmse:11.43637
[38] eval-rmse:11.41062
[39] eval-rmse:11.39647
[40] eval-rmse:11.38460
[41] eval-rmse:11.36462
[42] eval-rmse:11.35773
[43] eval-rmse:11.34883
[44] eval-rmse:11.33324
[45] eval-rmse:11.32182
[46] eval-rmse:11.30214
[47] eval-rmse:11.29839
[48] eval-rmse:11.29037
[49] eval-rmse:11.28069
[50] eval-rmse:11.28416
[51] eval-rmse:11.27195
[52] eval-rmse:11.26908
[53] eval-rmse:11.26170
[54] eval-rmse:11.25626
[55] eval-rmse:11.25088
[56] eval-rmse:11.24764
[57] eval-rmse:11.25019
[58] eval-rmse:11.24559
[59] eval-rmse:11.24785
[60] eval-rmse:11.24209
[61] eval-rmse:11.23997
[62] eval-rmse:11.23640
[63] eval-rmse:11.23210
[64] eval-rmse:11.22754
[65] eval-rmse:11.22665
[66] eval-rmse:11.22289
[67] eval-rmse:11.22488
[68] eval-rmse:11.22205
[69] eval-rmse:11.21975
[70] eval-rmse:11.21823
[71] eval-rmse:11.21501
[72] eval-rmse:11.21335
[73] eval-rmse:11.21102
[74] eval-rmse:11.20847
[75] eval-rmse:11.20606
[76] eval-rmse:11.20453
[77] eval-rmse:11.20465
[78] eval-rmse:11.20386
[79] eval-rmse:11.20386
[80] eval-rmse:11.20193
[81] eval-rmse:11.20137
[82] eval-rmse:11.19993
[83] eval-rmse:11.20025
[84] eval-rmse:11.19950

[85] eval-rmse:11.19829
[86] eval-rmse:11.19654
[87] eval-rmse:11.19647
[88] eval-rmse:11.19652
[89] eval-rmse:11.19585
[90] eval-rmse:11.19526
[91] eval-rmse:11.19481
[92] eval-rmse:11.19416
[93] eval-rmse:11.19373
[94] eval-rmse:11.19299
[95] eval-rmse:11.19249
[96] eval-rmse:11.19213
[97] eval-rmse:11.19166
[98] eval-rmse:11.19163
[99] eval-rmse:11.19119
[100] eval-rmse:11.19129
[101] eval-rmse:11.19096
[102] eval-rmse:11.19047
[103] eval-rmse:11.19076
[104] eval-rmse:11.19023
[105] eval-rmse:11.19075
[106] eval-rmse:11.19004
[107] eval-rmse:11.19025
[108] eval-rmse:11.19031
[109] eval-rmse:11.19041
[110] eval-rmse:11.19056
[111] eval-rmse:11.19089
[112] eval-rmse:11.19107
[113] eval-rmse:11.19119
[114] eval-rmse:11.19146
[115] eval-rmse:11.19166
[116] eval-rmse:11.19177
[117] eval-rmse:11.19209
[118] eval-rmse:11.19182
[119] eval-rmse:11.19144
[120] eval-rmse:11.19172
[121] eval-rmse:11.19162
[122] eval-rmse:11.19169
[123] eval-rmse:11.19169
[124] eval-rmse:11.19157
[125] eval-rmse:11.19129
[126] eval-rmse:11.19139
[127] eval-rmse:11.19151
[128] eval-rmse:11.19151
[129] eval-rmse:11.19154
[130] eval-rmse:11.19153
[131] eval-rmse:11.19160
[132] eval-rmse:11.19164
[133] eval-rmse:11.19176
[134] eval-rmse:11.19177
[135] eval-rmse:11.19183
[136] eval-rmse:11.19175
[137] eval-rmse:11.19174
[138] eval-rmse:11.19173
[139] eval-rmse:11.19173
[140] eval-rmse:11.19167
[141] eval-rmse:11.19167
[142] eval-rmse:11.19164
[143] eval-rmse:11.19167

```
[144] eval-rmse:11.19169
[145] eval-rmse:11.19168
[146] eval-rmse:11.19167
[147] eval-rmse:11.19167
[148] eval-rmse:11.19169
[149] eval-rmse:11.19168
[150] eval-rmse:11.19165
[151] eval-rmse:11.19165
[152] eval-rmse:11.19166
[153] eval-rmse:11.19164
[154] eval-rmse:11.19165
[155] eval-rmse:11.19165
[156] eval-rmse:11.19170
Fold 1 - RMSE: 11.191699008717476
```

Training on fold 2...

Epoch 1/50

```
4/4  1s 84ms/step - loss: 13487.8174 - val_loss: 13328.2607 - learning_rate: 0.0010
```

Epoch 2/50

```
4/4  0s 20ms/step - loss: 13054.1338 - val_loss: 13311.2734 - learning_rate: 0.0010
```

Epoch 3/50

```
4/4  0s 21ms/step - loss: 13039.1328 - val_loss: 13293.8867 - learning_rate: 0.0010
```

Epoch 4/50

```
4/4  0s 22ms/step - loss: 13616.7246 - val_loss: 13276.3418 - learning_rate: 0.0010
```

Epoch 5/50

```
4/4  0s 21ms/step - loss: 12811.1924 - val_loss: 13258.7910 - learning_rate: 0.0010
```

Epoch 6/50

```
4/4  0s 21ms/step - loss: 13041.9658 - val_loss: 13241.1973 - learning_rate: 0.0010
```

Epoch 7/50

```
4/4  0s 21ms/step - loss: 12977.1729 - val_loss: 13223.6162 - learning_rate: 0.0010
```

Epoch 8/50

```
4/4  0s 22ms/step - loss: 13367.2969 - val_loss: 13206.0156 - learning_rate: 0.0010
```

Epoch 9/50

```
4/4  0s 23ms/step - loss: 12835.4805 - val_loss: 13188.4727 - learning_rate: 0.0010
```

Epoch 10/50

```
4/4  0s 21ms/step - loss: 13335.4668 - val_loss: 13170.9082 - learning_rate: 0.0010
```

Epoch 11/50

```
4/4  0s 21ms/step - loss: 13189.2354 - val_loss: 13155.1338 - learning_rate: 9.0000e-04
```

Epoch 12/50

```
4/4  0s 19ms/step - loss: 13027.0986 - val_loss: 13140.9600 - learning_rate: 8.1000e-04
```

Epoch 13/50

```
4/4  0s 20ms/step - loss: 13447.3135 - val_loss: 13128.1885 - learning_rate: 7.2900e-04
```

Epoch 14/50

```
4/4  0s 19ms/step - loss: 13303.1582 - val_loss: 13116.7305 - learning_rate: 6.5610e-04
```

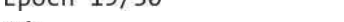
Epoch 15/50

4/4  0s 22ms/step - loss: 13291.0010 - val_loss: 13106.4258 - learning_rate: 5.9049e-04
Epoch 16/50

4/4  0s 22ms/step - loss: 12966.1836 - val_loss: 13097.1719 - learning_rate: 5.3144e-04
Epoch 17/50

4/4  0s 23ms/step - loss: 12685.4912 - val_loss: 13088.8535 - learning_rate: 4.7830e-04
Epoch 18/50

4/4  0s 24ms/step - loss: 13626.8623 - val_loss: 13081.3350 - learning_rate: 4.3047e-04
Epoch 19/50

4/4  0s 20ms/step - loss: 13420.0137 - val_loss: 13074.5898 - learning_rate: 3.8742e-04
Epoch 20/50

4/4  0s 29ms/step - loss: 13182.0674 - val_loss: 13068.5293 - learning_rate: 3.4868e-04
Epoch 21/50

4/4  0s 43ms/step - loss: 13008.8154 - val_loss: 13063.0771 - learning_rate: 3.1381e-04
Epoch 22/50

4/4  0s 40ms/step - loss: 13038.0596 - val_loss: 13058.1738 - learning_rate: 2.8243e-04
Epoch 23/50

4/4  0s 45ms/step - loss: 13381.7324 - val_loss: 13053.7529 - learning_rate: 2.5419e-04
Epoch 24/50

4/4  0s 40ms/step - loss: 13193.0205 - val_loss: 13049.7793 - learning_rate: 2.2877e-04
Epoch 25/50

4/4  0s 41ms/step - loss: 12898.2227 - val_loss: 13046.2090 - learning_rate: 2.0589e-04
Epoch 26/50

4/4  0s 42ms/step - loss: 13165.7529 - val_loss: 13042.9902 - learning_rate: 1.8530e-04
Epoch 27/50

4/4  0s 42ms/step - loss: 12832.8574 - val_loss: 13040.1035 - learning_rate: 1.6677e-04
Epoch 28/50

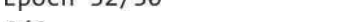
4/4  0s 41ms/step - loss: 12737.7920 - val_loss: 13037.5000 - learning_rate: 1.5009e-04
Epoch 29/50

4/4  0s 31ms/step - loss: 12957.6934 - val_loss: 13035.1562 - learning_rate: 1.3509e-04
Epoch 30/50

4/4  0s 43ms/step - loss: 13267.2715 - val_loss: 13033.0439 - learning_rate: 1.2158e-04
Epoch 31/50

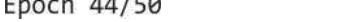
4/4  0s 43ms/step - loss: 13373.6973 - val_loss: 13031.1436 - learning_rate: 1.0942e-04
Epoch 32/50

4/4  0s 42ms/step - loss: 13065.6592 - val_loss: 13029.4355 - learning_rate: 9.8477e-05
Epoch 33/50

4/4  0s 22ms/step - loss: 13202.0176 - val_loss: 13027.8984 - learning_rate: 8.8629e-05
Epoch 34/50

4/4  0s 25ms/step - loss: 12957.3115 - val_loss: 13026.5176 - learning_rate: 7.9766e-05


```

Epoch 35/50
4/4  0s 20ms/step - loss: 12929.3633 - val_loss: 13025.2715 - learning_rate:
7.1790e-05
Epoch 36/50
4/4  0s 22ms/step - loss: 12802.8066 - val_loss: 13024.1533 - learning_rate:
6.4611e-05
Epoch 37/50
4/4  0s 22ms/step - loss: 13127.3896 - val_loss: 13023.1445 - learning_rate:
5.8150e-05
Epoch 38/50
4/4  0s 21ms/step - loss: 12927.1416 - val_loss: 13022.2363 - learning_rate:
5.2335e-05
Epoch 39/50
4/4  0s 22ms/step - loss: 12815.5820 - val_loss: 13021.4209 - learning_rate:
4.7101e-05
Epoch 40/50
4/4  0s 20ms/step - loss: 12638.2930 - val_loss: 13020.6875 - learning_rate:
4.2391e-05
Epoch 41/50
4/4  0s 21ms/step - loss: 12802.4541 - val_loss: 13020.0264 - learning_rate:
3.8152e-05
Epoch 42/50
4/4  0s 22ms/step - loss: 13255.4873 - val_loss: 13019.4287 - learning_rate:
3.4337e-05
Epoch 43/50
4/4  0s 24ms/step - loss: 13048.9834 - val_loss: 13018.8926 - learning_rate:
3.0903e-05
Epoch 44/50
4/4  0s 19ms/step - loss: 12874.2979 - val_loss: 13018.4111 - learning_rate:
2.7813e-05
Epoch 45/50
4/4  0s 21ms/step - loss: 12899.5137 - val_loss: 13017.9785 - learning_rate:
2.5032e-05
Epoch 46/50
4/4  0s 22ms/step - loss: 13190.8369 - val_loss: 13017.5859 - learning_rate:
2.2528e-05
Epoch 47/50
4/4  0s 22ms/step - loss: 13044.4961 - val_loss: 13017.2363 - learning_rate:
2.0276e-05
Epoch 48/50
4/4  0s 20ms/step - loss: 12974.7793 - val_loss: 13016.9189 - learning_rate:
1.8248e-05
Epoch 49/50
4/4  0s 22ms/step - loss: 12858.3564 - val_loss: 13016.6348 - learning_rate:
1.6423e-05
Epoch 50/50
4/4  0s 22ms/step - loss: 12802.3115 - val_loss: 13016.3779 - learning_rate:
1.4781e-05
4/4  0s 6ms/step
1/1  0s 28ms/step
[0] eval-rmse:17.07303
[1] eval-rmse:16.31903
[2] eval-rmse:15.53437
[3] eval-rmse:14.88360
[4] eval-rmse:14.33468
[5] eval-rmse:13.93796
[6] eval-rmse:13.45746
[7] eval-rmse:13.09836
[8] eval-rmse:12.64825

```

[9] eval-rmse:12.43691
[10] eval-rmse:12.20569
[11] eval-rmse:12.04431
[12] eval-rmse:11.74123
[13] eval-rmse:11.53626
[14] eval-rmse:11.40687
[15] eval-rmse:11.20267
[16] eval-rmse:10.97195
[17] eval-rmse:10.77708
[18] eval-rmse:10.67673
[19] eval-rmse:10.53874
[20] eval-rmse:10.41826
[21] eval-rmse:10.30476
[22] eval-rmse:10.20712
[23] eval-rmse:10.07502
[24] eval-rmse:9.94749
[25] eval-rmse:9.82274
[26] eval-rmse:9.71684
[27] eval-rmse:9.65801
[28] eval-rmse:9.60272
[29] eval-rmse:9.52782
[30] eval-rmse:9.47467
[31] eval-rmse:9.44216
[32] eval-rmse:9.41218
[33] eval-rmse:9.38737
[34] eval-rmse:9.31735
[35] eval-rmse:9.26104
[36] eval-rmse:9.20130
[37] eval-rmse:9.14031
[38] eval-rmse:9.10007
[39] eval-rmse:9.05152
[40] eval-rmse:9.03711
[41] eval-rmse:9.00942
[42] eval-rmse:8.98282
[43] eval-rmse:8.95893
[44] eval-rmse:8.93702
[45] eval-rmse:8.90445
[46] eval-rmse:8.87489
[47] eval-rmse:8.84682
[48] eval-rmse:8.80615
[49] eval-rmse:8.78176
[50] eval-rmse:8.76396
[51] eval-rmse:8.74421
[52] eval-rmse:8.73502
[53] eval-rmse:8.71445
[54] eval-rmse:8.69593
[55] eval-rmse:8.67644
[56] eval-rmse:8.66277
[57] eval-rmse:8.64706
[58] eval-rmse:8.63588
[59] eval-rmse:8.62004
[60] eval-rmse:8.60956
[61] eval-rmse:8.59632
[62] eval-rmse:8.58388
[63] eval-rmse:8.57310
[64] eval-rmse:8.56288
[65] eval-rmse:8.55203
[66] eval-rmse:8.54277
[67] eval-rmse:8.53727

[68] eval-rmse:8.52870
[69] eval-rmse:8.51829
[70] eval-rmse:8.51055
[71] eval-rmse:8.50475
[72] eval-rmse:8.49744
[73] eval-rmse:8.49306
[74] eval-rmse:8.48874
[75] eval-rmse:8.48362
[76] eval-rmse:8.47970
[77] eval-rmse:8.47450
[78] eval-rmse:8.47080
[79] eval-rmse:8.46496
[80] eval-rmse:8.46057
[81] eval-rmse:8.45674
[82] eval-rmse:8.45259
[83] eval-rmse:8.44928
[84] eval-rmse:8.44665
[85] eval-rmse:8.44408
[86] eval-rmse:8.44086
[87] eval-rmse:8.43858
[88] eval-rmse:8.43585
[89] eval-rmse:8.43362
[90] eval-rmse:8.43100
[91] eval-rmse:8.42891
[92] eval-rmse:8.42709
[93] eval-rmse:8.42546
[94] eval-rmse:8.42342
[95] eval-rmse:8.42158
[96] eval-rmse:8.42051
[97] eval-rmse:8.41902
[98] eval-rmse:8.41773
[99] eval-rmse:8.41631
[100] eval-rmse:8.41545
[101] eval-rmse:8.41441
[102] eval-rmse:8.41281
[103] eval-rmse:8.41201
[104] eval-rmse:8.41176
[105] eval-rmse:8.41091
[106] eval-rmse:8.40964
[107] eval-rmse:8.40878
[108] eval-rmse:8.40785
[109] eval-rmse:8.40739
[110] eval-rmse:8.40647
[111] eval-rmse:8.40568
[112] eval-rmse:8.40471
[113] eval-rmse:8.40409
[114] eval-rmse:8.40319
[115] eval-rmse:8.40261
[116] eval-rmse:8.40186
[117] eval-rmse:8.40114
[118] eval-rmse:8.40046
[119] eval-rmse:8.40004
[120] eval-rmse:8.39935
[121] eval-rmse:8.39898
[122] eval-rmse:8.39812
[123] eval-rmse:8.39787
[124] eval-rmse:8.39752
[125] eval-rmse:8.39721
[126] eval-rmse:8.39693

[127] eval-rmse:8.39667
[128] eval-rmse:8.39643
[129] eval-rmse:8.39605
[130] eval-rmse:8.39578
[131] eval-rmse:8.39553
[132] eval-rmse:8.39529
[133] eval-rmse:8.39508
[134] eval-rmse:8.39484
[135] eval-rmse:8.39445
[136] eval-rmse:8.39406
[137] eval-rmse:8.39389
[138] eval-rmse:8.39352
[139] eval-rmse:8.39336
[140] eval-rmse:8.39303
[141] eval-rmse:8.39271
[142] eval-rmse:8.39253
[143] eval-rmse:8.39224
[144] eval-rmse:8.39212
[145] eval-rmse:8.39204
[146] eval-rmse:8.39193
[147] eval-rmse:8.39180
[148] eval-rmse:8.39170
[149] eval-rmse:8.39159
[150] eval-rmse:8.39148
[151] eval-rmse:8.39139
[152] eval-rmse:8.39131
[153] eval-rmse:8.39123
[154] eval-rmse:8.39116
[155] eval-rmse:8.39109
[156] eval-rmse:8.39104
[157] eval-rmse:8.39100
[158] eval-rmse:8.39093
[159] eval-rmse:8.39087
[160] eval-rmse:8.39082
[161] eval-rmse:8.39076
[162] eval-rmse:8.39069
[163] eval-rmse:8.39065
[164] eval-rmse:8.39062
[165] eval-rmse:8.39058
[166] eval-rmse:8.39054
[167] eval-rmse:8.39050
[168] eval-rmse:8.39045
[169] eval-rmse:8.39041
[170] eval-rmse:8.39037
[171] eval-rmse:8.39034
[172] eval-rmse:8.39031
[173] eval-rmse:8.39028
[174] eval-rmse:8.39025
[175] eval-rmse:8.39022
[176] eval-rmse:8.39020
[177] eval-rmse:8.39017
[178] eval-rmse:8.39015
[179] eval-rmse:8.39013
[180] eval-rmse:8.39011
[181] eval-rmse:8.39009
[182] eval-rmse:8.39007
[183] eval-rmse:8.39005
[184] eval-rmse:8.39004
[185] eval-rmse:8.39002

[186] eval-rmse:8.39001
[187] eval-rmse:8.38999
[188] eval-rmse:8.38998
[189] eval-rmse:8.38997
[190] eval-rmse:8.38995
[191] eval-rmse:8.38994
[192] eval-rmse:8.38993
[193] eval-rmse:8.38992
[194] eval-rmse:8.38991
[195] eval-rmse:8.38990
[196] eval-rmse:8.38989
[197] eval-rmse:8.38989
[198] eval-rmse:8.38988
[199] eval-rmse:8.38987
[200] eval-rmse:8.38986
[201] eval-rmse:8.38985
[202] eval-rmse:8.38983
[203] eval-rmse:8.38982
[204] eval-rmse:8.38981
[205] eval-rmse:8.38981
[206] eval-rmse:8.38981
[207] eval-rmse:8.38981
[208] eval-rmse:8.38981
[209] eval-rmse:8.38981
[210] eval-rmse:8.38981
[211] eval-rmse:8.38981
[212] eval-rmse:8.38981
[213] eval-rmse:8.38981
[214] eval-rmse:8.38981
[215] eval-rmse:8.38981
[216] eval-rmse:8.38981
[217] eval-rmse:8.38981
[218] eval-rmse:8.38981
[219] eval-rmse:8.38981
[220] eval-rmse:8.38981
[221] eval-rmse:8.38981
[222] eval-rmse:8.38981
[223] eval-rmse:8.38981
[224] eval-rmse:8.38981
[225] eval-rmse:8.38981
[226] eval-rmse:8.38981
[227] eval-rmse:8.38981
[228] eval-rmse:8.38981
[229] eval-rmse:8.38981
[230] eval-rmse:8.38981
[231] eval-rmse:8.38981
[232] eval-rmse:8.38981
[233] eval-rmse:8.38981
[234] eval-rmse:8.38981
[235] eval-rmse:8.38981
[236] eval-rmse:8.38981
[237] eval-rmse:8.38981
[238] eval-rmse:8.38981
[239] eval-rmse:8.38981
[240] eval-rmse:8.38981
[241] eval-rmse:8.38981
[242] eval-rmse:8.38981
[243] eval-rmse:8.38981
[244] eval-rmse:8.38981

```
[245] eval-rmse:8.38981
[246] eval-rmse:8.38981
[247] eval-rmse:8.38981
[248] eval-rmse:8.38981
[249] eval-rmse:8.38981
[250] eval-rmse:8.38981
[251] eval-rmse:8.38981
[252] eval-rmse:8.38981
[253] eval-rmse:8.38981
[254] eval-rmse:8.38981
Fold 2 - RMSE: 8.389808974719402
```

Training on fold 3...

Epoch 1/50

```
4/4  1s 83ms/step - loss: 13649.5020 - val_loss: 13530.8750 - learning_rate: 0.0010
```

Epoch 2/50

```
4/4  0s 21ms/step - loss: 13646.8271 - val_loss: 13524.2979 - learning_rate: 0.0010
```

Epoch 3/50

```
4/4  0s 21ms/step - loss: 12946.3730 - val_loss: 13517.2715 - learning_rate: 0.0010
```

Epoch 4/50

```
4/4  0s 20ms/step - loss: 13311.7881 - val_loss: 13510.1309 - learning_rate: 0.0010
```

Epoch 5/50

```
4/4  0s 21ms/step - loss: 13860.2490 - val_loss: 13502.9004 - learning_rate: 0.0010
```

Epoch 6/50

```
4/4  0s 25ms/step - loss: 13424.3721 - val_loss: 13495.6465 - learning_rate: 0.0010
```

Epoch 7/50

```
4/4  0s 20ms/step - loss: 13672.1699 - val_loss: 13488.3721 - learning_rate: 0.0010
```

Epoch 8/50

```
4/4  0s 20ms/step - loss: 13049.0605 - val_loss: 13481.1104 - learning_rate: 0.0010
```

Epoch 9/50

```
4/4  0s 24ms/step - loss: 13537.5039 - val_loss: 13473.8320 - learning_rate: 0.0010
```

Epoch 10/50

```
4/4  0s 22ms/step - loss: 13748.6680 - val_loss: 13466.5557 - learning_rate: 0.0010
```

Epoch 11/50

```
4/4  0s 20ms/step - loss: 13562.2686 - val_loss: 13460.0166 - learning_rate: 9.0000e-04
```

Epoch 12/50

```
4/4  0s 23ms/step - loss: 13015.2539 - val_loss: 13454.1582 - learning_rate: 8.1000e-04
```

Epoch 13/50

```
4/4  0s 20ms/step - loss: 13566.2773 - val_loss: 13448.8691 - learning_rate: 7.2900e-04
```

Epoch 14/50

```
4/4  0s 22ms/step - loss: 13249.5742 - val_loss: 13444.1211 - learning_rate: 6.5610e-04
```

Epoch 15/50

```
4/4  0s 20ms/step - loss: 13476.6357 - val_loss: 13439.8447 - learning_rate: 5.9049e-04
```

Epoch 16/50

4/4  0s 19ms/step - loss: 13245.4414 - val_loss: 13436.0059 - learning_rate: 5.3144e-04
Epoch 17/50

4/4  0s 22ms/step - loss: 13620.3369 - val_loss: 13432.5469 - learning_rate: 4.7830e-04
Epoch 18/50

4/4  0s 23ms/step - loss: 13556.3164 - val_loss: 13429.4355 - learning_rate: 4.3047e-04
Epoch 19/50

4/4  0s 22ms/step - loss: 13579.8369 - val_loss: 13426.6387 - learning_rate: 3.8742e-04
Epoch 20/50

4/4  0s 22ms/step - loss: 13891.6377 - val_loss: 13394.6445 - learning_rate: 3.4868e-04
Epoch 21/50

4/4  0s 22ms/step - loss: 13289.8857 - val_loss: 13392.0781 - learning_rate: 3.1381e-04
Epoch 22/50

4/4  0s 20ms/step - loss: 13325.8730 - val_loss: 13389.6895 - learning_rate: 2.8243e-04
Epoch 23/50

4/4  0s 22ms/step - loss: 13684.4072 - val_loss: 13387.5156 - learning_rate: 2.5419e-04
Epoch 24/50

4/4  0s 36ms/step - loss: 13299.4434 - val_loss: 13385.5605 - learning_rate: 2.2877e-04
Epoch 25/50

4/4  0s 22ms/step - loss: 13067.4414 - val_loss: 13383.8066 - learning_rate: 2.0589e-04
Epoch 26/50

4/4  0s 22ms/step - loss: 13234.4385 - val_loss: 13382.2324 - learning_rate: 1.8530e-04
Epoch 27/50

4/4  0s 84ms/step - loss: 13783.5000 - val_loss: 13380.8184 - learning_rate: 1.6677e-04
Epoch 28/50

4/4  0s 83ms/step - loss: 13577.7314 - val_loss: 13379.5537 - learning_rate: 1.5009e-04
Epoch 29/50

4/4  0s 21ms/step - loss: 13299.1748 - val_loss: 13378.4238 - learning_rate: 1.3509e-04
Epoch 30/50

4/4  0s 35ms/step - loss: 13225.6426 - val_loss: 13377.4082 - learning_rate: 1.2158e-04
Epoch 31/50

4/4  0s 22ms/step - loss: 13706.1719 - val_loss: 13376.4980 - learning_rate: 1.0942e-04
Epoch 32/50

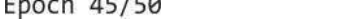
4/4  0s 64ms/step - loss: 13500.3604 - val_loss: 13375.6816 - learning_rate: 9.8477e-05
Epoch 33/50

4/4  0s 54ms/step - loss: 13047.1084 - val_loss: 13374.9512 - learning_rate: 8.8629e-05
Epoch 34/50

4/4  0s 24ms/step - loss: 13495.2461 - val_loss: 13374.2930 - learning_rate: 7.9766e-05
Epoch 35/50

4/4  0s 20ms/step - loss: 13313.4980 - val_loss: 13373.7051 - learning_rate: 7.1790e-05

```

Epoch 36/50
4/4  0s 22ms/step - loss: 13668.5605 - val_loss: 13373.1758 - learning_rate:
6.4611e-05
Epoch 37/50
4/4  0s 22ms/step - loss: 13514.7432 - val_loss: 13372.7002 - learning_rate:
5.8150e-05
Epoch 38/50
4/4  0s 66ms/step - loss: 13231.0820 - val_loss: 13372.2725 - learning_rate:
5.2335e-05
Epoch 39/50
4/4  0s 63ms/step - loss: 13494.3945 - val_loss: 13371.8896 - learning_rate:
4.7101e-05
Epoch 40/50
4/4  0s 20ms/step - loss: 13702.0459 - val_loss: 13371.5430 - learning_rate:
4.2391e-05
Epoch 41/50
4/4  0s 23ms/step - loss: 13286.5967 - val_loss: 13371.2344 - learning_rate:
3.8152e-05
Epoch 42/50
4/4  0s 20ms/step - loss: 13398.8916 - val_loss: 13370.9551 - learning_rate:
3.4337e-05
Epoch 43/50
4/4  0s 21ms/step - loss: 13279.2285 - val_loss: 13370.7070 - learning_rate:
3.0903e-05
Epoch 44/50
4/4  0s 24ms/step - loss: 13382.2578 - val_loss: 13370.4814 - learning_rate:
2.7813e-05
Epoch 45/50
4/4  0s 22ms/step - loss: 13769.6094 - val_loss: 13370.2783 - learning_rate:
2.5032e-05
Epoch 46/50
4/4  0s 20ms/step - loss: 13615.5010 - val_loss: 13370.0967 - learning_rate:
2.2528e-05
Epoch 47/50
4/4  0s 22ms/step - loss: 13055.9160 - val_loss: 13369.9336 - learning_rate:
2.0276e-05
Epoch 48/50
4/4  0s 22ms/step - loss: 13287.2754 - val_loss: 13369.7861 - learning_rate:
1.8248e-05
Epoch 49/50
4/4  0s 22ms/step - loss: 12960.6846 - val_loss: 13369.6533 - learning_rate:
1.6423e-05
Epoch 50/50
4/4  0s 24ms/step - loss: 13124.4600 - val_loss: 13369.5352 - learning_rate:
1.4781e-05
4/4  0s 7ms/step
1/1  0s 28ms/step
[0] eval-rmse:18.71190
[1] eval-rmse:18.05443
[2] eval-rmse:16.99912
[3] eval-rmse:16.54015
[4] eval-rmse:16.20447
[5] eval-rmse:15.77694
[6] eval-rmse:15.20195
[7] eval-rmse:14.93481
[8] eval-rmse:14.53941
[9] eval-rmse:14.29988
[10] eval-rmse:14.05461
[11] eval-rmse:13.87401

```

```
[12] eval-rmse:13.85855
[13] eval-rmse:13.86571
[14] eval-rmse:13.82265
[15] eval-rmse:13.79861
[16] eval-rmse:13.76339
[17] eval-rmse:13.72800
[18] eval-rmse:13.76430
[19] eval-rmse:13.83554
[20] eval-rmse:13.85022
[21] eval-rmse:13.89202
[22] eval-rmse:13.89011
[23] eval-rmse:13.95670
[24] eval-rmse:13.98885
[25] eval-rmse:14.04722
[26] eval-rmse:14.07543
[27] eval-rmse:14.11479
[28] eval-rmse:14.15639
[29] eval-rmse:14.22000
[30] eval-rmse:14.25695
[31] eval-rmse:14.30843
[32] eval-rmse:14.36498
[33] eval-rmse:14.40220
[34] eval-rmse:14.44634
[35] eval-rmse:14.48371
[36] eval-rmse:14.52242
[37] eval-rmse:14.55689
[38] eval-rmse:14.59534
[39] eval-rmse:14.62402
[40] eval-rmse:14.66559
[41] eval-rmse:14.70878
[42] eval-rmse:14.74236
[43] eval-rmse:14.78889
[44] eval-rmse:14.82133
[45] eval-rmse:14.84624
[46] eval-rmse:14.86970
[47] eval-rmse:14.90619
[48] eval-rmse:14.94241
[49] eval-rmse:14.97512
[50] eval-rmse:14.99047
[51] eval-rmse:15.02319
[52] eval-rmse:15.04363
[53] eval-rmse:15.07230
[54] eval-rmse:15.08896
[55] eval-rmse:15.11214
[56] eval-rmse:15.12666
[57] eval-rmse:15.14163
[58] eval-rmse:15.15688
[59] eval-rmse:15.17230
[60] eval-rmse:15.18605
[61] eval-rmse:15.20408
[62] eval-rmse:15.21747
[63] eval-rmse:15.23052
[64] eval-rmse:15.24258
[65] eval-rmse:15.25514
[66] eval-rmse:15.26505
Fold 3 - RMSE: 15.275687006547919
```

Training on fold 4...

Epoch 1/50

4/4  1s 79ms/step - loss: 13608.5127 - val_loss: 13263.2090 - learning_rate: 0.0010
Epoch 2/50

4/4  0s 20ms/step - loss: 13620.6270 - val_loss: 13252.2129 - learning_rate: 0.0010
Epoch 3/50

4/4  0s 22ms/step - loss: 13393.5703 - val_loss: 13240.9570 - learning_rate: 0.0010
Epoch 4/50

4/4  0s 23ms/step - loss: 13670.7754 - val_loss: 13229.6025 - learning_rate: 0.0010
Epoch 5/50

4/4  0s 24ms/step - loss: 13483.0439 - val_loss: 13176.0039 - learning_rate: 0.0010
Epoch 6/50

4/4  0s 20ms/step - loss: 13488.5850 - val_loss: 13164.1387 - learning_rate: 0.0010
Epoch 7/50

4/4  0s 22ms/step - loss: 13323.4785 - val_loss: 13152.0576 - learning_rate: 0.0010
Epoch 8/50

4/4  0s 21ms/step - loss: 13782.7734 - val_loss: 13139.8828 - learning_rate: 0.0010
Epoch 9/50

4/4  0s 21ms/step - loss: 13256.8779 - val_loss: 13127.6934 - learning_rate: 0.0010
Epoch 10/50

4/4  0s 21ms/step - loss: 13705.7881 - val_loss: 13115.4941 - learning_rate: 0.0010
Epoch 11/50

4/4  0s 22ms/step - loss: 13450.5732 - val_loss: 13104.5371 - learning_rate: 9.0000e-04
Epoch 12/50

4/4  0s 22ms/step - loss: 13205.3320 - val_loss: 13094.6943 - learning_rate: 8.1000e-04
Epoch 13/50

4/4  0s 22ms/step - loss: 13253.3398 - val_loss: 13085.8418 - learning_rate: 7.2900e-04
Epoch 14/50

4/4  0s 23ms/step - loss: 13742.7637 - val_loss: 13077.8691 - learning_rate: 6.5610e-04
Epoch 15/50

4/4  0s 21ms/step - loss: 13555.6064 - val_loss: 13070.7188 - learning_rate: 5.9049e-04
Epoch 16/50

4/4  0s 20ms/step - loss: 13422.8428 - val_loss: 13064.2939 - learning_rate: 5.3144e-04
Epoch 17/50

4/4  0s 20ms/step - loss: 13578.1279 - val_loss: 13058.5166 - learning_rate: 4.7830e-04
Epoch 18/50

4/4  0s 21ms/step - loss: 13683.6494 - val_loss: 13053.3223 - learning_rate: 4.3047e-04
Epoch 19/50

4/4  0s 22ms/step - loss: 13638.0732 - val_loss: 13048.6514 - learning_rate: 3.8742e-04
Epoch 20/50

4/4  0s 19ms/step - loss: 13153.1152 - val_loss: 13044.4658 - learning_rate: 3.4868e-04

Epoch 21/50
4/4  0s 19ms/step - loss: 13565.4395 - val_loss: 13040.6885 - learning_rate: 3.1381e-04

Epoch 22/50
4/4  0s 20ms/step - loss: 13034.7617 - val_loss: 13037.3018 - learning_rate: 2.8243e-04

Epoch 23/50
4/4  0s 23ms/step - loss: 13460.0928 - val_loss: 13034.2461 - learning_rate: 2.5419e-04

Epoch 24/50
4/4  0s 36ms/step - loss: 13658.7832 - val_loss: 13031.4971 - learning_rate: 2.2877e-04

Epoch 25/50
4/4  0s 20ms/step - loss: 13535.9180 - val_loss: 13029.0273 - learning_rate: 2.0589e-04

Epoch 26/50
4/4  0s 20ms/step - loss: 13521.5879 - val_loss: 13026.8076 - learning_rate: 1.8530e-04

Epoch 27/50
4/4  0s 20ms/step - loss: 13455.0723 - val_loss: 13024.8086 - learning_rate: 1.6677e-04

Epoch 28/50
4/4  0s 22ms/step - loss: 13503.1191 - val_loss: 13023.0117 - learning_rate: 1.5009e-04

Epoch 29/50
4/4  0s 21ms/step - loss: 13459.5771 - val_loss: 13021.3936 - learning_rate: 1.3509e-04

Epoch 30/50
4/4  0s 20ms/step - loss: 13427.8301 - val_loss: 13019.9395 - learning_rate: 1.2158e-04

Epoch 31/50
4/4  0s 21ms/step - loss: 13434.9033 - val_loss: 13018.6309 - learning_rate: 1.0942e-04

Epoch 32/50
4/4  0s 22ms/step - loss: 13488.4326 - val_loss: 13017.4531 - learning_rate: 9.8477e-05

Epoch 33/50
4/4  0s 24ms/step - loss: 13648.9980 - val_loss: 13016.3936 - learning_rate: 8.8629e-05

Epoch 34/50
4/4  0s 24ms/step - loss: 13735.9453 - val_loss: 13015.4395 - learning_rate: 7.9766e-05

Epoch 35/50
4/4  0s 22ms/step - loss: 13290.5166 - val_loss: 13014.5811 - learning_rate: 7.1790e-05

Epoch 36/50
4/4  0s 20ms/step - loss: 13324.2422 - val_loss: 13013.8086 - learning_rate: 6.4611e-05

Epoch 37/50
4/4  0s 21ms/step - loss: 13259.9668 - val_loss: 13013.1152 - learning_rate: 5.8150e-05

Epoch 38/50
4/4  0s 19ms/step - loss: 13448.7529 - val_loss: 13012.4902 - learning_rate: 5.2335e-05

Epoch 39/50
4/4  0s 20ms/step - loss: 12989.3340 - val_loss: 13011.9277 - learning_rate: 4.7101e-05

Epoch 40/50
4/4  0s 38ms/step - loss: 12930.1719 - val_loss: 13011.4229 - learning_rate:

```
4.2391e-05
Epoch 41/50
4/4 ————— 0s 21ms/step - loss: 13075.4268 - val_loss: 13010.9678 - learning_rate:
3.8152e-05
Epoch 42/50
4/4 ————— 0s 22ms/step - loss: 13335.3701 - val_loss: 13010.5566 - learning_rate:
3.4337e-05
Epoch 43/50
4/4 ————— 0s 22ms/step - loss: 13701.5547 - val_loss: 13010.1875 - learning_rate:
3.0903e-05
Epoch 44/50
4/4 ————— 0s 22ms/step - loss: 13468.1729 - val_loss: 13009.8555 - learning_rate:
2.7813e-05
Epoch 45/50
4/4 ————— 0s 20ms/step - loss: 13880.5889 - val_loss: 13009.5547 - learning_rate:
2.5032e-05
Epoch 46/50
4/4 ————— 0s 19ms/step - loss: 13117.8672 - val_loss: 13009.2871 - learning_rate:
2.2528e-05
Epoch 47/50
4/4 ————— 0s 22ms/step - loss: 13190.0635 - val_loss: 13009.0449 - learning_rate:
2.0276e-05
Epoch 48/50
4/4 ————— 0s 21ms/step - loss: 13510.6914 - val_loss: 13008.8271 - learning_rate:
1.8248e-05
Epoch 49/50
4/4 ————— 0s 21ms/step - loss: 13943.5244 - val_loss: 13008.6309 - learning_rate:
1.6423e-05
Epoch 50/50
4/4 ————— 0s 20ms/step - loss: 13714.0439 - val_loss: 13008.4551 - learning_rate:
1.4781e-05
4/4 ————— 0s 7ms/step
1/1 ————— 0s 29ms/step
[0]      eval-rmse:15.36666
[1]      eval-rmse:14.44169
[2]      eval-rmse:13.71985
[3]      eval-rmse:13.20343
[4]      eval-rmse:12.79552
[5]      eval-rmse:12.31893
[6]      eval-rmse:12.00914
[7]      eval-rmse:11.54817
[8]      eval-rmse:11.38056
[9]      eval-rmse:11.15908
[10]     eval-rmse:10.92011
[11]     eval-rmse:10.73975
[12]     eval-rmse:10.42196
[13]     eval-rmse:10.28302
[14]     eval-rmse:10.07493
[15]     eval-rmse:9.88746
[16]     eval-rmse:9.94177
[17]     eval-rmse:9.79050
[18]     eval-rmse:9.75470
[19]     eval-rmse:9.69857
[20]     eval-rmse:9.63198
[21]     eval-rmse:9.61789
[22]     eval-rmse:9.56757
[23]     eval-rmse:9.46415
[24]     eval-rmse:9.32415
[25]     eval-rmse:9.26758
```


[26] eval-rmse:9.30860
[27] eval-rmse:9.33578
[28] eval-rmse:9.34783
[29] eval-rmse:9.26785
[30] eval-rmse:9.26760
[31] eval-rmse:9.27827
[32] eval-rmse:9.30078
[33] eval-rmse:9.34084
[34] eval-rmse:9.27318
[35] eval-rmse:9.26251
[36] eval-rmse:9.27441
[37] eval-rmse:9.29051
[38] eval-rmse:9.30692
[39] eval-rmse:9.32950
[40] eval-rmse:9.36158
[41] eval-rmse:9.37000
[42] eval-rmse:9.39661
[43] eval-rmse:9.41848
[44] eval-rmse:9.38202
[45] eval-rmse:9.39729
[46] eval-rmse:9.43287
[47] eval-rmse:9.39638
[48] eval-rmse:9.41123
[49] eval-rmse:9.38009
[50] eval-rmse:9.38361
[51] eval-rmse:9.39688
[52] eval-rmse:9.37439
[53] eval-rmse:9.38825
[54] eval-rmse:9.36975
[55] eval-rmse:9.37217
[56] eval-rmse:9.35142
[57] eval-rmse:9.34179
[58] eval-rmse:9.33339
[59] eval-rmse:9.32585
[60] eval-rmse:9.31891
[61] eval-rmse:9.30745
[62] eval-rmse:9.29524
[63] eval-rmse:9.28430
[64] eval-rmse:9.27727
[65] eval-rmse:9.28835
[66] eval-rmse:9.28809
[67] eval-rmse:9.28864
[68] eval-rmse:9.28948
[69] eval-rmse:9.28924
[70] eval-rmse:9.28581
[71] eval-rmse:9.28243
[72] eval-rmse:9.28126
[73] eval-rmse:9.28398
[74] eval-rmse:9.28458
[75] eval-rmse:9.28520
[76] eval-rmse:9.28430
[77] eval-rmse:9.28480
[78] eval-rmse:9.28570
[79] eval-rmse:9.28785
[80] eval-rmse:9.28762
[81] eval-rmse:9.29046
[82] eval-rmse:9.29259
[83] eval-rmse:9.29183
[84] eval-rmse:9.29384

[85] eval-rmse:9.29330
Fold 4 - RMSE: 9.29330004852189

Training on fold 5...

Epoch 1/50

4/4  1s 82ms/step - loss: 13918.0254 - val_loss: 12984.5576 - learning_rate: 0.0010

Epoch 2/50

4/4  0s 21ms/step - loss: 13579.6611 - val_loss: 12976.9316 - learning_rate: 0.0010

Epoch 3/50

4/4  0s 21ms/step - loss: 13422.4111 - val_loss: 12969.2080 - learning_rate: 0.0010

Epoch 4/50

4/4  0s 22ms/step - loss: 13599.8115 - val_loss: 12961.4414 - learning_rate: 0.0010

Epoch 5/50

4/4  0s 22ms/step - loss: 13460.3379 - val_loss: 12953.6680 - learning_rate: 0.0010

Epoch 6/50

4/4  0s 23ms/step - loss: 14004.9316 - val_loss: 12945.8711 - learning_rate: 0.0010

Epoch 7/50

4/4  0s 20ms/step - loss: 13553.4004 - val_loss: 12938.0947 - learning_rate: 0.0010

Epoch 8/50

4/4  0s 22ms/step - loss: 13925.7520 - val_loss: 12930.3037 - learning_rate: 0.0010

Epoch 9/50

4/4  0s 22ms/step - loss: 13772.7129 - val_loss: 12922.5332 - learning_rate: 0.0010

Epoch 10/50

4/4  0s 21ms/step - loss: 13921.5713 - val_loss: 12914.7637 - learning_rate: 0.0010

Epoch 11/50

4/4  0s 21ms/step - loss: 13933.2275 - val_loss: 12894.4648 - learning_rate: 9.0000e-04

Epoch 12/50

4/4  0s 21ms/step - loss: 13625.3486 - val_loss: 12887.5312 - learning_rate: 8.1000e-04

Epoch 13/50

4/4  0s 25ms/step - loss: 13564.9180 - val_loss: 12881.1816 - learning_rate: 7.2900e-04

Epoch 14/50

4/4  0s 21ms/step - loss: 13240.1904 - val_loss: 12875.4307 - learning_rate: 6.5610e-04

Epoch 15/50

4/4  0s 23ms/step - loss: 13769.9092 - val_loss: 12870.2363 - learning_rate: 5.9049e-04

Epoch 16/50

4/4  0s 22ms/step - loss: 13692.1123 - val_loss: 12865.5684 - learning_rate: 5.3144e-04

Epoch 17/50

4/4  0s 23ms/step - loss: 13828.2783 - val_loss: 12861.3750 - learning_rate: 4.7830e-04

Epoch 18/50

4/4  0s 25ms/step - loss: 14244.3486 - val_loss: 12857.6064 - learning_rate: 4.3047e-04

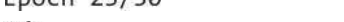
Epoch 19/50

4/4  0s 23ms/step - loss: 14098.9980 - val_loss: 12854.2295 - learning_rate: 3.8742e-04
Epoch 20/50

4/4  0s 20ms/step - loss: 13516.9746 - val_loss: 12851.2051 - learning_rate: 3.4868e-04
Epoch 21/50

4/4  0s 20ms/step - loss: 14163.6455 - val_loss: 12848.4814 - learning_rate: 3.1381e-04
Epoch 22/50

4/4  0s 22ms/step - loss: 13684.2734 - val_loss: 12846.0430 - learning_rate: 2.8243e-04
Epoch 23/50

4/4  0s 21ms/step - loss: 13523.9854 - val_loss: 12843.8516 - learning_rate: 2.5419e-04
Epoch 24/50

4/4  0s 22ms/step - loss: 13469.3770 - val_loss: 12841.8838 - learning_rate: 2.2877e-04
Epoch 25/50

4/4  0s 20ms/step - loss: 13565.4229 - val_loss: 12840.1123 - learning_rate: 2.0589e-04
Epoch 26/50

4/4  0s 22ms/step - loss: 13699.6445 - val_loss: 12838.5215 - learning_rate: 1.8530e-04
Epoch 27/50

4/4  0s 22ms/step - loss: 13446.3408 - val_loss: 12837.0918 - learning_rate: 1.6677e-04
Epoch 28/50

4/4  0s 22ms/step - loss: 13516.7930 - val_loss: 12835.8066 - learning_rate: 1.5009e-04
Epoch 29/50

4/4  0s 22ms/step - loss: 13159.0625 - val_loss: 12834.6533 - learning_rate: 1.3509e-04
Epoch 30/50

4/4  0s 20ms/step - loss: 13618.4229 - val_loss: 12833.6133 - learning_rate: 1.2158e-04
Epoch 31/50

4/4  0s 20ms/step - loss: 13218.9219 - val_loss: 12832.6797 - learning_rate: 1.0942e-04
Epoch 32/50

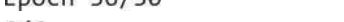
4/4  0s 37ms/step - loss: 13705.7061 - val_loss: 12831.8379 - learning_rate: 9.8477e-05
Epoch 33/50

4/4  0s 21ms/step - loss: 13553.9297 - val_loss: 12831.0820 - learning_rate: 8.8629e-05
Epoch 34/50

4/4  0s 22ms/step - loss: 13906.1797 - val_loss: 12830.4023 - learning_rate: 7.9766e-05
Epoch 35/50

4/4  0s 20ms/step - loss: 13804.8125 - val_loss: 12829.7900 - learning_rate: 7.1790e-05
Epoch 36/50

4/4  0s 25ms/step - loss: 13444.2422 - val_loss: 12829.2412 - learning_rate: 6.4611e-05
Epoch 37/50

4/4  0s 20ms/step - loss: 13651.8096 - val_loss: 12828.7480 - learning_rate: 5.8150e-05
Epoch 38/50

4/4  0s 22ms/step - loss: 13619.3486 - val_loss: 12828.3018 - learning_rate: 5.2335e-05

```
Epoch 39/50
4/4 ————— 0s 22ms/step - loss: 13399.3145 - val_loss: 12827.9014 - learning_rate:
4.7101e-05
Epoch 40/50
4/4 ————— 0s 22ms/step - loss: 13527.9336 - val_loss: 12827.5420 - learning_rate:
4.2391e-05
Epoch 41/50
4/4 ————— 0s 21ms/step - loss: 13491.1016 - val_loss: 12827.2197 - learning_rate:
3.8152e-05
Epoch 42/50
4/4 ————— 0s 22ms/step - loss: 13689.4678 - val_loss: 12826.9277 - learning_rate:
3.4337e-05
Epoch 43/50
4/4 ————— 0s 20ms/step - loss: 13723.7744 - val_loss: 12826.6660 - learning_rate:
3.0903e-05
Epoch 44/50
4/4 ————— 0s 20ms/step - loss: 13408.6797 - val_loss: 12826.4297 - learning_rate:
2.7813e-05
Epoch 45/50
4/4 ————— 0s 21ms/step - loss: 13615.1104 - val_loss: 12826.2178 - learning_rate:
2.5032e-05
Epoch 46/50
4/4 ————— 0s 23ms/step - loss: 13697.1387 - val_loss: 12826.0264 - learning_rate:
2.2528e-05
Epoch 47/50
4/4 ————— 0s 22ms/step - loss: 13734.2881 - val_loss: 12825.8545 - learning_rate:
2.0276e-05
Epoch 48/50
4/4 ————— 0s 20ms/step - loss: 13638.6504 - val_loss: 12825.7012 - learning_rate:
1.8248e-05
Epoch 49/50
4/4 ————— 0s 22ms/step - loss: 13259.8701 - val_loss: 12825.5605 - learning_rate:
1.6423e-05
Epoch 50/50
4/4 ————— 0s 22ms/step - loss: 13410.8018 - val_loss: 12825.4365 - learning_rate:
1.4781e-05
4/4 ————— 0s 7ms/step
1/1 ————— 0s 30ms/step
[0]      eval-rmse:17.48803
[1]      eval-rmse:17.00314
[2]      eval-rmse:16.56737
[3]      eval-rmse:16.35025
[4]      eval-rmse:16.29673
[5]      eval-rmse:16.36121
[6]      eval-rmse:15.81485
[7]      eval-rmse:15.33145
[8]      eval-rmse:15.16974
[9]      eval-rmse:15.00587
[10]     eval-rmse:14.61130
[11]     eval-rmse:14.37055
[12]     eval-rmse:14.16521
[13]     eval-rmse:13.95379
[14]     eval-rmse:13.65793
[15]     eval-rmse:13.56610
[16]     eval-rmse:13.52275
[17]     eval-rmse:13.36562
[18]     eval-rmse:13.25751
[19]     eval-rmse:13.16816
[20]     eval-rmse:13.07639
```

[21] eval-rmse:12.99128
[22] eval-rmse:12.88973
[23] eval-rmse:12.81083
[24] eval-rmse:12.75076
[25] eval-rmse:12.68826
[26] eval-rmse:12.64451
[27] eval-rmse:12.60957
[28] eval-rmse:12.57384
[29] eval-rmse:12.54275
[30] eval-rmse:12.51419
[31] eval-rmse:12.46529
[32] eval-rmse:12.46127
[33] eval-rmse:12.43914
[34] eval-rmse:12.41926
[35] eval-rmse:12.39415
[36] eval-rmse:12.37552
[37] eval-rmse:12.37801
[38] eval-rmse:12.35645
[39] eval-rmse:12.35432
[40] eval-rmse:12.34259
[41] eval-rmse:12.32533
[42] eval-rmse:12.31213
[43] eval-rmse:12.31872
[44] eval-rmse:12.30956
[45] eval-rmse:12.31108
[46] eval-rmse:12.30884
[47] eval-rmse:12.29948
[48] eval-rmse:12.29784
[49] eval-rmse:12.28764
[50] eval-rmse:12.28882
[51] eval-rmse:12.27007
[52] eval-rmse:12.25262
[53] eval-rmse:12.25235
[54] eval-rmse:12.26054
[55] eval-rmse:12.26265
[56] eval-rmse:12.26193
[57] eval-rmse:12.25985
[58] eval-rmse:12.25773
[59] eval-rmse:12.25589
[60] eval-rmse:12.25684
[61] eval-rmse:12.24582
[62] eval-rmse:12.24617
[63] eval-rmse:12.23942
[64] eval-rmse:12.23780
[65] eval-rmse:12.23655
[66] eval-rmse:12.23192
[67] eval-rmse:12.22568
[68] eval-rmse:12.22343
[69] eval-rmse:12.22495
[70] eval-rmse:12.22353
[71] eval-rmse:12.22223
[72] eval-rmse:12.22107
[73] eval-rmse:12.22082
[74] eval-rmse:12.22076
[75] eval-rmse:12.22190
[76] eval-rmse:12.22217
[77] eval-rmse:12.22334
[78] eval-rmse:12.22306
[79] eval-rmse:12.22325

[80] eval-rmse:12.22227
[81] eval-rmse:12.22205
[82] eval-rmse:12.22197
[83] eval-rmse:12.22119
[84] eval-rmse:12.22092
[85] eval-rmse:12.22106
[86] eval-rmse:12.22111
[87] eval-rmse:12.22131
[88] eval-rmse:12.22076
[89] eval-rmse:12.22028
[90] eval-rmse:12.22034
[91] eval-rmse:12.21966
[92] eval-rmse:12.21961
[93] eval-rmse:12.21975
[94] eval-rmse:12.21939
[95] eval-rmse:12.21904
[96] eval-rmse:12.21943
[97] eval-rmse:12.21937
[98] eval-rmse:12.21963
[99] eval-rmse:12.22003
[100] eval-rmse:12.22014
[101] eval-rmse:12.22068
[102] eval-rmse:12.22107
[103] eval-rmse:12.22142
[104] eval-rmse:12.22103
[105] eval-rmse:12.22159
[106] eval-rmse:12.22141
[107] eval-rmse:12.22128
[108] eval-rmse:12.22128
[109] eval-rmse:12.22082
[110] eval-rmse:12.22081
[111] eval-rmse:12.22136
[112] eval-rmse:12.22136
[113] eval-rmse:12.22118
[114] eval-rmse:12.22113
[115] eval-rmse:12.22111
[116] eval-rmse:12.22107
[117] eval-rmse:12.22127
[118] eval-rmse:12.22145
[119] eval-rmse:12.22165
[120] eval-rmse:12.22159
[121] eval-rmse:12.22124
[122] eval-rmse:12.22079
[123] eval-rmse:12.22074
[124] eval-rmse:12.22070
[125] eval-rmse:12.22028
[126] eval-rmse:12.22024
[127] eval-rmse:12.21981
[128] eval-rmse:12.21977
[129] eval-rmse:12.21977
[130] eval-rmse:12.21980
[131] eval-rmse:12.21983
[132] eval-rmse:12.21985
[133] eval-rmse:12.21989
[134] eval-rmse:12.21985
[135] eval-rmse:12.21987
[136] eval-rmse:12.21991
[137] eval-rmse:12.22002
[138] eval-rmse:12.22007

[139] eval-rmse:12.22015
[140] eval-rmse:12.22022
[141] eval-rmse:12.22027
[142] eval-rmse:12.22031
[143] eval-rmse:12.22033
[144] eval-rmse:12.22037
Fold 5 - RMSE: 12.220389916968065

K-Fold Cross-Validation Results:

Average RMSE: 11.27417699109495

Min RMSE: 8.389808974719402

Max RMSE: 15.275687006547919

Epoch 1/50

5/5  2s 66ms/step - loss: 13781.4385 - val_loss: 13276.6621 - learning_rate: 0.0010

Epoch 2/50

5/5  0s 18ms/step - loss: 13501.0635 - val_loss: 13260.5400 - learning_rate: 0.0010

Epoch 3/50

5/5  0s 18ms/step - loss: 13312.0410 - val_loss: 13243.9912 - learning_rate: 0.0010

Epoch 4/50

5/5  0s 18ms/step - loss: 13662.3340 - val_loss: 13227.2861 - learning_rate: 0.0010

Epoch 5/50

5/5  0s 20ms/step - loss: 13492.5586 - val_loss: 13210.5508 - learning_rate: 0.0010

Epoch 6/50

5/5  0s 18ms/step - loss: 13046.1602 - val_loss: 13193.8271 - learning_rate: 0.0010

Epoch 7/50

5/5  0s 18ms/step - loss: 13385.1523 - val_loss: 13177.0830 - learning_rate: 0.0010

Epoch 8/50

5/5  0s 18ms/step - loss: 13491.3770 - val_loss: 13160.3525 - learning_rate: 0.0010

Epoch 9/50

5/5  0s 18ms/step - loss: 13113.4297 - val_loss: 13143.6914 - learning_rate: 0.0010

Epoch 10/50

5/5  0s 17ms/step - loss: 13071.6113 - val_loss: 13127.0508 - learning_rate: 0.0010

Epoch 11/50

5/5  0s 18ms/step - loss: 13465.8164 - val_loss: 13082.0898 - learning_rate: 9.0000e-04

Epoch 12/50

5/5  0s 29ms/step - loss: 13121.9453 - val_loss: 13067.6865 - learning_rate: 8.1000e-04

Epoch 13/50

5/5  0s 20ms/step - loss: 13441.1074 - val_loss: 13054.5830 - learning_rate: 7.2900e-04

Epoch 14/50

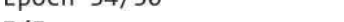
5/5  0s 17ms/step - loss: 12804.2910 - val_loss: 13042.8174 - learning_rate: 6.5610e-04

Epoch 15/50

5/5  0s 18ms/step - loss: 13585.2822 - val_loss: 13032.2061 - learning_rate: 5.9049e-04

Epoch 16/50

5/5  0s 17ms/step - loss: 13223.7607 - val_loss: 13022.7080 - learning_rate:

5.3144e-04
Epoch 17/50
5/5  0s 17ms/step - loss: 13122.5811 - val_loss: 13014.1885 - learning_rate:
4.7830e-04
Epoch 18/50
5/5  0s 17ms/step - loss: 13104.3643 - val_loss: 13006.5420 - learning_rate:
4.3047e-04
Epoch 19/50
5/5  0s 18ms/step - loss: 13335.5947 - val_loss: 12999.6641 - learning_rate:
3.8742e-04
Epoch 20/50
5/5  0s 19ms/step - loss: 13258.8711 - val_loss: 12993.4932 - learning_rate:
3.4868e-04
Epoch 21/50
5/5  0s 30ms/step - loss: 13111.1836 - val_loss: 12987.9639 - learning_rate:
3.1381e-04
Epoch 22/50
5/5  0s 18ms/step - loss: 12860.5898 - val_loss: 12982.9951 - learning_rate:
2.8243e-04
Epoch 23/50
5/5  0s 18ms/step - loss: 12996.8135 - val_loss: 12978.5264 - learning_rate:
2.5419e-04
Epoch 24/50
5/5  0s 18ms/step - loss: 12917.4902 - val_loss: 12974.5137 - learning_rate:
2.2877e-04
Epoch 25/50
5/5  0s 18ms/step - loss: 13314.6406 - val_loss: 12970.8984 - learning_rate:
2.0589e-04
Epoch 26/50
5/5  0s 17ms/step - loss: 13348.3799 - val_loss: 12967.6504 - learning_rate:
1.8530e-04
Epoch 27/50
5/5  0s 18ms/step - loss: 13392.4023 - val_loss: 12964.7305 - learning_rate:
1.6677e-04
Epoch 28/50
5/5  0s 19ms/step - loss: 13020.5977 - val_loss: 12962.1113 - learning_rate:
1.5009e-04
Epoch 29/50
5/5  0s 31ms/step - loss: 12697.9844 - val_loss: 12959.7549 - learning_rate:
1.3509e-04
Epoch 30/50
5/5  0s 18ms/step - loss: 13091.1387 - val_loss: 12957.6318 - learning_rate:
1.2158e-04
Epoch 31/50
5/5  0s 18ms/step - loss: 12789.0977 - val_loss: 12955.7256 - learning_rate:
1.0942e-04
Epoch 32/50
5/5  0s 19ms/step - loss: 12967.2588 - val_loss: 12954.0078 - learning_rate:
9.8477e-05
Epoch 33/50
5/5  0s 30ms/step - loss: 12947.6680 - val_loss: 12952.4629 - learning_rate:
8.8629e-05
Epoch 34/50
5/5  0s 35ms/step - loss: 12896.2773 - val_loss: 12951.0713 - learning_rate:
7.9766e-05
Epoch 35/50
5/5  0s 36ms/step - loss: 13208.5000 - val_loss: 12949.8203 - learning_rate:
7.1790e-05
Epoch 36/50

```

5/5 ————— 0s 33ms/step - loss: 12892.0020 - val_loss: 12948.6963 - learning_rate:
6.4611e-05
Epoch 37/50
5/5 ————— 0s 33ms/step - loss: 12858.1377 - val_loss: 12947.6855 - learning_rate:
5.8150e-05
Epoch 38/50
5/5 ————— 0s 34ms/step - loss: 13020.7148 - val_loss: 12946.7744 - learning_rate:
5.2335e-05
Epoch 39/50
5/5 ————— 0s 35ms/step - loss: 13171.9082 - val_loss: 12945.9561 - learning_rate:
4.7101e-05
Epoch 40/50
5/5 ————— 0s 34ms/step - loss: 13006.6992 - val_loss: 12945.2178 - learning_rate:
4.2391e-05
Epoch 41/50
5/5 ————— 0s 27ms/step - loss: 13117.5410 - val_loss: 12944.5547 - learning_rate:
3.8152e-05
Epoch 42/50
5/5 ————— 0s 37ms/step - loss: 13036.1045 - val_loss: 12943.9580 - learning_rate:
3.4337e-05
Epoch 43/50
5/5 ————— 0s 35ms/step - loss: 13024.1182 - val_loss: 12943.4209 - learning_rate:
3.0903e-05
Epoch 44/50
5/5 ————— 0s 18ms/step - loss: 12949.6396 - val_loss: 12942.9385 - learning_rate:
2.7813e-05
Epoch 45/50
5/5 ————— 0s 18ms/step - loss: 13037.1904 - val_loss: 12942.5049 - learning_rate:
2.5032e-05
Epoch 46/50
5/5 ————— 0s 28ms/step - loss: 12525.8018 - val_loss: 12942.1143 - learning_rate:
2.2528e-05
Epoch 47/50
5/5 ————— 0s 18ms/step - loss: 13032.8906 - val_loss: 12941.7627 - learning_rate:
2.0276e-05
Epoch 48/50
5/5 ————— 0s 18ms/step - loss: 13334.5098 - val_loss: 12941.4443 - learning_rate:
1.8248e-05
Epoch 49/50
5/5 ————— 0s 17ms/step - loss: 12325.4648 - val_loss: 12941.1602 - learning_rate:
1.6423e-05
Epoch 50/50
5/5 ————— 0s 18ms/step - loss: 13154.6992 - val_loss: 12940.9053 - learning_rate:
1.4781e-05
5/5 ————— 0s 5ms/step
2/2 ————— 0s 113ms/step
[0]      eval-rmse:19.79885
[1]      eval-rmse:18.27469
[2]      eval-rmse:16.83494
[3]      eval-rmse:15.54644
[4]      eval-rmse:14.39105
[5]      eval-rmse:13.33099
[6]      eval-rmse:12.44127
[7]      eval-rmse:11.65663
[8]      eval-rmse:10.97459
[9]      eval-rmse:10.36252
[10]     eval-rmse:9.90301
[11]     eval-rmse:9.48850
[12]     eval-rmse:9.09709

```

[13] eval-rmse:8.75067
[14] eval-rmse:8.63341
[15] eval-rmse:8.57674
[16] eval-rmse:8.47960
[17] eval-rmse:8.39275
[18] eval-rmse:8.28922
[19] eval-rmse:8.24123
[20] eval-rmse:8.21782
[21] eval-rmse:8.13954
[22] eval-rmse:8.06531
[23] eval-rmse:7.99881
[24] eval-rmse:7.94396
[25] eval-rmse:7.90026
[26] eval-rmse:7.87479
[27] eval-rmse:7.88122
[28] eval-rmse:7.85252
[29] eval-rmse:7.83358
[30] eval-rmse:7.85026
[31] eval-rmse:7.83413
[32] eval-rmse:7.85019
[33] eval-rmse:7.84745
[34] eval-rmse:7.84573
[35] eval-rmse:7.84728
[36] eval-rmse:7.85128
[37] eval-rmse:7.84755
[38] eval-rmse:7.85890
[39] eval-rmse:7.86468
[40] eval-rmse:7.87000
[41] eval-rmse:7.87612
[42] eval-rmse:7.87927
[43] eval-rmse:7.89218
[44] eval-rmse:7.90413
[45] eval-rmse:7.91180
[46] eval-rmse:7.92273
[47] eval-rmse:7.93220
[48] eval-rmse:7.93635
[49] eval-rmse:7.94479
[50] eval-rmse:7.95396
[51] eval-rmse:7.95473
[52] eval-rmse:7.95578
[53] eval-rmse:7.96206
[54] eval-rmse:7.96148
[55] eval-rmse:7.96743
[56] eval-rmse:7.97265
[57] eval-rmse:7.97356
[58] eval-rmse:7.97698
[59] eval-rmse:7.97940
[60] eval-rmse:7.97852
[61] eval-rmse:7.98157
[62] eval-rmse:7.98185
[63] eval-rmse:7.98418
[64] eval-rmse:7.98689
[65] eval-rmse:7.98861
[66] eval-rmse:7.98955
[67] eval-rmse:7.99183
[68] eval-rmse:7.99260
[69] eval-rmse:7.99436
[70] eval-rmse:7.99483
[71] eval-rmse:7.99625

```
[72]    eval-rmse:7.99789
[73]    eval-rmse:7.99902
[74]    eval-rmse:7.99777
[75]    eval-rmse:7.99789
[76]    eval-rmse:8.00256
[77]    eval-rmse:8.00704
[78]    eval-rmse:8.01124
```

Final Test RMSE: 8.014602133339082

```
In [ ]: # Make predictions on the test set using the final XGBoost model
y_test_pred = xgb_model_final.predict(dtest_final)

# Calculate RMSE manually
test_rmse = np.sqrt(np.mean(np.square(y_test_pred - y_test)))
print(f"Final Test RMSE: {test_rmse}")
```

Final Test RMSE: 8.014602133339082

```
In [ ]:
```

```
In [ ]: # Function to predict multiple values (e.g., 50 values) from the model
def predict_multiple_values(xgb_model_final, X_test, y_test, start_index=0, num_samples=50):
    # Select a batch of samples from X_test
    batch_samples = X_test[start_index:start_index+num_samples] # Select a batch of samples (e.g., 50)

    # Predict the output for these samples
    predictions = model.predict(batch_samples)

    # Get the actual values for these samples
    actual_values = y_test[start_index:start_index+num_samples]

    # Print the predictions and actual values for the first 50 samples
    for i in range(num_samples):
        print(f"Prediction {i+1}: Predicted Value = {predictions[i]}, Actual Value = {actual_values[i]}")

    return predictions, actual_values
```

```
In [ ]: # Call the function to predict 50 values (starting from index 0)
predicted_values, actual_values = predict_multiple_values(model, X_test, y_test, start_index=0, num_samples=50)
```


2/2 — 0s 87ms/step

Prediction 1: Predicted Value = [110.92162], Actual Value = 103
Prediction 2: Predicted Value = [113.96649], Actual Value = 102
Prediction 3: Predicted Value = [123.33864], Actual Value = 110
Prediction 4: Predicted Value = [114.35884], Actual Value = 99
Prediction 5: Predicted Value = [116.376236], Actual Value = 130
Prediction 6: Predicted Value = [124.93413], Actual Value = 112
Prediction 7: Predicted Value = [108.65557], Actual Value = 127
Prediction 8: Predicted Value = [115.9375], Actual Value = 96
Prediction 9: Predicted Value = [111.33491], Actual Value = 94
Prediction 10: Predicted Value = [113.460526], Actual Value = 110
Prediction 11: Predicted Value = [123.64275], Actual Value = 103
Prediction 12: Predicted Value = [119.79819], Actual Value = 124
Prediction 13: Predicted Value = [109.22665], Actual Value = 110
Prediction 14: Predicted Value = [109.339615], Actual Value = 106
Prediction 15: Predicted Value = [109.67463], Actual Value = 109
Prediction 16: Predicted Value = [125.34915], Actual Value = 130
Prediction 17: Predicted Value = [120.26686], Actual Value = 88
Prediction 18: Predicted Value = [108.85369], Actual Value = 88
Prediction 19: Predicted Value = [112.54408], Actual Value = 96
Prediction 20: Predicted Value = [123.63617], Actual Value = 183
Prediction 21: Predicted Value = [113.213005], Actual Value = 106
Prediction 22: Predicted Value = [123.31751], Actual Value = 110
Prediction 23: Predicted Value = [123.28581], Actual Value = 110
Prediction 24: Predicted Value = [110.12127], Actual Value = 183
Prediction 25: Predicted Value = [115.93299], Actual Value = 96
Prediction 26: Predicted Value = [114.38573], Actual Value = 100
Prediction 27: Predicted Value = [124.50636], Actual Value = 128
Prediction 28: Predicted Value = [114.36528], Actual Value = 99
Prediction 29: Predicted Value = [111.33156], Actual Value = 94
Prediction 30: Predicted Value = [124.99541], Actual Value = 124
Prediction 31: Predicted Value = [124.92204], Actual Value = 112
Prediction 32: Predicted Value = [115.98398], Actual Value = 106
Prediction 33: Predicted Value = [109.668076], Actual Value = 109
Prediction 34: Predicted Value = [123.63709], Actual Value = 103
Prediction 35: Predicted Value = [123.639244], Actual Value = 183
Prediction 36: Predicted Value = [117.77141], Actual Value = 146
Prediction 37: Predicted Value = [114.782555], Actual Value = 100
Prediction 38: Predicted Value = [125.35037], Actual Value = 130
Prediction 39: Predicted Value = [130.01385], Actual Value = 95
Prediction 40: Predicted Value = [124.905045], Actual Value = 110
Prediction 41: Predicted Value = [123.33284], Actual Value = 110
Prediction 42: Predicted Value = [113.932976], Actual Value = 102
Prediction 43: Predicted Value = [113.662834], Actual Value = 129
Prediction 44: Predicted Value = [124.90464], Actual Value = 112
Prediction 45: Predicted Value = [107.84265], Actual Value = 118
Prediction 46: Predicted Value = [114.79733], Actual Value = 113
Prediction 47: Predicted Value = [108.982506], Actual Value = 103
Prediction 48: Predicted Value = [117.79642], Actual Value = 146
Prediction 49: Predicted Value = [118.8222], Actual Value = 136
Prediction 50: Predicted Value = [113.65872], Actual Value = 129

```
In [ ]: #My code
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.models import Model
from tensorflow.keras.callbacks import LearningRateScheduler, EarlyStopping
from sklearn.model_selection import KFold
import numpy as np
```


Model Architecture with Improvements

```
def build_model(input_shape):
```

```
    # Input Layer
```

```
    input_signal = layers.Input(shape=input_shape, name='input_signal')
```

```
    # CNN Block 1 (for signal feature extraction)
```

```
    cnn_block_1 = layers.Conv1D(32, kernel_size=3, activation='relu', kernel_initializer='he_normal')
```

```
    cnn_block_1 = layers.BatchNormalization()(cnn_block_1)
```

```
    cnn_block_1 = layers.MaxPooling1D(pool_size=2)(cnn_block_1)
```

```
    cnn_block_1 = layers.Dropout(0.3)(cnn_block_1) # Adding dropout
```

```
    # CNN Block 2 (for additional feature extraction)
```

```
    cnn_block_2 = layers.Conv1D(64, kernel_size=5, activation='relu', kernel_initializer='he_normal')
```

```
    cnn_block_2 = layers.BatchNormalization()(cnn_block_2)
```

```
    cnn_block_2 = layers.MaxPooling1D(pool_size=2)(cnn_block_2)
```

```
    cnn_block_2 = layers.Dropout(0.3)(cnn_block_2) # Adding dropout
```

```
    # GRU Block (for sequential pattern recognition)
```

```
    gru_block = layers.GRU(64, return_sequences=True)(cnn_block_2) # Allowing the GRU to return
```

```
    gru_block = layers.GRU(64, return_sequences=False)(gru_block) # Adding another GRU Layer
```

```
    # Flatten the outputs from CNN and GRU blocks
```

```
    flatten_cnn = layers.Flatten()(cnn_block_2)
```

```
    flatten_gru = layers.Flatten()(gru_block)
```

```
    # Fully connected layers
```

```
    fc1 = layers.Dense(128, activation='relu', kernel_initializer='he_normal')(flatten_cnn)
```

```
    fc1 = layers.Dropout(0.4)(fc1) # Adding dropout to fully connected layer
```

```
    fc2 = layers.Dense(128, activation='relu', kernel_initializer='he_normal')(flatten_gru)
```

```
    fc2 = layers.Dropout(0.4)(fc2) # Adding dropout to fully connected layer
```

```
    # Concatenate the outputs from CNN and GRU
```

```
    concatenated = layers.concatenate([fc1, fc2], axis=-1)
```

```
    # Final regression output
```

```
    output = layers.Dense(1, activation='linear')(concatenated)
```

```
    # Create the model
```

```
    model = Model(inputs=input_signal, outputs=output)
```

```
    # Compile the model
```

```
    model.compile(optimizer='adam', loss='mean_squared_error', metrics=['mae'])
```

```
    return model
```

```
# Learning Rate Scheduler Function (if needed)
```

```
def scheduler(epoch, lr):
```

```
    if epoch < 10:
```

```
        return lr
```

```
    else:
```

```
        return lr * 0.9 # Reduce LR by 10% every 10 epochs
```

```
# RMSE Calculation function
```

```
def rmse(y_true, y_pred):
```

```
    return np.sqrt(np.mean(np.square(y_pred - y_true)))
```

```
# Define input shape based on the number of features and signal length
```

```
input_shape = (X_train.shape[1], 1) # Adjust based on your signal length
```

```

# K-Fold Cross-Validation
kf = KFold(n_splits=5, shuffle=True, random_state=42) # Define number of splits (K=5)
fold = 1
results = []

# Callbacks
lr_scheduler = LearningRateScheduler(scheduler)
early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

# Perform K-Fold Cross-Validation
for train_index, val_index in kf.split(X_train):
    print(f"\nTraining on fold {fold}...")

    # Split data into training and validation based on the fold indices
    X_train_fold, X_val_fold = X_train[train_index], X_train[val_index]
    y_train_fold, y_val_fold = y_train[train_index], y_train[val_index]

    # Build the model for the current fold
    model = build_model(input_shape)

    # Train the model for the current fold
    history = model.fit(X_train_fold, y_train_fold, epochs=50, batch_size=32,
                        validation_data=(X_val_fold, y_val_fold),
                        callbacks=[lr_scheduler, early_stopping])

    # Evaluate the model on the validation set of the current fold
    val_loss, val_mae = model.evaluate(X_val_fold, y_val_fold)
    print(f"Fold {fold} - Validation Loss: {val_loss}, Validation MAE: {val_mae}")

    # Make predictions on the validation set and calculate RMSE
    y_val_pred = model.predict(X_val_fold)
    fold_rmse = rmse(y_val_fold, y_val_pred)
    print(f"Fold {fold} - RMSE: {fold_rmse}")

    # Save the result for this fold (Loss, MAE, RMSE)
    results.append((val_loss, val_mae, fold_rmse))

    fold += 1

# After K-Fold, calculate the average, min, and max results
val_losses = [result[0] for result in results]
val_maes = [result[1] for result in results]
rmse_values = [result[2] for result in results]

print("\nK-Fold Cross-Validation Results:")
print(f"Average Validation Loss: {np.mean(val_losses)}")
print(f"Average Validation MAE: {np.mean(val_maes)}")
print(f"Average Validation RMSE: {np.mean(rmse_values)}")
print(f"Min RMSE: {np.min(rmse_values)}")
print(f"Max RMSE: {np.max(rmse_values)}")

# Optionally, retrain the model on the entire dataset after K-Fold (not necessary for validation)
final_model = build_model(input_shape)
final_model.fit(X_train, y_train, epochs=50, batch_size=32, validation_data=(X_val, y_val),
                callbacks=[lr_scheduler, early_stopping])

```

Training on fold 1...

Epoch 1/50

4/4  4s 241ms/step - loss: 7628.3472 - mae: 77.0443 - val_loss: 1535173.5000
- val_mae: 1237.3711 - learning_rate: 0.0010

Epoch 2/50

4/4  0s 71ms/step - loss: 2320.6934 - mae: 38.8146 - val_loss: 108414.1172 -
val_mae: 328.3441 - learning_rate: 0.0010

Epoch 3/50

4/4  0s 68ms/step - loss: 1650.8384 - mae: 34.3940 - val_loss: 173644.1094 -
val_mae: 415.7539 - learning_rate: 0.0010

Epoch 4/50

4/4  0s 98ms/step - loss: 784.8380 - mae: 22.9964 - val_loss: 166385.8125 - v
al_mae: 406.9324 - learning_rate: 0.0010

Epoch 5/50

4/4  1s 90ms/step - loss: 1041.4180 - mae: 26.0928 - val_loss: 46770.1641 - v
al_mae: 215.2412 - learning_rate: 0.0010

Epoch 6/50

4/4  1s 199ms/step - loss: 969.5867 - mae: 23.8388 - val_loss: 27357.7012 - v
al_mae: 164.3083 - learning_rate: 0.0010

Epoch 7/50

4/4  1s 86ms/step - loss: 805.7733 - mae: 22.6748 - val_loss: 40886.9727 - va
l_mae: 201.2459 - learning_rate: 0.0010

Epoch 8/50

4/4  0s 68ms/step - loss: 807.9630 - mae: 22.9399 - val_loss: 23307.2227 - va
l_mae: 151.5534 - learning_rate: 0.0010

Epoch 9/50

4/4  0s 65ms/step - loss: 963.9012 - mae: 25.2886 - val_loss: 6011.0747 - val
_mae: 75.6497 - learning_rate: 0.0010

Epoch 10/50

4/4  0s 57ms/step - loss: 888.2251 - mae: 23.3878 - val_loss: 9372.2969 - val
_mae: 95.2616 - learning_rate: 0.0010

Epoch 11/50

4/4  0s 57ms/step - loss: 538.8442 - mae: 18.1058 - val_loss: 8356.8340 - val
_mae: 89.7647 - learning_rate: 9.0000e-04

Epoch 12/50

4/4  0s 65ms/step - loss: 661.6750 - mae: 20.0821 - val_loss: 3162.2627 - val
_mae: 53.6880 - learning_rate: 8.1000e-04

Epoch 13/50

4/4  0s 68ms/step - loss: 599.6125 - mae: 17.8647 - val_loss: 2000.1440 - val
_mae: 41.5500 - learning_rate: 7.2900e-04

Epoch 14/50

4/4  0s 57ms/step - loss: 607.3618 - mae: 18.9115 - val_loss: 2315.3813 - val
_mae: 45.1425 - learning_rate: 6.5610e-04

Epoch 15/50

4/4  0s 68ms/step - loss: 561.4859 - mae: 19.2077 - val_loss: 2396.6223 - val
_mae: 46.0548 - learning_rate: 5.9049e-04

Epoch 16/50

4/4  0s 74ms/step - loss: 542.7488 - mae: 19.2136 - val_loss: 1678.8982 - val
_mae: 37.5535 - learning_rate: 5.3144e-04

Epoch 17/50

4/4  0s 73ms/step - loss: 643.2155 - mae: 19.4214 - val_loss: 1252.9518 - val
_mae: 31.5951 - learning_rate: 4.7830e-04

Epoch 18/50

4/4  0s 66ms/step - loss: 538.1362 - mae: 17.7520 - val_loss: 1063.9399 - val
_mae: 28.6698 - learning_rate: 4.3047e-04

Epoch 19/50

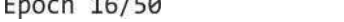
4/4  0s 64ms/step - loss: 555.0479 - mae: 19.1251 - val_loss: 628.6002 - val
_mae: 21.0544 - learning_rate: 3.8742e-04

Epoch 20/50

4/4 ————— 0s 76ms/step - loss: 732.0401 - mae: 19.9385 - val_loss: 557.9174 - val_mae: 19.8427 - learning_rate: 3.4868e-04
Epoch 21/50
4/4 ————— 0s 76ms/step - loss: 382.4808 - mae: 15.8471 - val_loss: 429.4281 - val_mae: 17.9016 - learning_rate: 3.1381e-04
Epoch 22/50
4/4 ————— 0s 67ms/step - loss: 542.8469 - mae: 17.3069 - val_loss: 540.1010 - val_mae: 19.6224 - learning_rate: 2.8243e-04
Epoch 23/50
4/4 ————— 0s 60ms/step - loss: 447.6719 - mae: 16.5301 - val_loss: 540.6075 - val_mae: 19.6511 - learning_rate: 2.5419e-04
Epoch 24/50
4/4 ————— 0s 60ms/step - loss: 575.9641 - mae: 19.3539 - val_loss: 474.5584 - val_mae: 18.6816 - learning_rate: 2.2877e-04
Epoch 25/50
4/4 ————— 0s 73ms/step - loss: 555.5287 - mae: 18.8911 - val_loss: 300.2211 - val_mae: 15.7877 - learning_rate: 2.0589e-04
Epoch 26/50
4/4 ————— 0s 63ms/step - loss: 485.6188 - mae: 17.0438 - val_loss: 268.6304 - val_mae: 14.0266 - learning_rate: 1.8530e-04
Epoch 27/50
4/4 ————— 0s 58ms/step - loss: 627.0782 - mae: 18.8518 - val_loss: 283.6104 - val_mae: 13.4857 - learning_rate: 1.6677e-04
Epoch 28/50
4/4 ————— 0s 57ms/step - loss: 560.8047 - mae: 18.0640 - val_loss: 282.9983 - val_mae: 13.5243 - learning_rate: 1.5009e-04
Epoch 29/50
4/4 ————— 0s 57ms/step - loss: 548.6624 - mae: 18.6153 - val_loss: 271.4862 - val_mae: 13.9007 - learning_rate: 1.3509e-04
Epoch 30/50
4/4 ————— 0s 60ms/step - loss: 507.8876 - mae: 17.9719 - val_loss: 270.6764 - val_mae: 13.9146 - learning_rate: 1.2158e-04
Epoch 31/50
4/4 ————— 0s 61ms/step - loss: 514.8126 - mae: 18.3342 - val_loss: 276.8701 - val_mae: 13.7283 - learning_rate: 1.0942e-04
1/1 ————— 0s 47ms/step - loss: 268.6304 - mae: 14.0266
Fold 1 - Validation Loss: 268.6304016113281, Validation MAE: 14.026586532592773
1/1 ————— 0s 228ms/step
Fold 1 - RMSE: 16.389948330456008

Training on fold 2...

Epoch 1/50
4/4 ————— 5s 290ms/step - loss: 7704.0430 - mae: 75.5586 - val_loss: 1348626.6250 - val_mae: 1159.2010 - learning_rate: 0.0010
Epoch 2/50
4/4 ————— 0s 99ms/step - loss: 1528.1068 - mae: 32.1401 - val_loss: 160421.8125 - val_mae: 399.4054 - learning_rate: 0.0010
Epoch 3/50
4/4 ————— 1s 83ms/step - loss: 1373.9232 - mae: 30.2140 - val_loss: 387921.7188 - val_mae: 621.5408 - learning_rate: 0.0010
Epoch 4/50
4/4 ————— 1s 74ms/step - loss: 1455.3287 - mae: 32.8822 - val_loss: 142229.6094 - val_mae: 376.1056 - learning_rate: 0.0010
Epoch 5/50
4/4 ————— 0s 74ms/step - loss: 698.5152 - mae: 20.4631 - val_loss: 30591.7461 - val_mae: 173.8110 - learning_rate: 0.0010
Epoch 6/50
4/4 ————— 0s 68ms/step - loss: 1255.1191 - mae: 28.6185 - val_loss: 49675.4570 - val_mae: 221.9132 - learning_rate: 0.0010

Epoch 7/50
4/4  0s 63ms/step - loss: 1046.6891 - mae: 25.0932 - val_loss: 50007.0078 - val_mae: 222.6750 - learning_rate: 0.0010
Epoch 8/50
4/4  0s 75ms/step - loss: 767.6334 - mae: 21.6789 - val_loss: 19135.1484 - val_mae: 137.1826 - learning_rate: 0.0010
Epoch 9/50
4/4  0s 66ms/step - loss: 714.3783 - mae: 20.7836 - val_loss: 13825.8701 - val_mae: 116.3318 - learning_rate: 0.0010
Epoch 10/50
4/4  0s 57ms/step - loss: 630.5214 - mae: 19.9096 - val_loss: 18159.2695 - val_mae: 133.6396 - learning_rate: 0.0010
Epoch 11/50
4/4  0s 65ms/step - loss: 784.8413 - mae: 22.1196 - val_loss: 9776.5371 - val_mae: 97.5109 - learning_rate: 9.0000e-04
Epoch 12/50
4/4  1s 154ms/step - loss: 750.3575 - mae: 21.8622 - val_loss: 6778.0186 - val_mae: 80.7648 - learning_rate: 8.1000e-04
Epoch 13/50
4/4  1s 156ms/step - loss: 601.0862 - mae: 18.5185 - val_loss: 6229.9897 - val_mae: 77.3221 - learning_rate: 7.2900e-04
Epoch 14/50
4/4  1s 159ms/step - loss: 721.6914 - mae: 20.8626 - val_loss: 4736.3877 - val_mae: 67.0278 - learning_rate: 6.5610e-04
Epoch 15/50
4/4  0s 124ms/step - loss: 577.6124 - mae: 18.6711 - val_loss: 2764.0962 - val_mae: 50.2948 - learning_rate: 5.9049e-04
Epoch 16/50
4/4  1s 80ms/step - loss: 657.4922 - mae: 18.5197 - val_loss: 2703.9141 - val_mae: 49.7118 - learning_rate: 5.3144e-04
Epoch 17/50
4/4  0s 58ms/step - loss: 668.3020 - mae: 19.2979 - val_loss: 2869.9519 - val_mae: 51.3593 - learning_rate: 4.7830e-04
Epoch 18/50
4/4  0s 62ms/step - loss: 605.9258 - mae: 18.6421 - val_loss: 1881.2866 - val_mae: 40.6805 - learning_rate: 4.3047e-04
Epoch 19/50
4/4  0s 64ms/step - loss: 686.2108 - mae: 20.8453 - val_loss: 1066.1582 - val_mae: 29.0668 - learning_rate: 3.8742e-04
Epoch 20/50
4/4  0s 60ms/step - loss: 555.2594 - mae: 18.3490 - val_loss: 1286.6235 - val_mae: 32.6235 - learning_rate: 3.4868e-04
Epoch 21/50
4/4  0s 68ms/step - loss: 575.9933 - mae: 18.5878 - val_loss: 2232.8604 - val_mae: 44.7804 - learning_rate: 3.1381e-04
Epoch 22/50
4/4  0s 71ms/step - loss: 686.4990 - mae: 21.1928 - val_loss: 1870.7780 - val_mae: 40.5620 - learning_rate: 2.8243e-04
Epoch 23/50
4/4  0s 75ms/step - loss: 633.9791 - mae: 19.9312 - val_loss: 737.4613 - val_mae: 22.9774 - learning_rate: 2.5419e-04
Epoch 24/50
4/4  0s 73ms/step - loss: 548.2912 - mae: 18.1457 - val_loss: 426.4572 - val_mae: 16.8067 - learning_rate: 2.2877e-04
Epoch 25/50
4/4  0s 57ms/step - loss: 564.9457 - mae: 18.9588 - val_loss: 476.9774 - val_mae: 17.7725 - learning_rate: 2.0589e-04
Epoch 26/50
4/4  0s 68ms/step - loss: 566.4389 - mae: 18.2379 - val_loss: 664.8480 - val_mae: 17.7725 - learning_rate: 2.0589e-04

mae: 21.5923 - learning_rate: 1.8530e-04
Epoch 27/50
4/4  0s 68ms/step - loss: 622.7915 - mae: 19.6736 - val_loss: 725.5850 - val_mae: 22.7131 - learning_rate: 1.6677e-04
Epoch 28/50
4/4  0s 57ms/step - loss: 616.2263 - mae: 19.8272 - val_loss: 430.9840 - val_mae: 16.9344 - learning_rate: 1.5009e-04
Epoch 29/50
4/4  0s 76ms/step - loss: 624.3829 - mae: 18.9616 - val_loss: 249.5645 - val_mae: 14.7454 - learning_rate: 1.3509e-04
Epoch 30/50
4/4  0s 65ms/step - loss: 551.6164 - mae: 17.7029 - val_loss: 237.0616 - val_mae: 14.5632 - learning_rate: 1.2158e-04
Epoch 31/50
4/4  0s 57ms/step - loss: 526.2556 - mae: 17.9717 - val_loss: 265.3676 - val_mae: 14.9292 - learning_rate: 1.0942e-04
Epoch 32/50
4/4  0s 90ms/step - loss: 637.7578 - mae: 20.4598 - val_loss: 293.4875 - val_mae: 15.2018 - learning_rate: 9.8477e-05
Epoch 33/50
4/4  1s 84ms/step - loss: 529.9539 - mae: 18.6955 - val_loss: 288.2280 - val_mae: 15.1527 - learning_rate: 8.8629e-05
Epoch 34/50
4/4  1s 83ms/step - loss: 567.0934 - mae: 18.6176 - val_loss: 257.8222 - val_mae: 14.8376 - learning_rate: 7.9766e-05
Epoch 35/50
4/4  1s 95ms/step - loss: 645.9583 - mae: 20.6208 - val_loss: 232.4491 - val_mae: 14.4747 - learning_rate: 7.1790e-05
Epoch 36/50
4/4  1s 75ms/step - loss: 714.7274 - mae: 20.2976 - val_loss: 217.1005 - val_mae: 14.0981 - learning_rate: 6.4611e-05
Epoch 37/50
4/4  0s 73ms/step - loss: 613.7021 - mae: 20.3652 - val_loss: 212.9978 - val_mae: 13.7306 - learning_rate: 5.8150e-05
Epoch 38/50
4/4  0s 59ms/step - loss: 664.8863 - mae: 19.3370 - val_loss: 218.0737 - val_mae: 13.4375 - learning_rate: 5.2335e-05
Epoch 39/50
4/4  0s 69ms/step - loss: 551.0507 - mae: 18.7838 - val_loss: 229.1929 - val_mae: 13.2972 - learning_rate: 4.7101e-05
Epoch 40/50
4/4  0s 57ms/step - loss: 566.0676 - mae: 18.1352 - val_loss: 237.0833 - val_mae: 13.2291 - learning_rate: 4.2391e-05
Epoch 41/50
4/4  0s 58ms/step - loss: 627.6569 - mae: 19.3737 - val_loss: 247.2678 - val_mae: 13.1555 - learning_rate: 3.8152e-05
Epoch 42/50
4/4  0s 61ms/step - loss: 582.4838 - mae: 19.5500 - val_loss: 250.4717 - val_mae: 13.1374 - learning_rate: 3.4337e-05
1/1  0s 56ms/step - loss: 212.9978 - mae: 13.7306
Fold 2 - Validation Loss: 212.997802734375, Validation MAE: 13.730592727661133
1/1  0s 238ms/step
Fold 2 - RMSE: 14.594444536363739

Training on fold 3...

Epoch 1/50

4/4  4s 236ms/step - loss: 8415.5117 - mae: 79.5451 - val_loss: 1022945.4375 - val_mae: 1010.0439 - learning_rate: 0.0010

Epoch 2/50

4/4 ————— 0s 75ms/step - loss: 1687.5122 - mae: 34.7338 - val_loss: 137877.1562 - val_mae: 370.4617 - learning_rate: 0.0010
Epoch 3/50

4/4 ————— 0s 71ms/step - loss: 1287.9554 - mae: 29.7529 - val_loss: 299081.5000 - val_mae: 545.9867 - learning_rate: 0.0010
Epoch 4/50

4/4 ————— 0s 57ms/step - loss: 1284.3909 - mae: 29.5641 - val_loss: 145701.4375 - val_mae: 380.8564 - learning_rate: 0.0010
Epoch 5/50

4/4 ————— 0s 65ms/step - loss: 649.6407 - mae: 20.0606 - val_loss: 42332.4727 - val_mae: 204.7973 - learning_rate: 0.0010
Epoch 6/50

4/4 ————— 0s 59ms/step - loss: 1126.0012 - mae: 27.4700 - val_loss: 51259.5859 - val_mae: 225.4963 - learning_rate: 0.0010
Epoch 7/50

4/4 ————— 0s 57ms/step - loss: 717.7958 - mae: 22.9527 - val_loss: 59124.9180 - val_mae: 242.2688 - learning_rate: 0.0010
Epoch 8/50

4/4 ————— 0s 74ms/step - loss: 647.6950 - mae: 20.4559 - val_loss: 34688.9453 - val_mae: 185.2925 - learning_rate: 0.0010
Epoch 9/50

4/4 ————— 0s 79ms/step - loss: 789.4864 - mae: 22.0181 - val_loss: 15249.2334 - val_mae: 122.3188 - learning_rate: 0.0010
Epoch 10/50

4/4 ————— 0s 68ms/step - loss: 752.8353 - mae: 20.6712 - val_loss: 19117.8789 - val_mae: 137.2265 - learning_rate: 0.0010
Epoch 11/50

4/4 ————— 0s 68ms/step - loss: 524.0151 - mae: 18.6162 - val_loss: 21157.1777 - val_mae: 144.5099 - learning_rate: 9.0000e-04
Epoch 12/50

4/4 ————— 0s 97ms/step - loss: 906.4252 - mae: 24.4328 - val_loss: 12633.4082 - val_mae: 111.3185 - learning_rate: 8.1000e-04
Epoch 13/50

4/4 ————— 0s 90ms/step - loss: 670.0572 - mae: 19.9316 - val_loss: 4617.6826 - val_mae: 66.3678 - learning_rate: 7.2900e-04
Epoch 14/50

4/4 ————— 1s 81ms/step - loss: 689.7787 - mae: 19.0919 - val_loss: 4746.1143 - val_mae: 67.3180 - learning_rate: 6.5610e-04
Epoch 15/50

4/4 ————— 1s 88ms/step - loss: 547.0414 - mae: 17.1838 - val_loss: 7406.4189 - val_mae: 84.7463 - learning_rate: 5.9049e-04
Epoch 16/50

4/4 ————— 1s 98ms/step - loss: 637.4426 - mae: 21.2165 - val_loss: 6992.1870 - val_mae: 82.2608 - learning_rate: 5.3144e-04
Epoch 17/50

4/4 ————— 1s 79ms/step - loss: 628.6169 - mae: 20.7907 - val_loss: 4195.6494 - val_mae: 63.0897 - learning_rate: 4.7830e-04
Epoch 18/50

4/4 ————— 0s 71ms/step - loss: 620.0344 - mae: 20.0374 - val_loss: 1946.7437 - val_mae: 41.7259 - learning_rate: 4.3047e-04
Epoch 19/50

4/4 ————— 0s 68ms/step - loss: 713.3562 - mae: 20.2089 - val_loss: 2025.5256 - val_mae: 42.6538 - learning_rate: 3.8742e-04
Epoch 20/50

4/4 ————— 0s 69ms/step - loss: 624.2599 - mae: 19.2259 - val_loss: 1995.3510 - val_mae: 42.3213 - learning_rate: 3.4868e-04
Epoch 21/50

4/4 ————— 0s 72ms/step - loss: 511.0335 - mae: 17.0419 - val_loss: 1655.9058 - val_mae: 38.1264 - learning_rate: 3.1381e-04

Epoch 22/50
4/4  0s 70ms/step - loss: 539.2043 - mae: 18.3912 - val_loss: 1551.2141 - val_mae: 36.7370 - learning_rate: 2.8243e-04

Epoch 23/50
4/4  0s 78ms/step - loss: 557.0504 - mae: 18.4941 - val_loss: 1445.8479 - val_mae: 35.2937 - learning_rate: 2.5419e-04

Epoch 24/50
4/4  0s 67ms/step - loss: 525.2075 - mae: 18.8907 - val_loss: 1216.2615 - val_mae: 32.1401 - learning_rate: 2.2877e-04

Epoch 25/50
4/4  0s 70ms/step - loss: 620.8504 - mae: 18.7728 - val_loss: 836.3737 - val_mae: 26.3893 - learning_rate: 2.0589e-04

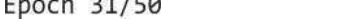
Epoch 26/50
4/4  0s 72ms/step - loss: 573.0794 - mae: 18.9035 - val_loss: 577.1876 - val_mae: 21.4796 - learning_rate: 1.8530e-04

Epoch 27/50
4/4  0s 73ms/step - loss: 659.8785 - mae: 20.4613 - val_loss: 461.9614 - val_mae: 19.2173 - learning_rate: 1.6677e-04

Epoch 28/50
4/4  0s 69ms/step - loss: 509.7300 - mae: 17.1060 - val_loss: 452.3279 - val_mae: 19.0389 - learning_rate: 1.5009e-04

Epoch 29/50
4/4  0s 65ms/step - loss: 557.7900 - mae: 19.0886 - val_loss: 387.4815 - val_mae: 17.7625 - learning_rate: 1.3509e-04

Epoch 30/50
4/4  0s 74ms/step - loss: 384.2839 - mae: 16.0755 - val_loss: 334.0059 - val_mae: 16.6774 - learning_rate: 1.2158e-04

Epoch 31/50
4/4  0s 77ms/step - loss: 524.3125 - mae: 18.9113 - val_loss: 301.8496 - val_mae: 16.0137 - learning_rate: 1.0942e-04

Epoch 32/50
4/4  0s 75ms/step - loss: 539.2258 - mae: 18.6631 - val_loss: 268.8421 - val_mae: 15.2243 - learning_rate: 9.8477e-05

Epoch 33/50
4/4  0s 62ms/step - loss: 572.2711 - mae: 19.3520 - val_loss: 240.4819 - val_mae: 14.4002 - learning_rate: 8.8629e-05

Epoch 34/50
4/4  0s 79ms/step - loss: 584.1526 - mae: 19.3376 - val_loss: 229.9229 - val_mae: 14.0341 - learning_rate: 7.9766e-05

Epoch 35/50
4/4  0s 69ms/step - loss: 387.3199 - mae: 16.4735 - val_loss: 235.1510 - val_mae: 14.2174 - learning_rate: 7.1790e-05

Epoch 36/50
4/4  0s 71ms/step - loss: 549.5320 - mae: 17.7933 - val_loss: 234.0578 - val_mae: 14.1770 - learning_rate: 6.4611e-05

Epoch 37/50
4/4  0s 64ms/step - loss: 489.2127 - mae: 17.7017 - val_loss: 227.8766 - val_mae: 13.9489 - learning_rate: 5.8150e-05

Epoch 38/50
4/4  0s 75ms/step - loss: 527.7209 - mae: 18.7983 - val_loss: 214.8549 - val_mae: 13.4316 - learning_rate: 5.2335e-05

Epoch 39/50
4/4  0s 69ms/step - loss: 532.4568 - mae: 19.1702 - val_loss: 200.3828 - val_mae: 12.7544 - learning_rate: 4.7101e-05

Epoch 40/50
4/4  0s 73ms/step - loss: 563.5922 - mae: 19.6181 - val_loss: 191.8374 - val_mae: 12.0501 - learning_rate: 4.2391e-05

Epoch 41/50
4/4  0s 64ms/step - loss: 559.9914 - mae: 18.3419 - val_loss: 189.9359 - val_mae: 11.7544 - learning_rate: 3.8243e-05

mae: 11.2808 - learning_rate: 3.8152e-05
Epoch 42/50
4/4  0s 59ms/step - loss: 546.1406 - mae: 18.0289 - val_loss: 195.8293 - val_mae: 10.6756 - learning_rate: 3.4337e-05
Epoch 43/50
4/4  0s 62ms/step - loss: 624.2868 - mae: 19.4358 - val_loss: 206.6892 - val_mae: 10.4510 - learning_rate: 3.0903e-05
Epoch 44/50
4/4  0s 58ms/step - loss: 530.5536 - mae: 18.3133 - val_loss: 218.2584 - val_mae: 10.4066 - learning_rate: 2.7813e-05
Epoch 45/50
4/4  0s 73ms/step - loss: 616.7100 - mae: 19.9813 - val_loss: 226.2301 - val_mae: 10.4568 - learning_rate: 2.5032e-05
Epoch 46/50
4/4  0s 69ms/step - loss: 587.9606 - mae: 18.4284 - val_loss: 233.5321 - val_mae: 10.5331 - learning_rate: 2.2528e-05
1/1  0s 48ms/step - loss: 189.9359 - mae: 11.2808
Fold 3 - Validation Loss: 189.93588256835938, Validation MAE: 11.280802726745605
1/1  0s 234ms/step
Fold 3 - RMSE: 13.781722833394973

Training on fold 4...

Epoch 1/50
4/4  5s 249ms/step - loss: 8350.6670 - mae: 78.4811 - val_loss: 548805.3750 - val_mae: 739.4398 - learning_rate: 0.0010
Epoch 2/50
4/4  0s 68ms/step - loss: 1367.4506 - mae: 29.0479 - val_loss: 181092.8594 - val_mae: 424.4943 - learning_rate: 0.0010
Epoch 3/50
4/4  0s 61ms/step - loss: 988.4772 - mae: 26.2941 - val_loss: 240092.0938 - val_mae: 488.9617 - learning_rate: 0.0010
Epoch 4/50
4/4  0s 63ms/step - loss: 809.6198 - mae: 22.5573 - val_loss: 50786.3789 - val_mae: 224.3350 - learning_rate: 0.0010
Epoch 5/50
4/4  0s 60ms/step - loss: 1159.0187 - mae: 26.1543 - val_loss: 57412.5469 - val_mae: 238.6422 - learning_rate: 0.0010
Epoch 6/50
4/4  0s 77ms/step - loss: 705.1876 - mae: 20.8646 - val_loss: 50103.3711 - val_mae: 222.8741 - learning_rate: 0.0010
Epoch 7/50
4/4  0s 77ms/step - loss: 654.2105 - mae: 20.1648 - val_loss: 18023.3828 - val_mae: 133.0222 - learning_rate: 0.0010
Epoch 8/50
4/4  0s 60ms/step - loss: 780.4922 - mae: 21.9590 - val_loss: 19029.6270 - val_mae: 136.7641 - learning_rate: 0.0010
Epoch 9/50
4/4  0s 70ms/step - loss: 683.0046 - mae: 20.2195 - val_loss: 19474.8027 - val_mae: 138.3760 - learning_rate: 0.0010
Epoch 10/50
4/4  0s 75ms/step - loss: 614.9913 - mae: 20.4612 - val_loss: 9031.5127 - val_mae: 93.4119 - learning_rate: 0.0010
Epoch 11/50
4/4  0s 70ms/step - loss: 515.4796 - mae: 17.0476 - val_loss: 6747.1323 - val_mae: 80.2353 - learning_rate: 9.0000e-04
Epoch 12/50
4/4  0s 60ms/step - loss: 409.2618 - mae: 15.9730 - val_loss: 9999.3379 - val_mae: 98.2650 - learning_rate: 8.1000e-04
Epoch 13/50

4/4 ————— 0s 58ms/step - loss: 636.1282 - mae: 19.6576 - val_loss: 9578.3438 - val_mae: 96.0428 - learning_rate: 7.2900e-04
Epoch 14/50

4/4 ————— 0s 75ms/step - loss: 605.3291 - mae: 18.7688 - val_loss: 3664.2363 - val_mae: 57.8841 - learning_rate: 6.5610e-04
Epoch 15/50

4/4 ————— 0s 59ms/step - loss: 527.7860 - mae: 18.0966 - val_loss: 3692.8760 - val_mae: 58.1544 - learning_rate: 5.9049e-04
Epoch 16/50

4/4 ————— 0s 74ms/step - loss: 586.5709 - mae: 19.0461 - val_loss: 3286.0981 - val_mae: 54.6001 - learning_rate: 5.3144e-04
Epoch 17/50

4/4 ————— 0s 63ms/step - loss: 629.5632 - mae: 20.7082 - val_loss: 1315.2317 - val_mae: 32.3145 - learning_rate: 4.7830e-04
Epoch 18/50

4/4 ————— 0s 70ms/step - loss: 529.9203 - mae: 17.6010 - val_loss: 1633.1692 - val_mae: 36.7824 - learning_rate: 4.3047e-04
Epoch 19/50

4/4 ————— 0s 59ms/step - loss: 578.3533 - mae: 19.1930 - val_loss: 1949.3052 - val_mae: 40.8117 - learning_rate: 3.8742e-04
Epoch 20/50

4/4 ————— 0s 62ms/step - loss: 556.6569 - mae: 18.3839 - val_loss: 1251.3186 - val_mae: 31.5018 - learning_rate: 3.4868e-04
Epoch 21/50

4/4 ————— 0s 65ms/step - loss: 575.3888 - mae: 17.9054 - val_loss: 728.4919 - val_mae: 23.6309 - learning_rate: 3.1381e-04
Epoch 22/50

4/4 ————— 0s 64ms/step - loss: 556.6213 - mae: 18.9184 - val_loss: 641.6002 - val_mae: 22.0837 - learning_rate: 2.8243e-04
Epoch 23/50

4/4 ————— 0s 63ms/step - loss: 629.8201 - mae: 19.8422 - val_loss: 589.3647 - val_mae: 21.2374 - learning_rate: 2.5419e-04
Epoch 24/50

4/4 ————— 0s 64ms/step - loss: 547.5195 - mae: 19.1378 - val_loss: 511.6811 - val_mae: 20.0078 - learning_rate: 2.2877e-04
Epoch 25/50

4/4 ————— 0s 77ms/step - loss: 667.0607 - mae: 19.8623 - val_loss: 484.2749 - val_mae: 19.6158 - learning_rate: 2.0589e-04
Epoch 26/50

4/4 ————— 0s 64ms/step - loss: 675.0148 - mae: 19.6867 - val_loss: 437.4604 - val_mae: 18.8748 - learning_rate: 1.8530e-04
Epoch 27/50

4/4 ————— 0s 74ms/step - loss: 471.0949 - mae: 16.7405 - val_loss: 372.2279 - val_mae: 17.6605 - learning_rate: 1.6677e-04
Epoch 28/50

4/4 ————— 0s 66ms/step - loss: 561.3436 - mae: 18.6166 - val_loss: 311.8197 - val_mae: 16.2472 - learning_rate: 1.5009e-04
Epoch 29/50

4/4 ————— 0s 66ms/step - loss: 577.5662 - mae: 18.1427 - val_loss: 254.7083 - val_mae: 13.8259 - learning_rate: 1.3509e-04
Epoch 30/50

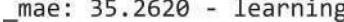
4/4 ————— 0s 59ms/step - loss: 528.5946 - mae: 17.7007 - val_loss: 256.1530 - val_mae: 13.2076 - learning_rate: 1.2158e-04
Epoch 31/50

4/4 ————— 0s 82ms/step - loss: 484.6226 - mae: 16.2883 - val_loss: 255.7258 - val_mae: 13.2850 - learning_rate: 1.0942e-04
Epoch 32/50

4/4 ————— 0s 92ms/step - loss: 675.0598 - mae: 19.9717 - val_loss: 256.7355 - val_mae: 13.8503 - learning_rate: 9.8477e-05

Epoch 33/50
4/4  1s 83ms/step - loss: 426.7325 - mae: 17.2328 - val_loss: 258.0310 - val_mae: 13.9696 - learning_rate: 8.8629e-05
Epoch 34/50
4/4  0s 92ms/step - loss: 425.1199 - mae: 16.6567 - val_loss: 256.3814 - val_mae: 13.6575 - learning_rate: 7.9766e-05
1/1  0s 78ms/step - loss: 254.7083 - mae: 13.8259
Fold 4 - Validation Loss: 254.7082977294922, Validation MAE: 13.825932502746582
1/1  0s 342ms/step
Fold 4 - RMSE: 15.95958326270079

Training on fold 5...

Epoch 1/50
4/4  5s 233ms/step - loss: 8028.2222 - mae: 80.3933 - val_loss: 1096868.0000 - val_mae: 1045.1510 - learning_rate: 0.0010
Epoch 2/50
4/4  0s 80ms/step - loss: 1403.1968 - mae: 29.9527 - val_loss: 105509.4609 - val_mae: 323.1730 - learning_rate: 0.0010
Epoch 3/50
4/4  0s 66ms/step - loss: 1162.5446 - mae: 28.3577 - val_loss: 179830.9844 - val_mae: 422.7398 - learning_rate: 0.0010
Epoch 4/50
4/4  0s 69ms/step - loss: 1090.0222 - mae: 27.0264 - val_loss: 74779.5938 - val_mae: 271.8361 - learning_rate: 0.0010
Epoch 5/50
4/4  0s 69ms/step - loss: 596.7691 - mae: 19.6203 - val_loss: 40399.4258 - val_mae: 198.9358 - learning_rate: 0.0010
Epoch 6/50
4/4  0s 76ms/step - loss: 518.7901 - mae: 19.1296 - val_loss: 34850.5312 - val_mae: 184.4595 - learning_rate: 0.0010
Epoch 7/50
4/4  0s 69ms/step - loss: 568.3295 - mae: 18.7150 - val_loss: 18399.9707 - val_mae: 132.6693 - learning_rate: 0.0010
Epoch 8/50
4/4  0s 66ms/step - loss: 564.1527 - mae: 19.8473 - val_loss: 14853.9102 - val_mae: 118.5715 - learning_rate: 0.0010
Epoch 9/50
4/4  0s 64ms/step - loss: 510.8038 - mae: 18.8970 - val_loss: 12296.9326 - val_mae: 107.2398 - learning_rate: 0.0010
Epoch 10/50
4/4  0s 66ms/step - loss: 505.0752 - mae: 17.8958 - val_loss: 6623.0088 - val_mae: 76.3080 - learning_rate: 0.0010
Epoch 11/50
4/4  0s 68ms/step - loss: 413.8832 - mae: 16.0891 - val_loss: 6038.8550 - val_mae: 72.3482 - learning_rate: 9.0000e-04
Epoch 12/50
4/4  0s 64ms/step - loss: 594.8898 - mae: 19.8128 - val_loss: 4494.3379 - val_mae: 61.8403 - learning_rate: 8.1000e-04
Epoch 13/50
4/4  0s 68ms/step - loss: 463.5160 - mae: 18.1093 - val_loss: 3538.7388 - val_mae: 55.4334 - learning_rate: 7.2900e-04
Epoch 14/50
4/4  0s 80ms/step - loss: 395.6592 - mae: 15.8237 - val_loss: 1711.0409 - val_mae: 38.3311 - learning_rate: 6.5610e-04
Epoch 15/50
4/4  0s 64ms/step - loss: 449.8590 - mae: 16.5563 - val_loss: 1491.3900 - val_mae: 35.2620 - learning_rate: 5.9049e-04
Epoch 16/50
4/4  0s 70ms/step - loss: 365.3396 - mae: 15.5797 - val_loss: 1962.8363 - val

_mae: 41.3374 - learning_rate: 5.3144e-04
Epoch 17/50
4/4  0s 67ms/step - loss: 451.3793 - mae: 16.9172 - val_loss: 1156.4844 - val_mae: 29.4800 - learning_rate: 4.7830e-04
Epoch 18/50
4/4  0s 95ms/step - loss: 502.1533 - mae: 17.7513 - val_loss: 897.1928 - val_mae: 23.7942 - learning_rate: 4.3047e-04
Epoch 19/50
4/4  0s 87ms/step - loss: 438.7259 - mae: 16.5340 - val_loss: 931.0905 - val_mae: 24.7606 - learning_rate: 3.8742e-04
Epoch 20/50
4/4  0s 92ms/step - loss: 385.1531 - mae: 15.9852 - val_loss: 1014.2480 - val_mae: 26.5880 - learning_rate: 3.4868e-04
Epoch 21/50
4/4  1s 93ms/step - loss: 353.2218 - mae: 14.9035 - val_loss: 869.9130 - val_mae: 22.8550 - learning_rate: 3.1381e-04
Epoch 22/50
4/4  1s 106ms/step - loss: 426.1108 - mae: 16.8071 - val_loss: 826.5793 - val_mae: 20.9945 - learning_rate: 2.8243e-04
Epoch 23/50
4/4  0s 106ms/step - loss: 422.3547 - mae: 16.0793 - val_loss: 824.2634 - val_mae: 21.2347 - learning_rate: 2.5419e-04
Epoch 24/50
4/4  0s 62ms/step - loss: 416.3271 - mae: 16.3708 - val_loss: 867.7095 - val_mae: 22.7810 - learning_rate: 2.2877e-04
Epoch 25/50
4/4  0s 72ms/step - loss: 426.5392 - mae: 16.5761 - val_loss: 835.9830 - val_mae: 21.5801 - learning_rate: 2.0589e-04
Epoch 26/50
4/4  0s 62ms/step - loss: 351.3214 - mae: 14.3858 - val_loss: 832.5036 - val_mae: 20.8507 - learning_rate: 1.8530e-04
Epoch 27/50
4/4  0s 71ms/step - loss: 357.7843 - mae: 15.4709 - val_loss: 849.8808 - val_mae: 20.9301 - learning_rate: 1.6677e-04
Epoch 28/50
4/4  0s 70ms/step - loss: 351.6573 - mae: 15.1224 - val_loss: 874.3843 - val_mae: 21.1581 - learning_rate: 1.5009e-04
1/1  0s 50ms/step - loss: 824.2634 - mae: 21.2347
Fold 5 - Validation Loss: 824.263427734375, Validation MAE: 21.234663009643555
1/1  0s 257ms/step
Fold 5 - RMSE: 28.70998761833528

K-Fold Cross-Validation Results:

Average Validation Loss: 350.10716247558594

Average Validation MAE: 14.81971549987793

Average Validation RMSE: 17.887137316250158

Min RMSE: 13.781722833394973

Max RMSE: 28.70998761833528

Epoch 1/50

4/4  4s 223ms/step - loss: 8132.3442 - mae: 77.7134 - val_loss: 752426.6250 - val_mae: 865.6510 - learning_rate: 0.0010

Epoch 2/50

4/4  0s 79ms/step - loss: 1618.9799 - mae: 32.8607 - val_loss: 195238.0469 - val_mae: 440.6022 - learning_rate: 0.0010

Epoch 3/50

4/4  1s 71ms/step - loss: 1113.8723 - mae: 26.7332 - val_loss: 330255.4375 - val_mae: 573.4219 - learning_rate: 0.0010

Epoch 4/50

4/4  0s 76ms/step - loss: 1312.7587 - mae: 28.9398 - val_loss: 74722.0469 - v

al_mae: 272.2043 - learning_rate: 0.0010

Epoch 5/50

4/4  0s 65ms/step - loss: 881.9872 - mae: 22.1941 - val_loss: 41400.0273 - va

l_mae: 202.2507 - learning_rate: 0.0010

Epoch 6/50

4/4  0s 59ms/step - loss: 620.1428 - mae: 19.0376 - val_loss: 68587.3047 - va

l_mae: 260.8021 - learning_rate: 0.0010

Epoch 7/50

4/4  0s 75ms/step - loss: 718.0179 - mae: 22.3751 - val_loss: 30701.6758 - va

l_mae: 173.9560 - learning_rate: 0.0010

Epoch 8/50

4/4  0s 76ms/step - loss: 536.2233 - mae: 18.3752 - val_loss: 13386.7168 - va

l_mae: 114.1064 - learning_rate: 0.0010

Epoch 9/50

4/4  0s 72ms/step - loss: 655.3442 - mae: 20.0298 - val_loss: 20230.0898 - va

l_mae: 140.9410 - learning_rate: 0.0010

Epoch 10/50

4/4  0s 65ms/step - loss: 551.6042 - mae: 18.8299 - val_loss: 11511.5957 - va

l_mae: 105.7276 - learning_rate: 0.0010

Epoch 11/50

4/4  0s 77ms/step - loss: 657.7761 - mae: 19.3112 - val_loss: 7307.5225 - val

_mae: 83.5871 - learning_rate: 9.0000e-04

Epoch 12/50

4/4  0s 72ms/step - loss: 608.6169 - mae: 19.3387 - val_loss: 8786.6475 - val

_mae: 91.9715 - learning_rate: 8.1000e-04

Epoch 13/50

4/4  0s 91ms/step - loss: 507.4504 - mae: 18.5452 - val_loss: 5134.4429 - val

_mae: 69.3755 - learning_rate: 7.2900e-04

Epoch 14/50

4/4  1s 94ms/step - loss: 538.5804 - mae: 17.9489 - val_loss: 3807.4980 - val

_mae: 59.2234 - learning_rate: 6.5610e-04

Epoch 15/50

4/4  1s 89ms/step - loss: 670.4318 - mae: 19.9791 - val_loss: 2953.6880 - val

_mae: 52.0690 - learning_rate: 5.9049e-04

Epoch 16/50

4/4  0s 94ms/step - loss: 498.7182 - mae: 17.1536 - val_loss: 2621.9731 - val

_mae: 49.0222 - learning_rate: 5.3144e-04

Epoch 17/50

4/4  1s 98ms/step - loss: 448.3608 - mae: 16.8811 - val_loss: 2257.4490 - val

_mae: 45.3009 - learning_rate: 4.7830e-04

Epoch 18/50

4/4  1s 76ms/step - loss: 612.5913 - mae: 19.0505 - val_loss: 2091.1819 - val

_mae: 43.4571 - learning_rate: 4.3047e-04

Epoch 19/50

4/4  0s 75ms/step - loss: 546.2827 - mae: 17.9475 - val_loss: 1721.7804 - val

_mae: 39.0793 - learning_rate: 3.8742e-04

Epoch 20/50

4/4  0s 77ms/step - loss: 586.3668 - mae: 19.3537 - val_loss: 1150.4028 - val

_mae: 31.0683 - learning_rate: 3.4868e-04

Epoch 21/50

4/4  0s 68ms/step - loss: 476.1397 - mae: 17.3973 - val_loss: 1121.2126 - val

_mae: 30.5567 - learning_rate: 3.1381e-04

Epoch 22/50

4/4  0s 76ms/step - loss: 577.5326 - mae: 18.7789 - val_loss: 1079.1086 - val

_mae: 29.8664 - learning_rate: 2.8243e-04

Epoch 23/50

4/4  0s 60ms/step - loss: 597.0236 - mae: 18.4481 - val_loss: 1160.8728 - val

_mae: 31.2255 - learning_rate: 2.5419e-04

Epoch 24/50

4/4 ————— 0s 61ms/step - loss: 587.0303 - mae: 19.1852 - val_loss: 1101.1472 - val_mae: 30.2716 - learning_rate: 2.2877e-04
Epoch 25/50

4/4 ————— 0s 76ms/step - loss: 536.8317 - mae: 19.5637 - val_loss: 735.9532 - val_mae: 23.9581 - learning_rate: 2.0589e-04
Epoch 26/50

4/4 ————— 0s 66ms/step - loss: 475.0698 - mae: 17.1607 - val_loss: 542.0558 - val_mae: 19.8409 - learning_rate: 1.8530e-04
Epoch 27/50

4/4 ————— 0s 69ms/step - loss: 561.4478 - mae: 18.5792 - val_loss: 527.3737 - val_mae: 19.4999 - learning_rate: 1.6677e-04
Epoch 28/50

4/4 ————— 0s 61ms/step - loss: 507.9600 - mae: 17.7362 - val_loss: 571.5165 - val_mae: 20.4837 - learning_rate: 1.5009e-04
Epoch 29/50

4/4 ————— 0s 60ms/step - loss: 658.4516 - mae: 20.0748 - val_loss: 586.9733 - val_mae: 20.8128 - learning_rate: 1.3509e-04
Epoch 30/50

4/4 ————— 0s 76ms/step - loss: 548.4027 - mae: 19.0834 - val_loss: 468.0995 - val_mae: 18.0050 - learning_rate: 1.2158e-04
Epoch 31/50

4/4 ————— 0s 78ms/step - loss: 553.3160 - mae: 18.8666 - val_loss: 384.4160 - val_mae: 16.2066 - learning_rate: 1.0942e-04
Epoch 32/50

4/4 ————— 0s 68ms/step - loss: 513.1725 - mae: 17.9010 - val_loss: 349.9499 - val_mae: 15.3594 - learning_rate: 9.8477e-05
Epoch 33/50

4/4 ————— 0s 66ms/step - loss: 511.1353 - mae: 18.0398 - val_loss: 344.1049 - val_mae: 15.1853 - learning_rate: 8.8629e-05
Epoch 34/50

4/4 ————— 0s 60ms/step - loss: 518.8600 - mae: 17.7909 - val_loss: 349.6287 - val_mae: 15.3521 - learning_rate: 7.9766e-05
Epoch 35/50

4/4 ————— 0s 63ms/step - loss: 490.6287 - mae: 17.6343 - val_loss: 348.4805 - val_mae: 15.3170 - learning_rate: 7.1790e-05
Epoch 36/50

4/4 ————— 0s 65ms/step - loss: 487.7240 - mae: 17.0658 - val_loss: 336.2145 - val_mae: 14.9220 - learning_rate: 6.4611e-05
Epoch 37/50

4/4 ————— 0s 78ms/step - loss: 479.8369 - mae: 18.4950 - val_loss: 320.6228 - val_mae: 14.2694 - learning_rate: 5.8150e-05
Epoch 38/50

4/4 ————— 0s 76ms/step - loss: 459.6926 - mae: 16.9172 - val_loss: 312.6107 - val_mae: 13.7756 - learning_rate: 5.2335e-05
Epoch 39/50

4/4 ————— 0s 67ms/step - loss: 524.9183 - mae: 18.8745 - val_loss: 309.0930 - val_mae: 13.4881 - learning_rate: 4.7101e-05
Epoch 40/50

4/4 ————— 0s 64ms/step - loss: 566.4651 - mae: 18.8758 - val_loss: 306.8513 - val_mae: 13.1518 - learning_rate: 4.2391e-05
Epoch 41/50

4/4 ————— 0s 65ms/step - loss: 470.4537 - mae: 16.8359 - val_loss: 306.5874 - val_mae: 12.8619 - learning_rate: 3.8152e-05
Epoch 42/50

4/4 ————— 0s 61ms/step - loss: 506.3005 - mae: 17.9279 - val_loss: 306.6512 - val_mae: 12.7695 - learning_rate: 3.4337e-05
Epoch 43/50

4/4 ————— 0s 77ms/step - loss: 419.3147 - mae: 16.7861 - val_loss: 306.2431 - val_mae: 12.8636 - learning_rate: 3.0903e-05

Epoch 44/50

4/4  0s 71ms/step - loss: 482.0371 - mae: 17.1252 - val_loss: 306.2724 - val_mae: 13.0186 - learning_rate: 2.7813e-05

Epoch 45/50

4/4  0s 59ms/step - loss: 504.1823 - mae: 17.8026 - val_loss: 306.3095 - val_mae: 13.0027 - learning_rate: 2.5032e-05

Epoch 46/50

4/4  0s 74ms/step - loss: 572.5278 - mae: 18.2241 - val_loss: 306.4326 - val_mae: 12.8762 - learning_rate: 2.2528e-05

Epoch 47/50

4/4  0s 60ms/step - loss: 409.4178 - mae: 16.3797 - val_loss: 307.1555 - val_mae: 12.7227 - learning_rate: 2.0276e-05

Epoch 48/50

4/4  0s 59ms/step - loss: 479.0528 - mae: 18.1719 - val_loss: 308.7180 - val_mae: 12.5545 - learning_rate: 1.8248e-05

Out[]: <keras.src.callbacks.history.History at 0x7f4048189b10>

In []: `from sklearn.metrics import r2_score # Importing r2_score`

Inside the K-Fold loop after predictions:

```
for train_index, val_index in kf.split(X_train):
    print(f"\nTraining on fold {fold}...")
```

Split data into training and validation based on the fold indices

```
X_train_fold, X_val_fold = X_train[train_index], X_train[val_index]
y_train_fold, y_val_fold = y_train[train_index], y_train[val_index]
```

Build the model for the current fold

```
model = build_model(input_shape)
```

Train the model for the current fold

```
history = model.fit(X_train_fold, y_train_fold, epochs=50, batch_size=32,
                    validation_data=(X_val_fold, y_val_fold),
                    callbacks=[lr_scheduler, early_stopping])
```

Evaluate the model on the validation set of the current fold

```
val_loss, val_mae = model.evaluate(X_val_fold, y_val_fold)
print(f"Fold {fold} - Validation Loss: {val_loss}, Validation MAE: {val_mae}")
```

Make predictions on the validation set and calculate RMSE

```
y_val_pred = model.predict(X_val_fold)
fold_rmse = rmse(y_val_fold, y_val_pred)
print(f"Fold {fold} - RMSE: {fold_rmse}")
```

Calculate R² for the current fold

```
r2 = r2_score(y_val_fold, y_val_pred)
print(f"Fold {fold} - R2: {r2}")
```

Save the result for this fold (Loss, MAE, RMSE, R²)

```
#results.append((val_loss, val_mae, fold_rmse, r2)) # Ensure 4 values are appended
results.append((val_mae, fold_rmse, r2))
```

```
fold += 1
```

After K-Fold, calculate the average, min, and max results

```
#val_losses = [result[0] for result in results]
```

```
val_maes = [result[0] for result in results]
```

```
rmse_values = [result[1] for result in results]
```

```
r2_values = [result[2] for result in results]
```

```
print("\nK-Fold Cross-Validation Results:")
#print(f"Average Validation Loss: {np.mean(val_losses)}")
print(f"Average Validation MAE: {np.mean(val_maes)}")
print(f"Average Validation RMSE: {np.mean(rmse_values)}")
print(f"Average R2: {np.mean(r2_values)}")
print(f"Min RMSE: {np.min(rmse_values)}")
print(f"Max RMSE: {np.max(rmse_values)}")
print(f"Min R2: {np.min(r2_values)}")
print(f"Max R2: {np.max(r2_values)}")
```


Training on fold 9...

Epoch 1/50

4/4  5s 233ms/step - loss: 8305.4082 - mae: 79.2733 - val_loss: 1067301.7500
- val_mae: 1032.2889 - learning_rate: 0.0010

Epoch 2/50

4/4  0s 70ms/step - loss: 1406.8281 - mae: 31.7250 - val_loss: 121770.6953 -
val_mae: 348.4603 - learning_rate: 0.0010

Epoch 3/50

4/4  0s 81ms/step - loss: 1298.8771 - mae: 28.4228 - val_loss: 340095.4375 -
val_mae: 582.6935 - learning_rate: 0.0010

Epoch 4/50

4/4  1s 68ms/step - loss: 1524.3379 - mae: 33.6893 - val_loss: 122734.7812 -
val_mae: 349.8127 - learning_rate: 0.0010

Epoch 5/50

4/4  0s 62ms/step - loss: 729.7260 - mae: 20.9976 - val_loss: 36911.3789 - va
l_mae: 191.3367 - learning_rate: 0.0010

Epoch 6/50

4/4  0s 69ms/step - loss: 789.3710 - mae: 21.5328 - val_loss: 52296.4375 - va
l_mae: 228.0107 - learning_rate: 0.0010

Epoch 7/50

4/4  0s 58ms/step - loss: 732.0455 - mae: 21.6478 - val_loss: 42005.0352 - va
l_mae: 204.1902 - learning_rate: 0.0010

Epoch 8/50

4/4  0s 74ms/step - loss: 667.6406 - mae: 20.9494 - val_loss: 16291.0771 - va
l_mae: 126.4177 - learning_rate: 0.0010

Epoch 9/50

4/4  0s 80ms/step - loss: 731.1271 - mae: 21.1276 - val_loss: 11979.4180 - va
l_mae: 108.0199 - learning_rate: 0.0010

Epoch 10/50

4/4  0s 59ms/step - loss: 530.1679 - mae: 18.4839 - val_loss: 17903.0918 - va
l_mae: 132.6021 - learning_rate: 0.0010

Epoch 11/50

4/4  0s 73ms/step - loss: 616.0859 - mae: 19.8547 - val_loss: 9904.7451 - val
_mae: 97.9690 - learning_rate: 9.0000e-04

Epoch 12/50

4/4  0s 63ms/step - loss: 639.8785 - mae: 19.5039 - val_loss: 4413.7363 - val
_mae: 64.1770 - learning_rate: 8.1000e-04

Epoch 13/50

4/4  0s 79ms/step - loss: 631.9885 - mae: 19.8991 - val_loss: 4352.8975 - val
_mae: 63.7947 - learning_rate: 7.2900e-04

Epoch 14/50

4/4  0s 64ms/step - loss: 623.8530 - mae: 19.4949 - val_loss: 3902.0864 - val
_mae: 60.2085 - learning_rate: 6.5610e-04

Epoch 15/50

4/4  0s 73ms/step - loss: 574.4601 - mae: 18.5812 - val_loss: 3458.3672 - val
_mae: 56.4011 - learning_rate: 5.9049e-04

Epoch 16/50

4/4  0s 69ms/step - loss: 616.8352 - mae: 19.0540 - val_loss: 3913.2668 - val
_mae: 60.2448 - learning_rate: 5.3144e-04

Epoch 17/50

4/4  0s 67ms/step - loss: 634.7372 - mae: 21.1259 - val_loss: 3090.0786 - val
_mae: 52.9262 - learning_rate: 4.7830e-04

Epoch 18/50

4/4  0s 63ms/step - loss: 472.8171 - mae: 16.1030 - val_loss: 2317.3225 - val
_mae: 45.0059 - learning_rate: 4.3047e-04

Epoch 19/50

4/4  0s 73ms/step - loss: 546.7811 - mae: 17.8860 - val_loss: 1148.2938 - val
_mae: 29.2301 - learning_rate: 3.8742e-04

Epoch 20/50

4/4 ————— 0s 78ms/step - loss: 638.3012 - mae: 20.8215 - val_loss: 558.6957 - val_mae: 19.9178 - learning_rate: 3.4868e-04
Epoch 21/50
4/4 ————— 0s 60ms/step - loss: 705.8624 - mae: 19.3150 - val_loss: 776.1049 - val_mae: 23.3241 - learning_rate: 3.1381e-04
Epoch 22/50
4/4 ————— 0s 58ms/step - loss: 506.6588 - mae: 17.9835 - val_loss: 1453.4901 - val_mae: 34.1856 - learning_rate: 2.8243e-04
Epoch 23/50
4/4 ————— 0s 59ms/step - loss: 485.3655 - mae: 17.6899 - val_loss: 1649.2046 - val_mae: 36.9503 - learning_rate: 2.5419e-04
Epoch 24/50
4/4 ————— 0s 58ms/step - loss: 606.0009 - mae: 19.4548 - val_loss: 1053.1189 - val_mae: 27.8117 - learning_rate: 2.2877e-04
Epoch 25/50
4/4 ————— 0s 69ms/step - loss: 557.8220 - mae: 18.4070 - val_loss: 683.0699 - val_mae: 21.8027 - learning_rate: 2.0589e-04
1/1 ————— 0s 49ms/step - loss: 558.6957 - mae: 19.9178
Fold 9 - Validation Loss: 558.6956787109375, Validation MAE: 19.917818069458008
1/1 ————— 0s 234ms/step
Fold 9 - RMSE: 23.63674551081852
Fold 9 - R²: -1.1210260391235352

Training on fold 10...

Epoch 1/50
4/4 ————— 5s 235ms/step - loss: 8026.7617 - mae: 78.0985 - val_loss: 773275.6875 - val_mae: 877.8126 - learning_rate: 0.0010
Epoch 2/50
4/4 ————— 1s 79ms/step - loss: 1154.6272 - mae: 27.3398 - val_loss: 123834.0781 - val_mae: 350.8843 - learning_rate: 0.0010
Epoch 3/50
4/4 ————— 0s 66ms/step - loss: 1345.7859 - mae: 29.2995 - val_loss: 225327.5312 - val_mae: 473.7060 - learning_rate: 0.0010
Epoch 4/50
4/4 ————— 0s 63ms/step - loss: 949.7798 - mae: 25.8729 - val_loss: 58259.6914 - val_mae: 240.4828 - learning_rate: 0.0010
Epoch 5/50
4/4 ————— 0s 74ms/step - loss: 803.7652 - mae: 20.4948 - val_loss: 31006.0723 - val_mae: 175.1072 - learning_rate: 0.0010
Epoch 6/50
4/4 ————— 0s 58ms/step - loss: 622.3649 - mae: 18.2677 - val_loss: 42883.2422 - val_mae: 206.2127 - learning_rate: 0.0010
Epoch 7/50
4/4 ————— 0s 65ms/step - loss: 818.5237 - mae: 23.8352 - val_loss: 19812.0957 - val_mae: 139.7053 - learning_rate: 0.0010
Epoch 8/50
4/4 ————— 0s 63ms/step - loss: 536.7620 - mae: 17.5976 - val_loss: 13845.8037 - val_mae: 116.4939 - learning_rate: 0.0010
Epoch 9/50
4/4 ————— 0s 73ms/step - loss: 654.9652 - mae: 19.2196 - val_loss: 10106.3076 - val_mae: 99.2056 - learning_rate: 0.0010
Epoch 10/50
4/4 ————— 0s 68ms/step - loss: 609.2612 - mae: 19.5363 - val_loss: 4894.7622 - val_mae: 68.2064 - learning_rate: 0.0010
Epoch 11/50
4/4 ————— 0s 74ms/step - loss: 590.6310 - mae: 19.1467 - val_loss: 4678.7437 - val_mae: 66.6266 - learning_rate: 9.0000e-04
Epoch 12/50
4/4 ————— 0s 73ms/step - loss: 616.5710 - mae: 19.9374 - val_loss: 4219.1836 - val

_mae: 63.1030 - learning_rate: 8.1000e-04
Epoch 13/50
4/4  0s 69ms/step - loss: 599.0474 - mae: 19.6347 - val_loss: 1282.8347 - val
_mae: 32.5390 - learning_rate: 7.2900e-04
Epoch 14/50
4/4  0s 57ms/step - loss: 569.8426 - mae: 17.9581 - val_loss: 1903.5013 - val
_mae: 40.9342 - learning_rate: 6.5610e-04
Epoch 15/50
4/4  0s 58ms/step - loss: 551.9564 - mae: 17.6732 - val_loss: 2594.7698 - val
_mae: 48.6376 - learning_rate: 5.9049e-04
Epoch 16/50
4/4  0s 64ms/step - loss: 656.9124 - mae: 21.4708 - val_loss: 1040.8951 - val
_mae: 28.6405 - learning_rate: 5.3144e-04
Epoch 17/50
4/4  0s 66ms/step - loss: 622.2424 - mae: 19.3541 - val_loss: 688.1184 - val
_mae: 22.0187 - learning_rate: 4.7830e-04
Epoch 18/50
4/4  0s 69ms/step - loss: 739.7510 - mae: 20.1362 - val_loss: 709.4175 - val
_mae: 22.3645 - learning_rate: 4.3047e-04
Epoch 19/50
4/4  0s 67ms/step - loss: 521.3505 - mae: 17.5890 - val_loss: 530.0580 - val
_mae: 19.1120 - learning_rate: 3.8742e-04
Epoch 20/50
4/4  0s 65ms/step - loss: 617.5027 - mae: 19.4466 - val_loss: 268.4070 - val
_mae: 14.6852 - learning_rate: 3.4868e-04
Epoch 21/50
4/4  0s 63ms/step - loss: 563.2733 - mae: 17.9896 - val_loss: 220.0170 - val
_mae: 13.8719 - learning_rate: 3.1381e-04
Epoch 22/50
4/4  0s 69ms/step - loss: 641.2278 - mae: 19.3977 - val_loss: 254.2257 - val
_mae: 14.5190 - learning_rate: 2.8243e-04
Epoch 23/50
4/4  0s 57ms/step - loss: 602.7463 - mae: 19.0920 - val_loss: 237.4349 - val
_mae: 14.2913 - learning_rate: 2.5419e-04
Epoch 24/50
4/4  0s 60ms/step - loss: 628.6296 - mae: 19.3068 - val_loss: 252.6072 - val
_mae: 14.5420 - learning_rate: 2.2877e-04
Epoch 25/50
4/4  0s 77ms/step - loss: 555.0873 - mae: 18.1156 - val_loss: 214.9123 - val
_mae: 13.6716 - learning_rate: 2.0589e-04
Epoch 26/50
4/4  0s 62ms/step - loss: 434.0678 - mae: 16.8511 - val_loss: 220.5278 - val
_mae: 13.2221 - learning_rate: 1.8530e-04
Epoch 27/50
4/4  0s 71ms/step - loss: 537.6763 - mae: 17.6815 - val_loss: 229.5642 - val
_mae: 13.0874 - learning_rate: 1.6677e-04
Epoch 28/50
4/4  0s 58ms/step - loss: 491.2682 - mae: 16.6238 - val_loss: 259.0168 - val
_mae: 13.1220 - learning_rate: 1.5009e-04
Epoch 29/50
4/4  0s 58ms/step - loss: 594.1484 - mae: 19.0542 - val_loss: 260.0629 - val
_mae: 13.1471 - learning_rate: 1.3509e-04
Epoch 30/50
4/4  0s 59ms/step - loss: 479.6461 - mae: 17.2962 - val_loss: 254.6795 - val
_mae: 13.1314 - learning_rate: 1.2158e-04
1/1  0s 56ms/step - loss: 214.9123 - mae: 13.6716
Fold 10 - Validation Loss: 214.91229248046875, Validation MAE: 13.67161750793457
1/1  0s 351ms/step
Fold 10 - RMSE: 14.659887101500031

Fold 10 - R²: 0.11388808488845825

Training on fold 11...

Epoch 1/50

4/4  5s 224ms/step - loss: 8628.7734 - mae: 80.2508 - val_loss: 1556976.7500
- val_mae: 1246.0037 - learning_rate: 0.0010

Epoch 2/50

4/4  1s 78ms/step - loss: 2993.0645 - mae: 45.4780 - val_loss: 101286.2344 -
val_mae: 317.3612 - learning_rate: 0.0010

Epoch 3/50

4/4  0s 62ms/step - loss: 1732.3517 - mae: 35.4196 - val_loss: 167148.7344 -
val_mae: 408.0026 - learning_rate: 0.0010

Epoch 4/50

4/4  0s 69ms/step - loss: 957.3543 - mae: 25.4066 - val_loss: 165854.8906 - v
al_mae: 406.4406 - learning_rate: 0.0010

Epoch 5/50

4/4  0s 74ms/step - loss: 1374.9243 - mae: 31.2903 - val_loss: 42997.0391 - v
al_mae: 206.5014 - learning_rate: 0.0010

Epoch 6/50

4/4  0s 65ms/step - loss: 783.0679 - mae: 22.0067 - val_loss: 23162.9863 - va
l_mae: 151.2264 - learning_rate: 0.0010

Epoch 7/50

4/4  0s 58ms/step - loss: 868.0739 - mae: 22.6947 - val_loss: 37303.6328 - va
l_mae: 192.2991 - learning_rate: 0.0010

Epoch 8/50

4/4  0s 59ms/step - loss: 650.2927 - mae: 20.5501 - val_loss: 26067.8789 - va
l_mae: 160.5633 - learning_rate: 0.0010

Epoch 9/50

4/4  0s 74ms/step - loss: 667.5745 - mae: 20.2942 - val_loss: 10357.8486 - va
l_mae: 100.6045 - learning_rate: 0.0010

Epoch 10/50

4/4  0s 64ms/step - loss: 751.0022 - mae: 21.0734 - val_loss: 9259.9111 - val
_mae: 95.0452 - learning_rate: 0.0010

Epoch 11/50

4/4  0s 69ms/step - loss: 618.7465 - mae: 19.2031 - val_loss: 11272.4951 - va
l_mae: 105.0853 - learning_rate: 9.0000e-04

Epoch 12/50

4/4  0s 62ms/step - loss: 726.9308 - mae: 20.4080 - val_loss: 9785.6875 - val
_mae: 97.7663 - learning_rate: 8.1000e-04

Epoch 13/50

4/4  0s 74ms/step - loss: 648.7267 - mae: 19.9410 - val_loss: 4320.6636 - val
_mae: 64.0890 - learning_rate: 7.2900e-04

Epoch 14/50

4/4  0s 64ms/step - loss: 673.7518 - mae: 19.2805 - val_loss: 3483.4597 - val
_mae: 57.1768 - learning_rate: 6.5610e-04

Epoch 15/50

4/4  0s 60ms/step - loss: 582.9560 - mae: 18.2424 - val_loss: 5699.0122 - val
_mae: 73.9767 - learning_rate: 5.9049e-04

Epoch 16/50

4/4  0s 58ms/step - loss: 685.3917 - mae: 21.0733 - val_loss: 3767.5444 - val
_mae: 59.5329 - learning_rate: 5.3144e-04

Epoch 17/50

4/4  0s 77ms/step - loss: 609.1572 - mae: 20.4883 - val_loss: 2421.2764 - val
_mae: 46.9262 - learning_rate: 4.7830e-04

Epoch 18/50

4/4  0s 74ms/step - loss: 583.0327 - mae: 19.2068 - val_loss: 1603.2103 - val
_mae: 37.2685 - learning_rate: 4.3047e-04

Epoch 19/50

4/4  0s 63ms/step - loss: 511.8466 - mae: 17.4388 - val_loss: 1267.4266 - val

_mae: 32.4857 - learning_rate: 3.8742e-04
Epoch 20/50
4/4  0s 57ms/step - loss: 669.8481 - mae: 20.3146 - val_loss: 1328.9071 - val
_mae: 33.4296 - learning_rate: 3.4868e-04
Epoch 21/50
4/4  0s 59ms/step - loss: 553.0771 - mae: 18.6882 - val_loss: 1841.1108 - val
_mae: 40.3418 - learning_rate: 3.1381e-04
Epoch 22/50
4/4  0s 70ms/step - loss: 523.8575 - mae: 17.8891 - val_loss: 2208.1982 - val
_mae: 44.6501 - learning_rate: 2.8243e-04
Epoch 23/50
4/4  0s 58ms/step - loss: 573.6951 - mae: 18.9596 - val_loss: 1642.7681 - val
_mae: 37.8404 - learning_rate: 2.5419e-04
Epoch 24/50
4/4  0s 63ms/step - loss: 585.9088 - mae: 20.1865 - val_loss: 963.9030 - val
_mae: 27.9492 - learning_rate: 2.2877e-04
Epoch 25/50
4/4  0s 76ms/step - loss: 674.6087 - mae: 21.3712 - val_loss: 753.8972 - val
_mae: 24.4405 - learning_rate: 2.0589e-04
Epoch 26/50
4/4  0s 86ms/step - loss: 591.2842 - mae: 18.8554 - val_loss: 625.7036 - val
_mae: 22.0260 - learning_rate: 1.8530e-04
Epoch 27/50
4/4  0s 87ms/step - loss: 781.7946 - mae: 21.1137 - val_loss: 660.9000 - val
_mae: 22.6771 - learning_rate: 1.6677e-04
Epoch 28/50
4/4  0s 89ms/step - loss: 548.6340 - mae: 18.0039 - val_loss: 701.8936 - val
_mae: 23.4714 - learning_rate: 1.5009e-04
Epoch 29/50
4/4  1s 87ms/step - loss: 500.0380 - mae: 18.2589 - val_loss: 685.1877 - val
_mae: 23.1461 - learning_rate: 1.3509e-04
Epoch 30/50
4/4  0s 93ms/step - loss: 589.3688 - mae: 19.1985 - val_loss: 579.7948 - val
_mae: 21.2785 - learning_rate: 1.2158e-04
Epoch 31/50
4/4  1s 102ms/step - loss: 543.1683 - mae: 17.9909 - val_loss: 474.1952 - val
_mae: 19.4520 - learning_rate: 1.0942e-04
Epoch 32/50
4/4  1s 78ms/step - loss: 575.5191 - mae: 17.9701 - val_loss: 393.6383 - val
_mae: 17.8120 - learning_rate: 9.8477e-05
Epoch 33/50
4/4  0s 74ms/step - loss: 666.9490 - mae: 21.2861 - val_loss: 352.6382 - val
_mae: 16.8822 - learning_rate: 8.8629e-05
Epoch 34/50
4/4  0s 63ms/step - loss: 831.3940 - mae: 22.2186 - val_loss: 337.9975 - val
_mae: 16.5595 - learning_rate: 7.9766e-05
Epoch 35/50
4/4  0s 79ms/step - loss: 674.4141 - mae: 19.9118 - val_loss: 317.2642 - val
_mae: 16.0731 - learning_rate: 7.1790e-05
Epoch 36/50
4/4  0s 65ms/step - loss: 480.9203 - mae: 17.2288 - val_loss: 299.7462 - val
_mae: 15.6291 - learning_rate: 6.4611e-05
Epoch 37/50
4/4  0s 68ms/step - loss: 655.7324 - mae: 20.1833 - val_loss: 303.8013 - val
_mae: 15.7417 - learning_rate: 5.8150e-05
Epoch 38/50
4/4  0s 67ms/step - loss: 482.7982 - mae: 17.4864 - val_loss: 294.8163 - val
_mae: 15.5058 - learning_rate: 5.2335e-05
Epoch 39/50

4/4 ————— 0s 79ms/step - loss: 482.0471 - mae: 17.4940 - val_loss: 270.4779 - val_mae: 14.9006 - learning_rate: 4.7101e-05
Epoch 40/50
4/4 ————— 0s 74ms/step - loss: 579.8084 - mae: 18.1799 - val_loss: 249.2103 - val_mae: 14.2875 - learning_rate: 4.2391e-05
Epoch 41/50
4/4 ————— 0s 74ms/step - loss: 699.1600 - mae: 19.8211 - val_loss: 234.5075 - val_mae: 13.8646 - learning_rate: 3.8152e-05
Epoch 42/50
4/4 ————— 0s 82ms/step - loss: 676.8032 - mae: 20.1322 - val_loss: 226.9034 - val_mae: 13.6214 - learning_rate: 3.4337e-05
Epoch 43/50
4/4 ————— 0s 66ms/step - loss: 478.3742 - mae: 16.6824 - val_loss: 220.5887 - val_mae: 13.3982 - learning_rate: 3.0903e-05
Epoch 44/50
4/4 ————— 0s 63ms/step - loss: 532.5935 - mae: 18.6761 - val_loss: 213.4534 - val_mae: 13.1103 - learning_rate: 2.7813e-05
Epoch 45/50
4/4 ————— 0s 77ms/step - loss: 635.4699 - mae: 19.6945 - val_loss: 205.0710 - val_mae: 12.6901 - learning_rate: 2.5032e-05
Epoch 46/50
4/4 ————— 0s 64ms/step - loss: 502.1243 - mae: 17.2447 - val_loss: 199.6398 - val_mae: 12.3263 - learning_rate: 2.2528e-05
Epoch 47/50
4/4 ————— 0s 63ms/step - loss: 622.9406 - mae: 19.5588 - val_loss: 196.2864 - val_mae: 11.9974 - learning_rate: 2.0276e-05
Epoch 48/50
4/4 ————— 0s 77ms/step - loss: 584.5947 - mae: 19.2415 - val_loss: 194.5346 - val_mae: 11.7160 - learning_rate: 1.8248e-05
Epoch 49/50
4/4 ————— 0s 66ms/step - loss: 493.0761 - mae: 18.3491 - val_loss: 194.0306 - val_mae: 11.5934 - learning_rate: 1.6423e-05
Epoch 50/50
4/4 ————— 0s 64ms/step - loss: 533.7434 - mae: 17.7839 - val_loss: 193.9647 - val_mae: 11.5111 - learning_rate: 1.4781e-05
1/1 ————— 0s 47ms/step - loss: 193.9647 - mae: 11.5111
Fold 11 - Validation Loss: 193.96473693847656, Validation MAE: 11.511072158813477
1/1 ————— 0s 226ms/step
Fold 11 - RMSE: 13.927122158460046
Fold 11 - R²: -0.01829814910888672

Training on fold 12...

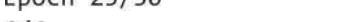
Epoch 1/50
4/4 ————— 7s 308ms/step - loss: 8935.4141 - mae: 81.2584 - val_loss: 544922.1875 - val_mae: 736.2350 - learning_rate: 0.0010
Epoch 2/50
4/4 ————— 1s 81ms/step - loss: 1364.3567 - mae: 30.3518 - val_loss: 79809.5000 - val_mae: 281.1609 - learning_rate: 0.0010
Epoch 3/50
4/4 ————— 1s 66ms/step - loss: 1274.5347 - mae: 28.2358 - val_loss: 205351.9219 - val_mae: 451.9177 - learning_rate: 0.0010
Epoch 4/50
4/4 ————— 0s 69ms/step - loss: 1713.2377 - mae: 34.1463 - val_loss: 71797.7500 - val_mae: 266.7985 - learning_rate: 0.0010
Epoch 5/50
4/4 ————— 0s 74ms/step - loss: 838.9636 - mae: 22.7108 - val_loss: 20093.4434 - val_mae: 140.3710 - learning_rate: 0.0010
Epoch 6/50
4/4 ————— 0s 59ms/step - loss: 802.6249 - mae: 22.1391 - val_loss: 34719.9531 - val_mae: 138.8510 - learning_rate: 0.0010

l_mae: 185.1998 - learning_rate: 0.0010
Epoch 7/50
4/4  0s 63ms/step - loss: 983.0050 - mae: 25.3562 - val_loss: 25146.7656 - val_mae: 157.4085 - learning_rate: 0.0010
Epoch 8/50
4/4  0s 69ms/step - loss: 617.0287 - mae: 20.2797 - val_loss: 9814.3066 - val_mae: 97.5061 - learning_rate: 0.0010
Epoch 9/50
4/4  0s 65ms/step - loss: 803.0508 - mae: 22.6417 - val_loss: 9331.9141 - val_mae: 95.0395 - learning_rate: 0.0010
Epoch 10/50
4/4  0s 68ms/step - loss: 672.1095 - mae: 21.4026 - val_loss: 8485.0820 - val_mae: 90.5208 - learning_rate: 0.0010
Epoch 11/50
4/4  0s 67ms/step - loss: 481.0233 - mae: 17.0551 - val_loss: 3482.7039 - val_mae: 56.7152 - learning_rate: 9.0000e-04
Epoch 12/50
4/4  0s 63ms/step - loss: 655.8925 - mae: 19.9895 - val_loss: 3156.4451 - val_mae: 53.7894 - learning_rate: 8.1000e-04
Epoch 13/50
4/4  0s 64ms/step - loss: 570.6921 - mae: 19.0845 - val_loss: 2494.3000 - val_mae: 47.2729 - learning_rate: 7.2900e-04
Epoch 14/50
4/4  0s 65ms/step - loss: 580.8737 - mae: 19.2736 - val_loss: 1851.0509 - val_mae: 39.9249 - learning_rate: 6.5610e-04
Epoch 15/50
4/4  0s 61ms/step - loss: 494.5362 - mae: 17.5471 - val_loss: 1954.2114 - val_mae: 41.1896 - learning_rate: 5.9049e-04
Epoch 16/50
4/4  0s 65ms/step - loss: 526.8792 - mae: 19.2346 - val_loss: 1444.7314 - val_mae: 34.4960 - learning_rate: 5.3144e-04
Epoch 17/50
4/4  0s 76ms/step - loss: 540.5735 - mae: 18.8553 - val_loss: 810.3942 - val_mae: 25.4032 - learning_rate: 4.7830e-04
Epoch 18/50
4/4  0s 62ms/step - loss: 463.3078 - mae: 16.2864 - val_loss: 1091.6569 - val_mae: 29.7882 - learning_rate: 4.3047e-04
Epoch 19/50
4/4  0s 70ms/step - loss: 613.0810 - mae: 19.7046 - val_loss: 1017.9339 - val_mae: 28.6804 - learning_rate: 3.8742e-04
Epoch 20/50
4/4  0s 74ms/step - loss: 542.6888 - mae: 19.0100 - val_loss: 612.9493 - val_mae: 21.8435 - learning_rate: 3.4868e-04
Epoch 21/50
4/4  0s 68ms/step - loss: 562.7228 - mae: 18.7527 - val_loss: 452.3107 - val_mae: 19.2428 - learning_rate: 3.1381e-04
Epoch 22/50
4/4  0s 77ms/step - loss: 551.7210 - mae: 18.4793 - val_loss: 431.7058 - val_mae: 18.9244 - learning_rate: 2.8243e-04
Epoch 23/50
4/4  0s 75ms/step - loss: 493.9081 - mae: 17.3173 - val_loss: 367.8645 - val_mae: 17.7084 - learning_rate: 2.5419e-04
Epoch 24/50
4/4  0s 66ms/step - loss: 614.3710 - mae: 19.8531 - val_loss: 278.2586 - val_mae: 15.2712 - learning_rate: 2.2877e-04
Epoch 25/50
4/4  0s 61ms/step - loss: 559.3787 - mae: 18.8378 - val_loss: 289.6848 - val_mae: 15.6831 - learning_rate: 2.0589e-04
Epoch 26/50

4/4 ————— 0s 71ms/step - loss: 526.0140 - mae: 17.9867 - val_loss: 291.4266 - val_mae: 15.7414 - learning_rate: 1.8530e-04
Epoch 27/50
4/4 ————— 0s 70ms/step - loss: 530.2488 - mae: 17.4286 - val_loss: 286.5713 - val_mae: 15.5935 - learning_rate: 1.6677e-04
Epoch 28/50
4/4 ————— 0s 71ms/step - loss: 455.5963 - mae: 16.5255 - val_loss: 278.7884 - val_mae: 15.3342 - learning_rate: 1.5009e-04
Epoch 29/50
4/4 ————— 0s 77ms/step - loss: 601.2828 - mae: 18.4882 - val_loss: 274.2334 - val_mae: 15.1659 - learning_rate: 1.3509e-04
Epoch 30/50
4/4 ————— 0s 75ms/step - loss: 507.9286 - mae: 18.1201 - val_loss: 265.7389 - val_mae: 14.7948 - learning_rate: 1.2158e-04
Epoch 31/50
4/4 ————— 0s 67ms/step - loss: 677.7521 - mae: 19.5249 - val_loss: 253.9370 - val_mae: 13.8813 - learning_rate: 1.0942e-04
Epoch 32/50
4/4 ————— 0s 62ms/step - loss: 503.6049 - mae: 17.8797 - val_loss: 256.4682 - val_mae: 12.9579 - learning_rate: 9.8477e-05
Epoch 33/50
4/4 ————— 0s 71ms/step - loss: 494.8096 - mae: 17.0445 - val_loss: 263.5362 - val_mae: 12.7118 - learning_rate: 8.8629e-05
Epoch 34/50
4/4 ————— 0s 92ms/step - loss: 423.7408 - mae: 16.6810 - val_loss: 275.6456 - val_mae: 12.6518 - learning_rate: 7.9766e-05
Epoch 35/50
4/4 ————— 1s 84ms/step - loss: 618.3863 - mae: 19.1351 - val_loss: 289.2218 - val_mae: 12.7354 - learning_rate: 7.1790e-05
Epoch 36/50
4/4 ————— 1s 90ms/step - loss: 690.5670 - mae: 19.4961 - val_loss: 299.3535 - val_mae: 12.8199 - learning_rate: 6.4611e-05
1/1 ————— 0s 82ms/step - loss: 253.9370 - mae: 13.8813
Fold 12 - Validation Loss: 253.93699645996094, Validation MAE: 13.881336212158203
1/1 ————— 0s 394ms/step
Fold 12 - RMSE: 15.935400082458992
Fold 12 - R²: -0.16850197315216064

Training on fold 13...

Epoch 1/50
4/4 ————— 4s 229ms/step - loss: 7690.9917 - mae: 74.5637 - val_loss: 539181.2500 - val_mae: 733.0803 - learning_rate: 0.0010
Epoch 2/50
4/4 ————— 0s 69ms/step - loss: 965.4561 - mae: 25.1601 - val_loss: 186985.8125 - val_mae: 431.3055 - learning_rate: 0.0010
Epoch 3/50
4/4 ————— 0s 74ms/step - loss: 636.7362 - mae: 19.7696 - val_loss: 168620.2812 - val_mae: 409.5150 - learning_rate: 0.0010
Epoch 4/50
4/4 ————— 0s 63ms/step - loss: 840.1277 - mae: 23.9398 - val_loss: 28444.5508 - val_mae: 166.3837 - learning_rate: 0.0010
Epoch 5/50
4/4 ————— 0s 71ms/step - loss: 949.0214 - mae: 24.5644 - val_loss: 42244.2734 - val_mae: 203.6288 - learning_rate: 0.0010
Epoch 6/50
4/4 ————— 0s 75ms/step - loss: 515.6973 - mae: 18.2086 - val_loss: 26442.2500 - val_mae: 160.2629 - learning_rate: 0.0010
Epoch 7/50
4/4 ————— 0s 67ms/step - loss: 417.9032 - mae: 17.2448 - val_loss: 15034.7754 - va

l_mae: 119.6157 - learning_rate: 0.0010
Epoch 8/50
4/4  0s 65ms/step - loss: 411.9264 - mae: 16.8033 - val_loss: 9834.4863 - val_mae: 95.4445 - learning_rate: 0.0010
Epoch 9/50
4/4  0s 74ms/step - loss: 340.5301 - mae: 14.4384 - val_loss: 7771.5625 - val_mae: 83.9109 - learning_rate: 0.0010
Epoch 10/50
4/4  0s 74ms/step - loss: 323.8788 - mae: 14.2966 - val_loss: 4441.3442 - val_mae: 61.4535 - learning_rate: 0.0010
Epoch 11/50
4/4  0s 77ms/step - loss: 483.4669 - mae: 18.5105 - val_loss: 3736.2744 - val_mae: 56.8834 - learning_rate: 9.0000e-04
Epoch 12/50
4/4  0s 76ms/step - loss: 313.7995 - mae: 14.3379 - val_loss: 2158.9751 - val_mae: 43.8004 - learning_rate: 8.1000e-04
Epoch 13/50
4/4  0s 79ms/step - loss: 378.0361 - mae: 15.6159 - val_loss: 1992.0553 - val_mae: 41.9584 - learning_rate: 7.2900e-04
Epoch 14/50
4/4  0s 68ms/step - loss: 370.9554 - mae: 15.8229 - val_loss: 1545.3419 - val_mae: 36.2973 - learning_rate: 6.5610e-04
Epoch 15/50
4/4  0s 78ms/step - loss: 432.0212 - mae: 17.3211 - val_loss: 911.6057 - val_mae: 24.3122 - learning_rate: 5.9049e-04
Epoch 16/50
4/4  0s 59ms/step - loss: 481.3494 - mae: 17.8840 - val_loss: 1290.8126 - val_mae: 32.0302 - learning_rate: 5.3144e-04
Epoch 17/50
4/4  0s 69ms/step - loss: 414.4630 - mae: 16.4833 - val_loss: 1883.0062 - val_mae: 40.5005 - learning_rate: 4.7830e-04
Epoch 18/50
4/4  0s 77ms/step - loss: 576.7986 - mae: 19.6054 - val_loss: 873.7877 - val_mae: 22.3659 - learning_rate: 4.3047e-04
Epoch 19/50
4/4  0s 72ms/step - loss: 435.3814 - mae: 15.8298 - val_loss: 873.9216 - val_mae: 22.2145 - learning_rate: 3.8742e-04
Epoch 20/50
4/4  0s 87ms/step - loss: 349.7108 - mae: 14.5740 - val_loss: 1058.5570 - val_mae: 27.4399 - learning_rate: 3.4868e-04
Epoch 21/50
4/4  0s 90ms/step - loss: 450.4263 - mae: 17.6625 - val_loss: 1015.3373 - val_mae: 26.5821 - learning_rate: 3.1381e-04
Epoch 22/50
4/4  0s 96ms/step - loss: 425.5193 - mae: 16.1395 - val_loss: 865.8605 - val_mae: 21.1204 - learning_rate: 2.8243e-04
Epoch 23/50
4/4  1s 108ms/step - loss: 506.2495 - mae: 17.5603 - val_loss: 857.3380 - val_mae: 20.7804 - learning_rate: 2.5419e-04
Epoch 24/50
4/4  0s 93ms/step - loss: 470.3465 - mae: 17.8667 - val_loss: 842.9352 - val_mae: 20.9268 - learning_rate: 2.2877e-04
Epoch 25/50
4/4  1s 96ms/step - loss: 432.8300 - mae: 17.7975 - val_loss: 854.9270 - val_mae: 22.2035 - learning_rate: 2.0589e-04
Epoch 26/50
4/4  0s 84ms/step - loss: 466.8942 - mae: 16.8474 - val_loss: 838.7405 - val_mae: 21.6788 - learning_rate: 1.8530e-04
Epoch 27/50

```

4/4 ----- 0s 66ms/step - loss: 436.2010 - mae: 16.7776 - val_loss: 827.2542 - val_
mae: 20.6777 - learning_rate: 1.6677e-04
Epoch 28/50
4/4 ----- 0s 59ms/step - loss: 433.7257 - mae: 17.6213 - val_loss: 828.1580 - val_
mae: 20.5446 - learning_rate: 1.5009e-04
Epoch 29/50
4/4 ----- 0s 79ms/step - loss: 444.0800 - mae: 16.8716 - val_loss: 825.9655 - val_
mae: 20.4995 - learning_rate: 1.3509e-04
Epoch 30/50
4/4 ----- 0s 59ms/step - loss: 372.0147 - mae: 16.3798 - val_loss: 828.3115 - val_
mae: 20.4340 - learning_rate: 1.2158e-04
Epoch 31/50
4/4 ----- 0s 70ms/step - loss: 357.4136 - mae: 15.2070 - val_loss: 842.7020 - val_
mae: 20.3043 - learning_rate: 1.0942e-04
Epoch 32/50
4/4 ----- 0s 63ms/step - loss: 391.7558 - mae: 16.0115 - val_loss: 863.2891 - val_
mae: 20.3005 - learning_rate: 9.8477e-05
Epoch 33/50
4/4 ----- 0s 59ms/step - loss: 405.8424 - mae: 16.5268 - val_loss: 863.1920 - val_
mae: 20.2914 - learning_rate: 8.8629e-05
Epoch 34/50
4/4 ----- 0s 60ms/step - loss: 358.2792 - mae: 15.9255 - val_loss: 864.0586 - val_
mae: 20.3002 - learning_rate: 7.9766e-05
1/1 ----- 0s 52ms/step - loss: 825.9655 - mae: 20.4995
Fold 13 - Validation Loss: 825.9654541015625, Validation MAE: 20.499486923217773
1/1 ----- 0s 255ms/step
Fold 13 - RMSE: 28.73961563956698
Fold 13 - R²: -0.11099541187286377

```

K-Fold Cross-Validation Results:

```

Average Validation MAE: 282.8998275756836
Average Validation RMSE: 15.818857161264916
Average R²: 14.310513667604019
Min RMSE: 10.944875717163086
Max RMSE: 28.73961563956698
Min R²: -1.1210260391235352
Max R²: 28.897985022009024

```

```

In [ ]: # Reshape X_test to the correct shape for LSTM: (samples, features, time_steps)
X_test_resaped = X_test.reshape((X_test.shape[0], X_test.shape[1], 1)) # (54, 2142, 1)

# Ensure y_test is 1D (it seems to already be 1D, but let's double-check)
y_test = y_test.flatten() # Flatten in case it is not 1D

# Check model summary to verify the expected input shape
final_model.summary()

# Evaluate the model on the test set
try:
    test_loss, test_mae = final_model.evaluate(X_test_resaped, y_test)
    print(f"\nFinal Test Loss: {test_loss}, Final Test MAE: {test_mae}")
except Exception as e:
    print(f"Error during evaluation: {e}")

# Make predictions on the test set
try:
    y_test_pred = final_model.predict(X_test_resaped)

    # Calculate RMSE manually
    test_rmse = rmse(y_test, y_test_pred)

```



```

print(f"Final Test RMSE: {test_rmse}")
except Exception as e:
    print(f"Error during prediction: {e}")

```

Model: "functional_35"

Layer (type)	Output Shape	Param #	Connected to
input_signal (InputLayer)	(None, 2142, 1)	0	-
conv1d_38 (Conv1D)	(None, 2140, 32)	128	input_signal[0][0]
batch_normalization_38 (BatchNormalization)	(None, 2140, 32)	128	conv1d_38[0][0]
max_pooling1d_38 (MaxPooling1D)	(None, 1070, 32)	0	batch_normalizati
dropout_68 (Dropout)	(None, 1070, 32)	0	max_pooling1d_38[
conv1d_39 (Conv1D)	(None, 1066, 64)	10,304	dropout_68[0][0]
batch_normalization_39 (BatchNormalization)	(None, 1066, 64)	256	conv1d_39[0][0]
max_pooling1d_39 (MaxPooling1D)	(None, 533, 64)	0	batch_normalizati
dropout_69 (Dropout)	(None, 533, 64)	0	max_pooling1d_39[
gru_36 (GRU)	(None, 533, 64)	24,960	dropout_69[0][0]
gru_37 (GRU)	(None, 64)	24,960	gru_36[0][0]
flatten_38 (Flatten)	(None, 34112)	0	dropout_69[0][0]
flatten_39 (Flatten)	(None, 64)	0	gru_37[0][0]
dense_65 (Dense)	(None, 128)	4,366,464	flatten_38[0][0]
dense_66 (Dense)	(None, 128)	8,320	flatten_39[0][0]
dropout_70 (Dropout)	(None, 128)	0	dense_65[0][0]
dropout_71 (Dropout)	(None, 128)	0	dense_66[0][0]
concatenate_19 (Concatenate)	(None, 256)	0	dropout_70[0][0], dropout_71[0][0]
dense_67 (Dense)	(None, 1)	257	concatenate_19[0]

Total params: 13,306,949 (50.76 MB)

Trainable params: 4,435,585 (16.92 MB)

Non-trainable params: 192 (768.00 B)

Optimizer params: 8,871,172 (33.84 MB)

Error during evaluation: as_list() is not defined on an unknown TensorShape.

2/2 ————— 0s 43ms/step

Final Test RMSE: 21.766790799292643

```
In [ ]: # Calculate the average RMSE across all folds
avg_rmse = np.mean(rmse_values)
print(f"\nAverage K-Fold RMSE: {avg_rmse}")
#return avg_rmse
```

Average K-Fold RMSE: 15.818857161264916

```
In [ ]: import numpy as np
import pandas as pd
from sklearn.model_selection import KFold
from sklearn.metrics import mean_absolute_error, mean_squared_error
from math import sqrt

# Function to Load data and labels from the previously processed data
# Assuming that 'X' contains the features and 'y' contains the labels
def load_data():
    # You can load the pre-split or augmented data here
    X = pd.read_csv('/content/PPG_Data/split_X_train.csv') # Replace with your actual data file
    y = pd.read_csv('/content/PPG_Data/split_y_train.csv') # Replace with your actual label file
    return X.values, y.values

# Load data
X, y = load_data()

# Initialize KFold
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# Store the results of each fold
mae_results = []
rmse_results = []

# 10-Fold Cross-Validation
for fold, (train_idx, val_idx) in enumerate(kf.split(X)):
    print(f"Training fold {fold+1}")

    # Split data into training and validation sets for this fold
    X_train, X_val = X[train_idx], X[val_idx]
    y_train, y_val = y[train_idx], y[val_idx]

    # Define your model here (adjust based on your architecture)
    model = build_model(input_shape) # Placeholder for your model-building function

    # Train the model
    model.fit(X_train, y_train, epochs=20, batch_size=32, validation_data=(X_val, y_val))

    # Predict on the validation set
    y_pred = model.predict(X_val)

    # Calculate MAE and RMSE
    mae = mean_absolute_error(y_val, y_pred)
    rmse = sqrt(mean_squared_error(y_val, y_pred))

    # Store the results for this fold
    mae_results.append(mae)
    rmse_results.append(rmse)
```

```
print(f"Fold {fold+1} - MAE: {mae:.4f} mg/dL, RMSE: {rmse:.4f} mg/dL")

# Summary of the 10-Fold Results
print("\n10-Fold Cross-Validation Results:")
print(f"MAE - Min: {min(mae_results):.4f} mg/dL, Max: {max(mae_results):.4f} mg/dL, Avg: {np.mean(mae_results):.4f} mg/dL")
print(f"RMSE - Min: {min(rmse_results):.4f} mg/dL, Max: {max(rmse_results):.4f} mg/dL, Avg: {np.mean(rmse_results):.4f} mg/dL")

# Example Output:
# MAE - Min: 0.76 mg/dL, Max: 3.75 mg/dL, Avg: 1.98 mg/dL
# RMSE - Min: 1.26 mg/dL, Max: 7.10 mg/dL, Avg: 3.42 mg/dL
```

Training fold 1

Epoch 1/20

4/4  4s 212ms/step - loss: 8067.4976 - mae: 75.9529 - val_loss: 502983.6875 - val_mae: 707.9941

Epoch 2/20

4/4  0s 74ms/step - loss: 1271.0818 - mae: 29.4275 - val_loss: 183960.4062 - val_mae: 427.8860

Epoch 3/20

4/4  0s 59ms/step - loss: 738.8511 - mae: 21.7049 - val_loss: 246425.7969 - val_mae: 495.4479

Epoch 4/20

4/4  0s 59ms/step - loss: 1054.9324 - mae: 26.5357 - val_loss: 61353.9609 - val_mae: 246.5958

Epoch 5/20

4/4  0s 69ms/step - loss: 940.0266 - mae: 23.8112 - val_loss: 49736.9922 - val_mae: 221.8973

Epoch 6/20

4/4  0s 58ms/step - loss: 562.6653 - mae: 18.5658 - val_loss: 61165.9922 - val_mae: 246.2966

Epoch 7/20

4/4  0s 58ms/step - loss: 762.1208 - mae: 22.9873 - val_loss: 18719.6250 - val_mae: 135.2954

Epoch 8/20

4/4  0s 57ms/step - loss: 745.7371 - mae: 21.2838 - val_loss: 17268.0586 - val_mae: 129.8553

Epoch 9/20

4/4  0s 61ms/step - loss: 565.0233 - mae: 17.9812 - val_loss: 20498.7812 - val_mae: 141.7413

Epoch 10/20

4/4  0s 70ms/step - loss: 604.0273 - mae: 19.0436 - val_loss: 6356.8506 - val_mae: 77.3038

Epoch 11/20

4/4  0s 58ms/step - loss: 650.5244 - mae: 19.5085 - val_loss: 6997.7383 - val_mae: 81.3312

Epoch 12/20

4/4  0s 57ms/step - loss: 476.3395 - mae: 16.6611 - val_loss: 6749.1729 - val_mae: 79.7894

Epoch 13/20

4/4  0s 69ms/step - loss: 456.0274 - mae: 17.0685 - val_loss: 4192.3257 - val_mae: 61.7542

Epoch 14/20

4/4  0s 61ms/step - loss: 439.3757 - mae: 16.4878 - val_loss: 3590.7410 - val_mae: 56.6980

Epoch 15/20

4/4  0s 58ms/step - loss: 676.2109 - mae: 20.4011 - val_loss: 2284.5854 - val_mae: 44.3439

Epoch 16/20

4/4  0s 68ms/step - loss: 416.2477 - mae: 15.7978 - val_loss: 2324.4800 - val_mae: 44.6395

Epoch 17/20

4/4  0s 79ms/step - loss: 550.3314 - mae: 18.2192 - val_loss: 2025.6377 - val_mae: 41.1698

Epoch 18/20

4/4  0s 86ms/step - loss: 375.7581 - mae: 15.1876 - val_loss: 1113.1038 - val_mae: 28.5461

Epoch 19/20

4/4  0s 72ms/step - loss: 467.9603 - mae: 16.9538 - val_loss: 1823.4902 - val_mae: 38.7050

Epoch 20/20

4/4 ————— 0s 88ms/step - loss: 677.1835 - mae: 20.7034 - val_loss: 1721.5243 - val_mae: 37.6053

WARNING:tensorflow:6 out of the last 15 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7f410d490e00> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

1/1 ————— 0s 323ms/step
Fold 1 - MAE: 37.6053 mg/dL, RMSE: 41.4913 mg/dL
Training fold 2
Epoch 1/20
4/4 ————— 4s 210ms/step - loss: 7815.1528 - mae: 75.5744 - val_loss: 1080131.2500
- val_mae: 1037.8933
Epoch 2/20
4/4 ————— 0s 62ms/step - loss: 1189.3123 - mae: 27.7764 - val_loss: 169290.6875 -
val_mae: 410.5667
Epoch 3/20
4/4 ————— 0s 59ms/step - loss: 1237.2371 - mae: 27.1211 - val_loss: 331111.6250 -
val_mae: 574.5568
Epoch 4/20
4/4 ————— 0s 70ms/step - loss: 1130.5803 - mae: 27.5137 - val_loss: 82108.5000 - v
al_mae: 285.6627
Epoch 5/20
4/4 ————— 0s 60ms/step - loss: 776.3866 - mae: 21.8293 - val_loss: 48141.8125 - va
l_mae: 218.4488
Epoch 6/20
4/4 ————— 0s 69ms/step - loss: 771.8926 - mae: 21.5519 - val_loss: 61785.3203 - va
l_mae: 247.6975
Epoch 7/20
4/4 ————— 0s 74ms/step - loss: 562.7969 - mae: 19.5481 - val_loss: 26060.6680 - va
l_mae: 160.3268
Epoch 8/20
4/4 ————— 0s 58ms/step - loss: 636.4147 - mae: 19.4034 - val_loss: 18239.5820 - va
l_mae: 133.8172
Epoch 9/20
4/4 ————— 0s 73ms/step - loss: 456.8747 - mae: 16.7920 - val_loss: 15851.0000 - va
l_mae: 124.6068
Epoch 10/20
4/4 ————— 0s 69ms/step - loss: 634.0591 - mae: 20.1411 - val_loss: 10324.6270 - va
l_mae: 100.0475
Epoch 11/20
4/4 ————— 0s 58ms/step - loss: 683.7705 - mae: 19.2910 - val_loss: 6829.6592 - val
_mae: 80.7490
Epoch 12/20
4/4 ————— 0s 61ms/step - loss: 663.1086 - mae: 19.9610 - val_loss: 4521.5703 - val
_mae: 65.1159
Epoch 13/20
4/4 ————— 0s 58ms/step - loss: 499.0977 - mae: 17.3377 - val_loss: 3856.1333 - val
_mae: 60.1366
Epoch 14/20
4/4 ————— 0s 70ms/step - loss: 591.7652 - mae: 18.6980 - val_loss: 2545.9512 - val
_mae: 48.6732
Epoch 15/20
4/4 ————— 0s 69ms/step - loss: 512.9595 - mae: 17.3179 - val_loss: 2013.4620 - val
_mae: 43.0573
Epoch 16/20
4/4 ————— 0s 61ms/step - loss: 541.2472 - mae: 17.9996 - val_loss: 2184.1116 - val
_mae: 44.9570
Epoch 17/20
4/4 ————— 0s 57ms/step - loss: 547.8647 - mae: 18.7075 - val_loss: 1003.9839 - val
_mae: 29.1414
Epoch 18/20
4/4 ————— 0s 70ms/step - loss: 596.9131 - mae: 18.8600 - val_loss: 1172.4700 - val
_mae: 31.9534
Epoch 19/20
4/4 ————— 0s 68ms/step - loss: 522.1737 - mae: 17.7927 - val_loss: 850.1053 - val_

mae: 26.2690
Epoch 20/20
4/4  0s 60ms/step - loss: 547.2708 - mae: 18.2334 - val_loss: 807.7628 - val_mae: 25.4193
1/1  0s 223ms/step
Fold 2 - MAE: 25.4193 mg/dL, RMSE: 28.4212 mg/dL
Training fold 3
Epoch 1/20
4/4  5s 220ms/step - loss: 7992.8379 - mae: 76.8631 - val_loss: 558310.3750 - val_mae: 745.7996
Epoch 2/20
4/4  0s 74ms/step - loss: 1229.8269 - mae: 28.2308 - val_loss: 136674.1875 - val_mae: 368.7037
Epoch 3/20
4/4  0s 69ms/step - loss: 893.5941 - mae: 23.6065 - val_loss: 212676.7500 - val_mae: 460.2258
Epoch 4/20
4/4  0s 60ms/step - loss: 1105.1190 - mae: 26.2646 - val_loss: 50348.4961 - val_mae: 223.3707
Epoch 5/20
4/4  0s 60ms/step - loss: 720.5695 - mae: 20.8818 - val_loss: 37313.5000 - val_mae: 192.1065
Epoch 6/20
4/4  0s 58ms/step - loss: 510.4142 - mae: 18.5044 - val_loss: 37523.5391 - val_mae: 192.6909
Epoch 7/20
4/4  0s 57ms/step - loss: 582.6371 - mae: 19.1387 - val_loss: 20126.0625 - val_mae: 140.6061
Epoch 8/20
4/4  0s 60ms/step - loss: 688.8631 - mae: 20.4590 - val_loss: 11628.1514 - val_mae: 106.2452
Epoch 9/20
4/4  0s 58ms/step - loss: 600.6526 - mae: 18.8185 - val_loss: 8635.7148 - val_mae: 91.1297
Epoch 10/20
4/4  0s 69ms/step - loss: 613.9332 - mae: 18.8118 - val_loss: 7357.2256 - val_mae: 83.8766
Epoch 11/20
4/4  0s 68ms/step - loss: 412.4901 - mae: 16.0639 - val_loss: 3844.3398 - val_mae: 59.7281
Epoch 12/20
4/4  0s 61ms/step - loss: 547.4401 - mae: 18.3184 - val_loss: 2934.9690 - val_mae: 52.0186
Epoch 13/20
4/4  0s 68ms/step - loss: 733.2134 - mae: 19.3603 - val_loss: 3552.2197 - val_mae: 57.3584
Epoch 14/20
4/4  0s 68ms/step - loss: 614.1671 - mae: 19.3126 - val_loss: 1772.1907 - val_mae: 39.8289
Epoch 15/20
4/4  0s 69ms/step - loss: 460.0972 - mae: 17.6183 - val_loss: 1493.3654 - val_mae: 36.2350
Epoch 16/20
4/4  0s 61ms/step - loss: 515.1412 - mae: 17.5724 - val_loss: 1261.6423 - val_mae: 32.9049
Epoch 17/20
4/4  0s 68ms/step - loss: 608.2446 - mae: 18.1024 - val_loss: 874.8368 - val_mae: 26.3033
Epoch 18/20

4/4 ————— 0s 57ms/step - loss: 463.6959 - mae: 17.0385 - val_loss: 571.8232 - val_mae: 19.6885
Epoch 19/20

4/4 ————— 0s 69ms/step - loss: 477.1699 - mae: 17.7285 - val_loss: 708.9618 - val_mae: 22.7632
Epoch 20/20

4/4 ————— 0s 61ms/step - loss: 491.9087 - mae: 17.3793 - val_loss: 720.4871 - val_mae: 22.9982

1/1 ————— 0s 223ms/step
Fold 3 - MAE: 22.9982 mg/dL, RMSE: 26.8419 mg/dL
Training fold 4
Epoch 1/20

4/4 ————— 5s 268ms/step - loss: 7984.4829 - mae: 76.5110 - val_loss: 789676.0625 - val_mae: 887.3058
Epoch 2/20

4/4 ————— 1s 87ms/step - loss: 1411.4009 - mae: 29.6684 - val_loss: 148626.6562 - val_mae: 384.5004
Epoch 3/20

4/4 ————— 1s 73ms/step - loss: 1479.0280 - mae: 31.5906 - val_loss: 318031.2188 - val_mae: 562.9927
Epoch 4/20

4/4 ————— 0s 70ms/step - loss: 1424.7776 - mae: 31.3103 - val_loss: 63211.5938 - val_mae: 250.4109
Epoch 5/20

4/4 ————— 0s 69ms/step - loss: 1050.5388 - mae: 25.3063 - val_loss: 29683.1543 - val_mae: 171.1343
Epoch 6/20

4/4 ————— 0s 58ms/step - loss: 820.2905 - mae: 22.8783 - val_loss: 62071.0547 - val_mae: 248.2380
Epoch 7/20

4/4 ————— 0s 71ms/step - loss: 803.7780 - mae: 23.7885 - val_loss: 34977.9766 - val_mae: 185.9934
Epoch 8/20

4/4 ————— 0s 71ms/step - loss: 743.4000 - mae: 21.4153 - val_loss: 8441.5059 - val_mae: 90.1570
Epoch 9/20

4/4 ————— 0s 70ms/step - loss: 758.4037 - mae: 22.1650 - val_loss: 11814.3506 - val_mae: 107.2380
Epoch 10/20

4/4 ————— 0s 57ms/step - loss: 501.0512 - mae: 17.8983 - val_loss: 14766.5234 - val_mae: 120.2079
Epoch 11/20

4/4 ————— 0s 71ms/step - loss: 655.8016 - mae: 20.4503 - val_loss: 5359.0513 - val_mae: 71.1847
Epoch 12/20

4/4 ————— 0s 60ms/step - loss: 556.6709 - mae: 18.0252 - val_loss: 1780.4932 - val_mae: 38.7596
Epoch 13/20

4/4 ————— 0s 59ms/step - loss: 691.8293 - mae: 19.6231 - val_loss: 3302.8770 - val_mae: 54.9700
Epoch 14/20

4/4 ————— 0s 58ms/step - loss: 572.8002 - mae: 19.6296 - val_loss: 1799.7058 - val_mae: 39.0687
Epoch 15/20

4/4 ————— 0s 70ms/step - loss: 451.1381 - mae: 16.4786 - val_loss: 682.1761 - val_mae: 23.6003
Epoch 16/20

4/4 ————— 0s 71ms/step - loss: 726.5234 - mae: 20.4157 - val_loss: 1477.7485 - val_mae: 34.7820

Epoch 17/20
4/4  0s 69ms/step - loss: 630.2964 - mae: 19.4571 - val_loss: 703.4202 - val_mae: 23.8260
Epoch 18/20
4/4  0s 60ms/step - loss: 508.3765 - mae: 17.6440 - val_loss: 460.2081 - val_mae: 19.1443
Epoch 19/20
4/4  0s 70ms/step - loss: 550.4442 - mae: 17.6963 - val_loss: 539.1419 - val_mae: 20.7861
Epoch 20/20
4/4  0s 58ms/step - loss: 490.4552 - mae: 18.3529 - val_loss: 266.3026 - val_mae: 14.2764
1/1  0s 232ms/step
Fold 4 - MAE: 14.2764 mg/dL, RMSE: 16.3188 mg/dL
Training fold 5
Epoch 1/20
4/4  4s 204ms/step - loss: 8000.1670 - mae: 76.5330 - val_loss: 928038.6250 - val_mae: 961.5959
Epoch 2/20
4/4  0s 74ms/step - loss: 1282.6262 - mae: 29.3286 - val_loss: 129567.8828 - val_mae: 358.7431
Epoch 3/20
4/4  0s 69ms/step - loss: 1088.8711 - mae: 25.5198 - val_loss: 249780.0000 - val_mae: 498.5692
Epoch 4/20
4/4  0s 89ms/step - loss: 1259.5883 - mae: 28.8809 - val_loss: 87296.5938 - val_mae: 294.3301
Epoch 5/20
4/4  1s 86ms/step - loss: 875.7717 - mae: 23.4063 - val_loss: 34822.4844 - val_mae: 185.3292
Epoch 6/20
4/4  1s 84ms/step - loss: 634.9075 - mae: 19.5204 - val_loss: 38175.1172 - val_mae: 194.2020
Epoch 7/20
4/4  1s 85ms/step - loss: 441.8565 - mae: 16.6761 - val_loss: 30282.9219 - val_mae: 172.7762
Epoch 8/20
4/4  0s 92ms/step - loss: 530.1347 - mae: 18.6764 - val_loss: 15235.3105 - val_mae: 121.7981
Epoch 9/20
4/4  0s 62ms/step - loss: 569.4257 - mae: 19.1774 - val_loss: 10481.9941 - val_mae: 100.4287
Epoch 10/20
4/4  0s 69ms/step - loss: 611.6792 - mae: 18.8407 - val_loss: 10200.0957 - val_mae: 99.0079
Epoch 11/20
4/4  0s 59ms/step - loss: 506.2182 - mae: 18.3563 - val_loss: 8012.8008 - val_mae: 87.3736
Epoch 12/20
4/4  0s 69ms/step - loss: 484.0908 - mae: 17.0774 - val_loss: 2938.5635 - val_mae: 51.6439
Epoch 13/20
4/4  0s 70ms/step - loss: 608.1486 - mae: 19.8948 - val_loss: 3777.9697 - val_mae: 58.9278
Epoch 14/20
4/4  0s 58ms/step - loss: 532.4969 - mae: 19.5757 - val_loss: 2336.2612 - val_mae: 46.1373
Epoch 15/20
4/4  0s 60ms/step - loss: 561.0967 - mae: 17.6947 - val_loss: 1157.7751 - val_mae: 46.1373

```

_mae: 31.4349
Epoch 16/20
4/4 ————— 0s 58ms/step - loss: 577.0057 - mae: 18.2274 - val_loss: 1381.0369 - val_
_mae: 34.7381
Epoch 17/20
4/4 ————— 0s 59ms/step - loss: 602.1469 - mae: 19.8459 - val_loss: 812.9980 - val_
mae: 25.3395
Epoch 18/20
4/4 ————— 0s 60ms/step - loss: 521.6210 - mae: 17.5404 - val_loss: 522.9639 - val_
mae: 19.2174
Epoch 19/20
4/4 ————— 0s 70ms/step - loss: 472.5269 - mae: 17.6140 - val_loss: 1073.4681 - val_
_mae: 29.6986
Epoch 20/20
4/4 ————— 0s 69ms/step - loss: 503.0553 - mae: 17.3718 - val_loss: 322.3452 - val_
mae: 14.0466
1/1 ————— 0s 225ms/step
Fold 5 - MAE: 14.0466 mg/dL, RMSE: 17.9540 mg/dL

```

10-Fold Cross-Validation Results:

```

MAE - Min: 14.0466 mg/dL, Max: 37.6053 mg/dL, Avg: 22.8692 mg/dL
RMSE - Min: 16.3188 mg/dL, Max: 41.4913 mg/dL, Avg: 26.2054 mg/dL

```

```

In [ ]: import numpy as np
import pandas as pd
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Function to calculate MAPE
def mean_absolute_percentage_error(y_true, y_pred):
    # Avoid division by zero by using np.finfo
    return np.mean(np.abs((y_true - y_pred) / y_true)) * 100

# Function to evaluate the model
def evaluate_model(model, X_test, y_test):
    # Predict the output on the test set
    y_pred = model.predict(X_test)

    # Calculate the metrics
    mae = mean_absolute_error(y_test, y_pred)
    mape = mean_absolute_percentage_error(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    r2 = r2_score(y_test, y_pred)

    # Print the evaluation metrics
    print(f"Model Evaluation Metrics:")
    print(f"Mean Absolute Error (MAE): {mae:.4f}")
    print(f"Mean Absolute Percentage Error (MAPE): {mape:.4f}%")
    print(f"Root Mean Squared Error (RMSE): {rmse:.4f}")
    print(f"R² Score: {r2:.4f}")

    return mae, mape, rmse, r2

# Example of how to evaluate a trained model
# Assuming that the model is already trained and you have X_test and y_test
# You can call this function to evaluate your model

# Example for a trained model (replace with actual test data and trained model)
# model = your_trained_model
# X_test = your_test_features
# y_test = your_test_labels

```



```
# Call the evaluate function
evaluate_model(model, X_test, y_test)
```

1/2 ————— 0s 122ms/step

WARNING:tensorflow:6 out of the last 7 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x79ba333d1580> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

2/2 ————— 1s 417ms/step

Model Evaluation Metrics:

Mean Absolute Error (MAE): 13.0509

Mean Absolute Percentage Error (MAPE): 11.6566%

Root Mean Squared Error (RMSE): 20.4937

R² Score: 0.0839

```
Out[ ]: (13.050942420959473,
        11.656601471505281,
        20.493686355068526,
        0.0839085578918457)
```

```
In [ ]: # Function to predict a single value from the model
def predict_single_value(model, X_test, y_test, index=0):
    # Select a single sample from X_test
    single_sample = X_test[index:index+1] # Select a single sample (index-based)

    # Predict the output for this single sample
    prediction = model.predict(single_sample)

    # Get the actual value for this sample
    actual_value = y_test[index]

    # Print the prediction and actual value
    print(f"Predicted Value: {prediction[0]}")
    print(f"Actual Value: {actual_value}")

    return prediction[0], actual_value
```

```
In [ ]: # Call the function to predict a single value (e.g., the 0th sample)
predicted_value, actual_value = predict_single_value(model, X_test, y_test, index=0)
```

1/1 ————— 1s 752ms/step

Predicted Value: [109.479034]

Actual Value: 103

```
In [ ]: # Function to predict multiple values (e.g., 50 values) from the model
def predict_multiple_values(model, X_test, y_test, start_index=0, num_samples=50):
    # Select a batch of samples from X_test
    batch_samples = X_test[start_index:start_index+num_samples] # Select a batch of samples (e.g., 50 samples)

    # Predict the output for these samples
    predictions = model.predict(batch_samples)

    # Get the actual values for these samples
    actual_values = y_test[start_index:start_index+num_samples]
```

```
# Print the predictions and actual values for the first 50 samples
for i in range(num_samples):
    print(f"Prediction {i+1}: Predicted Value = {predictions[i]}, Actual Value = {actual_valu

return predictions, actual_values
```

```
In [ ]: # Call the function to predict 50 values (starting from index 0)
predicted_values, actual_values = predict_multiple_values(model, X_test, y_test, start_index=0, i
```

2/2 — 0s 116ms/step

Prediction 1: Predicted Value = [109.47907], Actual Value = 103
Prediction 2: Predicted Value = [110.59109], Actual Value = 102
Prediction 3: Predicted Value = [116.92168], Actual Value = 110
Prediction 4: Predicted Value = [110.93603], Actual Value = 99
Prediction 5: Predicted Value = [112.47089], Actual Value = 130
Prediction 6: Predicted Value = [125.69503], Actual Value = 112
Prediction 7: Predicted Value = [120.5676], Actual Value = 127
Prediction 8: Predicted Value = [111.713165], Actual Value = 96
Prediction 9: Predicted Value = [108.301315], Actual Value = 94
Prediction 10: Predicted Value = [110.037636], Actual Value = 110
Prediction 11: Predicted Value = [124.64377], Actual Value = 103
Prediction 12: Predicted Value = [111.89162], Actual Value = 124
Prediction 13: Predicted Value = [119.06144], Actual Value = 110
Prediction 14: Predicted Value = [118.65339], Actual Value = 106
Prediction 15: Predicted Value = [114.32812], Actual Value = 109
Prediction 16: Predicted Value = [115.09415], Actual Value = 130
Prediction 17: Predicted Value = [110.70096], Actual Value = 88
Prediction 18: Predicted Value = [118.109024], Actual Value = 88
Prediction 19: Predicted Value = [109.22319], Actual Value = 96
Prediction 20: Predicted Value = [126.259834], Actual Value = 183
Prediction 21: Predicted Value = [110.280685], Actual Value = 106
Prediction 22: Predicted Value = [116.92236], Actual Value = 110
Prediction 23: Predicted Value = [116.92548], Actual Value = 110
Prediction 24: Predicted Value = [118.40086], Actual Value = 183
Prediction 25: Predicted Value = [111.710556], Actual Value = 96
Prediction 26: Predicted Value = [110.97395], Actual Value = 100
Prediction 27: Predicted Value = [115.171524], Actual Value = 128
Prediction 28: Predicted Value = [110.940346], Actual Value = 99
Prediction 29: Predicted Value = [108.296486], Actual Value = 94
Prediction 30: Predicted Value = [125.38256], Actual Value = 124
Prediction 31: Predicted Value = [125.699425], Actual Value = 112
Prediction 32: Predicted Value = [111.598694], Actual Value = 106
Prediction 33: Predicted Value = [114.30421], Actual Value = 109
Prediction 34: Predicted Value = [124.66219], Actual Value = 103
Prediction 35: Predicted Value = [126.26971], Actual Value = 183
Prediction 36: Predicted Value = [122.596664], Actual Value = 146
Prediction 37: Predicted Value = [108.831856], Actual Value = 100
Prediction 38: Predicted Value = [115.08807], Actual Value = 130
Prediction 39: Predicted Value = [119.65001], Actual Value = 95
Prediction 40: Predicted Value = [125.48766], Actual Value = 110
Prediction 41: Predicted Value = [116.91798], Actual Value = 110
Prediction 42: Predicted Value = [110.618126], Actual Value = 102
Prediction 43: Predicted Value = [111.12159], Actual Value = 129
Prediction 44: Predicted Value = [125.71091], Actual Value = 112
Prediction 45: Predicted Value = [111.60733], Actual Value = 118
Prediction 46: Predicted Value = [112.589554], Actual Value = 113
Prediction 47: Predicted Value = [118.13957], Actual Value = 103
Prediction 48: Predicted Value = [122.579056], Actual Value = 146
Prediction 49: Predicted Value = [110.38217], Actual Value = 136
Prediction 50: Predicted Value = [111.133865], Actual Value = 129

```
In [ ]: # Assuming your model is already defined as 'model1'
        model.save('/content/glucose_model.h5') # Save with a new name
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

```
In [ ]: from google.colab import files
```

```
# For H5 format  
files.download('/content/glucose_model.h5')
```

```
In [ ]: # Save only the model weights (you can specify the file name and format)  
model.save_weights('/content/glucose_model.weights.h5') # H5 format
```

```
In [ ]: from google.colab import files
```

```
# For H5 format  
files.download('/content/glucose_model.weights.h5')
```

```
In [ ]: print(f"Expected input shape: {model_glucose.input_shape}")  
print(f"Expected input shape for BP model: {model_bp.input_shape}")
```

Expected input shape: (None, 2142, 1)

Expected input shape for BP model: (None, 1)

```
In [ ]: from tensorflow.keras.models import load_model  
import numpy as np
```

```
# Load the two models  
model_glucose = load_model('/content/glucose_model.h5') # Model for glucose prediction  
model_bp = load_model('/content/bp_model.h5') # Model for blood pressure prediction  
  
# Check the expected input shape for both models  
print(f"Expected input shape for glucose model: {model_glucose.input_shape}")  
print(f"Expected input shape for BP model: {model_bp.input_shape}")  
  
# Reshape the single sample to match the input shape for each model  
  
# For glucose model (expects shape (None, 2142, 1)), reshape the input to have 2142 features with  
single_sample_glucose = X_test[0].reshape(1, 2142, 1) # 1 sample, 2142 features, 1 channel  
  
# For BP model (expects shape (None, 1)), select the first feature for the BP prediction  
single_sample_bp = X_test[0][0].reshape(1, 1) # Select first feature (assuming 2142 features) for  
  
# Predict glucose using the first model  
glucose_prediction = model_glucose.predict(single_sample_glucose)  
  
# Predict blood pressure using the second model  
bp_prediction = model_bp.predict(single_sample_bp)  
  
# Print the predictions for glucose and blood pressure  
print(f"Predicted Glucose: {glucose_prediction[0]}")  
print(f"Predicted BP: {bp_prediction[0]}")  
  
# Optionally, if you want to compare these predictions to actual values:  
actual_glucose = y_test[0]  
actual_bp = y_test[1] # Assuming y_test contains both glucose and blood pressure  
  
print(f"Actual Glucose: {actual_glucose}")  
print(f"Actual BP: {actual_bp}")
```

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.com
pile_metrics` will be empty until you train or evaluate the model.

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.com
pile_metrics` will be empty until you train or evaluate the model.

Expected input shape for glucose model: (None, 2142, 1)

Expected input shape for BP model: (None, 1)

1/1  0s 368ms/step

1/1  0s 81ms/step

Predicted Glucose: [103.41918]

Predicted BP: [109.73297]

Actual Glucose: 103

Actual BP: 102